KingbaseES 数据库管理员指南

金仓数据库管理系统 KingbaseES

文档版本：V9（V009R001C010）

发布日期：2025 年 12 月 3 日

中电科金仓（北京）科技股份有限公司

目　录

第 1 章　数据库管理概述 1
1.1 KingbaseES 数据库的用户类型 1
1.1.1 数据库管理员 1
1.1.2 安全管理员 2
1.1.3 应用开发人员 2
1.1.4 应用管理员 2
1.1.5 数据库用户 2
1.2 数据库管理员的任务 3
1.2.1 评估数据库使用的硬件资源 3
1.2.2 安装数据库软件 3
1.2.3 数据库规划 3
1.2.4 创建数据库 3
1.2.5 备份数据库 4
1.2.6 创建用户 4
1.2.7 实施数据库设计 4
1.2.8 备份功能完整的数据库 4
1.2.9 性能调优 4
1.3 SQL 语句 4
1.3.1 向数据库提交命令和 SQL 5
1.3.2 关于 Ksql 5
1.3.3 连接数据库 5
1.3.4 关于使用 Ksql 连接到数据库 5
1.3.5 Step 1：打开命令窗口 6
1.3.6 Step 2：设置操作系统环境变量 6
1.3.7 Step 3：启动 Ksql 6
1.3.8 Step 4：Ksql 连接到 KingbaseES 数据库 6
1.4 确定数据库的软件版本 7
1.4.1 获取当前数据库的版本信息 7
1.5 数据库管理员的安全和权限 7
1.5.1 管理员用户 7
1.5.2 管理员权限 8

1.6 数据库管理员认证 8
1.7 数据操作工具 8
1.7.1 数据加载工具 9
1.7.2 数据导出导入工具 9

第 2 章 配置管理 10
2.1 设置参数 10
2.1.1 参数名称和值 10
2.1.2 通过配置文件设置参数 11
2.1.3 通过 SQL 设置参数 12
2.1.4 通过 Shell 设置参数 13
2.1.5 管理配置文件内容 13
2.1.6 不同模式下参数差异 14
2.2 参数作用域 15
2.2.1 INTERNAL—内部级 15
2.2.2 KINGBASE—实例级 15
2.2.3 SIGHUP—会话级 15
2.2.4 SUPERUSER_BACKEND—超级用户后台级 16
2.2.5 BACKEND—后台级 16
2.2.6 SUPERUSER—超级用户级 16
2.2.7 USER—用户级 16
2.3 配置文件 16
2.4 连接和认证 18
2.4.1 连接设置 18
2.4.2 安全和认证 25
2.4.3 SSL 27
2.5 资源消耗 33
2.5.1 内存 33
2.5.2 磁盘 37
2.5.3 内核资源使用 38
2.5.4 基于代价的清理延迟 38
2.5.5 后台写入器 41
2.5.6 异步行为 42
2.5.7 全局临时表 46
2.6 预写式日志 46
2.6.1 设置 46
2.6.2 检查点 54
2.6.3 归档 56
2.6.4 归档文件恢复 57
2.6.5 恢复目标 59
2.6.6 lz4 压缩说明 62
2.7 复制 62

2.7.1 发送服务器 63
2.7.2 主控服务器 66
2.7.3 后备服务器 68
2.7.4 订阅者 73
2.8 查询计划 74
2.8.1 规划器方法配制 74
2.8.2 规划器代价常量 80
2.8.3 遗传查询优化 85
2.8.4 其他规划器选项 88
2.9 错误报告和日志 93
2.9.1 在哪里记录日志 93
2.9.2 什么时候记录日志 100
2.9.3 记录什么到日志 102
2.9.4 使用 CSV 格式的日志输出 110
2.9.5 进程标题 111
2.10 运行时统计数据 112
2.10.1 查询和索引统计收集器 112
2.10.2 统计监控 117
2.11 自动清理 117
2.12 客户端连接默认 122
2.12.1 语句动作 122
2.12.2 本地化和格式化 133
2.12.3 共享库预先加载 137
2.12.4 其他默认值 139
2.13 锁管理 140
2.14 版本和平台兼容性 142
2.14.1 KingbaseES 老版本 142
2.14.2 平台和客户端兼容性 146
2.14.3 一般特性兼容性 147
2.15 错误处理 154
2.16 预置选项 155
2.17 定制选项 161
2.18 开发者选项 163
2.19 短选项 173
2.20 控制文件 174
2.20.1 控制文件多路备份 174
2.21 函数别名作为列名 174
2.22 数据库审计 175
2.23 标记和强制访问控制 177
2.24 性能调优 178

第 3 章 存储管理 179

3.1 控制文件管理 179
3.1.1 什么是控制文件？ 179
3.1.2 控制文件准则 180
3.1.2.1 控制文件路径 180
3.1.2.2 控制文件的大小 180
3.1.3 创建控制文件 180
3.1.3.1 控制文件内部信息的创建 180
3.1.4 备份控制文件 180
3.1.5 读取控制文件信息 181
3.2 重做日志管理 181
3.2.1 什么是重做日志？ 181
3.2.1.1 WalWriter 进程 182
3.2.1.2 重做日志的内容 182
3.2.1.3 KingbaseES 数据库如何写入重做日志 182
3.2.2 规划重做日志 183
3.2.2.1 重做日志段文件的大小 183
3.2.2.2 重做日志文件的块大小 183
3.2.2.3 重做日志文件的数量 184
3.2.2.4 重做日志记录延迟拷贝 184
3.2.3 手动切换重做日志 184
3.2.4 验证重做日志文件中的块 185
3.2.5 清除重做日志文件 185
3.3 归档重做日志文件管理 186
3.3.1 什么是归档重做日志？ 186
3.3.2 开启和关闭归档重做日志 186
3.3.2.1 开启归档重做日志 187
3.3.2.2 关闭归档重做日志 187
3.3.3 控制归档 187
3.3.3.1 设置数据库归档模式 187
3.3.3.2 更改数据库归档模式 188
3.3.3.3 执行归档 188
3.3.4 指定归档目标 188
3.3.4.1 设置存档目标的初始化参数 189
3.3.4.2 归档的状态 189
3.3.5 查看归档重做日志状态 189
3.4 表空间管理 190
3.4.1 表空间的管理指南 190
3.4.1.1 使用多个表空间 190
3.4.1.2 将表空间分配给用户 191
3.4.2 创建表空间 191
3.4.2.1 关于创建表空间 191
3.4.2.2 加密表空间 192

3.4.2.3 临时表空间 192
3.4.3 修改表空间可用性 192
3.4.3.1 使表空间脱机 192
3.4.3.2 使表空间联机 193
3.4.4 使用只读表空间 193
3.4.4.1 关于只读表空间 193
3.4.4.2 使表空间只读 194
3.4.4.3 使只读表空间可读写 194
3.4.5 修改和维护表空间 194
3.4.6 重命名表空间 195
3.4.7 删除表空间 195
3.4.8 查看表空间相关信息 195
3.5 数据文件和临时文件管理 196
3.5.1 数据文件的管理指南 196
3.5.1.1 关于数据文件 196
3.5.1.2 选择适当的文件位置 197
3.5.1.3 将数据文件和重做日志文件分开存储 197

第 4 章 内存管理 198
4.1 关于内存管理 198
4.2 内存结构概述 198
4.3 手动配置内存 199
4.3.1 系统全局区配置 199
4.3.2 进程全局区配置 200

第 5 章 进程管理 202
5.1 关于独立服务进程和共享服务进程 202
5.2 管理 KingbaseES 的独立服务进程 202
5.3 关于 KingbaseES 的后台进程 202
5.4 终止会话 202
5.4.1 指定需要终止的会话 202
5.4.2 终止会话 203
5.4.3 取消会话执行的 SQL 语句 203
5.5 进程和会话视图 203

第 6 章 用户管理 205
6.1 数据库在使用之前设置安全策略的重要性 205
6.2 管理用户 206
6.3 用户的权限和角色 206
6.4 审计数据库活动 207
6.5 预定义的账户 207

第 7 章 监控管理 208

7.1 监控错误和警报 208
7.1.1 使用 kbbadger 208
7.1.2 基本能力 208
7.1.3 使用示例 210
7.1.4 适用场景 211
7.2 监控数据库性能 211
7.2.1 监控锁 211
7.2.1.1 使用 sys_locks 212
7.2.1.2 监控死锁 212
7.2.2 监控等待事件 213
7.2.2.1 等待事件 214
7.2.3 性能监控视图 216

第 8 章 数据库对象管理 220
8.1 模式对象的管理 220
8.1.1 关于默认模式 221
8.1.2 分析表和索引以及聚类 221
8.1.2.1 关于分析表、索引和聚类 221
8.1.2.2 使用 sys_stat_statement 搜集执行语句的统计信息 222
8.1.2.3 分析表、索引、聚类和物化视图 222
8.1.3 清空表 223
8.1.3.1 使用 DELETE 清空表 223
8.1.3.2 使用 DROP 和 CREATE 清空表 223
8.1.3.3 使用 TRUNCATE 清空表 224
8.1.4 启用和禁用触发器 224
8.1.4.1 关于启用和禁用触发器 224
8.1.4.2 启用触发器 225
8.1.4.3 禁用触发器 226
8.1.5 管理完整性约束 226
8.1.5.1 完整性约束状态 227
8.1.5.1.1 关于完整性约束状态 227
8.1.5.1.2 关于禁用约束 228
8.1.5.1.3 关于启用约束 228
8.1.5.1.4 完整性约束的有效使用: 顺序 228
8.1.5.2 创建约束 229
8.1.5.2.1 在创建表时指定约束 229
8.1.5.2.2 在创建表后指定约束 230
8.1.5.3 修改或删除现有的约束 230
8.1.5.3.1 禁用和启用约束 231
8.1.5.3.2 重命名约束 232
8.1.5.3.3 删除约束 232
8.1.5.4 延迟约束检查 232

8.1.5.4.1 将所有约束设置为延迟 . . . 232
8.1.5.4.2 检查提交 (可选) . . . 233
8.1.5.5 查看约束信息 . . . 233
8.1.6 重命名模式对象 . . . 233
8.1.7 管理对象名称解析 . . . 234
8.1.8 切换到其他模式 . . . 234
8.1.9 显示模式对象的信息 . . . 234
8.2 表的管理 . . . 235
8.2.1 关于表 . . . 236
8.2.2 表管理指南 . . . 236
8.2.2.1 创建表之前的设计准则 . . . 237
8.2.2.2 指定表的存储位置 . . . 237
8.2.2.3 在创建表时考虑使用 UNLOGGED . . . 237
8.2.2.4 评估表格大小和相应的规划 . . . 238
8.2.3 创建表 . . . 238
8.2.3.1 创建表示例 . . . 238
8.2.3.2 创建临时表 . . . 239
8.2.3.2.1 临时表概述 . . . 240
8.2.3.2.2 创建临时表的注意事项 . . . 240
8.2.3.2.3 临时表事务控制 . . . 240
8.2.3.2.4 创建全局临时表 . . . 241
8.2.3.2.5 创建局部临时表 . . . 241
8.2.4 加载表 . . . 242
8.2.4.1 sys_bulkload 工具快速加载数据 . . . 243
8.2.4.2 使用 CREATE TABLE AS 语句 . . . 244
8.2.4.3 使用 INSERT 语句 . . . 245
8.2.4.4 使用 MERGE 语句 . . . 245
8.2.4.5 使用 COPY 语句 . . . 246
8.2.5 收集数据库的统计信息 . . . 247
8.2.6 修改表 . . . 248
8.2.6.1 增加列 . . . 249
8.2.6.2 移除列 . . . 249
8.2.6.3 增加约束 . . . 250
8.2.6.4 移除约束 . . . 250
8.2.6.5 修改列的默认值 . . . 250
8.2.6.6 修改列的数据类型 . . . 251
8.2.6.7 重命名列 . . . 251
8.2.6.8 重命名表 . . . 251
8.2.7 删除表 . . . 251
8.2.8 使用 FLASHBACK 删除和恢复表 . . . 252
8.2.8.1 什么是回收站 . . . 253
8.2.8.2 启用和禁用回收站 . . . 253

8.2.8.3 从回收站恢复表 253
8.2.9 管理分区表 254
8.2.10 表的数据视图 254
8.3 索引的管理 255
8.3.1 关于索引 255
8.3.2 索引的管理指南 256
8.3.2.1 在插入表数据后创建索引 257
8.3.2.2 适合创建索引的表和列 257
8.3.2.3 考虑排序索引列来改善性能 258
8.3.2.4 限制每个表的索引的数量 258
8.3.2.5 删除不再需要的索引 258
8.3.2.6 为索引指定表空间 258
8.3.2.7 评估索引大小和设置存储参数 258
8.3.2.8 在禁用或删除约束之前考虑成本 259
8.3.3 创建索引 259
8.3.3.1 显式创建索引 260
8.3.3.2 创建唯一索引 260
8.3.3.3 创建指定顺序的 B 树索引 261
8.3.3.4 创建 Hash 索引 261
8.3.3.5 创建基于函数的索引 261
8.3.3.6 创建多列索引 262
8.3.3.7 创建本地和全局索引 262
8.3.4 修改索引 263
8.3.4.1 关于修改索引 263
8.3.4.2 修改或重建索引 263
8.3.5 删除索引 265
8.3.6 索引视图 265
8.4 视图的管理 266
8.4.1 关于视图 266
8.4.2 创建视图和连接视图 267
8.4.3 创建视图 267
8.4.4 创建连接视图 269
8.4.5 创建 FORCE 视图 269
8.4.6 替换视图 270
8.4.7 在查询中使用视图 271
8.4.8 修改视图 271
8.4.9 删除视图 272
8.4.10 视图的数据字典视图 272
8.4.10.1 视图信息的查询 272
8.5 序列的管理 273
8.5.1 关于序列 273
8.5.2 创建序列 274

8.5.3 修改序列 274
8.5.4 使用序列 275
8.5.4.1 使用 NEXTVAL 生成序列号 275
8.5.4.2 使用 CURRVAL 序列号 276
8.5.5 缓存序列号 276
8.5.5.1 关于缓存序列号 277
8.5.5.2 序列缓存中的条目数 277
8.5.5.3 缓存中的序列号数量 277
8.5.6 删除序列 277
8.5.7 序列的数据字典视图 278
8.5.7.1 序列信息的查询 278
8.6 同义词的管理 279
8.6.1 关于同义词 279
8.6.2 创建同义词 279
8.6.3 在 DML 语句中使用同义词 280
8.6.4 删除同义词 280
8.6.5 同义词的数据字典视图 281
8.6.5.1 同义词信息的查询 281

第 9 章 查询优化与任务管理 282
9.1 全文索引 282
9.1.1 介绍 282
9.1.1.1 什么是一个文档 284
9.1.1.2 基本文本匹配 285
9.1.1.3 配置 287
9.1.2 表和索引 288
9.1.2.1 搜索一个表 288
9.1.2.2 创建索引 289
9.1.3 控制文本搜索 290
9.1.3.1 解析文档 290
9.1.3.2 解析查询 291
9.1.3.3 排名搜索结果 294
9.1.3.4 加亮结果 297
9.1.4 额外特性 298
9.1.4.1 操纵文档 298
9.1.4.2 操纵查询 299
9.1.4.3 用于自动更新的触发器 301
9.1.4.4 收集文档统计数据 303
9.1.5 解析器 304
9.1.6 词典 306
9.1.6.1 停用词 307
9.1.6.2 简单词典 307

9.1.6.3 同义词词典 309
9.1.6.4 分类词典 310
9.1.6.5 Ispell 词典 313
9.1.6.6 Snowball 词典 315
9.1.7 配置例子 315
9.1.8 中文分词 317
9.1.8.1 zhparser 中文分词插件 317
9.1.8.2 sys_jieba 中文分词插件 318
9.1.9 使用 ftutilx 插件 319
9.1.9.1 抽取内容 319
9.1.9.2 联合使用方案 319
9.1.9.3 注意事项 320
9.1.10 测试和调试文本搜索 320
9.1.10.1 配置测试 320
9.1.10.2 解析器测试 323
9.1.10.3 词典测试 324
9.1.11 GIN 和 GiST 索引类型 325
9.1.12 RUM 索引类型 326
9.1.12.1 常用操作符和函数 326
9.1.12.2 操作符类 327
9.1.13 ksql 支持 330
9.1.14 限制 331
9.2 规则系统 332
9.2.1 查询树 332
9.2.2 视图和规则系统 334
9.2.2.1 SELECT 规则如何工作 334
9.2.2.2 非 SELECT 语句中的视图规则 339
9.2.2.3 KingbaseES 中视图的能力 340
9.2.2.4 更新一个视图 341
9.2.3 物化视图 341
9.2.4 INSERT、UPDATE 和 DELETE 上的规则 345
9.2.4.1 更新规则如何工作 345
9.2.4.1.1 第一个规则循序渐进 346
9.2.4.2 与视图合作 350
9.2.5 规则和权限 357
9.2.6 规则和命令状态 359
9.2.7 规则 vs 触发器 359
9.3 调度管理 361
9.3.1 数据库调度概念 361
9.3.1.1 调度器概述 362
9.3.1.2 作业和支持的调度对象 362
9.3.1.3 关于作业的更多描述 364

9.3.1.3.1 作业类别 364
9.3.1.3.2 作业实例 365
9.3.1.3.3 作业参数 365
9.3.2 数据库作业调度 365
9.3.2.1 作业调度及其子程序 365
9.3.2.2 DBMS_SCHEDULER 包使用 366
9.3.3 管理数据库调度器 370
9.3.3.1 配置调度器 370
9.3.3.2 监控和管理调度程序 370
9.3.3.3 导入/导出计划任务 372
9.3.4 管理数据库自动维护任务 372
9.3.4.1 关于自动维护任务 372
9.3.4.2 使用自动作业功能 373
9.3.4.3 启用和禁用自动维护任务 373
9.3.4.4 修改自动作业间隔时间 373

第 10 章 问题排查 375
10.1 服务连续性 375
10.1.1 影响服务连续性的典型问题及原因概览 376
10.1.2 风险/问题监控、数据采集 378
10.1.2.1 监控 379
10.1.2.1.1 集群状态类问题 379
10.1.2.1.2 资源耗尽类问题 380
10.1.2.1.3 日志 382
10.1.2.2 数据采集 382
10.1.2.2.1 日志 382
10.1.2.2.2 健康检查 382
10.1.2.2.3 集群事件 382
10.1.3 风险/问题识别 383
10.1.3.1 识别 383
10.1.3.1.1 集群状态类问题 383
10.1.3.1.2 资源耗尽类问题 383
10.1.4 风险/问题溯因 384
10.1.4.1 信息收集 384
10.1.4.1.1 日志 384
10.1.4.1.2 数据库状态检查结果 384
10.1.4.1.3 其他信息 384
10.1.4.2 溯因 385
10.1.4.2.1 集群状态类问题 385
10.1.4.2.2 资源耗尽类问题 390
10.1.5 风险/问题处理 395
10.1.5.1 影响分析 395

10.1.5.2 处理 . . . 395
10.1.5.2.1 集群状态类问题 . . . 395
10.1.5.2.2 资源耗尽类问题 . . . 400
10.2 数据防丢失 . . . 404
10.2.1 概述 . . . 404
10.2.2 影响数据丢失的典型问题及原因概览 . . . 404
10.2.2.1 经典问题 . . . 405
10.2.2.1.1 基础数据丢失 . . . 405
10.2.2.1.2 备份数据丢失 . . . 405
10.2.2.2 典型原因 . . . 406
10.2.2.2.1 存储故障 . . . 406
10.2.2.2.2 数据损坏 . . . 406
10.2.2.2.3 人为损坏 . . . 406
10.2.2.2.4 软件缺陷 . . . 407
10.2.2.2.5 不可预防的损坏或灾难 . . . 407
10.2.2.2.6 其他错误 . . . 408
10.2.3 风险/问题监控、数据采集 . . . 408
10.2.3.1 监控思路 . . . 408
10.2.3.2 具体监控内容 . . . 409
10.2.3.3 数据采集 . . . 409
10.2.4 风险/问题识别 . . . 411
10.2.4.1 基础数据分析识别 . . . 411
10.2.4.2 备份数据分析识别 . . . 412
10.2.5 风险/问题溯因 . . . 412
10.2.5.1 基础数据访问异常 . . . 412
10.2.5.2 物理备份未配置 . . . 413
10.2.5.3 未归档 WAL 累积过多 . . . 413
10.2.5.4 WAL 日志归档失败 . . . 413
10.2.5.5 data 目录设置不当导致备份失败 . . . 414
10.2.5.6 使用索引时出现 index ···contains unexpected zero page at block ···故障 . . . 414
10.2.6 风险/问题处理 . . . 416
10.2.6.1 影响分析 . . . 416
10.2.6.2 常见数据故障及问题处理 . . . 416
10.3 数据库性能 . . . 417
10.3.1 风险/问题监控、数据采集 . . . 417
10.3.2 风险/问题识别 . . . 417
10.3.3 风险/问题溯因 . . . 418
10.3.3.1 信息收集 . . . 418
10.3.3.2 溯因 . . . 418
10.3.4 风险/问题处理 . . . 419
10.3.4.1 处理方法 . . . 419
10.3.4.2 常见风险问题处理 . . . 420

10.3.5 常用的性能诊断工具 420
10.3.5.1 nmon 420
10.3.5.2 KWR、KSH 报告 423
10.3.5.3 sys_stat_activity 视图 423
10.3.5.4 sys_stat_statement 插件 423
10.3.5.5 火焰图 423
10.4 常见命令 424
10.4.1 数据库启停 424
10.4.1.1 单机环境 424
10.4.1.2 集群环境 426
10.4.2 常用集群相关操作 430
10.4.3 常用 SQL 命令 431
10.4.3.1 数据库版本的查询 431
10.4.3.2 数据库连接情况 431
10.4.3.3 查看 license 到期时间 431
10.4.3.4 查看锁进程及对象 431
10.4.3.5 kill session 431
10.4.3.6 查看慢 sql 432
10.4.3.7 统计不同类型数据库对象数量 433
10.4.3.8 获取对象 DDL 434
10.4.3.9 查询表及表行数 435
10.4.3.10 查询包含 lob 字段的表 435
10.4.3.11 查询分区表及其对应的子分区 435
10.4.3.12 无效对象 436
10.4.4 数据库常用命令工具 436
10.4.4.1 ksql 命令的使用 436
10.4.4.2 memstat 437
10.4.4.3 sys_dump 437
10.4.5 操作系统常用命令 437
10.4.5.1 查看硬件配置 437
10.4.5.2 系统配置相关 443
10.4.5.3 查看系统运行状态 454
10.4.6 软件包安装及升级管理 461

版权声明 462

服务周期承诺 463

第1章　数据库管理概述

本章节包含以下内容：

- KingbaseES 数据库的用户类型
- 数据库管理员的任务
- SQL 语句
- 确定数据库的软件版本
- 数据库管理员的安全和权限
- 数据库管理员认证
- 数据操作工具

1.1　KingbaseES 数据库的用户类型

数据库的用户类型和各自的职责取决于数据库支持的业务。小型业务可以只有一位数据库管理员来支持应用开发人员和数据库用户；大型的业务对数据库的使用则可能需要多位数据库管理员分别负责不同的数据库管理职责。

1.1.1　数据库管理员

数据库至少需要一位数据库管理员，根据数据库支持的业务规模不同，管理员角色可能由单人或是一个团队承担。数据库管理员可能承担以下职责：

- 安装、升级数据库和相关软件
- 规划、分配存储资源
- 为应用开发人员设计的应用创建存储结构（例如表空间）
- 为应用开发人员设计的应用创建对象（例如表、视图、索引）
- 根据应用开发人员的需要修改存储结构和对象
- 创建用户、维护数据库安全

- 控制、监控用户对数据库的访问
- 监控、优化数据库性能
- 规划、实施备份策略、维护归档
- 处理故障、恢复服务
- 联系 KingbaseES 的技术服务渠道获取技术支持

1.1.2　安全管理员

有些系统会由一位或多位独立的安全管理员来创建用户，控制、监控用户对数据库的访问，维护数据库安全。有独立的安全管理员的情况下，数据库管理员就不再负责以上这些职责。

参考 用户和角色管理来分配安全相关的职责。

1.1.3　应用开发人员

应用开发人员设计并实现使用数据库的应用系统，他们的职责包括：

- 设计、开发应用系统
- 为应用系统设计数据库对象
- 估算应用对存储的容量需求
- 对数据库对象做对应应用的修改
- 向数据库管理员传递以上需求
- 调试、优化应用
- 明确应用对安全的需求

应用开发人员可以联合数据库管理员共同完成上面的一些工作。

1.1.4　应用管理员

数据库支撑的应用可能有一个或多个，每个应用都可以由专门的应用管理员负责。

1.1.5　数据库用户

数据库用户通过应用或数据库工具和数据库交互，典型用户的职责包括：

- 在权限允许的情况下登入数据库、修改或删除数据
- 查询数据生成报表

1.2　数据库管理员的任务

设计、实现和维护 KingbaseES 数据库需要完成以下一些任务：

1.2.1　评估数据库使用的硬件资源

评估数据库所需的硬件资源，应该包括：

- 所需要的磁盘容量和 I/O 能力
- 在需要的情况下，磁带的容量
- 需要的内存容量

1.2.2　安装数据库软件

数据库管理员需要在一台或多台服务器上安装数据库和相关的工具软件。

1.2.3　数据库规划

数据库管理员需要规划数据库的逻辑存储结构，关系对象存储策略，备份策略。

规划逻辑存储结构时考虑对数据库性能和管理任务的影响非常重要。例如，创建表空间前考虑表空间可能存储的物理文件数量、大小；存储数据的访问模式，表空间所在的物理设备的性能。规划逻辑存储结构时应考虑的影响包括：

- 数据库运行的服务器的性能
- 数据访问时的性能
- 数据库备份恢复操作的效率

关系对象存储策略可以直接影响这些对象访问、存储和管理的效率。规划时注意要考虑数据的增长。规划阶段也应该包括备份策略规划，这个阶段可以调整逻辑存储结构和关系对象存储策略来提升备份效率。

《系统原理》中的物理存储结构、逻辑存储结构和数据库对象管理章节中包括了对存储结构和关系对象的进一步描述。

《应用开发指南》中的数据库开发规范章节包括了一些常见的规划建议。

1.2.4　创建数据库

创建数据库并启动服务。

1.2.5 备份数据库

实施规划的备份策略，包括配置 WAL、归档，配置备份工具和定时任务，完成首次全量备份。

1.2.6 创建用户

创建用户并分配合适的权限，可以参考 用户和角色管理。

1.2.7 实施数据库设计

根据规划完成表空间和对象的创建。

1.2.8 备份功能完整的数据库

数据库设计实施后，数据库就具备了完整功能，再次备份数据库。除了按备份策略做周期性的备份，最好在变更实施后立刻做一次备份。

1.2.9 性能调优

数据库管理员需要持续的优化数据库性能，KingbaseES 数据库提供工具和方法诊断性能问题和优化调整。具体参见《KingbaseES 数据库性能调优指南 》。

1.3 SQL 语句

与 KingbaseES 数据库通信的主要方式是提交 SQL 语句。

- 向数据库提交命令和 SQL

  有几种方法可以将 SQL 语句和命令提交到 KingbaseES 数据库。

- 关于 Ksql

  Ksql 是 KingbaseES 数据库的主要命令行界面。您可以使用 Ksql 设置数据库初始化参数、创建和管理用户、创建和更改数据库对象 (例如表和索引)、插入和更新数据、运行 SQL 查询等等。

- 使用 Ksql 连接到数据库

  使用 Ksql 连接到 KingbaseES 数据库实例。

1.3.1　向数据库提交命令和 SQL

有几种方法可以将 SQL 语句和命令提交到 KingbaseES 数据库。

- 直接使用 Ksql 的命令行界面
- 间接使用图形用户界面

  例如通过 KingbaseES 数据库对象管理工具 KStudio 的图形界面来管理数据库，该工具在后台提交 SQL 语句和命令。

1.3.2　关于 Ksql

Ksql 是 KingbaseES 数据库的主要命令行界面。您可以使用 Ksql 设置数据库初始化参数、创建和管理用户、创建和更改数据库对象 (例如表和索引)、插入和更新数据、运行 SQL 查询等等。

在提交 SQL 语句和命令之前，您必须连接到数据库。使用 Ksql，您可以本地或者远程连接。本地连接意味着连接到运行 Ksql 的同一台计算机上运行的 KingbaseES 数据库。远程连接意味着通过网络连接到远程计算机上运行的 KingbaseES 数据库。这样的数据库成为远程的数据库。

1.3.3　连接数据库

使用 Ksql 连接到 KingbaseES 数据库实例。

- 关于使用 Ksql 连接到数据库

  您必须连接到 KingbaseES 数据库才能查询或修改该数据库中的数据。您可以连接到默认数据库或通过网络连接到其他数据库。

- Step 1：打开命令窗口

  在您的平台上执行必要的操作以打开一个窗口，您可以在其中输入操作系统命令。

- Step 2：设置操作系统环境变量

  根据您的平台，您可能必须在启动 Ksql 之前设置环境变量，或者至少验证它们是否正确。

- Step 3：启动 Ksql

  启动 Ksql。

- Step 4：Ksql 连接到 KingbaseES 数据库

  使用 Ksql 连接到 KingbaseES 数据库实例，可以用不同用户的身份重新连接。

1.3.4　关于使用 Ksql 连接到数据库

KingbaseES 数据库包含以下组件：KingbaseES 数据库实例，它是进程和内存的集合，以及一组包含用户数据和系统数据的磁盘文件。

由于主机上可能有多个 KingbaseES 实例，每个实例都有自己的一组数据文件，因此您必须确定连接的实例。对于本地连接，您可以通过设置操作系统环境变量来识别实例。对于远程连接，您可以通过指定网路和数据库服务名称来识别实例。对于本地和远程连接，您必须设置环境变量以帮助操作系统找到 Ksql 可执行文件并为可执行文件提供其支持文件和脚本路径。

1.3.5　Step 1：打开命令窗口

在您的平台上执行必要的操作以打开一个窗口，您可以在其中输入操作系统命令。

1.3.6　Step 2：设置操作系统环境变量

根据您的平台，您可能必须在启动 Ksql 之前设置环境变量，或者至少验证它们是否设置正确。

例如，在大多数平台上，您需要配置 PATH 环境变量以包含 KingbaseES 的 bin 目录。

- 在 Linux 上，通过输入操作系统命令来设置环境变量。

```
-- Setting Environment Variables in Linux (C Shell)
setenv LD_LIBRARY_PATH=/home/test/kb_install/lib/kingbase/

-- Setting Environment Variables in Linux (Bash Shell)
export LD_LIBRARY_PATH=/home/test/kb_install/lib/kingbase/
```

1.3.7　Step 3：启动 Ksql

要开始使用 Ksql, 您必须首先了解如何启动和停止 Ksql。

- 请确保在您的计算机上已经安装了 Ksql
- 登录到操作系统 (如果需要)
- 输入连接到数据库的命令，然后按 Enter 键

```
ksql -h hostname -U username -d dbname -p port
```

1.3.8　Step 4：Ksql 连接到 KingbaseES 数据库

当您启动 Ksql 时, 需要一个用户名和密码才能登录到 KingbaseES 数据库模式。您的用户名和密码是 KingbaseES 数据库授权的用户。

您可以使用连接命令对不同的用户进行连接，用户名和密码必须对该数据库有效。例如，连接 user1 用户：

```
-- Connect to a Local Database User
ksql -U username -d dbname -p port

-- Connect to a  Database with the Host IP Address
ksql -h hostname -U username -d dbname -p port
```

Ksql 连接数据库命命令语法

连接命令格式如下:

ksql [option…] [dbname [username]]

```
ksql -U username -d dbname -p port
```

更多相关 Ksql 使用的信息请参考《ksql 工具》。

1.4　确定数据库的软件版本

1.4.1　获取当前数据库的版本信息

可以通过以下 SQL 获取数据库版本信息：

```
SELECT version();
```

1.5　数据库管理员的安全和权限

1.5.1　管理员用户

KingbaseES 采用了三权分立的安全管理体制，系统初始化时会创建三个管理员用户：数据库管理员、安全管理员、审计管理员。

- 数据库管理员

  用户名和密码由初始化时设定，通常用户名指定为 system。主要负责执行数据库日常管理的各种操作和自主存取控制。

- 安全管理员

  用户名默认为 sso，主要负责强制访问规则的制定和管理，监督审计管理员和普通用户的操作，不能创建和操作普通对象。

- 审计管理员

用户名默认为 sao，主要负责数据库的审计，监督系统管理员和安全管理员的操作，不能创建和操作普通对象。

三权分立的安全管理体制是为了解决数据库超级用户权力过度集中的问题。此外，还可以创建普通用户来访问数据库，运行数据库应用。

1.5.2　管理员权限

管理员执行基本数据库操作所需要的权限是通过特殊的系统权限授予的，在创建用户时可以指定授权。KingbaseES 还提供一些特殊的管理特权，例如，ANY 权限和 SYSBACKUP 特权。

管理权限的内容如下表所示：

表 1.5.1: 管理权限

权限	描述
SUPERUSER	超级用户，越过数据库内的所有访问限制，是最强大的管理权限
CREATEDB	允许创建数据库
CREATEROLE	允许创建用户和角色
LOGIN	允许登录数据库
REPLICATION	是否为复制角色
ANY 权限	允许用户操作所有的某种类型的数据库对象的某种操作，不包括系统对象
SYSBACKUP	允许执行物理备份操作

更多权限管理内容，可参考《安全指南》。

1.6　数据库管理员认证

在 KingbaseES 数据库中，可以通过使用存储在数据库字典中的账户和口令对管理员进行身份验证，即口令验证。同时，还支持结合第三方服务 Kerberos、RADIUS、LDAP、SSL 等认证协议和 CA 等技术对数据库管理员进行强化身份验证。更多身份验证内容，可参考《安全指南》。

1.7　数据操作工具

KingbaseES 提供多种工具用于维护数据。

1.7.1　数据加载工具

使用方式参见《工具参考》中的 bulkload 。

1.7.2　数据导出导入工具

使用方式参见《工具参考》中的 sys_dump 、sys_restore 和 exp/imp 工具介绍。

第2章　配置管理

本部分提供了影响数据库系统行为的配置参数的完整参考信息，包含参数的配置方法。

2.1　设置参数

KingbaseES 服务器的配置主要通过设置配置文件中参数完成。在 KingbaseES 数据库中，数据库管理员可以通过以下三种方式来设置配置文件参数，从而配置服务器：

- 通过配置文件设置参数 。
- 通过 SQL 设置参数 。
- 通过 Shell 设置参数 。

2.1.1　参数名称和值

KingbaseES 中参数名大小写是不敏感的。参数类型可以为：布尔、字符串、整数、浮点数或枚举。参数类型决定了设置该参数的语法：

布尔

值可以被写成 `on`, `off`, `true`, `false`, `yes`, `no`, `y`, `n`, `1`,`0` （都是大小写不敏感的）。

字符串

通常值被包括在单引号内，值内部的任何单引号都需要被双写。不过，如果值是一个简单数字或者标识符，引号通常可以被省略。（匹配 SQL 关键字的值在某些上下文中需要引用。）

数字（整数和浮点）

数值参数可以指定为常见的整数和浮点格式; 如果参数是整数类型，则将小数四舍五入为最接近的整数。整数参数还接受十六进制输入 (以 `0x` 开头) 和八进制输入 (以 `0` 开头)，但是这些格式不能有分数。不要使用千位分隔符。除十六进制输入外，不需要引号。

带单位的数字

一些数字参数具有隐含单位，因为它们描述内存或时间量。单位可能是字节、千字节、块（通常是 8KB）、毫秒、秒或分钟。这些设置之一的一个未修饰的数字值将使用该设置的默认单位，默认单位可以通过引用 sys_settings.unit 来找到。为了方便，也可以显式地指定一个不同的单位，例如时间值可以是 120 ms，并且它们将被转换到参数的实际单位。要使用这个特性，注意值必须被写成一个字符串（带有引号）。单位名称是大小写敏感的，并且在数字值和单位之间可以有空白。

- 可用的内存单位是 B（字节）、kB（千字节）、MB（兆字节）和 GB（吉字节）。内存单位的乘数是 1024 而不是 1000。
- 可用的时间单位是 us (微秒)、ms（毫秒）、s（秒）、min（分钟）、h（小时）和 d（天）。

  如果一个分数值是用一个单位所指定，它将四舍五入为下一个较小单位的倍数（如果有下一个较小的单位）。例如，30.1GB 将被转换为 30822 MB 而非 32319628902B。如果参数是整数类型，则在进行任何单位转换之后，舍入到整数。

枚举

枚举类型的参数以与字符串参数相同的方式指定，但被限制到一组有限的值。这样一个参数可用的值可以在 sys_settings.enumvals 中找到。枚举参数值是大小写无关的。

2.1.2　通过配置文件设置参数

kingbase.conf 文件通常被保存在数据目录中（当数据库集簇目录被初始化时，将默认拷贝 kingbase.conf 文件至数据目录，为数据集簇提供了默认值）。直接编辑 kingbase.conf 文件是设置配置参数的最基本的方式。

以下为一个 kingbase.conf 文件的示例：

```
# This is a comment
log_connections = yes
log_destination = 'syslog'
search_path = '"$user", public'
shared_buffers = 128MB
```

每一行指定一个参数。名称和值之间的等号是可选的。空白无意义（除了在一个引号引用的参数值内）并且空行被忽略。井号（#）指示该行的剩余部分是一个注释。非简单标识符或者数字的参数值必须用单引号包围。要在参数值里嵌入单引号，要么写两个单引号（首选）或者在引号前放反斜线。如果文件包含相同参数名的多个条目，则最后一个条目的设置覆盖之前所有同名条目的设置。

以这种方式设定的参数为集簇提供了默认值。除非这些设置被覆盖，活动会话看到的就是这些设置。下面的小节描述了管理员或用户覆盖这些默认值的方法。

主服务器进程每次收到 SIGHUP 信号（最简单的方法是从命令行运行 sys_ctl reload 或调用 SQL 函数 sys_reload_conf() 来发送这个信号）后都会重新读取这个配置文件。主服务器进程还会把这个信号传播给所有正在运行的服务器进程，这样现有的会话也能采用新值（要等待它们完成当前正在执行的客户端命令之后才会发生）。另外，可以直接向一个单一服务器进程发送该信号。有些参数只能在服务器启动时设置，在配置文件中对这些条目的修改将被忽略，直到下次服务器重启。配置文件中的非法参数设置也会在 SIGHUP 处理过程中被忽略（但是会记录日志）。

除 kingbase.conf 之外，KingbaseES 数据目录还包含一个文件 kingbase.auto.conf，它和 kingbase.conf 具有相同的格式但不能被编辑。此文件保存了通过 ALTER SYSTEM 命令提供的设置。每当 kingbase.conf 被读取时，该文件也将被自动读取，并且其设置会以同样的方式生效。kingbase.auto.conf 中的设置会覆盖 kingbase.conf 中的设置。

外部工具也可以修改 kingbase.auto.conf。不建议在服务器运行时执行此操作，因为并发的 ALTER SYSTEM 命令可能会覆盖这些更改。这些工具可能只是简单地在末尾附加新的设置，或者它们可能选择删除重复的设置和/或注释 (如 ALTER SYSTEM)。

系统视图 sys_file_settings 可以有助于对配置文件中的更改进行提前测试，或者在 SIGHUP 信号没有达到预期效果时用来诊断问题。

2.1.3　通过 SQL 设置参数

KingbaseES 提供了三个 SQL 命令来建立配置默认值。

- ALTER SYSTEM 命令提供了一种改变全局默认值的从 SQL 可访问的方法，等效于直接编辑 kingbase.conf。
- ALTER DATABASE 命令允许针对一个数据库覆盖其全局设置。
- ALTER ROLE 命令允许用用户指定的值来覆盖全局设置和数据库设置。

只有当开始一个新的数据库会话时，用 ALTER DATABASE 和 ALTER ROLE 设置的值才会被应用。它们会覆盖从配置文件或服务器命令行获得的值，并且作为该会话后续的默认值。注意某些设置在服务器启动后不能被更改，并且因此不能被这些命令（或者下文列举的命令）设置。

当客户端连接到数据库，KingbaseES 会提供两个额外的 SQL 命令（以及等效的函数）用以影响会话本地的配置设置：

- SHOW 命令允许察看所有参数的当前值。对应的函数是 current_setting(setting_name text)。
- SET 命令允许修改对于一个会话可以本地设置的参数的当前值，它对其他会话没有影响。对应的函数是 set_config(setting_name, new_value, is_local) 。

此外，系统视图 sys_settings 可以被用来查看和改变会话本地的值：

- 查询这个视图与使用 SHOW ALL 相似，但是可以提供更多细节。它也更加灵活，因为可以为它指定过滤条件或者把它与其他关系进行连接。
- 在这个视图上使用 UPDATE 并且指定更新 setting 列，其效果等同于发出 SET 命令。例如，下面的命令

```
SET configuration_parameter TO DEFAULT;
```

等效于：

```
UPDATE sys_settings SET setting = reset_val WHERE name =  'configuration_parameter';
```

2.1.4　通过 Shell 设置参数

除了在数据库或者角色层面上设置全局默认值或者进行覆盖，还可以通过 shell 工具把设置传递给 KingbaseES。服务器和 libkci 客户端库都能通过 shell 接受参数值。

- 在服务器启动期间，可通过-c 命令行参数把参数设置传递给 kingbase 命令。例如：

  ```
  kingbase -c log_connections=yes -c log_destination='syslog'
  ```

  这种方式提供的设置会覆盖通过 kingbase.conf 或者 ALTER SYSTEM 提供的设置，因此除了重启服务器之外无法从全局上改变它们。

- 当通过 libkci 启动一个客户端会话时，可以使用 KINGBASE_OPTIONS 环境变量指定参数设置。这种方式建立的设置构成了会话生存期间的默认值，但是不会影响其他的会话。由于历史原因，KINGBASE_OPTIONS 的格式和启动 kingbase 命令时用到的相似，特别是-c 标志必须被指定。例如：

  ```
  env KINGBASE_OPTIONS="-c geqo=off -c statement_timeout=5min" ksql
  ```

  通过 shell 或者其他方式，其他客户端和库可能提供它们自己的机制，以便允许用户在不直接使用 SQL 命令的前提下修改会话设置。

2.1.5　管理配置文件内容

KingbaseES 提供了一些特性用于把复杂的 kingbase.conf 文件分解成子文件。这些特性在管理多个具有相关但不完全相同配置的服务器时，特别有用。

除了单个参数设置，kingbase.conf 文件可以包含包括指令，它指定要读入和处理的另一个文件，就好像该文件被插入到配置文件的这个点。此特性允许一个配置文件被划分成物理上独立的部分。包括指令看起来像：

```
include 'filename'
```

如果文件名不是一个绝对路径，它将作为包含引用配置文件的目录的相对位置。包括可以被嵌套。

也有一个 include_if_exists 指令，它的作用和 include 指令一样，不过当被引用的文件不存在或者无法被读取时其行为不同。一个通常的 include 将认为这是一个错误情况，而 include_if_exists 仅仅记录一个消息并且继续处理引用配置文件。

kingbase.conf 文件也可以包含 include_dir 指令，它指定要被包含的配置文件的一整个目录。它的用法类似：

```
include_dir 'directory'
```

非绝对目录名被当做包含引用配置文件的目录的相对路径。在该指定目录中，只有以后缀名 .conf 结尾的非目录文件才会被包括。以. 字符开头的文件名也会被忽略，因为在某些平台上它们是隐藏文件。一个包括目录中的多个文件被以文件名顺序处理（根据 C 区域规则排序，即数字在字母之前并且大写字母在小写字母之前）。

包括文件或目录可以被用来在逻辑上分隔数据库配置的各个部分，而不是用一个很大的 kingbase.conf 文件。考虑一个有两台数据库服务器的公司，每一个都有不同的内存量。很可能配置的元素都会被共享，例如用于日志的参数。但是两者关于内存的参数将会不同。并且还可能会有服务器相关的自定义。一种管理这类情况的方法是将站点的自定义配置修改分成三个文件。把下面的内容加入到 kingbase.conf 文件末尾来包括它们：

```
include 'shared.conf'
include 'memory.conf'
include 'server.conf'
```

所有的系统将会有相同的 shared.conf。每个有特定内存量的服务器可以共享相同的 memory.conf。可能 8GB 内存的服务器有一个，而 16GB 内存的服务器有另一个。并且 server.conf 可以装有真正服务器相关的配置信息。

另一种可能性是创建一个配置文件目录并把这个信息放到其中的文件里。例如，一个 conf.d 目录可以在 kingbase.conf 的末尾被引用：

```
include_dir 'conf.d'
```

然后命名 conf.d 目录中的文件如下所示：

```
00shared.conf
01memory.conf
02server.conf
```

这种命名习惯建立了这些文件被载入的清晰顺序。这是很重要的，因为在服务器读取配置文件时，对于一个特定的参数只有最后碰到的一个设置才会被使用。在这个例子中，conf.d/02server.conf 设置的东西将会覆盖在 conf.d/01memory.conf 中相同参数的值。

还可以使用这种配置目录方法，在命名文件时更有描述性：

```
00shared.conf
01memory-8GB.conf
02server-foo.conf
```

这种形式的安排为每个配置文件变体给定了一个唯一的名称。当多个服务器把它们的配置全部存储在一个位置（例如在一个版本控制仓库中）时，这可以帮助消除歧义（在版本控制下存储数据库配置文件是另一个值得考虑的好方法）。

2.1.6　不同模式下参数差异

KingbaseES 数据库有四种模式，分别是 Oracle 模式、PG 模式、MySQL 模式和 SQLServer 模式。在 PG 模式下，以下参数默认为 off 状态，且禁止将其打开。

表 2.1.1: 模式差异

ora_forbid_func_polymorphism	是否禁用多态 on 表示开启禁用多态，同模式下 (不包括 sys, sys_catalog 系统内置模式) 不能创建 同名的函数或存储过程。 off（默认）, 则表示启用多态
ora_input_emptystr_isnull	输入空字符串时的处理措施。 on 表示将输入的空字符串作为 null 值处理。 off 表示不处理
ora_numop_style	是否将 integers 操作符当做 numeric 操作符
ora_statement_level_rollback	是否启用语句级回滚
ora_style_nls_date_format	这个变量设置为 on 时，日期时间的 隐式输入与输出字符串格式 才会使用参数 nls_date_format、 nls_timestamp_format 和 nls_timestamp_tz_format 的配置

2.2　参数作用域

2.2.1　INTERNAL—内部级

只读参数，其中部分参数是 Kingbase 程序确定，或者使用一些不同的编辑选项确定的；有些参数是数据库实例初始化时就确定了的，之后就不能再改变此参数值了。这类参数值不能在 `kingbase.conf` 中配置，因为他们是由 Kingbase 程序在初始化实例时确定的。

2.2.2　KINGBASE—实例级

此类参数的改变需要重启 Kingbase 实例。在 kingbase.conf 改变此类参数后，需要重启 Kingbase 实例才能生效。

2.2.3　SIGHUP—会话级

在 kingbase.conf 文件中可改变这些参数的值，不需要重启数据库，只需要向 Kingbase 进程发送 SIGHUP 信号，让其读取新的参数值。Kingbase 进程收到 SIGHUP 信号后，也会向它的子进程发送 SIGUP 信号，让新的参数在所有的进程中都生效。

2.2.4　SUPERUSER_BACKEND—超级用户后台级

在 kingbase.conf 文件中可改变这些参数的值，不需要重启数据库，只需要向 Kingbase 进程发送 SIGHUP 信号，让其读取新的参数值。新的配置只会出现在这之后的新连接中，在已有的连接中，这些参数的值不会改变。这些参数的值也可以在由超级用户建立新连接时，由连接的一些参数改变。

2.2.5　BACKEND—后台级

在 kingbase.conf 文件中可改变这些参数的值，不需要重启数据库，只需要向 Kingbase 进程发送 SIGHUP 信号，让其读取新的参数值。新的配置只会出现在这之后的新连接中，在已有的连接中，这些参数的值不会改变。

2.2.6　SUPERUSER—超级用户级

此类参数可以由超级用户使用 set 来改变。而超级用户改变参数值时，只会影响自身的连接配置，不会影响其他用户。向 Kingbase 进程发送 SIGHUP 信号，也只会影响后续建立的连接，不会影响现在的连接。

2.2.7　USER—用户级

普通用户可使用 set 命令通过此类参数来改变本连接中的配置值。除了普通用户都可以改变外，这类参数与 SUPERUSER 类参数没有区别。

2.3　配置文件

KingbaseES 主服务器的配置主要通过修改配置文件 `kingbase.conf` 完成。而配置文件 `sys_hba.conf` 和 `sys_ident.conf` 主要用于控制 强身份验证。当数据库集簇目录被初始化时，上述三个配置文件将被默认拷贝在数据目录（通常为/opt/Kingbase/ES/V9/data）中。本节描述的参数允许配置文件存放在其他位置（可以简化管理，配置文件被独立放置时，更易于保证它得到恰当的备份）。

`data_directory`

属性	描述
类型	string
默认值	ConfigDir
级别	KINGBASE

指定用于数据存储的目录。这个选项只能在服务器启动时设置。

`config_file`

属性	描述
类型	string
默认值	ConfigDir
级别	KINGBASE

指定主服务器配置文件（`kingbase.conf`）。此参数只能在 `kingbase` 命令行上设置。

`hba_file`

属性	描述
类型	string
默认值	ConfigDir/sys_hba.conf
级别	KINGBASE

指定基于主机认证配置文件（`sys_hba.conf`）。此参数只能在服务器启动的时候设置。

`ident_file`

属性	描述
类型	string
默认值	ConfigDir/ident_file.conf
级别	KINGBASE

指定用于用户名称映射的配置文件（`sys_ident.conf`）。此参数只能在服务器启动的时候设置。另见 用户名映射。

`external_pid_file`

属性	描述
类型	string
默认值	,,
级别	KINGBASE

指定可被服务器创建的用于管理程序的额外进程 ID（PID）文件。此参数只能在服务器启动的时候设置。

数据库在默认安装时不会显式设置以上参数。而是由命令行参数-D 或者环境变量 KINGBASE_DATA 指定数据目录，上述的配置文件均能在数据目录中找到。

如果想把配置文件放在别的地方而不是数据目录中，那么 kingbase`' '`-D 命令行选项或者环境变量 KINGBASE_DATA 必须指向包含配置文件的目录，并且 kingbase.conf 中（或者命令行上）的 data_directory 参数必须显示数据目录实际存放的地方。请注意，data_directory 将覆盖-D 和 KINGBASE_DATA 指定的数据目录位置，但是不覆盖配置文件的位置。

可使用选项 config_file、hba_file 和/或 ident_file 单独指定配置文件名称和位置。config_file 只能在 kingbase 命令行上指定，但是其他文件可以在主配置文件中设置。如果所有三个参数外加 data_directory 被显式地设置，则不必指定-D 或 KINGBASE_DATA。

在设置这些参数时，相对路径将被认为是相对于 kingbase 启动路径的路径。

2.4 连接和认证

2.4.1 连接设置

listen_addresses

属性	描述
类型	string
默认值	*
级别	KINGBASE

指定服务器在哪些 TCP/IP 地址上监听客户端连接。值的形式是一个逗号分隔的主机名和/或数字 IP 地址列表。特殊项 * 对应所有可用 IP 接口。“0.0.0.0”允许监听所有 IPv4 地址,“::”允许监听所有 IPv6 地址。如果列表为空，服务器将根本不会监听任何 IP 接口，在这种情况中只能使用 Unix 域套接字来连接它。默认值是 local-

host，它只允许建立本地 TCP/IP “环回”连接。虽然客户端认证（强身份验证）允许细粒度地控制谁能访问服务器，listen_addresses 控制哪些接口接受连接尝试，这能帮助在不安全网络接口上阻止重复的恶意连接请求。这个参数只能在服务器启动时设置。

port

属性	描述
类型	integer
默认值	54321
级别	KINGBASE

服务器监听的 TCP 端口；默认是 54321 。请注意服务器会同一个端口号监听所有的 IP 地址。这个参数只能在服务器启动时设置。

max_connections

属性	描述
类型	integer
默认值	100
级别	KINGBASE

决定数据库的最大并发连接数。默认值通常是 100 个连接，但是如果内核设置不支持（initdb 时决定），可能会比这个数少。这个参数只能在服务器启动时设置。

当运行一个后备服务器时，必须设置这个参数等于或大于主服务器上的参数。否则，后备服务器上可能无法允许查询。

superuser_reserved_connections

属性	描述
类型	integer
默认值	3
级别	KINGBASE

决定为 KingbaseES 超级用户连接而保留的连接“槽”数。同时活跃的并发连接最多 max_connections 个。任何时候，活跃的并发连接数最多为 `max_connections` 减去 `superuser_reserved_connections`，新连接就只能由超级用户发起了，并且不会有新的复制连接被接受。

默认值是 3 。这个值必须小于 `max_connections` 的值。这个参数只能在服务器启动时设置。

`unix_socket_directories`

属性	描述
类型	string
默认值	/tmp
级别	KINGBASE

指定服务器用于监听来自客户端应用的连接的 Unix 域套接字目录。通过列出用逗号分隔的多个目录可以建立多个套接字。项之间的空白会被忽略，因此，如果需要在名字中包括空白或逗号，需要在目录名周围放上双引号。空值表示在任何 Unix 域套接字上都不会被监听，在这种情况中只能使用 TCP/IP 套接字来连接到服务器。默认值通常是`/tmp`，但是在编译时可以被改变。这个参数只能在服务器启动时设置。

除了套接字文件本身（名为`.s.KINGBASE.nnnn`，其中 `*nnnn*` 是服务器的端口号），一个名为`.s.KINGBASE.nnnn`.lock 的普通文件会在每一个 `unix_socket_directories` 目录中被创建。任何一个都不应该被手工移除。

Windows 下没有 Unix 域套接字，因此这个参数与 Windows 无关。

`unix_socket_group`

属性	描述
类型	string
默认值	,,
级别	KINGBASE

设置 Unix 域套接字的所属组（套接字的所属用户总是启动服务器的用户）。可以与选项 `unix_socket_permissions` 一起用于对 Unix 域连接进行访问控制。默认是一个空字符串，表示服务器用户的默认组。这个参数只能在服务器启动时设置。

Windows 下没有 Unix 域套接字，因此这个参数与 Windows 无关。

`unix_socket_permissions`

属性	描述
类型	integer
默认值	0777
级别	KINGBASE

设置 Unix 域套接字的访问权限。Unix 域套接字使用普通的 Unix 文件系统权限集。这个参数值应该是数字的形式，也就是系统调用 `chmod` 和 `umask` 接受的形式（如果使用自定义的八进制格式，数字必须以一个 `0`（零）开头）。

默认的权限是 `0777`，意思是任何人都可以连接。合理的候选是 `0770`（只有用户和同组的人可以访问，又见 `unix_socket_group`）和 `0700`（只有用户自己可以访问）（请注意，对于 Unix 域套接字，只有写权限有麻烦，因此没有对读取和执行权限的设置和收回）。

这个访问控制机制与 强身份验证中的用户认证没有关系。

这个参数只能在服务器启动时设置。

这个参数与完全忽略套接字权限的系统无关，尤其是自版本 10 以上的 Solaris。在那些系统上，可以通过把 `unix_socket_directories` 指向一个把搜索权限限制给指定用户的目录来实现相似的效果。因为 Windows 下没有 Unix 域套接字，因此这个参数也与 Windows 无关。

`bonjour`

属性	描述
类型	boolean
默认值	off
级别	KINGBASE

通过 Bonjour 广告服务器的存在。默认值是关闭。这个参数只能在服务器启动时设置。

`bonjour_name`

属性	描述
类型	string
默认值	,,
级别	KINGBASE

指定 Bonjour 服务名称。空字符串''（默认值）表示使用计算机名。如果编译时没有打开 Bonjour 支持那么将忽略这个参数。这个参数只能在服务器启动时设置。

`stats_collector_port`

属性	描述
类型	integer
默认值	0
级别	KINGBASE

配置 stats collector 进程启动时使用的 UDP 端口。默认值是 0，可以使用任意 UDP 端口启动进程。这个参数只能在服务器启动时设置。

`tcp_keepalives_idle`

属性	描述
类型	integer
默认值	0
级别	USER

指定不活动多少秒之后通过 TCP 向客户端发送一个 keepalive 消息。0 值表示使用默认值。这个参数只有在支持 `TCP_KEEPIDLE` 或等效套接字选项的系统或 Windows 上才可以使用。在其他系统上，它必须为零。在通过 Unix 域套接字连接的会话中，这个参数被忽略并且总是读作零。

备注:

在 Windows 上，值若为 0，系统会将该参数设置为 2 小时，因为 Windows 不支持读取系统默认值。

`tcp_keepalives_interval`

属性	描述
类型	integer
默认值	0

-续表-

属性	描述
级别	USER

指定在多少秒之后重发一个还没有被客户端告知已收到的 TCP keepalive 消息。0 值表示使用系统默认值。这个参数只有在支持 `TCP_KEEPINTVL` 或等效套接字选项的系统或 Windows 上才可以使用。在其他系统上，必须为零。在通过 Unix 域套接字连接的会话中，这个参数被忽略并总被读作零。

备注:

在 Windows 上，值若为 0，系统会将该参数设置为 1 秒，因为 Windows 不支持读取系统默认值。

`tcp_keepalives_count`

属性	描述
类型	integer
默认值	0
级别	USER

指定与客户端的服务器连接被认为死掉之前允许丢失的 TCP keepalive 数量。0 值表示使用系统默认值。这个参数只有在支持 `TCP_KEEPCNT` 或等效套接字选项的系统上才可以使用。在其他系统上，必须为零。在通过 Unix 域套接字连接的会话中，这个参数被忽略并总被读作零。

备注:

Windows 不支持该参数，且必须为零。

`tcp_user_timeout`

属性	描述
类型	integer
默认值	0
级别	USER

在强制关闭连接之前，指定传输的数据可能保持未确认的毫秒数。值 0 使用系统默认值。此参数仅支持 TCP_USER_TIMEOUT；在其他系统上，它必须是零。在通过 unix 域套接字连接的会话中，此参数将被忽略，并且始终读取为零。

备注:

在 Windows 和 Linux 2.6.36 或更早版本的 Linux 中不支持此参数。

`client_idle_timeout`

属性	描述
类型	integer
默认值	0
级别	USER

指定客户端空闲多少秒之后断开与服务器的连接。0 值表示关闭这个功能，大于 0 的值表示开启这个功能并设置自动断开连接的空闲时间。连接断开后需要重新连接数据库，建议不要设置的时间太短。

备注:

该值的最大范围是 1800 秒，超出范围后会报错提示。

`connections_limit_per_user`

属性	描述
类型	boolean
默认值	true
级别	USER

参数 connections_limit_per_user 设置为 true 时，普通用户默认连接数限制为 5，超级用户默认连接数限制为 8。connections_limit_per_user 设置为 false 时，所有用户默认的最大连接限制均为-1，并且超级用户不受连接数限制。

备注:

注意这个限制仅针对于普通连接，预备事务和后台工作者连接都不受这一限制管辖。

2.4.2 安全和认证

`authentication_timeout`

属性	描述
类型	integer
默认值	1min
级别	SIGHUP

完成客户端认证的最长时间，以秒计。如果一个客户端没有在这段时间里完成认证协议，服务器将关闭连接。这样就避免了出问题的客户端无限制地占有一个连接。默认值是 1 分钟（`1m`）。这个参数只能在服务器命令行上或者在 `kingbase.conf` 文件中设置。

`password_encryption`

属性	描述
类型	enum
默认值	scram-sha-256
级别	USER

当在 CREATE ROLE(Oracle)、CREATE ROLE(SQLServer) 或者 ALTER ROLE(Oracle)、ALTER ROLE(SQLServer) 中指定了口令时，这个参数决定用于加密该口令的算法。默认值是 `scram-sha-256`。目前支持的口令加密算法有 `scram-sha-256` 、`md5` 、`sm3` 、`scram-sm3` 、`sm4` 。

注意老的客户端可能缺少对 SCRAM 认证机制的支持，因此无法使用用 SCRAM-SHA-256 加密的口令。详情请参考 口令加密认证。

`krb_server_keyfile`

属性	描述
类型	string
默认值	,,
级别	SIGHUP

设置 Kerberos 服务器密钥文件的位置。详情请参考 GSSAPI 身份验证。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。

`krb_caseins_users`

属性	描述
类型	boolean
默认值	off
级别	SIGHUP

设置是否应该以大小写不敏感的方式对待 GSSAPI 用户名。默认值是 `off`（大小写敏感）。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。

`db_user_namespace`

属性	描述
类型	boolean
默认值	off
级别	SIGHUP

这个参数启用针对每个数据库的用户名。这个参数默认是关掉的。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。

如果这个参数为打开，应该把用户创建成 `username@dbname` 的形式。当一个连接客户端传来 `username` 时，`@` 和数据库实例名会被追加到用户名并且服务器会查找这个与数据库相关的用户名。注意在 SQL 环境中用含有 `@` 的名称创建用户时，需要把用户名放在引号内。

在这个参数被启用时，仍然可以创建平常的全局用户。而在客户端中指定这种用户时只需要简单地追加 `@`，例如

joe@。在服务器查找该用户名之前，@ 会被剥离掉。

db_user_namespace 会导致客户端和服务器的用户名表达形式不同。认证检查总是会以服务器的用户名表达形式来完成，因此认证方法必须针对服务器用户名而不是客户端用户名来配置。由于 md5 方法在客户端和服务器两端都使用用户名作为 salt，md5 不能与 db_user_namespace 同时使用。

备注:

这种特性的目的是在找到完整的解决方案之前提供一种临时的措施。在找到完整解决方案时，这个选项将被去除。

---

ca_cert_file

属性	描述
类型	string
默认值	,,
级别	KINGBASE

证书认证时，用于指定 ca 证书的名字。

cert_type

属性	描述
类型	string
默认值	,,
级别	KINGBASE

证书认证时，用于指定证书登录算法，取值：rsa、sm2。

2.4.3　SSL

有关设置 SSL 的更多信息请参考 传输加密。

ssl

属性	描述
类型	boolean
默认值	off
级别	SIGHUP

启用 SSL 连接。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。默认值是 `off`。

`ssl_ca_file`

属性	描述
类型	string
默认值	,,
级别	SIGHUP

指定包含 SSL 服务器证书颁发机构（CA）的文件名。相对路径是相对于数据目录的。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。默认值为空，表示没有载入 CA 文件，并且客户端证书验证没有被执行。

`ssl_cert_file`

属性	描述
类型	string
默认值	server.crt
级别	SIGHUP

指定包含 SSL 服务器证书的文件名。相对路径是相对于数据目录的。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。默认值是 `server.crt`。

`ssl_crl_file`

属性	描述
类型	string
默认值	,,
级别	SIGHUP

指定包含 SSL 服务器证书撤销列表（CRL）的文件名。其中的相对路径是相对于数据目录的。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。默认值是空，表示没有载入 CRL 文件。

`ssl_key_file`

属性	描述
类型	string
默认值	server.key
级别	SIGHUP

指定包含 SSL 服务器私钥的文件名。相对路径是相对于数据目录。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。默认值是 `server.key`。

`ssl_ciphers`

属性	描述
类型	string
默认值	HIGH:MEDIUM:+3DES:!aNULL
级别	SIGHUP

指定一个 SSL 密码列表，用于安全连接。这个设置的语法和所支持的值列表可以参见 OpenSSL 包中的 ciphers 手册页。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。默认值是 `HIGH:MEDIUM:+3DES:!aNULL`。默认值通常是一种合理的选择，除非用户有特定的安全性需求。

默认值的解释：

`HIGH`
使用来自 `HIGH` 组的密码的密码组（例如 AES, Camellia, 3DES）

`MEDIUM`
使用来自 `MEDIUM` 组的密码的密码组（例如 RC4, SEED）

`+3DES`
OpenSSL 对 `HIGH` 的默认排序是有问题的，因为它认为 3DES 比 AES128 更高。这是错误的，因为 3DES 提供的安全性比 AES128 低，并且它也更加慢。`+3DES` 被重新排序在所有其他 `HIGH` 和 `MEDIUM` 密码之后。

`!aNULL`
禁用不做认证的匿名密码组。这类密码组容易收到中间人攻击，因此不应被使用。

可用的密码组细节可能会随着 OpenSSL 版本变化。可使用命令 `openssl ciphers -v 'HIGH:MEDIUM:+3DES:!aNULL'`来查看当前安装的 OpenSSL 版本的实际细节。注意这个列表是根据服务器密钥类型在运行时过滤过的。

`ssl_prefer_server_ciphers`

属性	描述
类型	boolean
默认值	on
级别	SIGHUP

指定是否使用服务器的 SSL 密码首选项，而不是用客户端的。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。默认值是 `on`。

老的 KingbaseES 版本没有这个设置并且总是使用客户端的首选项。这个设置主要用于与那些版本的向后兼容性。使用服务器的首选项通常会更好，因为服务器更可能会被合适地配置。

`ssl_ecdh_curve`

属性	描述
类型	string
默认值	prime256v1
级别	SIGHUP

指定用在 ECDH 密钥交换中的曲线名称。它需要被所有连接的客户端支持。它不需要与服务器椭圆曲线密钥使用的曲线相同。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。默认值是 `prime256v1`。

OpenSSL 命名了最常见的曲线：`prime256v1` (NIST P-256)、`secp384r1` (NIST P-384)、`secp521r1` (NIST P-521)。`openssl ecparam -list_curves` 命令可以显示可用曲线的完整列表。不过并不是所有的都在 TLS 中可用。

`ssl_min_protocol_version`

属性	描述
类型	enum
默认值	TLSv1
级别	SIGHUP

设置要使用的最小 SSL/TLS 协议版本。当前的有效值是：`TLSv1`、`TLSv1.1`、`TLSv1.2`、`TLSv1.3`。老版本的 OpenSSL 不支持所有的值; 如果选择不支持的设置，将引发错误。TLS 1.0 之前的协议版本 (即 SSL 版本 2 和 3) 总是禁用的。

默认值是 `TLSv1`，主要支持旧版本的 OpenSSL 库。如果所有软件组件都支持较新的协议版本，则需要将该值设置为较高的值。

`ssl_max_protocol_version`

属性	描述
类型	enum
默认值	，，
级别	SIGHUP

设置要使用的最大 SSL/TLS 协议版本。有效值为 ssl_min_protocol_version，添加一个空字符串，允许任何协议版本。默认允许任何版本。设置最大协议版本主要用于测试，或者如果某个组件在使用较新的协议时出现问题，则设置最大协议版本非常有用。

`ssl_dh_params_file`

属性	描述
类型	string
默认值	，，
级别	SIGHUP

指定含有用于 SSL 密码的所谓临时 DH 家族的 Diffie-Hellman 参数的文件名。默认值为空，这种情况下将使用内置的默认 DH 参数。使用自定义的 DH 参数可以降低攻击者破解众所周知的内置 DH 参数的风险。可以用命令

`openssl dhparam -out dhparams.pem 2048` 创建自己的 DH 参数文件。这个参数只能在 `kingbase.conf` 文件中或服务器命令行上进行设置。

`ssl_passphrase_command`

属性	描述
类型	string
默认值	,,
级别	SIGHUP

设置当需要一个密码（例如一个私钥）来解密 SSL 文件时会调用的一个外部命令。默认情况下，这个参数为空，表示使用内建的提示机制。

该命令必须将密码打印到标准输出并且以代码 0 退出。在该参数值中，`%p` 被替换为一个提示字符串（要得到文字%，应该写成%%）。注意该提示字符串将可能含有空格，因此要确保加上适当的引号。如果输出的末尾有单一的新行，它会被剥离掉。

该命令实际上并不一定要提示用户输入一个密码。它可以从文件中读取密码、从钥匙链得到密码等等。确保选中的机制足够安全是用户的责任。

这个参数只能在 `kingbase.conf` 文件中或服务器命令行上进行设置。

`ssl_passphrase_command_supports_reload`

属性	描述
类型	boolean
默认值	off
级别	SIGHUP

这个参数决定在配置重载期间如果一个密钥文件需要口令时，是否也调用 `ssl_passphrase_command` 设置的密码命令。如果这个参数为 off（默认），那么在重载期间将忽略 `ssl_passphrase_command`，如果在此期间需要密码则 SSL 配置将不会被重载。对于要求一个 TTY（当服务器正在运行时可能是不可用的）来进行提示的命令，这种设置是合适的。例如，如果密码是从一个文件中得到的，将这个参数设置为 on 可能是合适的。

这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。

2.5　资源消耗

2.5.1　内存

shared_buffers

属性	描述
类型	integer
默认值	128MB
级别	KINGBASE

设置数据库服务器将使用的共享内存缓冲区量。默认通常是 128 兆字节（128MB），但是如果内核设置不支持（在 initdb 时决定），那么可以会更少。这个设置必须至少为 128 千字节（BLCKSZ 的非默认值将改变最小值，最小值为 BLCKSZ 的 16 倍）。不过为了更好的性能，通常会使用明显高于最小值的设置。

如果有一个专用的 1GB 或更多内存的数据库服务器，一个合理的 shared_buffers 开始值是系统内存的 25%。即使更大的 shared_buffers 有效，也会造成一些工作负载，但因为 KingbaseES 同样依赖操作系统的高速缓冲区，将 shared_buffers 设置为超过 40% 的 RAM 不太可能比一个小点值工作得更好。为了能把对写大量新的或改变的数据的处理分布在一个较长的时间段内，shared_buffers 更大的设置通常要求对 max_wal_size 也做相应增加。

如果系统内存小于 1GB，一个较小的 RAM 百分数是合适的，这样可以为操作系统留下足够的空间。

huge_pages

属性	描述
类型	enum
默认值	try
级别	KINGBASE

控制是否为主共享内存区域请求巨型页。有效值是 try（默认）、on 以及 off。如果 huge_pages 被设置为 try，则服务器将尝试请求巨型页，但是如果失败会退回到默认的方式。如果为 on，请求巨型页失败将使得服务器无法启动。如果为 off，则不会请求巨型页。

当前，只有 Linux 和 Windows 上支持这个设置。在其他系统上这个参数被设置为 try 时，它会被忽略。

巨型页面的使用会导致更小的页面表以及花费在内存管理上的 CPU 时间更少，从而提高性能。

巨型页在 Windows 上被称为大页面。要使用大页面，需要为运行 KingbaseES 的 Windows 用户账号分配 Lock Pages in Memory 的用户权限。可以使用 Windows 的组策略工具（gpedit.msc）来分配用户权限 Lock Pages in Memory。为了在命令窗口以单进程（而不是 Windows 服务）的方式启动数据库服务器，命令窗口必须以管理员身份运行或者禁用用户访问控制（UAC）。当 UAC 被启用时，普通的命令窗口会在启动时收回用户权限 Lock Pages in Memory。

注意这种设置仅影响主共享内存区域。Linux、FreeBSD 以及 Illumos 之类的操作系统也能为普通内存分配自动使用巨型页（也被称为“超级”页或者“大”页面），而不需要来自 KingbaseES 的显式请求。在 Linux 上，这被称为“transparent huge pages”（THP，透明巨型页）。已知这种特性对某些 Linux 版本上的某些用户会导致 KingbaseES 的性能退化，因此当前并不鼓励使用它（与 `huge_pages` 的显式使用不同）。

`temp_buffers`

属性	描述
类型	integer
默认值	8MB
级别	USER

设置每个数据库会话使用的临时缓冲区的最大数目。这些都是会话的本地缓冲区，只用于访问临时表。默认是 8 兆字节（`8MB`）。这个设置可以在独立的会话内部被改变，但是只有在会话第一次使用临时表之前才能改变；在会话中随后企图改变该值是无效的。

一个会话将按照 `temp_buffers` 给出的限制根据需要分配临时缓冲区。如果在一个并不需要大量临时缓冲区的会话里设置一个大的数值，其开销只是一个缓冲区描述符，或者说 `temp_buffers` 每增加一则增加大概 64 字节。不过，如果一个缓冲区被实际使用，那么它就会额外消耗 8192 字节（或者 `BLCKSZ` 字节）。

`max_prepared_transactions`

属性	描述
类型	integer
默认值	0
级别	KINGBASE

用于控制可同时处于“prepared”状态事务（预备事务）的最大数量。取值为 0（默认值），则会禁用预备事务。

- 如果在实际应用中无需使用预备事务，建议将此参数设置为 0，以避免意外创建预备事务。

- 如果正在使用预备事务，则 max_prepared_transactions 取值应不小于 max_connections 的设置，以确保每个会话都能有一个预备事务待处理。
- 当运行后备服务器时，务必确保此参数取值不小于主服务器上的参数设置。否则，后备服务器上将无法执行查询操作。

备注:

修改此参数需要重启服务器。

work_mem

属性	描述
类型	integer
默认值	4MB
级别	USER

指定在写到临时磁盘文件之前被内部排序操作和哈希表使用的内存量。该值默认为 4 兆字节（4MB）。注意对于一个复杂查询，可能会并行运行好几个排序或者哈希操作；每个操作都会被允许使用这个参数指定的内存量，然后才会开始写数据到临时文件。同样，几个正在运行的会话可能并发进行这样的操作。因此被使用的总内存可能是 work_mem 值的好几倍，在选择这个值时一定要记住这一点。ORDER BY、DISTINCT 和归并连接都要用到排序操作。哈希连接、基于哈希的聚集以及基于哈希的 IN 子查询处理中都要用到哈希表。

maintenance_work_mem

属性	描述
类型	integer
默认值	64MB
级别	USER

指定在维护性操作（例如 VACUUM、CREATE INDEX 和 ALTER TABLE ADD FOREIGN KEY）中使用的最大的内存量。其默认值是 64 兆字节（64MB）。因为在一个数据库会话中，一个时刻只有一个这样的操作可以被执行，并且一个数据库安装通常不会有太多这样的操作并发执行，把这个数值设置得比 work_mem 大很多是安全的。更大的设置可以改进清理和恢复数据库转储的性能。

注意当自动清理运行时，可能会分配最多达这个内存的 autovacuum_max_workers 倍，因此要小心不要把该默认值设置得太高。通过独立地设置 autovacuum_work_mem 可能会对控制这种情况有所帮助。

`autovacuum_work_mem`

属性	描述
类型	integer
默认值	-1
级别	SIGHUP

指定每个自动清理工作者进程能使用的最大内存量。其默认值为 -1，表示转而使用 maintenance_work_mem 的值。当运行在其他上下文环境中时，这个设置对 `VACUUM` 的行为没有影响。

`max_stack_depth`

属性	描述
类型	integer
默认值	3MB
级别	SUPERUSER

指定服务器的执行堆栈的最大安全深度。这个参数的理想设置是由内核强制的实际栈尺寸限制（`ulimit -s` 所设置的或者本地等价物），减去大约一兆字节的安全边缘。需要这个安全边缘是因为在服务器中并非所有例程都检查栈深度，只是在关键的可能递规的例程中才进行检查。默认设置是 3 兆字节（`3MB`），这个值相对比较小并且不可能导致崩溃。但是，这个值可能太小了，以至于无法执行复杂的函数。只有超级用户可以修改这个设置。

把 `max_stack_depth` 参数设置得高于实际的内核限制将意味着一个失控的递归函数可能会导致一个独立的后端进程崩溃。在 KingbaseES 能够检测内核限制的平台上，服务器将不允许把这个参数设置为一个不安全的值。不过，并非所有平台都能提供该信息，因此要小心设置此值。

`shared_memory_type`

属性	描述
类型	enum
默认值	mmap
级别	KINGBASE

指定服务器应该用于主共享内存区域的共享内存实现，该区域包含 KingbaseES 的共享缓冲区和其他共享数据。可能的值是 mmap (为匿名共享内存分配使用 mmap)，sysv (为系统 V 分配共享内存 shmget) 和 windows (为 Windows 共享内存)。并非所有平台都支持所有值；第一个受支持的选项是该平台的默认选项。使用 sysv 选项在任何平台上都不是默认的，这通常是不鼓励的，因为它通常需要非默认的内核设置来允许大量的分配 (参考系统全局区配置).

dynamic_shared_memory_type

属性	描述
类型	enum
默认值	posix
级别	SUPERUSER

指定服务器应该使用的动态共享内存实现。可能的值是 posix（用于使用 shm_open 分配的 POSIX 共享内存）、sysv （用于通过 shmget 分配的 System V 共享内存）、windows（用于 Windows 共享内存）、mmap （使用存储在数据目录中的内存映射文件模拟共享内存）。并非所有平台上都支持所有值，平台上第一个支持的选项就是其默认值。在任何平台上 mmap 选项都不是默认值，通常不鼓励使用它，因为操作系统会反复地把修改过的页面写回到磁盘上，从而增加了系统的 I/O 负载。不过当 sys_dynshmem 目录被存储在一个 RAM 盘时或者没有其他共享内存功能可用时，它还是有用的。

copy_buffered_bytes

属性	描述
类型	integer
默认值	65535B
级别	USER

设置 Copy From 刷新缓冲区的大小。如果存储的元组的大小超过设置的缓冲区大小，则刷新缓冲区，将数据写入相应的表中。

2.5.2　磁盘

temp_file_limit

属性	描述
类型	integer
默认值	-1
级别	SUPERUSER

指定一个进程能用于临时文件（如排序和哈希临时文件，或者用于保持游标的存储文件）的最大磁盘空间量。一个试图超过这个限制的事务将被取消。这个值以千字节计，并且-1（默认值）意味着没有限制。只有超级用户能够修改这个设置。

这个设置约束着一个给定 KingbaseES 进程在任何瞬间所使用的所有临时文件的总空间。应该注意的是，与在查询执行中在幕后使用的临时文件相反，显式临时表所用的磁盘空间不被这个设置所限制。

2.5.3　内核资源使用

`max_files_per_process`

属性	描述
类型	integer
默认值	1000
级别	KINGBASE

设置每个服务器子进程允许同时打开的最大文件数目。默认是 1000 个文件。如果内核强制一个安全的针对每个进程的限制，则不需要设置此值。但是在一些平台上（特别是大多数 BSD 系统），如果很多进程都尝试打开很多文件，内核将允许独立进程打开比个系统真正可以支持的数目大得多得文件数。如果返回“Too many open files”这样的失败提示信息，可尝试减小这个设置。这个参数只能在服务器启动时设置。

2.5.4　基于代价的清理延迟

在 VACUUM(Oracle)、VACUUM(SQLServer) 和 ANALYZE(Oracle)、ANALYZE(SQLServer) 命令的执行过程中，系统维持着一个内部计数器来跟踪各种被执行的 I/O 操作的估算开销。当累计的代价达到一个限制（由 `vacuum_cost_limit` 指定），执行这些操作的进程将按照 `vacuum_cost_delay` 所指定的休眠一小段时间。然后它将重置计数器并继续执行。

这个特性的出发点是允许管理员降低这些命令对并发的数据库活动产生的 I/O 影响。在很多情况下，`VACUUM` 和

ANALYZE 等维护命令能否快速完成并不重要，而非常重要的是这些命令不会对系统执行其他数据库操作的能力产生显著的影响。基于代价的清理延迟提供了一种方式让管理员能够保证这一点。

对于手动发出的 VACUUM 命令，该特性默认被禁用。要启用它，只要把 vacuum_cost_delay 变量设为一个非零值。

vacuum_cost_delay

属性	描述
类型	real
默认值	0
级别	USER

进程超过代价限制后将休眠的时间长度，以毫秒计。其默认值为 0，这将禁用基于代价的清理延迟特性。正值将启用基于代价的清理。

在使用基于代价的清理时，vacuum_cost_delay 的合适值通常很小，可能不到 1 毫秒。虽然可以将 vacuum_cost_delay 设置为毫秒级的值，但是在较老的平台上可能无法准确地测量这种延迟。在这样的平台上，增加 VACUUM 的节流资源消耗超过 1ms 需要改变其他的真空成本参数。尽管如此，也应该保持 vacuum_cost_delay 的大小与平台一致；大的延迟是没有帮助的。

vacuum_cost_page_hit

属性	描述
类型	integer
默认值	1
级别	USER

清理一个在共享缓存中找到的缓冲区的估计代价。它表示锁住缓冲池、查找共享哈希表和扫描页内容的代价。默认值为 1。

vacuum_cost_page_miss

属性	描述
类型	integer
默认值	10

-续表-

属性	描述
级别	USER

清理一个必须从磁盘上读取的缓冲区的代价。它表示锁住缓冲池、查找共享哈希表、从磁盘读取需要的块以及扫描其内容的代价。默认值为 10。

`vacuum_cost_page_dirty`

属性	描述
类型	integer
默认值	20
级别	USER

清理修改一个之前干净的块时需要花费的估计代价。它表示再次把脏块刷出到磁盘所需要的额外 I/O。默认值为 20。

`vacuum_cost_limit`

属性	描述
类型	integer
默认值	200
级别	USER

导致清理进程休眠的累计代价。默认值为 200。

备注:

有些操作会保持关键性的锁，这样可以尽快完成。基于代价的清理延迟在这类操作期间不会发生。因此有可能代价会累计至大大超过指定的限制。

2.5.5　后台写入器

有一个独立的服务器进程，叫做“后台写入器”，它的功能就是发出写“脏”（新的或修改过的）共享缓冲区的命令。它写出共享缓冲区，这样让处理用户查询的服务器进程很少或者永不等待写动作的发生。不过，后台写入器确实会增加 I/O 的总负荷，因为虽然在每个检查点间隔中一个重复弄脏的页面可能只会写出一次，但在同一个间隔中后台写入器可能会把它写出好几次。在这一小节讨论的参数可以被用于调节本地需求的行为。

`bgwriter_delay`

属性	描述
类型	integer
默认值	200ms
级别	SIGHUP

指定后台写入器活动轮次之间的延迟。在每个轮次中，写入器都会为一定数量的脏缓冲区发出写操作（可以用下面的参数控制）。然后它就休眠 `bgwriter_delay` 毫秒，然后重复动作。默认值是 200 毫秒（`200ms`）。注意在许多系统上，休眠延迟的有效解析度是 10 毫秒；因此，为 `bgwriter_delay` 设置一个不是 10 的倍数的值与把它设置为下一个更高的 10 的倍数是一样的效果。这个选项只能在服务器命令行上或者在 `kingbase.conf` 文件中设置。

`bgwriter_lru_maxpages`

属性	描述
类型	integer
默认值	100
级别	SIGHUP

在每个轮次中，不超过这么多个缓冲区将被后台写入器写出。把这个参数设置为零可禁用后台写出（注意被一个独立、专用辅助进程管理的检查点不受影响）。默认值是 100 个缓冲区。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

`bgwriter_lru_multiplier`

属性	描述
类型	real
默认值	2

-续表-

属性	描述
级别	SIGHUP

每一轮次要写的脏缓冲区的数目基于最近几个轮次中服务器进程需要的新缓冲区的数目。最近所需的平均值乘以 bgwriter_lru_multiplier 可以估算下一轮次中将会需要的缓冲区数目。脏缓冲区将被写出直到有很多干净可重用的缓冲区（然而，每一轮次中写出的缓冲区数不超过 bgwriter_lru_maxpages）。因此，设置为 1 表示一种“刚刚好的”策略，这种策略会写出正好符合预测值的数目的缓冲区。更大的值可以为需求高峰提供某种缓冲，而更小的值则需要服务进程来处理一些写出操作。默认值是 2。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

bgwriter_flush_after

属性	描述
类型	integer
默认值	512KB
级别	SIGHUP

不管何时 bgwriter 写入了超过 bgwriter_flush_after 字节，尝试强制 OS 把这些写发送到底层存储上。这样做将限制内核页缓存中脏数据的量，降低了在检查点末尾发出一个 fsync 时或者 OS 在后台大批量写回数据时卡住的可能性。那常常会导致大幅度压缩的事务延迟，但是也有一些情况（特别是负载超过 shared_buffers 但小于 OS 页面高速缓存）的性能会降低。这种设置可能会在某些平台上没有效果。合法的范围在 0（禁用受控写回）和 2MB 之间。Linux 上的默认值是 512kB，其他平台上是 0（如果 BLCKSZ 不是 512kB，则默认值和最大值会按比例缩放至这个值）。这个参数只能在 kingbase.conf 文件中或者服务器命令行上设置。

较小的 bgwriter_lru_maxpages 和 bgwriter_lru_multiplier 可以降低由后台写入器造成的额外 I/O 开销。但更可能的是，服务器进程将必须自己发出写入操作，这会延迟交互式查询。

2.5.6　异步行为

effective_io_concurrency

属性	描述
类型	integer

-续表-

属性	描述
默认值	1
级别	USER

设置 KingbaseES 可以同时被执行的并发磁盘 I/O 操作的数量。调高这个值，可以增加任何单个 KingbaseES 会话试图并行发起的 I/O 操作的数目。允许的范围是 1 到 1000，或 0 表示禁用异步 I/O 请求。当前这个设置仅影响位图堆扫描。

对于磁盘驱动器，这个设置的一个很好的出发点是组成一个被用于该数据库的 RAID 0 条带或 RAID 1 镜像的独立驱动器数量（对 RAID 5 而言，校验驱动器不计入）。但是，如果数据库经常忙于在并发会话中发出的多个查询，较低的值可能足以使磁盘阵列繁忙。比保持磁盘繁忙所需的值更高的值只会造成额外的 CPU 开销。SSD 以及其他基于内存的存储常常能处理很多并发请求，因此它们的最佳值可能是数百。

异步 I/O 依赖于一个有效的 `posix_fadvise` 函数（一些操作系统可能没有）。如果不存在这个函数，将这个参数设置为除 0 之外的任何东西将导致错误。在一些操作系统上（如 Solaris）虽然提供了这个函数，但它不会做任何事情。

在支持的系统上默认值为 1，否则为 0。对于一个特定表空间中的表，可以通过设定该表空间的同名参数（见 ALTER TABLESPACE(Oracle)、ALTER TABLESPACE(MySQL)、ALTER TABLESPACE(SQLServer) ）可以覆盖这个值。

`max_worker_processes`

属性	描述
类型	integer
默认值	8
级别	KINGBASE

设置系统能够支持的后台进程的最大数量。这个参数只能在服务器启动时设置。默认值为 8。

在运行一个后备服务器时，必须把这个参数设置为等于或者高于主控服务器上的值。否则，后备服务器上可能不会允许查询。

在更改这个值时，考虑也对 max_parallel_workers 、max_parallel_maintenance_workers 以及 max_parallel_workers_per_gather 进行调整。

`max_parallel_workers_per_gather`

属性	描述
类型	integer
默认值	2
级别	USER

设置单个 `Gather` 或者 `Gather Merge` 节点能够开始的工作者的最大数量。并行工作者会从 max_worker_processes 建立的进程池中取得，数量由 max_parallel_workers 限制。注意所要求的工作者数量在运行时可能实际无法被满足。如果这种事情发生，该计划将会以比预期更少的工作者运行，这可能会不太高效。默认值是 2。把这个值设置为 0 将会禁用并行查询执行。

注意并行查询可能消耗比非并行查询更多的资源，因为每一个工作者进程时一个完全独立的进程，它对系统产生的影响大致和一个额外的用户会话相同。在为这个设置选择值时，以及配置其他控制资源利用的设置（例如 work_mem）时，应该把这个因素考虑在内。`work_mem` 之类的资源限制会被独立地应用于每一个工作者，这意味着所有进程的总资源利用可能会比单个进程时高得多。例如，一个使用 4 个工作者的并行查询使用的 CPU 时间、内存、I/O 带宽可能是不使用工作者时的 5 倍之多。

`max_parallel_maintenance_workers`

属性	描述
类型	integer
默认值	2
级别	USER

设置单一工具性命令能够启动的并行工作者的最大数目。当前，唯一一种支持使用并行工作者的工具性命令是 `CREATE INDEX`，并且只有在构建 B-树索引时才能并行。并行工作者从由 max_worker_processes 创建的进程池中取出，数量由 max_parallel_workers 控制。注意实际在运行时所请求数量的工作者可能不可用。如果发生这种情况，工具性操作将使用比预期数量少的工作者运行。默认值为 2。将这个值设置为 0 可以禁用工具性命令对并行工作者的使用。

注意并行工具性命令不应该消耗比同等数量非并行操作更多的内存。这种策略与并行查询不同，并行查询的资源限制通常是应用在每个工作者进程上。并行工具性命令把资源限制 `maintenance_work_mem` 当作对整个工具性命令的限制，而不管其中用到了多少个并行工作者进程。不过，并行工具性命令实际上可能仍会消耗更多的 CPU 资源和 I/O 带宽。

`max_parallel_workers`

属性	描述
类型	integer
默认值	8
级别	USER

设置系统为并行操作所支持的工作者的最大数量。默认值为 8。在增加或者减小这个值时，也要考虑对 max_parallel_maintenance_workers 以及 max_parallel_workers_per_gather 进行调整。此外，要注意将这个值设置得大于 max_worker_processes 将不会产生效果，因为并行工作者进程都是从 max_worker_processes 所建立的工作者进程池中取出来的。

`backend_flush_after`

属性	描述
类型	integer
默认值	0
级别	USER

只要一个后端写入了超过 `backend_flush_after` 字节，就会尝试强制 OS 把这些写发送到底层存储。这样做将会限制内核页高速缓存中的脏数据数量，降低在检查点末尾发出 `fsync` 时或者 OS 在后台大批写回数据时卡住的可能性。这常常会导致极大降低的事务延迟，但是也有一些情况中（特别是负载超过 shared_buffers 但低于 S 的页面高速缓存时），性能可能会下降。这个设置可能在某些平台上没有效果。合法的范围位于 `0`（禁用受控写回）和 `2MB` 之间。默认是 `0`（即没有强制写回）。（如果 `BLCKSZ` 不是 8kB，最大值会按比例缩放到它）。

`old_snapshot_threshold`

属性	描述
类型	integer
默认值	-1
级别	KINGBASE

设置在使用快照时，一个快照可以被使用而没有发生 `snapshot too old` 错误风险的最小时间。这个参数只能在服务器启动时设置。

如果超过该阈值，旧数据将被清理掉。这可以有助于阻止长时间使用的快照造成的快照膨胀。为了阻止由于本来对该快照可见的数据被清理导致的不正确结果，当快照比这个阈值更旧并且该快照被用来读取一个该快照建立以来被修改过的页面时，将会产生一个错误。

值为-1 会禁用这个特性，并且这个值是默认值。对于生产工作有用的值可能从几个小时到几天。该设置将被转换成分钟粒度，并且小数字（例如 0 或者 1min）被允许只是因为它们有时对于测试有用。虽然允许高达 60d 的设置，但是请注意很多负载情况下，很短的时间帧里就可能发生极大的膨胀或者事务 ID 回卷。

当这个特性被启用时，关系末尾的被清出的空间不能被释放给操作系统，因为那可能会移除用于检测 snapshot too old 情况所需的信息。所有分配给关系的空间还将与该关系关联在一起便于重用，除非它们被显式地释放（例如，用 VACUUM FULL）。

这个设置不会尝试保证在任何特殊情况下都会生成错误。事实上，如果（例如）可以从一个已经物化了一个结果集的游标中生成正确的结果，即便被引用表中的底层行已经被清理掉也不会生成错误。某些表不能被过早地安全清除，并且因此将不受这个设置的影响，例如系统目录。对于这些表，这个设置将不能降低膨胀，也不能降低在扫描时产生 snapshot too old 错误的可能性。

2.5.7　全局临时表

max_pending_gtfiles_per_backend

属性	描述
类型	integer
默认值	32
级别	KINGBASE

设置一个会话内最多允许的全局临时表数据和索引的最大文件数量，默认值为 32，最小值为 1，最大值为 1024。这个文件数量并不是实际磁盘看到的文件数目，而是一些依附于这个表的扩展关系文件，比如索引、大对象等。一般情况下，一个全局临时表只会占用参数中一个配额，如果为创建临时表创建了索引，或者全局临时表中包含了大对象列，则会占用多个配额，具体取决于索引和大对象的数量。这个参数只能在服务器启动时设置。设置过大会造成资源浪费。

2.6　预写式日志

2.6.1　设置

wal_level

属性	描述
类型	enum
默认值	replica
级别	KINGBASE

wal_level 决定多少信息写入到 WAL 中。默认值是 replica，它会写入足够的数据以支持 WAL 归档和复制，包括在后备服务器上运行只读查询。minimal 会去掉除从崩溃或者立即关机中进行恢复所需的信息之外的所有记录。最后，logical 会增加支持逻辑解码所需的信息。每个层次包括所有更低层次记录的信息。

在 minimal 级别中，某些批量操作的 WAL 日志可以被安全地跳过，这可以使那些操作更快。这种优化可以应用的操作包括：

- CREATE TABLE AS
- CREATE INDE
- CLUSTER
- COPY 到在同一个事务中被创建或截断的表中

但最少的 WAL 不会包括足够的信息来从基础备份和 WAL 日志中重建数据，因此，要启用 WAL 归档（archive_mode ）和流复制，必须使用 replica 或更高级别。

在 logical 层，与 replica 相同的信息会被记录，外加上允许从 WAL 抽取逻辑修改集所需的信息。使用 logical 级别操作将增加 WAL 容量，特别是如果为了 REPLICA IDENTITY FULL 配置了很多表并且执行了很多 UPDATE 和 DELETE 语句时。

在 V8R2 之前的版本中，这个参数也允许值 archive 和 hot_standby。现在仍然接受这些值，但是它们会被映射到 replica。

这个参数可以在服务器启动时设置，也可以在线设置。使用在线设置时，此参数只能在 replica 和 logical 两个级别之间切换。如果 old_snapshot_threshold >= 0 或逻辑复制槽存在，不允许在线设置此参数。

fsync

属性	描述
类型	boolean
默认值	on
级别	SIGHUP

当启用此参数，KingbaseES 服务器将尝试确保所有更新通过发出 fsync() 系统调用或其他等效方法（参见 wal_sync_method）物理写入磁盘，以此保障了数据库集簇在操作系统或者硬件崩溃后能恢复到一致的状态。

虽然关闭 fsync 可以带来性能提升，但当发生断电或系统崩溃时可能造成不可恢复的数据损坏。因此，仅在能轻松从外部数据中重建整个数据库时，才建议关闭 fsync。例如，从备份文件中初始加载一个新数据库集簇、在数据库被删掉并重建之后使用新数据库集簇来处理批量数据，或者对于经常重建且不作为故障恢复备用的只读数据库克隆。仅凭高质量硬件不足以成为关闭 fsync 的理由。

若将 fsync 从关闭状态切换为开启状态时，为了确保可靠恢复，必须强制内核中所有已修改的缓冲区持久化存储。这可以在集簇被关闭时、运行 initdb --sync-only 开启 fsync 时、执行 sync 时、卸载文件系统时或者重启服务器时完成。

在许多场景下，对于不重要的事务，关闭 synchronous_commit 可以获得与关闭 fsync 相似的性能收益，而不会带来数据损坏的风险。

fsync 只能在 kingbase.conf 文件中或在服务器命令行上设置。如果确认需要关闭这个参数，请也考虑同时关闭 full_page_writes。

synchronous_commit

属性	描述
类型	enum
默认值	on
级别	USER

指定在命令返回“success”指示给客户端之前，一个事务是否需要等待 WAL 记录被写入磁盘。合法的值是 on、remote_apply、remote_write、local 和 off。默认的并且安全的设置是 on。当设置为 off 时，在向客户端报告成功和真正保证事务不会被服务器崩溃威胁之间会有延迟（最大的延迟是 wal_writer_delay 的三倍）。不同于 fsync ，将这个参数设置为 off 不会产生数据库不一致性的风险：一个操作系统或数据库崩溃可能会造成一些最近据说已提交的事务丢失，但数据库状态是一致的，就像这些事务已经被干净地中止。因此，当性能比完全确保事务的持久性更重要时，关闭 synchronous_commit 可以作为一个有效的代替手段。

如果 synchronous_standby_names 为非空，这个参数也控制事务提交是否将等待它们的 WAL 记录被复制到后备服务器上。当这个参数被设置为 on 时，直到来自于当前同步的后备服务器的回复指示它们已经收到了事务的提交记录并将其刷入了磁盘，主服务器上的事务才会提交。这保证事务将不会被丢失，除非主服务器和所有同步后备都遭受到了数据库存储损坏的问题。当被设置为 remote_apply 时，提交将会等待，直到来自当前的同步后备的回复指示它们已经收到了该事务的提交记录并且已经应用了该事务，这样该事务才变得对后备上的查询可见。当这个参数被设置为 remote_write 时，提交将等待，直到来自当前的同步后备的回复指示它们已经收到了该事务的提交记录并且已经把该记录写出到它们的操作系统，这种设置足以保证数据在后备服务器的 KingbaseES 实例崩溃时得以保存，但是不能保证后备服务器遭受操作系统级别崩溃时数据能被保持，因为数据不一定必须要在后备机上达到稳定存储。最后，设置 local 会导致提交等待本地刷写到磁盘而不是复制完成。在使用同步复制时这通常不是我们想要的效果，但是为了完整性，还是提供了这样一个选项。

如果 synchronous_standby_names 为空，设置 on、remote_apply、remote_write 和 local 都提供了同样的同步级别：事务提交只等待本地刷写磁盘。

这个参数可以随时被修改；任何一个事务的行为由其提交时生效的设置决定。因此，可以同步提交一些事务，同时异步提交其他事务。例如，当默认是相反时，实现一个单一多语句事务的异步提交，在事务中发出 SET LOCAL synchronous_commit TO OFF。

wal_sync_method

属性	描述
类型	enum
默认值	fsync
级别	SIGHUP

该参数用于强制将 WAL 更新到磁盘。如果 fsync 是关闭状态，则此设置可忽略，因为 WAL 文件更新不会被强制。可选取值为：

- open_datasync（用 open() 选项 O_DSYNC 写 WAL 文件）
- fdatasync（在每次提交时调用 fdatasync()）
- fsync（在每次提交时调用 fsync()）
- fsync_writethrough（在每次提交时调用 fsync()，强制任何磁盘写高速缓存的直通写）
- open_sync（用 open() 选项 O_SYNC 写 WAL 文件）

open_* 选项也可以使用 O_DIRECT（如果可用）。不是在所有平台上都能使用所有这些选择。默认值是列表中第一个被平台支持的那个，不过 fdatasync 是 Linux 中的默认值。默认值不一定是最理想的；有可能需要修改这个设置或系统配置的其他方面来创建一个崩溃-安全的配置，或达到最佳性能。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

full_page_writes

属性	描述
类型	boolean
默认值	on
级别	SIGHUP

当这个参数为打开时，KingbaseES 服务器在一个检查点之后的页面的第一次修改期间将每个页面的全部内容写

到 WAL 中。这么做是因为在操作系统崩溃期间正在处理的一次页写入可能只有部分完成，从而导致在一个磁盘页面中混合有新旧数据。在崩溃后的恢复期间，通常存储在 WAL 中的行级改变数据不足以完全恢复这样一个页面。存储完整的页面映像可以保证页面被正确存储，但代价是增加了必须被写入 WAL 的数据量（因为 WAL 重放总是从一个检查点开始，所以在检查点后每个页面的第一次改变时这样做就够了。因此，一种减小全页面写开销的方法是增加检查点间隔参数值）。

把这个参数关闭会加快正常操作，但是在系统失败后可能导致不可恢复的数据损坏，或者静默的数据损坏。其风险类似于关闭 `fsync`，但是风险较小。并且只有在可关闭 `fsync` 的情况下才应该关闭它。

关闭这个选项并不影响用于时间点恢复（PITR）的 WAL 归档使用。

这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。默认值是 `on`。

`wal_log_hints`

属性	描述
类型	boolean
默认值	off
级别	KINGBASE

当这个参数为 `on` 时，KingbaseES 服务器一个检查点之后页面被第一次修改期间把该磁盘页面的整个内容都写入 WAL，即使对所谓的提示位做非关键修改也会这样做。

如果启用了数据校验和，提示位更新总是会被 WAL 记录并且这个设置会被忽略。可以使用这个设置测试如果数据库启用了数据校验和，会有多少额外的 WAL 记录发生。

这个参数只能在服务器启动时设置。默认值是 `off`。

`wal_compression`

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

此参数允许使用指定的压缩方法压缩 WAL。启用后，KingbaseES 服务器会在 full_page_writes 处于打开状态或进行基本备份期间，对写入 WAL 中的完整页面图像进行压缩。压缩后的页面图像将在 WAL 重放期间解压缩。支持的解压缩方法包括 pglz、lz4（需编译时启用 --with-lz4）和 zstd（需编译时启用 --with-zstd）。默认值是 off（未启用压缩）。只有超级用户和具有相应权限的用户才能更改此设置。

启用压缩可以在不增加数据不可恢复损坏风险的前提下减少 WAL 的生成量，但代价是额外消耗一些 CPU 资源，用于在 WAL 记录时进行压缩，以及在 WAL 重放时进行解压缩。

`wal_buffers`

属性	描述
类型	integer
默认值	-1
级别	KINGBASE

用于还未写入磁盘的 WAL 数据的共享内存量。默认值 -1 选择等于 shared_buffers 的 1/32 的尺寸（大约 3%），但是不小于 `64kB` 也不大于 WAL 段的尺寸（通常为）。如果自动的选择太大或太小可以手工设置该值，但是任何小于 `32kB` 的正值都将被当作 `32kB`。这个参数只能在服务器启动时设置。

在每次事务提交时，WAL 缓冲区的内容被写出到磁盘，因此极大的值不可能提供显著的收益。不过，把这个值设置为几个兆字节可以在一个繁忙的服务器（其中很多客户端会在同一时间提交）上提高写性能。由默认设置 -1 选择的自动调节将在大部分情况下得到合理的结果。

`wal_writer_delay`

属性	描述
类型	integer
默认值	200ms
级别	SIGHUP

指定 WAL 写入器刷写 WAL 的频繁程度。在刷写 WAL 之后它会睡眠 `wal_writer_delay` 毫秒，除非被一个异步提交事务唤醒。假如上一次刷写发生在少于 `wal_writer_delay` 毫秒以前并且从上一次刷写发生以来产生了少于 `wal_writer_flush_after` 字节的 WAL，则 WAL 将只被写入到操作系统而不是被刷到磁盘。默认值是 200 毫秒（`200ms`）。注意在很多系统上，有效的睡眠延迟粒度是 10 毫秒，把 `wal_writer_delay` 设置为一个不是 10 的倍数的值，其效果和把它设置为大于该值的下一个 10 的倍数产生的效果相同。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。

`wal_writer_flush_after`

属性	描述
类型	integer

-续表-

属性	描述
默认值	1MB
级别	KINGBASE

指定 WAL 写入器刷写 WAL 的频繁程度。如果上一次刷写发生在少于 `wal_writer_delay` 毫秒以前并且从上一次刷写发生以来产生了少于 `wal_writer_flush_after` 字节的 WAL，则 WAL 将只被写入到操作系统而不是被刷到磁盘。如果 `wal_writer_flush_after` 被设置为 `0`，则 WAL 数据会被立即刷写。默认是 `1MB`。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。

`commit_delay`

属性	描述
类型	integer
默认值	0
级别	SUPERUSER

在一次 WAL 刷写被发起之前，`commit_delay` 增加一个时间延迟，以微妙计。如果系统负载足够高，使得在一个给定间隔内有额外的事务准备好提交，那么通过允许更多事务通过一个单次 WAL 刷写来提交能够提高组提交的吞吐量。但是，它也把每次 WAL 刷写的潜伏期增加到了最多 `commit_delay` 微秒。因为如果没有其他事务准备好提交，就会浪费一次延迟，只有在当一次刷写将要被发起时有至少 `commit_siblings` 个其他活动事务时，才会执行一次延迟。另外，如果 `fsync` 被禁用，则将不会执行任何延迟。默认的 `commit_delay` 是零（无延迟）。只有超级用户才能修改这个设置。

在 KingbaseES 的 V8R2 发布之前，`commit_delay` 的行为不同并且效果更差：它只影响提交，而不是所有 WAL 刷写，并且即使在 WAL 刷写马上就要完成时也会等待一整个配置的延迟。从 KingbaseES V8R2 中开始，第一个准备好刷写的进程会等待配置的间隔，而后续的进程只等到领先者完成刷写操作。

`commit_siblings`

属性	描述
类型	integer
默认值	5
级别	USER

在执行 commit_delay 延迟时，要求的并发活动事务的最小数目。大一些的值会导致在延迟间隔期间更可能有至少另外一个事务准备好提交。默认值是五个事务。

enable_xlog_insert_lock_free

属性	描述
类型	boolean
默认值	off
级别	USER

是否开启 WAL 插入的无锁化优化。开启此参数后，后端进程不在刷写 WAL ，而是等待 WAL 写入器刷写对应的 WAL 完成。同时，wal_insert 等待事件会变为 wal_flush_wait 等待事件。开启无锁化优化后，WAL 写入器将会更加积极的刷写，后端进程也会在完成日志复制后，第一时间通知刷写。此参数默认值是 off。这个参数只能在 kingbase.conf 文件中设置，通过重启生效。启用此参数，将会使 commit_delay，commit_siblings，wal_writer_flush_after 等参数失效。

wal_writer_idle_flush_before_sleep

属性	描述
类型	integer
默认值	200
级别	KINGBASE

指定 WAL 写入器在多少次空闲刷写后，进入睡眠状态，它会睡眠 wal_writer_delay 毫秒，除非当日志复制结束后被唤醒。当开启 enable_xlog_insert_lock_free 时才有意义。默认是 200 次。这个参数只能在 kingbase.conf 文件中或者服务器命令行上设置。

llogminer_enable_pbe

属性	描述
类型	boolean
默认值	on
级别	KINGBASE

参数控制是否记录 PBE 本地更新操作 wal , 当取值为 no 时，表示服务端会以独立的日志记录更新的前项数据，保证逻辑同步软件能正常同步更新数据，取值为 off 时，表示不以独立的形式记录本地更新 wal。

2.6.2　检查点

`checkpoint_timeout`

属性	描述
类型	integer
默认值	5min
级别	SIGHUP

自动 WAL 检查点之间的最长时间，以秒计。合理的范围在 30 秒到 1 天之间。默认是 5 分钟（`5min`）。增加这个参数的值会增加崩溃恢复所需的时间。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

`checkpoint_completion_target`

属性	描述
类型	real
默认值	0.5
级别	SIGHUP

指定检查点完成的目标，作为检查点之间总时间的一部分。默认是 0.5。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

`checkpoint_flush_after`

属性	描述
类型	integer
默认值	256KB
级别	SIGHUP

在执行检查点时，只要有 checkpoint_flush_after 字节被写入，就尝试强制 OS 把这些写发送到底层存储。这样做将会限制内核页面高速缓存中的脏数据数量，降低在检查点末尾发出 fsync 或者 OS 在后台大批量写回数据时被卡住的可能性。那常常会导致大幅度压缩的事务延迟，但是也有一些情况（特别是负载超过 shared_buffers 但小于 OS 页面高速缓存）的性能会降低。这种设置可能会在某些平台上没有效果。合法的范围在 0（禁用强制写回）和 2MB 之间。Linux 上的默认值是 256kB，其他平台上是 0（如果 BLCKSZ 不是 8kB，则默认值和最大值会按比例缩放到它）。这个参数只能在 kingbase.conf 文件中或者服务器命令行上设置。

checkpoint_warning

属性	描述
类型	integer
默认值	30s
级别	SIGHUP

如果由于填充 WAL 段文件导致的检查点之间的间隔低于这个参数表示的秒数，那么就向服务器日志写一个消息（它建议增加 max_wal_size 的值）。默认值是 30 秒（30s）。零则关闭警告。如果 checkpoint_timeout 低于 checkpoint_warning，则不会有警告产生。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

max_wal_size

属性	描述
类型	integer
默认值	1GB
级别	SIGHUP

在自动 WAL 检查点之间允许 WAL 增长到的最大尺寸。这是一个软限制，在特殊的情况下 WAL 尺寸可能会超过 max_wal_size，例如在重度负荷下、archive_command 失败或者高的 wal_keep_segments 设置。默认为 1 GB。增加这个参数可能导致崩溃恢复所需的时间。这个参数只能在 kingbase.conf 或者服务器命令行中设置。

min_wal_size

属性	描述
类型	integer
默认值	80MB

-续表-

属性	描述
级别	SIGHUP

只要 WAL 磁盘用量保持在这个设置之下，在检查点时旧的 WAL 文件总是被回收以便未来使用，而不是直接被删除。这可以被用来确保有足够的 WAL 空间被保留来应付 WAL 使用的高峰，例如运行大型的批处理任务。默认是 80 MB。这个参数只能在 `kingbase.conf` 或者服务器命令行中设置。

2.6.3　归档

`archive_mode`

属性	描述
类型	enum
默认值	always
级别	KINGBASE

当启用 `archive_mode` 时，可以通过设置 archive_command 命令将完成的 WAL 段发送到归档存储。除用于禁用的 `off` 之外，还有两种模式：`on` 和 `always`。在普通操作期间，这两种模式之间没有区别，但是当设置为 `always` 时，WAL 归档器在归档恢复或者后备模式下也会被启用。在 `always` 模式下，所有从归档恢复的或者用流复制传来的文件将被（再次）归档。

`archive_mode` 和 `archive_command` 是独立的变量，这样可以在不影响归档模式的前提下修改 `archive_command`。这个参数只能在服务器启动时设置。当 `wal_level` 被设置为 `minimal` 时，`archive_mode` 不能被启用。

`archive_command`

属性	描述
类型	string
默认值	'exit 0'
级别	SIGHUP

本地 shell 命令被执行来归档一个完成的 WAL 文件段。字符串中的任何%p 被替换成要被归档的文件的路径名，而%f 只被文件名替换（路径名是相对于服务器的工作目录，即集簇的数据目录）。如果要在命令里嵌入一个真正的%字符，可以使用%%。有一点很重要，该命令只在成功时返回一个零作为退出状态。

这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。除非服务器启动时启用了 archive_mode，否则它会被忽略。如果 archive_mode 被启用时，archive_command 默认为’exit 0’，WAL 归档会被临时禁用，但服务器仍会继续累计 WAL 段文件，期待着一个命令被提供。将 archive_command 设置为一个只返回真但不做任何事的命令（例如/bin/true 或 Windows 上的 REM）实际上会禁用归档，也会打破归档恢复所需的 WAL 文件链，因此只有在极少数情况下才能用。

archive_timeout

属性	描述
类型	integer
默认值	0
级别	KINGBASE

archive_command 仅在已完成的 WAL 段上调用。因此，如果服务器只产生很少的 WAL 流量（或产生流量的周期很长），那么在事务完成和它被安全地记录到归档存储之间将有一个很长的延迟。为了限制未归档数据存在的时间，可以设置 archive_timeout 来强制服务器来周期性地切换到一个新的 WAL 段文件。当这个参数被设置为大于零时，只要从上次段文件切换后过了参数所设置的那么多秒并且已经有过任何数据库活动（包括一个单一检查点），服务器将切换到一个新的段文件（如果没有数据库活动则会跳过检查点）。注意，由于强制切换而提早关闭的被归档文件仍然与完整的归档文件长度相同。因此，使用非常短的 archive_timeout 是不明智的—它将占用巨大的归档存储。一分钟左右的 archive_timeout 设置通常比较合理。如果希望数据能被更快地从主服务器上复制下来，应该考虑使用流复制而不是归档。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

2.6.4　归档文件恢复

本节描述仅在恢复期间适用的设置。如果希望执行任何后续恢复，则必须重置它们。

“Recovery”包括将服务器用作备用服务器或用于执行目标恢复。通常，备用模式用于提供高可用性和/或读可伸缩性，而目标恢复用于从数据丢失中恢复。

要在备用模式下启动服务器，请在数据目录中创建一个名为 standby.signal 的备用文件。服务器将开始恢复，不会停止恢复时的 WAL 归档，但是将继续尝试通过连接到发送服务器来进行恢复，具体方法是通过 primary_conninfo 设置或使用 restore_command 来获取新的 WAL 段。对于这种模式，本节中的参数是有意义的。来自恢复目标 的参数也将被应用，但通常在此模式下并不有用。

若要以目标恢复模式启动服务器，请在数据目录中创建一个名为 recovery.signal 的文件。如果文

件 standby.signal 和 recovery.signal 都被创建，备用模式优先。当 WAL 归档完全重播，或者达到 recovery_target 时，目标恢复模式结束。在此模式下，来自本节和恢复目标 的参数将被应用。

restore_command

属性	描述
类型	string
默认值	,,
级别	KINGBASE

执行本地 shell 命令来检索 WAL 文件系列的归档段。此参数对于存档恢复是必需的，但对于流式复制是可选的。字符串中的任何%f 都将被从存档中检索的文件名称替换，而任何%p 都将被服务器上的复制目标路径名称替换。(路径名相对于当前工作目录，即，集群的数据目录。) 任何%r 都将被包含最后一个有效重启点的文件名称所代替。这是为了允许恢复可重新启动而必须保留的最早的文件，因此可以使用此信息将存档截断为支持从当前恢复重新启动所需的最小值。%r 通常只用于温备配置。写%% 将嵌入一个实际的 % 字符。

请注意命令只有在执行成功时才返回零退出状态，如果命令被要求提供档案中不存在的文件名，则必须返回非零退出状态。示例如下：

```
restore_command = 'cp /mnt/server/archivedir/%f "%p"'
restore_command = 'copy "C:\\server\\archivedir\\%f" "%p"'  # Windows
```

一个例外是, 如果命令终止了一个信号 (除了 SIGTERM, 它用作数据库服务器关闭的一部分) 或者一个错误的外壳 (如命令未找到)，然后恢复将中止，服务器将不会启动。

此参数只能在服务器启动时设置。

archive_cleanup_command

属性	描述
类型	string
默认值	,,
级别	SIGHUP

此可选参数指定将在每个重新开始点执行的 shell 命令。archive_cleanup_command 的目的是提供一种机制来清理备用服务器不再需要的旧的归档 WAL 文件。任意%r 用包含最后一个有效重新启动点的文件的名称替换。通过保留并维护最早的文件，能够恢复至重启点，因此，所有%r 之前的文件都可以被安全删除。此信息可用于将存档截断为支持从当前还原重新启动所需的最小值。sys_archivecleanup 模块通常用于 archive_cleanup_command 对于 single-

standby 的配置，例如：

```
archive_cleanup_command = 'sys_archivecleanup /mnt/server/archivedir %r'
```

但是请注意，如果多个备用服务器从相同的存档目录恢复，则需要确保在任何服务器不再需要 WAL 文件之前不会删除它们。archive_cleanup_command 通常在温备配置中使用。编写%%，在命令中嵌入一个实际的% 字符。

如果该命令返回非零退出状态，则将写入警告日志消息。一个例外是，如果命令因一个信号或 shell 错误 (例如命令未找到) 而终止，将引发一个致命错误。

此参数只能在 kingbase.conf 文件或在服务器命令行中设置。

recovery_end_command

属性	描述
类型	string
默认值	' '
级别	SIGHUP

此参数指定仅在恢复结束时执行一次的 shell 命令。这个参数是可选的。recovery_end_command 的目的是为复制或恢复后的清理提供一种机制。任何%r 都被包含最后一个有效重启点的文件名称替换，就像 archive_cleanup_command 。

如果该命令返回非零退出状态，则将写入一条警告日志消息，数据库将继续启动。一个例外是，如果命令因信号或 shell 错误 (例如未找到命令) 而终止，数据库将无法继续启动。

此参数只能在 kingbase.conf 文件或在服务器命令行中设置。

2.6.5 恢复目标

默认情况下，恢复将恢复到 WAL 日志的末尾。以下参数可用于指定较早的停止点。至多 recovery_target、recovery_target_lsn、recovery_target_name、recovery_target_time 或 recovery_target_xid 中的一个可以被使用；如果在配置文件中指定了其中一个以上，则会引发错误。这些参数只能在服务器启动时设置。

recovery_target

属性	描述
类型	string
默认值	immediate
级别	KINGBASE

此参数指定一旦达到一致状态，即越早恢复越好。从联机备份恢复时，这意味着备份结束的时间点。

从技术上讲，这是一个字符串参数，但是`'immediate'`是当前唯一允许的值。

`recovery_target_name`

属性	描述
类型	string
默认值	,,
级别	KINGBASE

此参数指定将执行恢复的指定恢复点 (使用 `sys_create_restore_point()` 创建)。

`recovery_target_time`

属性	描述
类型	string
默认值	timestamp
级别	KINGBASE

此参数指定将进行恢复的时间戳。精确的停止点也会受到 recovery_target_inclusive 影响。

`recovery_target_xid`

属性	描述
类型	string
默认值	,,
级别	KINGBASE

此参数指定将进行恢复的事务 ID。请记住，虽然在事务开始时按顺序分配事务 id，但事务可以按不同的数字顺序完成。将被恢复的事务是那些在指定事务之前提交的事务 (可选包括)。精确的停止点也会受到 recovery_target_inclusive 影响。

`recovery_target_lsn`

属性	描述
类型	string
默认值	' '
级别	KINGBASE

此参数指定将进行恢复的写前日志位置的 LSN。精确的停止点也会受到 recovery_target_inclusive 影响。使用系统数据类型 sys_lsn 解析此参数。

以下选项进一步指定恢复目标，并影响到达目标时的情况:

`recovery_target_inclusive`

属性	描述
类型	boolean
默认值	on
级别	KINGBASE

指定是否仅在指定的恢复目标之后停止 (`on`), 或者就在恢复目标之前 (`off`)。适用于当 recovery_target_lsn 、recovery_target_time 或 recovery_target_xid 都是确定的。此设置控制是否将分别具有目标 WAL 位置 (LSN)、提交时间或事务 ID 的事务包含在恢复中。默认是 `on`。

`recovery_target_timeline`

属性	描述
类型	string
默认值	latest
级别	KINGBASE

指定恢复到特定时间线。该值可以是数字时间线 ID 或特殊值。值 `current` 表示恢复到时间轴中执行基本备份时的时间点上。值 `latest` 表示恢复到时间轴上存档中找到的最近时间点上，这在备用服务器中非常有用。`latest` 是默认的。

通常只需要在复杂的重新恢复情况下设置此参数，在这种情况下，需要返回到在某个时间点恢复之后达到的状态。

`recovery_target_action`

属性	描述
类型	enum
默认值	pause
级别	KINGBASE

指定服务器到达恢复目标后应采取的操作。默认的是 `pause`，这意味着恢复将被暂停。`promote` 表示恢复过程结束，服务器开始接受连接。最后，`shutdown` 将在达到恢复目标后停止服务器。

使用 `pause` 设置的目的是允许对数据库执行查询，以检查这个恢复目标是否是最需要的恢复点。使用 `sys_wal_replay_resume()` 命令可以恢复暂停状态，然后导致复苏结束。如果此恢复目标不是所需的停止点，则关闭服务器，将恢复目标设置更改为稍后的目标并重新启动以继续恢复。

`shutdown` 设置对于在需要的确切重播点准备好实例非常有用。实例仍然能够重播更多的 WAL 记录 (实际上，在下一次启动它时，必须重播上一次检查点之后的 WAL 记录)。

注意，当 `recovery_target_action` 被设置为 `shutdown` 时，`recovery.signal` 将不会被删除，除非改变配置或者 `recovery.signal` 文件被手动删除。

如果没有设置恢复目标，则此设置无效。如果 hot_standby 未启用，则 `pause` 将执行与 `shutdown` 相同的设置。

2.6.6　lz4 压缩说明

概述：

lz4 是一种高性能的无损压缩算法，主要特点是压缩和解压缩速度快，适用于对效率要求较高的场景，诸如实时数据处理、日志存储、网络传输等场景。

用途：本产品中，lz4 用于减少存储空间和提高数据传输效率，适用于大量日志、缓存数据等场景。

2.7　复制

这些设置控制内建流复制特性的行为。服务器可以是主控服务器（primary server）或后备服务器（standby server）。主控服务器可以发送数据，而后备服务器通常是复制数据的接收者。当使用级联复制（cascaded replication）时，后备服务器也可以充当发送者，同时仍然是接收者。这些参数主要用于发送服务器和后备服务器，尽管其中某些参数仅在主控服务器上有意义。如果有必要，这些设置可以在集群中进行调整而不会出现问题。

2.7.1　发送服务器

这些参数可以在任何发送复制数据给一个或多个后备服务器的服务器上设置。主控服务器总是一个发送服务器，因此这些参数总是要在主控服务器上设置。这些参数的角色和含义不会在一个后备服务器变成主控服务器后改变。

`max_wal_senders`

属性	描述
类型	integer
默认值	10
级别	KINGBASE

指定来自后备服务器或流式基础备份客户端的并发连接的最大数量（即同时运行 WAL 发送进程的最大数）。默认值是 `10`。值 `0` 意味着禁用复制。如果流客户端（如备库的 WAL Receiver）意外断开连接，可能会在主库上留下一个孤立的连接槽（connection slot），因此这个参数应该设置得略高于最大客户端连接数，这样断开连接的客户端可以立刻重新连接。这个参数只能在服务器启动时被设置。此外，`wal_level` 必须设置为 `replica` 或更高级别以允许来自后备服务器的连接。

在运行备用服务器时，必须将此参数设置为与主控服务器相同或更高的值。否则，备用服务器将不允许查询。

`max_replication_slots`

属性	描述
类型	integer
默认值	10
级别	KINGBASE

指定服务器可以支持的复制槽最大数量。默认值为 10。这个参数只能在服务器启动时设置。将它设置为一个比当前已有复制槽要少的值会阻碍服务器启动。此外，若要启用复制槽功能，参数 `wal_level` 必须被设置为 `replica` 或更高。

`wal_keep_segments`

属性	描述
类型	integer
默认值	0

-续表-

属性	描述
级别	SIGHUP

指定在备用服务器需要为流复制获取旧日志文件段的情况下，sys_wal 目录下保留的日志文件段的最小数目。每个段通常是 16MB。如果连接到发送服务器的备用服务器落后了超过 wal_keep_segments 设置的值，发送服务器可能会移除一个备用服务器仍然需要的 WAL 段，在这种情况下复制连接将被中断。最终结果是下行连接也将失败（但如果启用了 WAL 归档，备用服务器可以从存档中获取该段来恢复）。

该参数仅设置 sys_wal 中保留的文件段的最小数目，系统可能会根据需要为 WAL 归档或检查点恢复而保留更多的段和空间。如果 wal_keep_segments 是 0（缺省），系统不会为主备目的保留任何多余的段，因此备用服务器可用的旧 WAL 段的数量取决于上一个检查点的位置和 WAL 归档状态。此参数只能在 kingbase.conf 文件或在服务器命令行中设置。

wal_init_zero

属性	描述
类型	boolean
默认值	on
级别	SUPERUSER

如果设置为 on(默认值)，这个选项会导致新的 WAL 文件被填满 0。在某些文件系统上，这可以确保在需要写入 WAL 记录之前分配空间。但是，Copy-On-Write (COW) 文件系统可能不会受益于这种技术，因此可以选择跳过不必要的工作。如果设置为 off，则在创建文件时只写入最后一个字节，使其具有预期的大小。

wal_recycle

属性	描述
类型	boolean
默认值	on
级别	SUPERUSER

如果设置为 on(默认值)，这个选项会通过重命名使 WAL 文件被回收，从而避免了创建新文件的需要。在 COW

文件系统上，创建新文件系统可能更快，因此提供了禁用此行为的选项。

`wal_sender_timeout`

属性	描述
类型	integer
默认值	60s 或 120s
级别	USER

中断那些停止活动超过指定毫秒数的复制连接。对发送服务器检测一个后备服务器崩溃或网络中断有用。其中单机默认值为 60 秒，KingbaseES RAC 默认值为 120 秒。对于分布在多个地理位置的集群，每个位置使用不同的值可以为集群管理带来更大的灵活性。对于具有低延迟网络连接的备用服务器，较小的值有助于更快地进行故障检测; 对于具有高延迟网络连接的远程服务器，较大的值有助于更好地判断备用服务器的健康状况。

`track_commit_timestamp`

属性	描述
类型	boolean
默认值	off
级别	KINGBASE

记录事务的提交时间。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。默认值是 `off`。

`log_supplement_replica_identity`

属性	描述
类型	string
默认值	nothing
级别	SIGHUP

KingbaseES 可以自动为无复制标识的表添加附加日志信息，从而为其补充上复制标识。`log_supplement_replica_identity` 用于为表配置复制标识模式。

- nothing：保持现有逻辑。
- full：对于 default 和 index 模式，如果表中缺少复制标识，那么用整个元组的属性数据 (oldTuple) 作为复制标识。
- auto：对于 default 模式，如果表中缺少复制标识，那么用整个元组的属性数据 (oldTuple) 作为复制标识。对于 index 模式，如果表中缺少复制标识，那么先检查下表中是否有主键，如果有主键，则用主键作为复制标识；如果没有主键，则用整个元组的属性数据 (oldTuple) 作为复制标识。

此参数对 nothing 复制标识模式和 full 复制标识模式的表没有影响。表级别的复制标识优先级高于该参数。当日志级别小于 logical 时，参数不生效。

2.7.2　主控服务器

这些参数可以在发送复制数据给一个或多个后备服务器的主控服务器上设置。注意除了这些参数之外，在主控服务器上必须设置合适的 wal_level ，并且也启用可选的 WAL 归档，这些参数值与后备服务器无关。

`synchronous_standby_names`

属性	描述
类型	string
默认值	,,
级别	SIGHUP

该参数指定一个支持同步复制的后备服务器的列表。系统将等待这些同步后备服务器确认接收到提交的数据后，才允许事务提交。同步后备服务器是那些名字出现在这个列表前面，并且当前已连接且正在实时流传输数据的服务器。配置多个同步后备服务器可以显著提高系统的可用性，并有效防止数据丢失。

后备服务器的名称由 `application_name` 设置确定，该设置应在后备服务器的连接信息中配置。

- 物理复制场景，应该在 `primary_conninfo` 参数中设置；默认值为 `cluster_name`(如果已设置)，否则为 `walreceiver`。
- 逻辑复制场景，可以在订阅的连接信息中设置 `application_name`。

对于其他类型的流复制消费者，请参考相关文档。

这个参数使用下面的语法之一来指定一个后备服务器列表：

```
[FIRST] num_sync ( standby_name [, ...] )
ANY num_sync ( standby_name [, ...] )
standby_name [, ...]
```

其中 num_sync 是事务需要等待其回复的同步后备服务器的数量，standby_name 是一个后备服务器的名称。FIRST 以及 ANY 指定从所列服务器中选取同步后备的方法。

关键词 FIRST 加上 num_sync 指定一种基于优先的同步复制，并且会让事务提交等待，直到它们的 WAL 记录被复制到基于优先级选择的 num_sync 台同步后备上为止。例如，设置 FIRST 3 (s1, s2, s3, s4) 将导致每次提交都等待来自三台较高优先级的后备服务器的答复，这三台后备服务器将从后备服务器 s1、s2、s3 以及 s4 中选出。在该列表中出现较早的后备服务器将被给予较高的优先级，并且将被考虑为同步后备。列表中出现的其他后备服务器表示潜在的同步后备。如果当前的任何同步后备因为某种原因断开连接，它将立刻被下一个最高优先级的后备服务器替代。关键词 FIRST 是可选的。

关键词 ANY 加上 num_sync 指定一种基于规定数量的同步复制，并且会让事务提交等待，直到它们的 WAL 记录被复制到所列出后备服务器中的至少 num_sync 台上为止。例如，设置 ANY 3 (s1, s2, s3, s4) 将导致每次提交会在收到 s1、s2、s3 以及 s4 中任意三台后备服务器的回答后立刻继续下去。

FIRST 和 ANY 是大小写不敏感的。如果这些关键词被用作后备服务器的名字，其 standby_name 必须被放在双引号内。

KingbaseES 版本 V8R2 之前使用过第三种语法，目前也仍然支持。它和 FIRST 和 num_sync 等于 1 的第一种语法相同。例如，FIRST 1 (s1, s2) 和 s1, s2 具有相同的含义：s1 或者 s2 会被选中作为同步后备服务器。

特殊项 * 匹配任意后备名称。

没有机制强制后备服务器名称的唯一性。在出现重复的情况下，匹配的后备之一将被认为是较高优先级，不过无法弄清到底是哪一个。

备注:

每一个 standby_name 都应该具有合法 SQL 标识符的形式，除非它是 * 。如果必要可以使用双引号。但是注意在比较 standby_name 和后备服务器应用程序名称时是大小写不敏感的（不管有没有双引号）。

如果这里没有指定同步后备服务器名称，那么同步复制不能被启用并且事务提交将不会等待复制。这是默认的配置。即便当同步复制被启用时，个体事务也可以被配置为不等待复制，做法是将 synchronous_commit 参数设置为 local 或 off。

这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

vacuum_defer_cleanup_age

属性	描述
类型	integer
默认值	0
级别	SIGHUP

指定 VACUUM 和 HOT 更新在清除“死亡”元组之前，应该推迟多久（以事务提交序列号个数计量）。默认值是零个事务，表示“死亡”元组将被尽可能快地清除，即当它们不再对任何打开的事务可见时尽快清除。在一个支持热后备服务器的主控服务器上，可以把这个参数设置为一个非零值。这允许后备服务器上的查询有更多时间来完成而不会由于先前的行清除产生冲突。不过，由于该值是以主控服务器上发生的写事务数量为衡量标准的，很难预测对后备服务器查询可用的附加时间到底是多少。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

可以通过设置后备服务器上的 hot_standby_feedback 来作为使用这个参数的一种替代方案。

这无法阻止已经达到 old_snapshot_threshold 所指定年龄的“死亡”元组被清除。

备注:

该参数不为 0 时，会直接阻止对系统中活跃长事务进行区间清理，只清理最前端部分“死亡”元组：

- 最老长事务为 RC 隔离级别，清理次老长事务之前死亡元组
- 最老长事务为 RR 隔离级别，清理最老长事务之前死亡元组

2.7.3 后备服务器

这些设置控制接收复制数据的一个后备服务器的行为。它们的值与主控服务器无关。

primary_conninfo

属性	描述
类型	string
默认值	,,
级别	KINGBASE

指定用于备用服务器与发送服务器连接的连接字符串。如果该字符串中有未指定的选项，则选中相应的环境变量。如果环境变量也没有设置，则使用默认值。

连接字符串应该指定发送服务器的主机名 (或地址)，以及端口号 (如果它与备用服务器的默认端口号不同)。还要指定与发送服务器上具有适当特权的角色对应的用户名。如果发送方需要口令身份验证，还需要提供口令。它可以在 primary_conninfo 字符串中提供，或是备用服务器上的单独的 ~/.kbpass 文件（使用 replication 作为数据库名称）。不要在 primary_conninfo 字符串中指定数据库名称。

此参数只能在服务器启动时设置。如果服务器没有处于备用模式，则此设置无效。

primary_slot_name

属性	描述
类型	string
默认值	,,
级别	KINGBASE

可选地指定在通过流复制连接到发送服务器以控制上游节点上的资源删除时使用的现有复制槽。此参数只能在服务器启动时设置。如果没有设置 `primary_conninfo`，则此设置无效。

`promote_trigger_file`

属性	描述
类型	string
默认值	,,
级别	SIGHUP

指定一个触发器文件，该文件的存在将结束备用系统中的恢复。即使未设置此值，仍然可以促进备用使用 `sys_ctl promote`，或者调用 `sys_promote`。此参数只能在 `kingbase.conf` 文件或在服务器命令行中设置。

`hot_standby`

属性	描述
类型	boolean
默认值	on
级别	KINGBASE

指定在恢复期间，是否能够连接并运行查询。默认值是 `on`。这个参数只能在服务器启动时设置。它只在归档恢复期间或后备服务器模式下才有效。

`max_standby_archive_delay`

属性	描述
类型	integer
默认值	30s
级别	SIGHUP

当热后备服务器处于活动状态时，这个参数决定取消那些与即将应用的 WAL 项冲突的后备服务器查询之前，后备服务器应该等待多久。当 WAL 数据被从 WAL 归档（并且因此不是当前的 WAL）时，max_standby_archive_delay 可以应用。默认值是 30 秒。如果没有指定，衡量单位是毫秒。值 -1 允许后备服务器一直等到冲突查询结束。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

注意，max_standby_archive_delay 与取消之前一个查询能够运行的最长时间不同；它表示应用任何一个 WAL 段数据能够被允许的最长总时间。因此，如果一个查询早于 WAL 段导致了显著的延迟，后续冲突查询将只有更少的时间。

max_standby_streaming_delay

属性	描述
类型	integer
默认值	30s
级别	SIGHUP

当热后备服务器处于活动状态时，这个参数决定取消那些与即将应用的 WAL 项冲突的后备服务器查询之前，后备服务器应该等待多久。当 WAL 数据正在通过流复制被接收时，max_standby_streaming_delay 可以应用。默认值是 30 秒。如果没有指定，衡量单位是毫秒。值 -1 允许后备服务器一直等到冲突查询结束。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

注意，max_standby_streaming_delay 与取消之前一个查询能够运行的最长时间不同；它表示在从主控服务器接收到 WAL 数据并立刻应用它能够被允许的最长总时间。因此，如果一个查询导致了显著的延迟，后续冲突查询将只有更少的时间，直到后备服务器再次赶上进度。

wal_receiver_status_interval

属性	描述
类型	integer
默认值	2s

-续表-

属性	描述
级别	SIGHUP

指定在后备服务器上的 WAL 接收进程向主控服务器或上游后备服务器发送有关复制进度信息的最小频度，发送的复制进度信息可以在主控服务器或上游后备服务器的 `sys_stat_replication` 视图中看到。后备服务器将报告自己已经写入文件缓存的最后 WAL 日志位置、已经刷新到磁盘的最后 WAL 日志位置以及已经重做完成的最后 WAL 日志位置。这个参数的值是每次发送复制进度信息的最大间隔，以秒为单位。每次写入或刷新的 WAL 日志位置改变时都会发送复制进度信息，如果没有 WAL 日志位置改变则至少按这个参数指定的间隔发送复制进度信息。将这个参数设置为零会禁用此功能，后备服务器将不会主动发送复制进度信息（如果主控服务器或上游后备服务器要求发送心跳或其他同步信息，会被动发送复制进度信息）。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。默认值是 2 秒。

`hot_standby_feedback`

属性	描述
类型	boolean
默认值	off
级别	SIGHUP

指定一个热后备服务器是否将会向主控服务器或上游后备服务器发送有关于后备服务器上当前正被执行的查询的反馈。这个参数可以被用来排除由于记录清除导致的查询取消，但是可能导致在主控服务器上用于某些负载的数据库膨胀。反馈消息的发送频度不会高于每个 `wal_receiver_status_interval` 周期发送一次。默认值是 `off`。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

如果使用级联复制，反馈将被向上游传递直到它最后到达主控服务器。后备服务器在接收到反馈之后除了传递给上游不会做任何其他操作。

这个设置不会覆盖主控服务器上的 `old_snapshot_threshold` 的行为，后备服务器上一个超过了主控服务器年龄阈值的快照可能会变得不可用，导致后备服务器上事务的取消。这是因为 `old_snapshot_threshold` 是为了对死亡行能够存在的时间给出一个绝对限制，不然就会因为一个后备服务器的配置而被违背。

`wal_receiver_timeout`

属性	描述
类型	integer

-续表-

属性	描述
默认值	60s
级别	SIGHUP

中止处于非活动状态超过指定毫秒数的复制链接。这对于正在接收的后备服务器检测主控服务器崩溃或网络断开有用。值零会禁用超时机制。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。默认值是 60 秒。

`wal_retrieve_retry_interval`

属性	描述
类型	integer
默认值	5s
级别	SIGHUP

指定当从任何来源（流复制、本地 `sys_wal` 或者 WAL 归档）都得不到 WAL 数据时，后备服务器应该等待多久才去重新尝试获取 WAL 数据。这个参数只能在 `kingbase.conf` 文件或者服务器命令行中设置。默认值是 5 秒。如果没有指定，则单位是毫秒。

这个参数对恢复中的节点需要为新 WAL 数据可用等待多少时间的配置有用。例如，在归档恢复中，通过减小这个参数的值可以让恢复更积极地检测新的 WAL 日志文件。在一个 WAL 活动较低的系统上，增加这个参数的值可以减少访问 WAL 归档所必需的请求数量，这对于例如云环境是有用的，在其中对于基础设施的访问时间也是被考虑的。

`recovery_min_apply_delay`

属性	描述
类型	integer
默认值	0
级别	SIGHUP

默认情况下，备用服务器将尽快从发送服务器恢复 WAL 记录。拥有一个延迟的数据副本可能会很有用，这为

纠正数据丢失错误提供了机会。此参数允许恢复延迟一段固定的时间 (如果没有指定单位，则以毫秒为单位)。例如，5min，那么只有当备用系统的时间比主系统报告的提交时间至少晚 5 分钟时，备用系统才会重新执行每个事务提交。

服务器之间的复制延迟可能超过此参数的值，在这种情况下不添加延迟。请注意，延迟是在主控服务器上写的 WAL 时间戳和备用服务器上的当前时间之间计算的。由于网络延迟或级联复制配置而导致的传输延迟可能会显著减少实际的等待时间。如果主备系统上的时钟不同步，则可能导致应用记录的恢复比预期的要早; 但这不是一个主要问题，因为此参数的有用设置比服务器之间的典型时间偏差大得多。

延迟只发生在事务提交的 WAL 记录上。其他记录会尽可能快地重放，这不是问题，因为 MVCC 可见性规则确保它们的效果在应用相应的提交记录之前是不可见的。

当处于恢复状态的数据库达到一致状态时，直到提升或触发备用数据库时，才会发生延迟。之后，备用服务器将结束恢复，不再等待。

此参数用于流式复制部署; 但是，如果指定了参数，则在所有情况下都将使用该参数。hot_standby_feedback 将延迟使用此功能，这可能会导致主机膨胀; 两者一起使用时要小心。

注意:

同步复制将被影响当 synchronous_commit 被设置为 remote_apply；每个 COMMIT 需要等待应用。

此参数只能在 kingbase.conf 文件或在服务器命令行中设置。

2.7.4 订阅者

这些设置控制逻辑复制订阅者的行为。它们在发布者上的值与此无关。

注意，配置参数 wal_receiver_timeout、wal_receiver_status_interval 以及 wal_retrieve_retry_interval 也影响逻辑复制工作者。

max_logical_replication_workers

属性	描述
类型	int
默认值	4
级别	KINGBASE

指定逻辑复制工作者的最大数目。这同时包括应用工作者和表同步工作者。

逻辑复制工作者是从 max_worker_processes 定义的池中取出的。

max_sync_workers_per_subscription

属性	描述
类型	integer
默认值	2
级别	SIGHUP

每个订阅的同步工作者的最大数目。这个参数控制订阅初始化期间或者新表增加时的初始数据拷贝的并行度。

当前，每个表只能有一个同步工作者。

同步工作者是从 `max_logical_replication_workers` 定义的池中取出的。

2.8　查询计划

2.8.1　规划器方法配制

这些配置参数影响查询优化器选择查询计划的暴力方法。如果优化器为一个特定查询选择的默认计划不是最优的，一种临时解决方案是使用这些配置参数之一来强制优化器选择一个不同的计划。提高优化器选择的计划质量的更好的方式包括调整规划器的代价常数、手工运行 ANALYZE、增加 `default_statistics_target` 配置参数的值以及使用 `ALTER TABLE SET STATISTICS` 增加为特定列收集的统计信息量。

关于 ANALYZE 使用方法，请参考 ANALYZE(Oracle)、ANALYZE(SQLServer) 。

`enable_bitmapscan`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器使用位图扫描计划类型。默认值是 `on`。

`enable_gathermerge`

属性	描述
类型	boolean
默认值	on
级别	USER

启用或者禁用查询计划器对收集归并计划类型的使用。默认值是 `on`。

`enable_hashagg`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁用查询计划器使用哈希聚集计划类型。默认值是 `on`。

`enable_hashjoin`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器使用哈希连接计划类型。默认值是 `on`。

`enable_indexscan`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器使用索引扫描计划类型。默认值是 `on`。

`enable_indexonlyscan`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器使用仅索引扫描计划类型。默认值是 `on`。

`enable_material`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或者禁止查询计划器使用物化。它不可能完全禁用物化，但是关闭这个变量将阻止规划器插入物化节点，除非为了保证正确性。默认值是 `on`。

`enable_mergejoin`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器使用归并连接计划类型。默认值是 `on`。

`enable_nestloop`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器使用嵌套循环连接计划。它不可能完全禁止嵌套循环连接，但是关闭这个变量将使得规划器尽可能优先使用其他方法。默认值是 `on`。

`enable_parallel_append`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器使用并行追加计划类型。默认值是 `on`。

`enable_parallel_hash`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器对并行哈希使用哈希连接计划类型。如果哈希连接计划也没有启用，这个参数没有效果。默认值是 `on`。

`enable_partition_pruning`

属性	描述
类型	boolean

-续表-

属性	描述
默认值	on
级别	USER

允许或者禁止查询计划器从查询计划中消除一个分区表的分区。这也控制着规划器产生允许执行器在查询执行期间移除（忽略）分区的查询计划的能力。默认值是 `on`。详情请参考管理分区表 。

`enable_partitionwise_join`

属性	描述
类型	boolean
默认值	off
级别	USER

允许或者禁止查询计划器使用面向分区的连接，这使得分区表之间的连接以连接匹配的分区的方式来执行。面向分区的连接当前只适用于连接条件包括所有分区键的情况，连接条件必须是相同的数据类型并且子分区集合要完全匹配。由于面向分区的连接规划在规划期间会使用可观的 CPU 时间和内存，所以默认值为 `off`。

`enable_partitionwise_aggregate`

属性	描述
类型	boolean
默认值	off
级别	USER

允许或者禁止查询计划器使用面向分区的分组或聚集，这使得在分区表上的分组或聚集可以在每个分区上分别执行。如果 `GROUP BY` 子句不包括分区键，只有部分聚集能够以基于每个分区的方式执行，并且 finalization 必须最后执行。由于面向分区的分组或聚集在规划期间会使用可观的 CPU 时间和内存，所以默认值为 `off`。

`enable_seqscan`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器使用顺序扫描计划类型。它不可能完全禁止顺序扫描，但是关闭这个变量将使得规划器尽可能优先使用其他方法。默认值是 `on`。

`enable_sort`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器使用显式排序步骤。它不可能完全禁止显式排序，但是关闭这个变量将使得规划器尽可能优先使用其他方法。默认值是 `on`。

`enable_tidscan`

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止查询计划器使用 TID 扫描计划类型。默认值是 `on`。

`enable_aggregate_pullup`

属性	描述
类型	boolean

-续表-

属性	描述
默认值	on
级别	USER

允许或禁止查询计划器提升聚集函数层级。对于子查询中引用了外层查询变量的聚集函数，当开启变量时，聚集函数将被提升到外层查询计算；当关闭变量时，聚集函数保留在子查询中计算。该参数仅在 oracle 模式下有效。默认值是 `on`。

2.8.2　规划器代价常量

这一节中描述的 * 代价 * 变量可以按照任意尺度衡量。我们只关心它们的相对值，将它们以相同的因子缩放不会影响规划器的选择。默认情况下，这些代价变量是基于顺序页面获取的代价的，即 `seq_page_cost` 被设置为 `1.0` 并且其他代价变量都参考它来设置。不过可以使用不同尺度，例如在一个特定机器上的真实执行时间。

备注:

不幸的是，没有一种良好的方法来决定代价变量的理想值。因此，它们最好是被作为某个特定安装的查询的平均值来对待。这也意味着，参照少数的测试值来改变它们是有较大风险的。

`seq_page_cost`

属性	描述
类型	real
默认值	1.0
级别	USER

设置规划器对一系列顺序磁盘页面获取中的一次的代价估计。默认值是 1.0。通过把表和索引放在一个特殊的表空间（要设置该表空间的同名参数）中可以覆盖这个值（见 ALTER TABLESPACE(Oracle)、ALTER TABLESPACE(MySQL)、ALTER TABLESPACE(SQLServer) ）。

`random_page_cost`

属性	描述
类型	real
默认值	4
级别	USER

设置规划器对一次非顺序获取磁盘页面的代价估计。默认值是 4。通过把表和索引放在一个特殊的表空间（要设置该表空间的同名参数）中可以覆盖这个值（见 ALTER TABLESPACE(Oracle)、ALTER TABLESPACE(MySQL)、ALTER TABLESPACE(SQLServer) ）。

降低此值（相对于 `seq_page_cost`）会使系统更倾向于选择索引扫描；而提高此值则会使索引扫描显得相对代价更高，系统会更少地选择索引扫描。可以一起提高或降低两个值来改变磁盘 I/O 代价相对于 CPU 代价的重要性，后者由下列参数描述。

对磁盘存储的随机访问通常比顺序访问要贵不止四倍。但是，由于对磁盘的大部分随机访问（例如被索引的读取）都被假定在高速缓冲中进行，所以使用了一个较低的默认值（4）。默认值可以被想成把随机访问建模为比顺序访问慢 40 倍，而期望 90% 的随机读取会被缓存。

90% 的缓冲率对负载是一个不正确的假设，可以增加 `random_page_cost` 来更好的反映随机存储读取的真正代价。相应地，如果数据可以完全放在高速缓存中（例如当数据库小于服务器总内存时），降低 `random_page_cost` 可能是合适的。为具有很低的随机读取代价的存储（例如固态驱动器）采用较低的 `random_page_cost` 值可能更好。

提示:

虽然允许将 `random_page_cost` 设置的比 seq_page_cost 小，但是物理上的实际情况并不受此影响。然而当所有数据库都位于内存中时，两者设置为相等是非常合理的，因为在此情况下，乱序抓取并不比顺序抓取开销更大。同样，在缓冲率很高的数据库上，应当相对于 CPU 开销同时降低这两个值，因为获取内存中的页比通常情况下的开销小许多。尽管系统可以把 `random_page_cost` 设置得小于 `seq_page_cost`，但是实际上没有意义。不过，如果数据库被整个缓存在 RAM 中，将它们设置为相等是有意义的，因为在那种情况中不按顺序访问页面是没有惩罚值的。同样，在一个高度缓存化的数据库中，应该相对于 CPU 参数降低这两个值，因为获取一个已经在 RAM 中的页面的代价要远小于通常情况下的代价。

`cpu_tuple_cost`

属性	描述
类型	real
默认值	0.01
级别	USER

设置规划器对一次查询中处理每一行的代价估计。默认值是 0.01。

`cpu_index_tuple_cost`

属性	描述
类型	real
默认值	0.005
级别	USER

设置规划器对一次索引扫描中处理每一个索引项的代价估计。默认值是 0.005。

`cpu_operator_cost`

属性	描述
类型	real
默认值	0.0025
级别	USER

设置规划器对于一次查询中处理每个操作符或函数的代价估计。默认值是 0.0025。

`parallel_setup_cost`

属性	描述
类型	real
默认值	1000
级别	USER

设置规划器对启动并行工作者进程的代价估计。默认是 1000。

`parallel_tuple_cost`

属性	描述
类型	real
默认值	0.1
级别	USER

设置规划器对于从一个并行工作者进程传递一个元组给另一个进程的代价估计。默认是 0.1。

`min_parallel_table_scan_size`

属性	描述
类型	integer
默认值	8MB
级别	USER

为必须扫描的表数据量设置一个最小值，扫描的表数据量超过这一个值才会考虑使用并行扫描。对于并行顺序扫描，被扫描的表数据量总是等于表的尺寸，但是在使用索引时，被扫描的表数据量通常会更小。默认值是 8 兆字节（8MB）。

`min_parallel_index_scan_size`

属性	描述
类型	integer
默认值	512kB
级别	USER

为必须扫描的索引数据量设置一个最小值，扫描的索引数据量超过这一个值时才会考虑使用并行扫描。注意并行索引扫描通常并不会触及整个索引，它是规划器认为该扫描会实际用到的相关页面的数量。默认值是 512 千字节（512kB）。

`effective_cache_size`

属性	描述
类型	integer
默认值	4GB
级别	USER

设置规划器对一个单一查询可用的有效磁盘缓冲区尺寸的假设。这个参数会被考虑在使用一个索引的代价估计中，更高的数值会使得索引扫描更可能被使用，更低的数值会使得顺序扫描更可能被使用。在设置这个参数时，还应该考虑 KingbaseES 的共享缓冲区以及将被用于 KingbaseES 数据文件的内核磁盘缓冲区，尽管有些数据可能在两个地方都存在。另外，还要考虑预计在不同表上的并发查询数目，因为它们必须共享可用的空间。这个参数对 KingbaseES 分配的共享内存尺寸没有影响，它也不会保留内核磁盘缓冲，它只用于估计的目的。系统也不会假设在查询之间数据会保留在磁盘缓冲中。默认值是 4 吉字节（`4GB`）。

`jit_above_cost`

属性	描述
类型	real
默认值	100000
级别	USER

设置激活 JIT 编译的查询代价，如果查询代价超过这个值就会激活 JIT 编译（如果启用了 JIT）。执行 JIT 会消耗一些规划时间，但是能够加速查询执行。将这个值设置为`-1` 会禁用 JIT 编译。默认值是 `100000`。

`jit_inline_above_cost`

属性	描述
类型	real
默认值	500000
级别	USER

设置 JIT 编译尝试内联函数和操作符的查询代价阈值，如果查询代价超过这个值，JIT 编译就会尝试内联。内联会增加规划时间，但是可以提高执行速度。将这个参数设置成小于 `jit_above_cost` 是没有意义的。将这个参数设置为`-1` 会禁用内联。默认值是 `500000`。

jit_optimize_above_cost

属性	描述
类型	real
默认值	500000
级别	USER

设置 JIT 编译应用优化的查询代价阈值，如果查询代价超过这个值，JIT 编译就会应用开销较大的优化。这类优化会增加规划时间，但是更能够改进执行速度。将这个参数设置成小于 jit_above_cost 是没有意义的，并且将它设置成大于 jit_inline_above_cost 也未必有益。将这个参数设置为-1 会禁用开销较大的优化。默认值是 500000。

hier_worktable_rehash_cost_factor

属性	描述
类型	real
默认值	0.1
级别	USER

设置层次查询中递归侧做哈希连接时，中间结果表作为连接外表的代价估算因子。默认值是 0.1。

2.8.3　遗传查询优化

GEQO 是一个使用探索式搜索来执行查询计划的算法。它可以降低负载查询的规划时间。同时，GEQO 的检索是随机的，因此它的规划可能会不可确定。遗传查询计划器（GEQO）是一种使用启发式搜索来进行查询计划的算法。它可以降低对于复杂查询（连接很多表的查询）的规划时间，但是代价是它产生的计划有时候要差于使用穷举搜索算法找到的计划。

geqo

属性	描述
类型	boolean
默认值	on
级别	USER

允许或禁止遗传查询优化。默认是启用。在生产环境中通常最好不要关闭它。geqo_threshold 变量提供了对 GEQO 更细粒度的控制。

geqo_threshold

属性	描述
类型	integer
默认值	12
级别	USER

只有当涉及的 FROM 项数量至少有这么多个的时候，才使用遗传查询优化（注意一个 FULL OUTER JOIN 只被计为一个 FROM 项）。默认值是 12。对于更简单的查询，通常会使用普通的穷举搜索规划器，但是对于有很多表的查询穷举搜索会花很长时间，通常比执行一个次优的计划带来的惩罚值还要长。因此，在查询尺寸上的一个阈值是管理 GEQO 使用的一种方便的方法。

geqo_effort

属性	描述
类型	integer
默认值	5
级别	USER

这个变量必须是一个范围从 1 到 10 的整数。缺省值是 5。增大此值将增加花在进行查询计划上面的时间，但是也很可能会提高选中更有效的查询计划的几率。因此需要妥善控制此值，来使得 GEQO 里规划时间和查询计划的有效性之间达到一个平衡。

geqo_effort 实际并不直接做任何事情；它只是被用来计算其他影响 GEQO 行为的变量（如下所述）的默认值，可以手工设置其他参数。

geqo_pool_size

属性	描述
类型	integer
默认值	0

-续表-

属性	描述
级别	USER

控制 GEQO 使用的池尺寸，它就是遗传种群中的个体数目，有用的值通常在 100 到 1000 之间。如果它被设置为零（默认设置）则会基于 `geqo_effort` 和查询中表的数量选择一个合适的值。

`geqo_generations`

属性	描述
类型	integer
默认值	0
级别	USER

控制 GEQO 使用的子代数目。子代的意思是算法的迭代次数。它必须大于等于 1，有用的值范围和池大小相同。如果设置为 0(缺省)，那么将基于 `geqo_pool_size` 选取合适的值。

`geqo_selection_bias`

属性	描述
类型	real
默认值	2
级别	USER

控制 GEQO 使用的选择偏好。选择偏好是种群中的选择压力。值可以是 1.5 到 2 之间，后者是默认值。

`geqo_seed`

属性	描述
类型	real
默认值	0.0

-续表-

属性	描述
级别	USER

控制 GEQO 使用的随机数生成器的初始值，随机数生成器用于在连接顺序搜索空间中选择随机路径。该值可以从 0 （默认值）到 1。变化该值会改变被探索的连接路径集合，并且可能导致找到一个更好或更差的路径。

2.8.4 其他规划器选项

`default_statistics_target`

属性	描述
类型	integer
默认值	100
级别	USER

为没有通过 `ALTER TABLE SET STATISTICS` 设置列相关目标的表列设置默认统计目标。更大的值增加了需要做 `ANALYZE` 的时间，但是可能会改善规划器的估计质量。默认值是 100。

`constraint_exclusion`

属性	描述
类型	enum
默认值	partition
级别	USER

`constraint_exclusion` 的允许值是 `on` （对所有表检查约束）、`off`（从不检查约束）和 `partition`（只对继承的子表和 `UNION ALL` 子查询检查约束）。`partition` 是默认设置。它通常被用于传统的继承树来提高性能。

当对一个特定表允许这个参数，规划器比较查询条件和表的 `CHECK` 约束，并且忽略那些条件违反约束的表扫描。例如：

```
CREATE TABLE parent(key integer, ...);
CREATE TABLE child1000(check (key between 1000 and 1999)) INHERITS
(parent);
CREATE TABLE child2000(check (key between 2000 and 2999)) INHERITS
(parent);
...
SELECT * FROM parent WHERE key = 2400;
```

在启用约束排除时，这个 SELECT 将完全不会扫描 child1000，从而提高性能。

目前，约束排除只在通过继承树实现表分区的情况中被默认启用。为所有表启用它会增加额外的规划开销，特别是在简单查询上并且不会产生任何好处。如果没有使用传统继承进行分区的表，则需要完全关闭它。(注意，分区表的等效特性是由一个单独的参数控制的，enable_partition_pruning 。)

cursor_tuple_fraction

属性	描述
类型	real
默认值	0.1
级别	USER

设置规划器对将被检索的一个游标的行的比例的估计。默认值是 0.1。更小的值使得规划器偏向为游标使用“快速开始”计划，它将很快地检索前几行但是可能需要很长时间来获取所有行。更大的值强调总的估计时间。最大设置为 1.0，游标将和普通查询完全一样地被规划，只考虑总估计时间并且不考虑前几行会被多快地返回。

from_collapse_limit

属性	描述
类型	integer
默认值	8
级别	USER

如果生成的 FROM 列表不超过这么多项，规划器将把子查询融合到上层查询。较小的值可以减少规划时间，但是可能会生成较差的查询计划。默认值是 8。

将这个值设置为 geqo_threshold 或更大，可能触发使用 GEQO 规划器，从而产生非最优计划。见遗传查询优化 。

jit

属性	描述
类型	boolean
默认值	on
级别	USER

决定如果可用，KingbaseES 是否可以使用 JIT 编译。默认值是 on。

join_collapse_limit

属性	描述
类型	integer
默认值	8
级别	USER

如果得出的列表中不超过这么多项，那么规划器将把显式 JOIN（除了 FULL JOIN）结构重写到 FROM 项列表中。较小的值可减少规划时间，但是可能会生成差些的查询计划。

默认情况下，这个变量被设置成和 from_collapse_limit 相同，这样适合大多数使用。把它设置为 1 可避免任何显式 JOIN 的重排序。因此查询中指定的显式连接顺序就是关系被连接的实际顺序。因为查询计划器并不是总能选取最优的连接顺序，高级用户可以选择暂时把这个变量设置为 1，然后显式地指定他们想要的连接顺序。

将这个值设置为 geqo_threshold 或更大，可能触发使用 GEQO 规划器，从而产生非最优计划。见遗传查询优化 。

parallel_leader_participation

属性	描述
类型	boolean
默认值	on
级别	USER

允许领导者进程执行 Gather 和 Gather Merge 节点之下的查询计划而不是等待工作者进程。默认值是 on。将这个值设置为 off 会降低工作者由于领导者读取元组速度不够快而被阻塞的可能性，但是要求领导者在产生第一个元组之前等待工作者进程启动。领导者能够帮助或者阻碍性能的程度取决于计划的类型、工作者的数量以及查询时长。

force_parallel_mode

属性	描述
类型	enum
默认值	off
级别	USER

允许为测试目的使用并行查询，即便是并不期望在性能上得到效益。force_parallel_mode 的允许值是 off（只在期望改进性能时才使用并行模式）、on（只要查询被认为是安全的，就强制使用并行查询）以及 regress（和 on 相似，但是有如下文所解释的额外行为改变）。

更具体地说，把这个值设置为 on 会在任何一个对于并行查询安全的查询计划顶端增加一个 Gather 节点，这样查询会在一个并行工作者中运行。即便当一个并行工作者不可用或者不能被使用时，诸如开始一个子事务等在并行查询环境中会被禁止的操作将会被禁止，除非规划器相信这样做会导致查询失败。当这个选项被设置时如果出现失败或者意料之外的结果，查询使用的某些函数可能需要被标记为 PARALLEL UNSAFE（或者可能是 PARALLEL RESTRICTED）。

把这个值设置为 regress 具有设置成 on 所有相同的效果，外加一些有助于自动回归测试的额外的效果。一般来说，来自于一个并行工作者的消息会包括一个上下文行指出这一点，但是设置为 regress 会消除这一行，这样输出就和非并行执行完全一样。同样，被这个设置加到计划上的 Gather 节点在 EXPLAIN 输出终会被隐藏起来，这样产生的输出匹配设置为 off 时产生的输出。

plan_cache_mode

属性	描述
类型	enum
默认值	auto
级别	USER

准备好的语句（无论是显式准备的还是隐式生成的，例如 PL/SQL）都可以使用定制计划或通用计划执行。使用特定的一组参数值为每次执行重新制定自定义计划，而通用计划不依赖于参数值，可以在执行过程中重用。因此，使用泛型计划可以节省计划时间，但是如果理想的计划严重依赖于参数值，那么泛型计划可能是低效的。

这些选项之间的选择通常是自动进行的，但是它可以被覆盖 plan_cache_mode 覆盖。允许的值是 auto(默认)，force_custom_plan 和 force_generic_plan。此设置将在执行缓存计划时考虑，而不是在准备计划时考虑。

exclude_reserved_words

属性	描述
类型	string
默认值	,,
级别	USER

指定需要忽略的关键字，使关键字可以用作表名或列名。但是会造成使用指定关键字的功能失效。

```
1. 设置该参数时不检查是否是关键字。
2. 当屏蔽掉关键字“level”时，执行使用关键字“level”的功能语句时，请将“level”替换成“KINGBASE_INTERNAL_
LEVEL”关键字。
```

输入时，关键字用单引号括起。指定关键字时可同时设定多个，词间通过“，”隔开。

例如：

```
set exclude_reserved_words= 'drop','in';
SET

show exclude_reserved_words;
exclude_reserved_words
------------------------
  drop, in
(1 row)

CREATE TABLE drop(in INT);
CREATE TABLE
SELECT in from drop;
in
----
(0 rows)

DROP table drop;
ERROR:  syntax error at or near "DROP"        ------功能受到限制
LINE 1: DROP table drop;
         ^


set exclude_reserved_words = 'level, in';
```

```
SHOW  TRANSACTION ISOLATION KINGBASE_INTERNAL_LEVEL;
 transaction_isolation
-----------------------
read committed
(1 row)

set exclude_reserved_words= '';
SET

show exclude_reserved_words;
exclude_reserved_words
------------------------

(1 row)
```

`enable_query_rule`

属性	描述
类型	boolean
默认值	off
级别	USER

允许用户使用 KingbaseES 的 Query Mapping 功能，该功能允许用户预先创建 SQL 语句的映射关系并储存在相应的系统表，当用户输入的 SQL 语句与所创建的映射关系相匹配时，替换成映射的 SQL 语句去实际执行。该参数的默认值是 `off`。当设置为 `on` 时，会启用 Query Mapping 功能。

2.9　错误报告和日志

2.9.1　在哪里记录日志

`log_destination`

属性	描述
类型	string
默认值	stderr

-续表-

属性	描述
级别	SIGHUP

KingbaseES 支持多种方法来记录服务器消息，包括 stderr、csvlog、syslog、alertlog。在 Windows 上还支持 eventlog。设置这个参数为一个由想要的日志目的地的列表，之间用逗号分隔。默认值是只记录到 stderr。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

如果 csvlog 被包括在 `log_destination` 中，日志项会以“逗号分隔值”（CSV）格式被输出，这样可以很方便地把日志载入到程序中。要产生 CSV 格式的日志输出，必须启用 logging_collector 。

当包括有 stderr 或 csvlog 时，会创建文件 `current_logfiles` 来记录当前正在被日志收集器使用的日志文件的位置以及相关的日志目的地。这提供了一种查找实例当前使用的日志的便利手段。这里是该文件内容的一个例子：

```
stderr log/kingbase.log
csvlog log/kingbase.csv
```

当包括有 alertlog 时，将开启告警日志，否则不会产生告警日志，默认为不开启。当开启告警日志后，数据库会实时产生告警记录，并记录到告警日志文件（以 _alert.log 结尾）中。同时，通过参数log_min_alert_statement 来指定最小告警级别，将日志等级大于等于 `log_min_alert_statement` 的系统日志记入到告警日志中，作为告警日志。

当由于轮转效应创建一个新的日志文件时以及 `log_destination` 被重载时，`current_logfiles` 文件会被重建。当 `log_destination` 中不包括 stderr 和 csvlog 时以及当日志收集器被禁用时，这个文件会被删除。

备注:

在大多数 Unix 系统上，需要修改系统的 syslog 守护进程的配置来使用 `log_destination` 的 syslog 选项。KingbaseES 可以在 syslog 设备 `LOCAL0` 到 `LOCAL7` 中记录（见 syslog_facility），但是大部分平台上的默认 syslog 配置会丢弃所有这种消息。需要增加这样的内容：

```
local0.*    /var/log/kingbase
```

到 syslog 守护进程的配置文件来让它工作。在 Windows 上，当使用 `log_destination` 的 `eventlog` 选项时，应该在操作系统中注册一个事件源和库，这样 Windows 事件查看器能够清楚地显示事件日志消息。

`logging_collector`

属性	描述
类型	boolean

-续表-

属性	描述
默认值	on
级别	KINGBASE

这个参数启用 * 日志收集器 *，它是一个捕捉被发送到 stderr 的日志消息的后台进程，并且它会将这些消息重定向到日志文件中。这种方法比记录到 syslog 通常更有用，因为某些类型的消息不会在 syslog 输出中出现（一个常见的例子是动态链接器错误消息；另一个例子是由 `archive_command` 等脚本产生的错误消息）。这个参数只能在服务器启动时设置。

备注:

也可以不使用日志收集器而把日志记录到 stderr，日志消息将只会去到服务器的 stderr 被定向到的位置。不过，那种方法只适合于低日志量，因为它没有提供方法来轮转日志文件。还有，在某些不使用日志收集器的平台上可能会导致丢失或者混淆日志输出，因为多个进程并发写入同一个日志文件时会覆盖彼此的输出。

备注:

日志收集器被设计成从来不会丢失消息。这意味着在极高的负载下，如果服务器进程试图在收集器已经落后时发送更多的日志消息，那么它会被阻塞。相反，syslog 倾向于在无法写入消息时丢掉消息，这意味着在这样的情况下它可能会无法记录某些消息，但是它不会阻塞系统的其他部分。

`log_directory`

属性	描述
类型	string
默认值	sys_log
级别	SIGHUP

当 `logging_collector` 被启用时，这个参数决定日志文件将被在哪个目录下创建。它可以被指定为一个绝对路径，也可以被指定为一个相对于集簇数据目录的相对路径。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。默认是 `sys_log`。

`log_filename`

属性	描述
类型	string
默认值	kingbase-%Y-%m-%d_%H%M%S.log
级别	SIGHUP

当 logging_collector 被启用时，这个参数设置被创建的日志文件的文件名。该值被视为一种 strftime 模式，因此% 转义可以被用来指定根据时间变化的文件名（注意如果有任何时区独立的% 转义，计算将在由 log_timezone 指定的时区中完成）。被支持的% 转义和开放组织的 strftime 说明中列举的类似。注意系统的 strftime 不会被直接使用，因此平台相关（非标准）的扩展无法工作。默认是 kingbase-%Y-%m-%d_%H%M%S.log。

如果不使用转义来指定一个文件名，则应该计划使用一个日志轮转工具来避免最终填满整个磁盘。在 8.4 发行之前，如果不存在% 转义，KingbaseES 将追加新日志文件创建时间的纪元，但是现在已经不再这样做了。

如果在 log_destination 中启用了 CSV 格式输出，.csv 将会被追加到时间戳日志文件名中来创建 CSV 格式输出（如果 log_filename 以.log 结尾，该后缀会被替换）。

这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

log_file_mode

属性	描述
类型	integer
默认值	0600
级别	SIGHUP

在 Unix 系统上，当 logging_collector 被启用时，这个参数设置日志文件的权限（在微软 Windows 上这个参数将被忽略）。这个参数值应当是一个数字形式的模式，它可以被 chmod 和 umask 系统调用接受（要使用通常的十进制格式，该数字必须以一个 0（零）开始）。

默认的权限是 0600，表示只有服务器拥有者才能读取或写入日志文件。其他常用的设置是 0640，它允许拥有者的组成员读取文件。不过需要修改 log_directory 为将文件存储在集簇数据目录之外的某个位置，才能利用这个设置。在任何情况下，让日志文件变成任何人都可读是不明智的，因为日志文件中可能包含敏感数据。

这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

log_rotation_age

属性	描述
类型	integer
默认值	1d
级别	SIGHUP

当 `logging_collector` 被启用时，这个参数决定一个个体日志文件的最长生命期。当这些分钟过去后，一个新的日志文件将被创建。将这个参数设置为零将禁用基于时间的新日志文件创建。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

`log_rotation_size`

属性	描述
类型	integer
默认值	10MB
级别	SIGHUP

当 `logging_collector` 被启用时，这个参数决定一个个体日志文件的最大尺寸。当这么多千字节被发送到一个日志文件后，将创建一个新的日志文件。将这个参数设置为零将禁用基于尺寸的新日志文件创建。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

`log_truncate_on_rotation`

属性	描述
类型	boolean
默认值	off
级别	SIGHUP

当 `logging_collector` 被启用时，这个参数将导致 KingbaseES 截断（覆盖而不是追加）任何已有的同名日志文件。不过，截断只在一个新文件由于基于时间的轮转被打开时发生，在服务器启动或基于尺寸的轮转时不会发生。如果被关闭，在所有情况下以前存在的文件将被追加。例如，使用这个设置和一个类似 `kingbase-%H.log` 的 `log_filename` 将导致产生 24 个每小时的日志文件，并且循环地覆盖它们。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

例子：要保留 7 天的日志，每天的一个日志文件被命令为 server_log.Mon、server_log.Tue 等等，并且自动用本周的日志覆盖上一周的日志。可以这样做：将 log_filename 设置为 server_log.%a、将 log_truncate_on_rotation 设置为 on 并且将 log_rotation_age 设置为 1440。

例子：要保留 24 小时的日志，每个小时一个日志文件，但是在日志文件尺寸超过 1GB 时轮转。可以这样做：将 log_filename 设置为 server_log.%H%M、将 log_truncate_on_rotation 设置为 on、将 log_rotation_age 设置为 60 并且将 log_rotation_size 设置为 1000000。在 log_filename 中包括%M 允许发生任何尺寸驱动的轮转来选择一个不同于每个小时的初始文件名的新文件名。

syslog_facility

属性	描述
类型	enum
默认值	LOCAL0
级别	SIGHUP

当启用了向 syslog 记录时，这个参数决定要使用的 syslog“设备”。可以在 LOCAL0、LOCAL1、LOCAL2、LOCAL3、LOCAL4、LOCAL5、LOCAL6、LOCAL7 中选择，默认值是 LOCAL0。还请参阅系统的 syslog 守护进程的文档。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

syslog_ident

属性	描述
类型	string
默认值	kingbase
级别	SIGHUP

当启用了向 syslog 记录时，这个参数决定用来标识 syslog 中的 KingbaseES 消息的程序名。默认值是 kingbase。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

syslog_sequence_numbers

属性	描述
类型	boolean
默认值	on

-续表-

属性	描述
级别	SIGHUP

当日志被记录到 syslog 并且这个设置为 on（默认）时，每一个消息会被加上一个增长的序号作为前缀（例如 [2]）。这种行为避开了很多 syslog 实现默认采用的“—上一个消息重复 N 次—”形式。在现代 syslog 实现中，抑制重复消息是可以配置的（例如 rsyslog 中的 `$RepeatedMsgReduction`），因此这个参数可能不是必需的。此外，如果想抑制重复消息，可以把这个参数设置为 off。

这个参数只能在 `kingbase.conf` 文件或者服务器命令行上设置。

`syslog_split_messages`

属性	描述
类型	boolean
默认值	on
级别	SIGHUP

当启用把日志记录到 syslog 时，这个参数决定消息如何送达 syslog。当设置为 on（默认）时，消息会被分成行，并且长的行也会被划分以便能够放到 1024 字节中，这是传统 syslog 实现一种典型的尺寸限制。当设置为 off 时，KingbaseES 服务器日志消息会被原样送达 syslog 服务，而处理可能的大体量消息的任务由 syslog 服务负责。

如果 syslog 最终被记录到一个文本文件中，那么两种设置的效果是一样的，但最好设置为 on，因为大部分 syslog 实现要么不能处理大型消息，要么需要做特殊的配置以处理大型消息。但是如果 syslog 最终写入到某种其他媒介，有必要让消息保持逻辑上的整体性（也更加有用）。

这个参数只能在 `kingbase.conf` 文件或者服务器命令行上设置。

`event_source`

属性	描述
类型	string
默认值	Kingbase
级别	KINGBASE

当启用了向事件日志记录时，这个参数决定用来标识日志中 KingbaseES 消息的程序名。默认值是 KingbaseES。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

2.9.2 什么时候记录日志

log_min_messages

属性	描述
类型	enum
默认值	warning
级别	SUPERUSER

控制哪些消息级别被写入到服务器日志。有效值是 DEBUG5、DEBUG4、DEBUG3、DEBUG2、DEBUG1、INFO、NOTICE、WARNING、ERROR、LOG、FATAL 和 PANIC。每个级别都包括以后的所有级别。级别越靠后，被发送的消息越少。默认值是 WARNING。注意 LOG 在这里有与 client_min_messages 中不同的排名。只有超级用户可以改变这个设置。

log_min_error_statement

属性	描述
类型	enum
默认值	error
级别	SUPERUSER

控制哪些导致一个错误情况的 SQL 语句被记录在服务器日志中。任何指定严重级别或更高级别的消息的当前 SQL 语句将被包括在日志项中。有效值是 DEBUG5、DEBUG4、DEBUG3、DEBUG2、DEBUG1、INFO、NOTICE、WARNING、ERROR、LOG、FATAL 和 PANIC。默认值是 ERROR，它表示导致错误、日志消息、致命错误或恐慌错误的语句将被记录在日志中。要有效地关闭记录失败语句，将这个参数设置为 PANIC。只有超级用户可以改变这个设置。

log_min_alert_statement

属性	描述
类型	enum
默认值	FATAL

-续表-

属性	描述
级别	SUPERUSER

指定最小错误告警级别。当开启告警日志（参见log_destination 中的 alertlog 参数）时，系统会将日志等级大于等于 `log_min_alert_statement` 的系统日志记入到告警日志中，作为告警日志。默认值为 FATAL。

`log_min_duration_statement`

属性	描述
类型	integer
默认值	-1
级别	SUPERUSER

如果语句运行至少指定的毫秒数，将导致记录每一个这种完成的语句的持续时间。将这个参数设置为零将打印所有语句的执行时间。设置为 -1（默认值）将停止记录语句持续时间。例如，如果设置它为 `250ms`，那么所有运行 250ms 或更久的 SQL 语句将被记录。启用这个参数可以有助于追踪应用中未优化的查询。只有超级用户可以改变这个设置。

对于使用扩展查询协议的客户端，解析、绑定和执行步骤的持续时间将被独立记录。

备注:

当把这个选项和 `log_statement` 一起使用时，已经被 `log_statement` 记录的语句文本不会在持续时间日志消息中重复。如果没有使用 syslog，推荐使用 `log_line_prefix` 记录 PID 或会话 ID，这样可以使用进程 ID 或会话 ID 把语句消息链接到后来的持续时间消息。

`log_transaction_sample_rate`

属性	描述
类型	real
默认值	0.0
级别	SUPERUSER

设置所有语句都被记录的事务的部分，以及由于其他原因而记录的语句。它适用于每个新事务，而不考虑其语句的持续时间。默认是 0，意思是不记录任何额外事务的语句。将其设置为 1 记录所有事务的所有语句。log_transaction_sample_rate 有助于跟踪事务的样本。

备注:

与所有语句日志记录选项一样，此选项可能会增加大量开销。

表”消息严重级别”解释了 KingbaseES 所使用的消息严重级别。如果日志输出被发送到 syslog 或 Windows 的 eventlog，严重级别会按照表中所示进行转换。

表 2.9.19: 消息严重级别

严重性	用法	syslog	eventlog
DEBUG1..DEBUG5	为开发者提供连续的更详细的信息。	DEBUG	INFORMATION
INFO	提供用户隐式要求的信息，例如来自 VACUUM VERBOSE 的输出。	INFO	INFORMATION
NOTICE	提供可能对用户有用的信息，例如长标识符截断提示。	NOTICE	INFORMATION
WARNING	提供可能出现的问题的警告，例如在一个事务块外 COMMIT。	NOTICE	WARNING
ERROR	报告一个导致当前命令中断的错误。	WARNING	ERROR
LOG	报告管理员可能感兴趣的信息，例如检查点活动。	INFO	INFORMATION
FATAL	报告一个导致当前会话中断的错误。	ERR	ERROR
PANIC	报告一个导致所有数据库会话中断的错误。	CRIT	ERROR

2.9.3 记录什么到日志

application_name

属性	描述
类型	string
默认值	‘’
级别	USER

application_name 可以是任意小于 NAMEDATALEN 个字符（标准编译中是 64 个字符）的字符串。这通常由一个应用通过到服务器的连接设置。该名称将被显示在 sys_stat_activity 视图中并被包括在 CSV 日志项中。它也会被通过 log_line_prefix 包括在普通日志项中。只有可打印 ASCII 字符能被使用在 application_name 之中。其他字符将被替换为问号（?）。

debug_print_parse

属性	描述
类型	boolean
默认值	off
级别	USER

debug_print_rewritten

属性	描述
类型	boolean
默认值	off
级别	USER

debug_print_plan

属性	描述
类型	boolean
默认值	off

-续表-

属性	描述
级别	USER

这个参数启用发出各种调试输出。当设置时，会打印生成的解析树，查询重写输出，或执行的每个查询的执行计划。这些信息是在 LOG 信息级别发出，因此默认的，它们会出现在服务器日志中，但不会发送给客户端。可以通过 client_min_messages 和/或 log_min_messages 来设置。这些参数缺省是 off。这些参数将会让多种调试输出被发出。当被设置时，它们为每一个被执行的查询打印结果分析树、查询重写器输出或执行计划。这些消息在 LOG 消息级别上被发出，因此默认情况下它们将出现在服务器日志中但不会被发送到客户端。可以通过调整 client_min_messages 和/或 log_min_messages 来改变这种情况。这些参数默认是关闭的。

debug_pretty_print

属性	描述
类型	boolean
默认值	on
级别	USER

当被设置时，debug_pretty_print 会缩进由 debug_print_parse、debug_print_rewritten 或 debug_print_plan 产生的输出。这将导致比关闭参数时使用的“紧凑”模式可读性更强但是更长的输出。它默认是打开的。

log_checkpoints

属性	描述
类型	boolean
默认值	off
级别	SIGHUP

导致检查点和重启点被记录在服务器日志中。一些统计信息也被包括在日志消息中，包括写入缓冲区的数据和写它们所花的时间。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。默认值是关闭。

log_connections

属性	描述
类型	boolean
默认值	off
级别	SU_BACKEND

导致每一次尝试对服务器的连接被记录，客户端认证的成功完成也会被记录。只有超级用户能在会话开始时更改这个参数，在会话中它不能被更改。默认为 `off`。

备注:

某些客户端程序（例如 ksql）在要求密码时会尝试连接两次，因此重复的“收到连接”消息并不一定表示一个错误。

`log_disconnections`

属性	描述
类型	boolean
默认值	off
级别	SIGHUP

导致会话终止被记录。日志输出提供的信息类似于 `log_connections`，不过还外加会话的持续时间。只有超级用户能在会话开始时更改这个参数，在会话中它不能被更改。默认为 `off`。

`log_duration`

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

导致每一个完成的语句的持续时间被记录。默认值是 `off`。只有超级用户可以改变这个设置。

对于使用扩展查询协议的客户端，解析、绑定和执行步骤的持续时间将被独立记录。

备注:

授权 log_duration 和设置 log_min_duration_statement 为零之间的区别是，超过 log_min_duration_statement 强制查询的文本被记录，但这个选项不会。因此，如果 log_duration 为 on 并且 log_min_duration_statement 为正值，所有持续时间都将被记录，但是只有超过阈值的语句才会被记录查询文本。这种行为有助于在高负载安装中收集统计信息。

---

log_error_verbosity

属性	描述
类型	enum
默认值	default
级别	SUPERUSER

控制为每一个被记录的消息要写入到服务器日志的细节量。有效值是 TERSE、DEFAULT 和 VERBOSE，每一个都为显示的消息增加更多域。TERSE 排除记录 DETAIL、HINT、QUERY 和 CONTEXT 错误信息。VERBOSE 输出包括 SQLSTATE 错误码（见 KingbaseES 错误代码）以及产生错误的源代码文件名、函数名和行号。只有超级用户能够更改这个设置。

log_hostname

属性	描述
类型	boolean
默认值	off
级别	SIGHUP

默认情况下，连接日志消息只显示连接主机的 IP 地址。打开这个参数将导致也记录主机名。注意根据主机名解析设置，这可能会导致很微小的性能损失。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

log_line_prefix

属性	描述
类型	string
默认值	’%m [%p] ’
级别	SIGHUP

这是一个 `printf` 风格的字符串，它在每个日志行的开头输出。`%` 字符开始“转义序列”，它将被按照下文描述的替换成状态信息。未识别的转义被忽略。其他字符被直接复制到日志行。某些转义只被会话进程识别并且被主服务器进程等后台进程当作空。通过指定一个在% 之后和该选项之前的数字可以让状态信息左对齐或右对齐。负值将导致在右边用空格填充状态信息已达到最小宽度，而正值则在左边填充。填充对于日志文件的人类可读性大有帮助。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。默认值是`'%m [%p] '`，它记录时间戳和进程 ID。

转义	效果	只限会话
%a	应用名	是
%u	用户名	是
%d	数据库名	是
%r	远程主机名或 IP 地址，以及远程端口	是
%h	远程主机名或 IP 地址	是
%p	进程 ID	否
%t	无毫秒的时间戳	否
%m	带毫秒的时间戳	否
%n	带毫秒的时间戳（作为 Unix 时间戳）	no
%i	命令标签：会话当前命令的类型	是
%e	SQLSTATE 错误代码	否
%c	会话 ID：见下文	否
%l	对每个会话或进程的日志行号，从 1 开始	否
%s	进程开始的时间戳	否
%v	虚拟事务 ID (backendID/localXID)	否
%x	事务 ID （如果未分配则为 0）	否

-续表-

转义	效果	只限会话
%q	不产生输出，但是告诉非会话进程在字符串的这一点停止；会话进程忽略	否
%%	纯文字%	否

%c 转义打印一个准唯一的会话标识符，它由两个 4 字节的十六进制数（不带先导零）组成，以点号分隔。这些数字是进程启动时间和进程 ID，因此%c 也可以被用作一种节省空间空间的方式来打印这些项目。例如，要从 sys_stat_activity 生成会话标识符，使用这个查询：

```
SELECT to_hex(trunc(EXTRACT(EPOCH FROM backend_start))::integer)
       || '.' ||  to_hex(pid)
FROM sys_stat_activity;
```

提示:

如果为 log_line_prefix 设置了非空值，通常应该让它的最后一个字符为空格，这样用以提供和日志行的剩余部分的视觉区别。也可以使用标点符号。

提示:

Syslog 产生自己的时间戳和进程 ID 信息，因此，如果记录到 syslog，则不会包括一些转义。

提示:

在包括仅在会话（后端）上下文中可用的信息（如用户名或者数据库名）时，%q 转义很有用。例如：

```
log_line_prefix = '%m [%p] %q%u@%d/%a '
```

log_lock_waits

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

控制当一个会话为获得一个锁等到超过 deadlock_timeout 时，是否要产生一个日志消息。这有助于决定是否所等待造成了性能低下。默认值是 off。只有超级用户可以更改这个设置。

log_statement

属性	描述
类型	enum
默认值	none
级别	SUPERUSER

控制哪些 SQL 语句被记录。有效值是 none (off)、ddl、mod 和 all（所有语句）。ddl 记录所有数据定义语句，例如 CREATE、ALTER 和 DROP 语句。mod 记录所有 ddl 语句，外加数据修改语句例如 INSERT, UPDATE、DELETE、TRUNCATE, 和 COPY FROM。如果 PREPARE、EXECUTE 和 EXPLAIN ANALYZE 包含合适类型的命令，它们也会被记录。对于使用扩展查询协议的客户端，当收到一个执行消息时会产生日志并且会包括绑定参数的值（任何内嵌的单引号会被双写）。

默认值是 none。只有超级用户可以改变这个设置。

备注:

即使使用 log_statement = all 设置，包含简单语法错误的语句也不会被记录。这是因为只有在完成基本语法解析并确定了语句类型之后才会发出日志消息。在扩展查询协议的情况下，在执行阶段之前（即在解析分析或规划期间）出错的语句也不会被记录。将 log_min_error_statement 设置为 ERROR（或更低）来记录这种语句。

---

log_replication_commands

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

导致每一个复制命令都被记录在服务器日志中。默认值是 off。只有超级用户可以更改这个设置。

log_temp_files

属性	描述
类型	integer
默认值	-1
级别	SUPERUSER

控制记录临时文件名和尺寸。临时文件可以被创建用来排序、哈希和存储临时查询结果。当每一个临时文件被删除时都会制作一个日志项。一个零值记录所有临时文件信息，而正值只记录尺寸大于或等于指定千字节数的文件。默认设置为 -1，它禁用这种记录。只有超级用户可以更改这个设置。

`log_timezone`

属性	描述
类型	string
默认值	'Atlantic/Reykjavik'
级别	SIGHUP

设置在服务器日志中写入的时间戳的时区。和 TimeZone 不同，这个值是集簇范围的，因此所有会话将报告一致的时间戳。内建默认值是 `GMT`，但是通常会被在 `kingbase.conf` 中覆盖。initdb 将安装一个对应于其系统环境的设置。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

2.9.4 使用 CSV 格式的日志输出

在 `log_destination` 列表中包括 `csvlog` 提供了一种便捷方式将日志文件导入到一个数据库表。这个选项发出逗号分隔值（CSV）格式的日志行，包括这些列：带毫秒的时间戳、用户名、数据库名、进程 ID、客户端主机、端口号、会话 ID、每个会话的行号、命令标签、会话开始时间、虚拟事务 ID、普通事务 ID、错误严重性、SQL-STATE 代码、错误消息、错误消息详情、提示、导致错误的内部查询（如果有）、错误位置所在的字符计数、错误上下文、导致错误的用户查询（如果有且被 `log_min_error_statement` 启用）、错误位置所在的字符计数、在 KingbaseES 源代码中错误的位置（如果 `log_error_verbosity` 被设置为 `verbose`）以及应用名。下面是一个定义用来存储 CSV 格式日志输出的样表：

```
CREATE TABLE kingbase_log
(
  log_time timestamp(3) with time zone,
  user_name text,
  database_name text,
```

```
  process_id integer,
  connection_from text,
  session_id text,
  session_line_num bigint,
  command_tag text,
  session_start_time timestamp with time zone,
  virtual_transaction_id text,
  transaction_id bigint,
  error_severity text,
  sql_state_code text,
  message text,
  detail text,
  hint text,
  internal_query text,
  internal_query_pos integer,
  context text,
  query text,
  query_pos integer,
  location text,
  application_name text,
  PRIMARY KEY (session_id, session_line_num)
);
```

使用 COPY FROM 命令将一个日志文件导入到这个表中：

```
COPY kingbase_log FROM '/full/path/to/logfile.csv' WITH csv;
```

可以做一些事情来简化导入 CSV 日志文件：

1. 设置 log_filename 和 log_rotation_age 为日志文件提供一种一致的、可预测的命名空间。
2. 将 log_rotation_size 设置为 0 来禁用基于尺寸的日志轮转，因为它使得日志文件名难以预测。
3. 将 log_truncate_on_rotation 设置为 on，这样在同一个文件中旧日志数据不会与新数据混杂。
4. 上述表定义包括一个主键声明。这有助于避免意外地两次导入相同的信息。COPY 命令一次提交所有它导入的数据，因此任何错误将导致整个导入失败。如果导入一个部分完成的日志文件并且稍后当它完全完成后再次导入，主键违背将导致导入失败。请等到日志完成且被关闭之后再导入。这个过程也可以避免意外地导入部分完成的行，这种行也将导致 COPY 失败。

2.9.5　进程标题

这些设置控制服务器进程的进程标题如何被修改。进程标题通常可以用 ps 或者 Windows 上的进程浏览器等程序来查看。

cluster_name

属性	描述
类型	string
默认值	,,
级别	KINGBASE

为各种目的设置标识此数据库集群 (实例) 的名称。集群名称出现在该集群中所有服务器进程的进程标题中。而且，它是备用连接的默认应用程序名称 (参考主服务器 中的 synchronous_standby_names)

为这个集簇中所有的服务器进程设置出现在进程标题中的集簇名称。这个名称可以是任何长度不超过 NAMEDATALEN 个字符（在标准编译中是 64 字符）的任何字符串。只有可打印的 ASCII 字符能被用在 cluster_name 值中。其他字符将被替换为问号（?）。如果这个参数被设置为空字符串 ''（也是默认值），将不会显示名称。这个参数只能在服务器启动时设置。

update_process_title

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

启用进程标题更新，每次服务器接收到一个新的 SQL 命令时都更新进程标题。在大部分平台上这个设置默认为 on，但是由于 Windows 上更新进程标题的开销更大，所以在 Windows 这个设置默认为 off。只有超级用户能更改这个设置。

2.10　运行时统计数据

2.10.1　查询和索引统计收集器

这些参数控制服务器范围的统计数据收集特性。当统计收集被启用时，被产生的数据可以通过 sys_stat 和 sys_statio 系统视图族访问。

track_activities

属性	描述
类型	boolean
默认值	on
级别	KINGBASE

启用对每个会话的当前执行命令的信息收集，还有命令开始执行的时间。这个参数默认为打开。注意即使被启用，这些信息也不是对所有用户可见，只有超级用户和拥有报告信息的会话的用户可见，因此它不会表现为一个安全风险。只有超级用户可以更改这个设置。

`track_activity_query_size`

属性	描述
类型	integer
默认值	1024
级别	KINGBASE

指定跟踪每个活动会话当前执行命令所保留的字节数，它们被用于 `sys_stat_activity.query` 域。默认值是 1024。这个参数只能在服务器启动时被设置。

`track_counts`

属性	描述
类型	boolean
默认值	on
级别	SUPERUSER

启用在数据库活动上的统计收集。这个参数默认为打开，因为自动清理守护进程需要被收集的信息。只有超级用户可以更改这个设置。

`track_io_timing`

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

启用对系统 I/O 调用的计时。这个参数默认为关闭，因为它将重复地向操作系统查询当前时间，这会在某些平台上导致显著的负荷。可以使用 `sys_test_timing` 工具来度量系统中计时的开销。I/O 计时信息被显示在 `sys_stat_database` 中、当 `BUFFERS` 选项被使用时的 EXPLAIN 输出中。只有超级用户可以更改这个设置。

`track_functions`

属性	描述
类型	enum
默认值	none
级别	SUPERUSER

启用跟踪函数调用计数和用时。指定 `pl` 只跟踪过程语言函数，指定 `all` 还会跟踪 SQL 和 C 语言函数。默认值是 `none`，它禁用函数统计跟踪。只有超级用户可以更改这个设置。

备注:

简单到足以被“内联”到调用查询中的 SQL 语言函数不会被跟踪，而不管这个设置。

`stats_temp_directory`

属性	描述
类型	string
默认值	sys_stat_tmp
级别	SIGHUP

设置存储临时统计数据的目录。这可以是一个相对于数据目录的路径或一个绝对路径。默认值是

sys_stat_tmp。在一个基于 RAM 的文件系统上指明这个参数将降低物理 I/O 需求，并且提高性能。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

track_instance_item

属性	描述
类型	string
默认值	”
级别	SUPERUSER

设置实例统计的级别，指定 instio 表示统计实例 IO，指定 instevent 表示统计实例事件，指定 instlock 表示统计实例锁，指定 instio_relid 表示在统计实例 IO 时统计对应的 relid。默认情况表示收集 instio、instevent、instlock。

备注:

instio 可以单独指定统计，但指定 instio_relid 的时候需与 instio 联动使用。

---

track_sql_item

属性	描述
类型	string
默认值	”
级别	SUPERUSER

设置 SQL 统计的级别，指定 sqltime 表示统计语句执行时间，指定 sqlwait 表示统计语句等待事件，指定 sqlio 表示统计语句 IO，指定 sqltime_query 表示统计语句执行时间时统计对应 queryid，指定 sqlwait_query 表示统计语句等待事件时统计对应 queryid，指定 sqlio_query 表示统计语句 IO 时统计对应 queryid。默认情况下收集全部统计信息。

备注:

sqltime 可以单独指定统计，但是指定统计 sqltime_query 的时候需要与 sqltime 联动使用；同理于 sqlio/sqlio_query，sqlwait/sqlwait_query。

---

track_sql

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

此参数用于跟踪数据库时间模型相关统计：sys_stat_sqltime（SQL 报文统计），sys_stat_sqlwait（SQL 等待事件），sys_stat_sqlio（SQL IO 统计），默认关闭。

如果关闭该开关，KWR 报告中数据库时间模型、SQL 报文统计、SQL 等待事件、SQL IO 等报告会缺少内容。建议开启该开关，并搭配：track_sql_item='sqltime, sqlwait, sqlio'。

track_instance

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

此参数用于跟踪数据库时间模型相关统计：sys_stat_instevent（关键活动），sys_stat_instio（实例 IO），sys_stat_instlock（轻量级锁活动），默认关闭。

如果关闭该开关，KWR 报告中数据库时间模型、实例 IO 统计、锁活动统计、关键活动统计等报告会缺少内容。建议开启该开关，并搭配：track_instance_item='instevent, instio, instlock'。

track_real_stats

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

此参数用于跟踪实时 TPS 和 QPS 统计，比如：`sys_stat_metric`，默认关闭。

备注:

实时 TPS 和 QPS 统计功能也依赖 `track_counts` 开关提供底层数据。

2.10.2 统计监控

`log_statement_stats`、`log_parser_stats` 、`log_planner_stats` 、`log_executor_stats`

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

这些参数是 KingbaseES 用于控制日志记录行为的配置选项，将不同模块的统计信息输出到日志中，主要用于调试和性能分析。这是一个功能较为基础的分析工具。类似于 Unix 的 `getrusage()` 系统功能。

- `log_statement_stats` 用于在每个 SQL 语句执行结束后，记录该语句的总体统计信息。
- `log_parser_stats` 用于记录 SQL 语句在解析阶段的统计信息
- `log_planner_stats` 用于记录 SQL 语句在查询计划生成阶段的统计信息。
- `log_executor_stats` 用于记录 SQL 语句在执行阶段的统计信息

以上所有参数都是默认禁用，只有超级用户可以更改这个设置。`log_statement_stats` 不能和其他任何针对每个模块统计的配置选项一起启用。

2.11 自动清理

这些设置控制 autovacuum 特性的行为。注意很多这些设置可以被针对每个表的设置所覆盖，见存储参数。

`autovacuum`

属性	描述
类型	boolean
默认值	on

-续表-

属性	描述
级别	SIGHUP

控制服务器是否运行自动清理启动器后台进程。默认为开启，不过要自动清理正常工作还需要启用 `track_counts` 。该参数只能在 `kingbase.conf` 文件或服务器命令行中设置，但是，通过更改表存储参数可以为表禁用自动清理。

注意即使该参数被禁用，系统也会在需要防止事务 ID 回卷时发起清理进程。

`log_autovacuum_min_duration`

属性	描述
类型	integer
默认值	-1
级别	SIGHUP

如果自动清理运行至少该值所指定的毫秒数，被自动清理执行的每一个动作都会被日志记录。将该参数设置为 0 会记录所有的自动清理动作。`-1`（默认值）将禁用对自动清理动作的记录。

例如，如果将它设置为 `250ms`，则所有运行 250ms 或更长时间的自动清理和分析将被记录。此外，当该参数被设置为除`-1` 外的任何值时，如果一个自动清理动作由于一个锁冲突或者被并发删除的关系而被跳过，将会为此记录一个消息。开启这个参数对于追踪自动清理活动非常有用。这个参数只能在 `kingbase.conf` 文件或者服务器命令行中设置。但是可以通过更改表的存储参数为个别表覆盖这个设置。

`autovacuum_max_workers`

属性	描述
类型	integer
默认值	3
级别	SIGHUP

指定能同时运行的自动清理进程（除了自动清理启动器之外）的最大数量。默认值为 3。该参数只能在服务器启动时设置。

autovacuum_naptime

属性	描述
类型	integer
默认值	1min
级别	SIGHUP

指定自动清理在任意给定数据库上运行的最小延迟。在每一轮中后台进程检查数据库并根据需要为数据库中的表发出 VACUUM 和 ANALYZE 命令。延迟以秒计，且默认值为 1 分钟（1min）。该参数只能在 kingbase.conf 文件或在服务器命令行上设置。

autovacuum_vacuum_threshold

属性	描述
类型	integer
默认值	50
级别	SIGHUP

指定能在一个表上触发 VACUUM 的被插入、被更新或被删除元组的最小数量。默认值为 50 个元组。该参数只能在 kingbase.conf 文件或在服务器命令中设置。但是对个别表可以通过修改表存储参数来覆盖该设置。

autovacuum_analyze_threshold

属性	描述
类型	integer
默认值	50
级别	SIGHUP

指定能在一个表上触发 ANALYZE 的被插入、被更新或被删除元组的最小数量。默认值为 50 个元组。该参数只能在 kingbase.conf 文件或在服务器命令中设置。但是对个别表可以通过修改表存储参数来覆盖该设置。

autovacuum_vacuum_scale_factor

属性	描述
类型	real
默认值	0.2
级别	SIGHUP

指定一个表尺寸的分数，在决定是否触发 VACUUM 时将它加到 autovacuum_vacuum_threshold 上。默认值为 0.2（表尺寸的 20%）。该参数只能在 kingbase.conf 文件或在服务器命令中设置。但是对个别表可以通过修改表存储参数来覆盖该设置。

autovacuum_analyze_scale_factor

属性	描述
类型	real
默认值	0.2
级别	SIGHUP

指定一个表尺寸的分数，在决定是否触发 ANALYZE 时将它加到 autovacuum_analyze_threshold 上。默认值为 0.2（表尺寸的 20%）。该参数只能在 kingbase.conf 文件或在服务器命令中设置。但是对个别表可以通过修改表存储参数来覆盖该设置。

autovacuum_freeze_max_age

属性	描述
类型	integer
默认值	200000000
级别	KINGBASE

指定在一个 VACUUM 操作被强制执行来防止表中事务 ID 回卷之前，一个表的 sys_class.relfrozenxid 域能保持的最大年龄（事务的）。注意即便自动清理被禁用，系统也将发起自动清理进程来阻止回卷。清理也允许从 sys_xact 子目录中移除旧文件，默认值为 2 亿事务。该参数只能在服务器启动时设置，但是对于个别表可以通过修改表存储参数来降低该设置。

autovacuum_multixact_freeze_max_age

属性	描述
类型	integer
默认值	400000000
级别	KINGBASE

指定在一个 VACUUM 操作被强制执行来防止表中多事务 ID 回卷之前，一个表的 sys_class.relminmxid 域能保持的最大年龄（多事务的）。注意即便自动清理被禁用，系统也将发起自动清理进程来阻止回卷。

清理多事务也允许从 sys_multixact/members 和 sys_multixact/offsets 子目录中移除旧文件，默认值为 4 亿事务。该参数只能在服务器启动时设置，但是对于个别表可以通过修改表存储参数来降低该设置。

autovacuum_vacuum_cost_delay

属性	描述
类型	real
默认值	2ms
级别	SIGHUP

指定用于自动 VACUUM 操作中的代价延迟值。如果值为-1，则使用 vacuum_cost_delay 值。默认值为 2 毫秒。该参数只能在 kingbase.conf 文件或在服务器命令中设置。但是对个别表可以通过修改表存储参数来覆盖该设置。

autovacuum_vacuum_cost_limit

属性	描述
类型	integer
默认值	-1
级别	SIGHUP

指定用于自动 VACUUM 操作中的代价限制值。如果指定-1（默认值），则使用 vacuum_cost_limit 值。注意该值被按比例地分配到运行中的自动清理工作者上（如果有多个），因此每一个工作者的限制值之和不会超过这个变量

中的值。该参数只能在 kingbase.conf 文件或在服务器命令中设置。但是对个别表可以通过修改表存储参数来覆盖该设置。

autovacuum_vacuum_by_snapshotcsn

属性	描述
类型	integer
默认值	-1
级别	USER

用于 VACUUM AUTOVACUUM 在执行清理操作时，可以越过长事务阻碍，正常清理垃圾数据。该参数为 vacuum by snapshotcsn 功能触发阈值，当系统中两个活跃事务之间累积的已提交事务数达到设定值时，会触发 vacuum by snapshotcsn 功能，越过活跃事务进行分段清理。该参数的默认值为-1，表示参数无效；用户可设置的范围为 0 到 2147483647，设置为 0 将一直触发功能。参数为表级参数，只对普通用户表生效。

为避免频繁触发功能，在正常业务期间允许系统按照原有清理方案运行，可以根据实际业务情况按以下两种方案设置 autovacuum_vacuum_by_snapshotcsn 参数值：

- 根据具有长事务业务系统日常最大事务并发量设置，建议将参数值设定为最大并发量 10-100 倍。如最大事务并发量为 1000，则设置为 10000 至 100000。
- 根据受长事务影响的业务表原有 autovacuum 参数设置，原有 autovacuum 参数包括：autovacuum_vacuum_threshold 和 autovacuum_vacuum_scale_factor 。Autovacuum 触发条件为：pg_stat_all_tables.n_dead_tup > autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor × pg_class.reltuples 。

  因此参数值建议根据业务表大小设置为 (autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor × pg_class.reltuples) 的 1-10 倍。

备注:

本功能会影响到闪回查询的实现，两者需结合业务需求，不能在同一张表同时设置。

2.12 客户端连接默认

2.12.1 语句动作

client_min_messages

属性	描述
类型	enum
默认值	NOTICE
级别	USER

该参数用于控制被发送给客户端的消息级别 。有效值是 DEBUG5、DEBUG4、DEBUG3、DEBUG2、DEBUG1、LOG、NOTICE、WARNING、ERROR。每个级别都包括其后的所有级别。级别越靠后，被发送的消息越少。默认值是 NOTICE。

注意 LOG 在这里有与 log_min_messages 中不同的排名。

INFO 级别的消息总是被发送到客户端。

search_path

属性	描述
类型	string
默认值	”$user”, public
级别	USER

这个变量指定当一个对象（表、数据类型、函数等）被用一个无模式限定的简单名称引用时，用于进行搜索该对象的模式顺序。当在不同模式中有同名对象时，将使用第一个在搜索路径中被找到的对象。一个不属于搜索路径中任何一个模式的对象只能通过用限定名（带点号）指定包含它的模式来引用。search_path 的值必需是一个逗号分隔的模式名列表。任何不是一个已有模式的名称，或者是一个用户不具有 USAGE 权限的模式，将被安静地忽略。

如果列表项之一是特殊名 $user，则具有 SESSION_USER 返回的名字的模式将取代它（如果有这样一个模式并且该用户有该模式的 USAGE 权限；如果没有，$user 会被忽略）。

同样，当前会话的临时表模式 sys_temp_nnn 也总是被搜索（如果存在）。它可以在路径中通过使用别名 sys_temp 显式列出。如果在路径中没有列出，那么会首先对它进行搜索（甚至是在 sys_catalog 之前）。然而，临时模式只被用来搜索关系（表、视图、序列等）和数据类型名。它从不用于搜索函数或操作符名。

当对象创建时没有指定一个特定目标模式，它们将被放置在 search_path 中第一个合法模式中。如果搜索路径为空将报告一个错误。

这个参数的缺省值是"$user", public。这种设置支持数据库（其中没有用户拥有私有模式，并且所有人共享使用 public）、每个用户私有模式及其组合的共享使用。其他效果可以通过全局或者针对每个用户修改默认搜索路径设置获得。

更多有关模式处理的信息，请参考数据库对象管理中的模式对象类型。特别地，只有当数据库只有一个用户或者

有少数的相互信任的用户时，默认配置是合适的。

搜索路径的当前有效值可以通过 SQL 函数 `current_schemas` 检查。它和检查 `search_path` 的值不太一样，因为 `current_schemas` 展示了 `search_path` 中的各项 schema 是如何被解析的。

`row_security`

属性	描述
类型	boolean
默认值	on
级别	USER

这个变量控制是否以抛出一个错误来代替应用一条行安全性策略。在设置为 `on` 时，策略正常应用。在设置为 `off` 时，原本会应用至少一个策略的查询将会失败。默认为 `on`。受限的行可见性会导致不正确的结果时，可将其改成 `off`。例如，sys_dump 默认会做这种更改。这个变量对能绕过每一条行安全性策略的角色（即超级用户和具有 `BYPASSRLS`）属性的角色没有效果。

更多关于行安全性策略的信息请见 CREATE POLICY(Oracle) 、CREATE POLICY(SQLServer)。

`default_table_access_method`

属性	描述
类型	string
默认值	heap
级别	USER

如果 `CREATE` 命令没有显式指定访问方法，则此参数指定在创建表或物化视图时使用的默认表访问方法，或者当 `SELECT ... INTO` 被使用，它不允许指定表访问方法。默认值是 `heap`。

`default_tablespace`

属性	描述
类型	string
默认值	' '
级别	USER

这个变量指定当一个 CREATE 命令没有显式指定一个表空间时，创建表的默认表空间。它还决定了一个分区关系将把未来的分区指向的表空间。

该值要么是一个表空间的名字，要么是一个指定使用当前数据库默认表空间的空字符串。如果该值和任何现有表空间的名字都不匹配，KingbaseES 将自动使用当前数据库的默认表空间。如果指定了一个非默认的表空间，用户必须对它有 CREATE 权限，否则创建企图将失败。

这个变量不被用于临时表，对临时表会使用 temp_tablespaces 。

当创建数据库时也会不使用这个变量。默认情况下，一个新数据库会从它的模板数据库继承其表空间设置。

default_index_tablespace

属性	描述
类型	string
默认值	，，
级别	USER

这个变量指定当一个 CREATE INDEX 命令没有显式指定一个表空间时，创建索引的默认表空间。

该值要么是一个表空间的名字，要么是一个指定使用当前数据库默认表空间的空字符串。如果该值和任何现有表空间的名字都不匹配，KingbaseES 将自动使用当前数据库的默认表空间。如果指定了一个非默认的表空间，用户必须对它有 CREATE 权限，否则创建企图将失败。

如果 default_index_tablespace 没有被设置，并且 CREATE INDEX 也没有显式的指定一个 TABLESPACE，那么 INDEX 的将会建立在 default_tablespace 所指定的表空间中。

temp_tablespaces

属性	描述
类型	string
默认值	，，
级别	USER

这个变量指定当一个 CREATE 命令没有显式指定一个表空间时，创建临时对象（临时表和临时表上的索引）的默认表空间。用于排序大型数据集的临时文件也被创建在这些表空间中。

该值是一个表空间名字的列表。当列表中有多于一个名称时，每次一个临时对象被创建时 KingbaseES 随机选择

列表中的一个成员。例外是在一个事务中，连续创建的临时对象被放置在里表中的连续表空间中。如果列表被选中元素是一个空字符串，KingbaseES 将自动使用当前数据库的默认表空间。

当 `temp_tablespaces` 被交互式地设置时，指定一个不存在的表空间是一种错误，类似于为用户指定一个不具有 `CREATE` 权限的表空间。不过，当使用一个之前设置的值时，不存在的表空间会被忽略，就像用户缺少 `CREATE` 权限的表空间一样。特殊地，使用一个在 `kingbase.conf` 中设置的值时，这条规则起效。

默认值是一个空字符串，它使得所有临时对象被创建在当前数据库的默认表空间中。

参阅 `default_tablespace` 。

`check_function_bodies`

属性	描述
类型	boolean
默认值	on
级别	USER

这个参数通常为打开。当设置为 `off` 时，它禁用 CREATE FUNCTION(Oracle)、CREATE FUNCTION(MySQL)、CREATE FUNCTION(SQLServer) 期间对函数体字符串的验证。禁用验证避免了验证处理的副作用并且避免了如向前引用导致的伪肯定。在代表其他用户载入函数之前设置这个参数为 `off`；sys_dump 会自动这样做。

`default_transaction_isolation`

属性	描述
类型	enum
默认值	read committed
级别	USER

每个 SQL 事务都有一个隔离级别，可以是“读未提交”、“读已提交”、“可重复读”或者“可序列化”。这个参数控制每个新事务的默认隔离级别。默认是“读已提交”。

更多信息请参考 SET TRANSACTION(Oracle)、SET TRANSACTION(MySQL)、SET TRANSACTION(SQLServer) 。

`default_transaction_read_only`

属性	描述
类型	boolean
默认值	off
级别	USER

一个只读的 SQL 事务不能修改非临时表。这个参数控制每个新事务的默认只读状态。默认是 `off`（读/写）。

更多信息请参考 SET TRANSACTION(Oracle)、SET TRANSACTION(MySQL)、SET TRANSACTION(SQLServer) 。

`default_transaction_deferrable`

属性	描述
类型	boolean
默认值	off
级别	USER

当运行在可序列化隔离级别时，一个可延迟只读 SQL 事务可以在它被允许继续之前延迟一段时间。但是，一旦它开始执行就不会产生任何用来保证可序列化性的负荷；因此序列化代码将没有任何理由因为并发更新而强制它中断，使得这个选项适合于长时间运行的只读事务。

这个参数控制每个新事务的默认可延迟状态。目前它对读写事务或者那些操作在低于可序列化隔离级别上的事务无效。默认值是 `off`。

更多信息请参考 SET TRANSACTION(Oracle)、SET TRANSACTION(MySQL)、SET TRANSACTION(SQLServer) 。

`session_replication_role`

属性	描述
类型	enum
默认值	origin
级别	USER

为当前会话控制复制相关的触发器和规则的触发。需要超级用户权限才能设置这个变量，并且会导致丢弃任何之前缓存下来的查询计划。可能的值有 origin（默认）、replica 和 local。

这个设置的预期用途是用于逻辑复制系统在其应用复制变更时将其设为 replica。更多信息请参考 ALTER_TABLE(Oracle)、ALTER_TABLE(SQLServer)、ALTER_TABLE(MySQL) 的子句 ENABLE TRIGGER 以及 ENABLE RULE。

KingbaseES 在内部会把设置 origin 和 local 同样对待。第三方复制系统可能会把这两个值用于其内部目的，例如把 local 用来标出一个不应复制其更改的会话。

因为外键被实现为触发器，将这个参数设置为 replica 还会禁用所有的外键检查，如果使用不当可能会让数据处于一种不一致的状态。

statement_timeout

属性	描述
类型	integer
默认值	0
级别	USER

中止任何使用了超过指定毫秒数的语句，从命令到达服务器开始计时。如果 log_min_error_statement 被设置为 ERROR 或更低，语句如果超时也会被记录。一个零值（默认）将关闭这个参数。

不推荐在 kingbase.conf 中设置 statement_timeout，因为它会影响所有会话。

lock_timeout

属性	描述
类型	integer
默认值	0
级别	USER

如果任何语句在试图获取表、索引、行或其他数据库对象上的锁时等待超过指定的毫秒数，该语句将被中止。该时间限制独立地应用于每一次锁获取尝试。该限制会应用到显式锁定请求（如 LOCK TABLE 或不带 NOWAIT 的 SELECT FOR UPDATE）和隐式获得的锁。如果 log_min_error_statement 被设置为 ERROR 或更低，超时的语句会被记录。一个零值（默认）将关闭这个参数。

与 statement_timeout 不同，这个超时只在等待锁时发生。注意，如果 statement_timeout 为非零，设置 lock_timeout 与之相同或更大的值没有意义，因为事务超时总是第一个被触发。

不推荐在 kingbase.conf 中设置 lock_timeout，因为它会影响所有会话。

idle_in_transaction_session_timeout

属性	描述
类型	integer
默认值	0
级别	USER

终止任何已经闲置超过这个参数所指定的时间（以毫秒计）的打开事务的会话。这使得该会话所持有的任何锁被释放，并且其所持有的连接槽可以被重用，它也允许只对这个事务可见的元组被清理。

默认值 0 会禁用这个特性。

vacuum_freeze_table_age

属性	描述
类型	integer
默认值	150000000
级别	USER

当表的 sys_class.relfrozenxid 域达到该设置指定的年龄时，VACUUM 会执行一次激进的扫描。激进的扫描与常规 VACUUM 的不同在于它会访问每一个可能包含未冻结 XID 或者 MXID 的页面，而不只是那些可能包含死亡元组的页面。默认值是 1.5 亿个事务。用户可以把这个值设置为从 0 到 20 亿，但 VACUUM 会自动地将有效值设置为 autovacuum_freeze_max_age 值的 95%，确保手动 VACUUM 有机会在反回卷自动清理之前执行。

vacuum_freeze_min_age

属性	描述
类型	integer
默认值	5000000
级别	USER

指定 VACUUM 在扫描表时用来决定是否冻结行版本的切断年龄（以事务计）。默认值是 5 百万个事务。尽管用户

可以将这个值设置为从 0 到 10 亿，但 VACUUM 会自动地将有效值设置为 autovacuum_freeze_max_age 值的一半，这样在强制执行的自动清理之间不会有过短的时间间隔。

vacuum_multixact_freeze_table_age

属性	描述
类型	integer
默认值	150000000
级别	USER

如果表的 sys_class.relminmxid 域超过了这个设置指定的年龄，VACUUM 会执行一次激进的扫描。激进的扫描与常规 VACUUM 的区别在于它会访问每一个可能包含未冻结 XID 或者 MXID 的页面，而不是只扫描那些可能包含死亡元组的页面。默认值是 1.5 亿个组合事务。尽管用户可以把这个值设置为从 0 到 20 亿，VACUUM 会悄悄地将有效值设置为 autovacuum_multixact_freeze_max_age 值的 95%，因此在表上启动一次反回卷自动清理之前有机会进行一次定期手动 VACUUM。

vacuum_multixact_freeze_min_age

属性	描述
类型	integer
默认值	50000000
级别	USER

指定 VACUUM 在扫描表时用来决定是否把组合事务 ID 替换为一个更新的事务 ID 或组合事务 ID 的切断年龄（以组合事务计）。默认值是 5 千万个组合事务。尽管用户可以将这个值设置为从 0 到 10 亿，VACUUM 会悄悄地将有效值设置为 autovacuum_multixact_freeze_max_age 值的一半，这样在强制执行的自动清理之间不会有过短的时间间隔。

vacuum_cleanup_index_scale_factor

属性	描述
类型	real
默认值	0.1
级别	USER

指定在以前的统计信息收集过程中计数到的堆元组总数的一个分数，插入不超过这一数量所代表的元组不会导致 VACUUM 清理阶段的索引扫描。这个设置当前仅适用于 B-树索引。如果没有元组从堆中删除，则当至少满足下列条件之一时，在 VACUUM 清理阶段仍会扫描 B-树索引：索引统计信息过时或者索引中包含在清理时可回收的已删除页。如果新近插入的元组数占上次统计信息收集时检测到的堆元组总数的比例超过 vacuum_cleanup_index_scale_factor，则认为索引信息已经过时。堆元组的总数被存放在索引的元页中。注意，直到 VACUUM 找不到死亡元组之前，元页中都不包括这个数据。因此只有在第二次以及之后的 VACUUM 周期检测不到死亡元组时，清理阶段的 B-树索引扫描才能被跳过。该值的取值范围可以是从 0 到 10000000000。当 vacuum_cleanup_index_scale_factor 被设置为 0 时，在 VACUUM 清理期间不会跳过索引扫描。默认值是 0.1。

vacuum_deleted_not_recycled_index_scale_factor

属性	描述
类型	real
默认值	0.05
级别	USER

当被删除索引页面占索引容量达到 vacuum_deleted_not_recycled_index_scale_factor 值后会触发回收。这个设置当前仅适用于 B-树索引。默认值是 0.05。

bytea_output

属性	描述
类型	enum
默认值	hex
级别	USER

设置 bytea 类型值的输出格式。有效值是 hex（默认）和 escape（传统的 KingbaseES 格式）。不管这个设置的值如何，bytea 类型总是接受这两种格式的输入。

xmlbinary

属性	描述
类型	enum
默认值	base64

-续表-

属性	描述
级别	USER

设置二进制值如何被编码为 XML。例如，这适用于通过 `xmlelement` 函数或 `xmlforest` 函数将 `bytea` 值转换到 XML 值。可能的值有 `base64` 和 `hex`，它们都是用 XML 模式标准定义的。默认值是 `base64`。更多关于 XML 相关函数的信息可参阅 XML 函数。

这里的实际选择都是根据爱好做出的，只受客户端应用中可能存在的限制的约束。两种方法都支持所有可能的值，尽管十六进制编码将比 base64 编码更大。

`xmloption`

属性	描述
类型	enum
默认值	CONTENT
级别	USER

当在 XML 和字符串值之间进行转换时，无论设置 `DOCUMENT` 或 `CONTENT` 都是隐式的。有效值是 `DOCUMENT` 和 `CONTENT`。缺省值是 `CONTENT`。详见 XML 类型。

根据 SQL 标准，设置这个选项的命令是：

```
SET XML OPTION { DOCUMENT | CONTENT };
```

这种语法在 KingbaseES 也可用。

`gin_pending_list_limit`

属性	描述
类型	integer
默认值	4MB
级别	USER

设置 fastupdate 被启用时可以使用的 GIN 待处理列表的最大尺寸。如果该列表增长到超过这个最大尺寸，会通过批量将其中的项移入主 GIN 数据结构来清理列表。默认值是四兆字节（4MB）。可以通过更改索引的存储参数来为个别 GIN 索引覆盖这个设置。

2.12.2　本地化和格式化

DateStyle

属性	描述
类型	string
默认值	ISO, YMD
级别	USER

设置日期和时间值的显示格式，以及解释有歧义的日期输入值的规则。由于历史原因，这个变量包含两个独立的部分：输出格式声明（ISO、kingbase、SQL 或 German）、输入/输出的年/月/日顺序（DMY、MDY 或 YMD）。这些可以被独立设置或者一起设置。关键字 Euro 和 European 是 DMY 的同义词；关键字 US、NonEuro 和 NonEuropean 是 MDY 的同义词。详见 SQL 基本元素 中的日期/时间类型。内建默认值是 ISO，MDY，但是 initdb 将用对应于选中的 lc_time 区域行为的设置初始化配置文件。

IntervalStyle

属性	描述
类型	enum
默认值	kingbase
级别	USER

设置间隔值的显示格式。值 sql_standard 将产生匹配 SQL 标准间隔文本的输出。当 DateStyle 参数被设置为 ISO 时，值 kingbase（默认）将产生匹配 KingbaseES 版本之前的输出。当 DateStyle 参数被设置为非 ISO 输出时，值 kingbase_verbose 会产生匹配 KingbaseES 版本之前的输出。值 iso_8601 会产生匹配在 ISO 8601 的 4.4.3.2 节中定义的“带标志符格式”的时间间隔的输出。

IntervalStyle 参数也可以影响对有歧义的间隔输入的解释。

TimeZone

属性	描述
类型	string
默认值	GMT
级别	USER

设置用于显示和解释时间戳的时区。内建默认值是 `GMT`，但是它通常会在 `kingbase.conf` 中被覆盖；initdb 将安装一个对应于其系统环境的设置。

`timezone_abbreviations`

属性	描述
类型	string
默认值	Default
级别	USER

设置服务器接受的日期时间输入中使用的时区缩写集合。默认值为`'Default'`，这个集合在全世界大多数地方都能工作。也还有`'Australia'`和`'India'`，以及可能为一种特定安装定义的其他集合。

`extra_float_digits`

属性	描述
类型	integer
默认值	1
级别	USER

此参数调整用于浮点值文本输出的位数，包括 `float4`、`float8` 和 geometric 数据类型。

如果值为 1(默认值) 或以上，则以最简洁的格式输出浮点值；详见 SQL 基本元素中的数据类型下的浮点类型。实际生成的位数只取决于输出的值，而不取决于此参数的值。`float8` 的值最多需要 17 位，`float4` 的值最多需要 9 位。这种格式既快速又精确，在正确读取时精确地保留了原始的二进制浮点值。对于历史兼容性，最多允许有 3 个值。

如果值为 0 或负数，则输出四舍五入为给定的十进制精度。使用的精度是根据该参数的值减少的类型 (`FLT_DIG`

或 DBL_DIG) 的标准位数。(例如，指定-1 将导致 float4 的值四舍五入为 5 位有效数字，float8 的值四舍五入为 14 位数字。) 这种格式比较慢，并且不会保留二进制浮点值的所有位，但是可能更易于阅读。

备注:

这个参数的设置，会改变 KingbaseES 中对浮点数的输出; 详见 SQL 基本元素中的数据类型下的数值类型。

---

client_encoding

属性	描述
类型	string
默认值	sql_ascii
级别	USER

设置客户端编码（字符集）。默认使用数据库编码。KingbaseES 服务器所支持的字符集在 被支持的字符集中描述。

lc_messages

属性	描述
类型	string
默认值	zh_CN.UTF-8
级别	SUPERUSER

设置消息显示的语言。可接受的值是系统相关的；详见 区域支持。如果这个变量被设置为空字符串，那么该值将以一种系统相关的方式从服务器的执行环境中继承。

在一些系统上，这个区域分类并不存在。仍然可以设置这个变量，只是不会有任何效果。同样，所期望语言的翻译消息也可能不存在。在这种情况下，消息以英文呈现。

只有超级用户可以改变这个设置。因为它同时影响发送到服务器日志和客户端的消息。一个不正确的值可能会降低服务器日志的可读性。

lc_monetary

属性	描述
类型	string
默认值	zh_CN.UTF-8
级别	USER

设置用于格式化货币量的区域，可接受的值是系统相关的。详见 区域支持。如果这个变量被设置为空字符串，那么该值将以一种系统相关的方式从服务器的执行环境中继承。

`lc_numeric`

属性	描述
类型	string
默认值	zh_CN.UTF-8
级别	USER

设置用于格式化数字的区域，可接受的值是系统相关的。详见 区域支持。如果这个变量被设置为空字符串，那么该值将以一种系统相关的方式从服务器的执行环境中继承。

`lc_time`

属性	描述
类型	string
默认值	zh_CN.UTF-8
级别	USER

设置用于格式化日期和时间的区域，可接受的值是系统相关的。详见 区域支持。如果这个变量被设置为空字符串（默认），那么该值将以一种系统相关的方式从服务器的执行环境中继承。

`default_text_search_config`

属性	描述
类型	string
默认值	pg_catalog.simple
级别	USER

选择被那些没有显式参数指定配置的文本搜索函数变体使用的文本搜索配置。内建默认值是 pg_catalog.simple，但是如果能够标识一个匹配区域的配置，initdb 将用对应于选中的 lc_ctype 区域的设置初始化配置文件。

2.12.3　共享库预先加载

为了载入附加的功能或者达到提高性能的目的，可用多个设置来预先载入共享库到服务器中。例如'$libdir/mylib'设置可能会导致 mylib.so（或者某些平台上的 mylib.sl）从安装的标准库目录被预装载。这些设置之间的区别在于生效的时间以及改变它们所需的特权。

可以用这个方法预装载 KingbaseES 的过程语言库，通常是使用'$libdir/plXXX'语法，其中的 XXX 是 kingbase、perl、tcl 或 python。

只有特别为与 KingbaseES 一起使用设计的共享库才能以这种方式载入。每一个 KingbaseES 支持的库都有一个“魔法块”，它会被检查以保证兼容性。由于这个原因，非 KingbaseES 无法以这种方式被载入。可以使用操作系统的工具（如 ld_preload）载入它。

总之，请参考指定模块的说明文档，使用推荐的方法载入。

local_preload_libraries

属性	描述
类型	string
默认值	,,
级别	USER

这个变量指定一个或者多个要在连接开始时预载入的共享库。它包含一个由逗号分隔的库名列表，其中每个名称都会按 LOAD 命令的方式解析。项之间的空格会被忽略，如果需要在库名中包含空格或者逗号，请把库名放在双引号内。这个参数值只在连接开始时生效。后续的更改不会有任何效果。如果一个指定的库没有找到，连接尝试将会失败。

任何用户都能设置这个选项。正因为如此，能被这样载入的库被严格限制为出现于安装的标准库目录中 plugins 子目录下的共享库（保证只有“安全的”库被安装到这里是数据库管理员的责任）。

local_preload_libraries 中的项可以显式指定这个目录，例如 $libdir/plugins/mylib，或者只是指定库的名称 mylib，两者的效果是相同的。

这个特性的目的是允许非特权用户在特定的会话中载入正在调试的或者性能度量库，而无需一个显式的 LOAD 命令。为了这个目的，通常通过使用客户端的 KINGBASE_OPTIONS 环境变量或者 ALTER ROLE SET 来设置这个参数。

不过，除非一个模块被特别设计成由非超级用户以这种方式使用，通常不推荐使用这个设置。应该看看 session_preload_libraries 。

session_preload_libraries

属性	描述
类型	string
默认值	,,
级别	SUPERUSER

这个变量指定一个或者多个要在连接开始时预载入的共享库。它包含一个由逗号分隔的库名列表，其中每个名称都会按 LOAD 命令的方式解析。项之间的空格会被忽略，如果需要在库名中包含空格或者逗号，请把库名放在双引号内。这个参数只在连接开始时起效。后续的改变没有效果。如果指定的库没有找到，连接尝试将会失败。只有超级用户能够更改这个设置。

这个特性的意图是允许在特定会话中载入调试用的或者测量性能的库，而不需要显式的给出一个 LOAD 命令。还有，无需重启服务器就能更改这个参数（但是只有新会话启动时才会生效），这样可以以这种方式更容易地增加新模块，即便它们会应用到所有会话。

和 shared_preload_libraries 不同，相对于在库被第一次使用时载入它，在会话开始时载入库并没有什么性能优势。不过，当使用连接池时这样做还是有一些优势。

shared_preload_libraries

属性	描述
类型	string
默认值	,,
级别	KINGBASE

这个变量指定一个或者多个要在服务器启动时预载入的共享库。它包含一个由逗号分隔的库名列表，其中每个名称都会按 LOAD 命令的方式解析。项之间的空格会被忽略，如果需要在库名中包含空格或者逗号，请把库名放在双引号内。这个参数只能在服务器启动时设置。如果指定的库没有找到，服务器将无法启动。

有些库需要执行只能在 kingbase 启动时发生的特定操作，例如分配共享内存、保留轻量级锁或者启动后台工作者。这些库必须通过这个参数在服务器启动时载入。每个库的详情请见文档。

其他库也能被预载入。通过预载入一个共享库，当该库被第一次使用时就可以避免库的启动时间。不过，启动每个新服务器进程的时间可能会略有增加，即使该进程从不使用该库。因此，推荐只把这个参数用于那些要在大多数会话中使用的库上。还有，改变这个参数要求重启服务器，因此对于短期的调试任务来说这不是好的选择，应该转用 session_preload_libraries 。

备注:

在 Windows 主机上，在服务器启动时预载入一个库并不会减少启动每个新服务器进程所需的时间；每一个服务器进程将会重新载入预载入的库。不过，对于那些要在 kingbase 启动时执行操作的库来说，Windows 主机上的 `shared_preload_libraries` 仍然有用。

`jit_provider`

属性	描述
类型	string
默认值	llvmjit
级别	KINGBASE

这个变量是要被使用的 JIT 提供者库的名称。默认是 `llvmjit`。这个参数只能在服务器启动时设置。

如果这个变量被设置为一个不存在的库，JIT 将不可用，但是也不会发生错误。这种特性允许在主 KingbaseES 包之外单独安装 JIT 支持。

2.12.4　其他默认值

`dynamic_library_path`

属性	描述
类型	string
默认值	$libdir
级别	SUPERUSER

如果需要打开一个可以动态装载的模块并且在 CREATE FUNCTION 或 LOAD 命令中指定的文件名没有目录部分（即名字中不包含斜线），那么系统将搜索这个路径以查找所需的文件。

dynamic_library_path 的值必须是一个冒号分隔（或者在 Windows 上以分号分隔）的绝对目录路径的列表。如果一个列表元素以特殊字符串开始，$libdir 会被替换为 KingbaseES 包中已编译好的库目录。这里是 KingbaseES 发布提供的模块被安装的位置（使用 sys_config --pkglibdir 来找到这个目录的名字）。例如：

```
dynamic_library_path = '/usr/local/lib/kingbase:
/home/my_project/lib:$libdir'
```

或者在 Windows 环境中：

```
dynamic_library_path = 'C:\tools\kingbase;H:\my_project\lib;$libdir'
```

这个参数的默认值是'$libdir'。如果该值被设置为一个空字符串，则关闭自动路径搜索。

这个参数可以在运行时由超级用户修改，但是这样修改的设置只能保持到这个客户端连接的结尾，因此这个方法应该保留给开发目的。建议在 kingbase.conf 配置文件中设置这个参数。

gin_fuzzy_search_limit

属性	描述
类型	integer
默认值	0
级别	USER

GIN 索引返回的集合尺寸的软上限。

2.13　锁管理

deadlock_timeout

属性	描述
类型	integer
默认值	1s
级别	SUPERUSER

这是进行死锁检测之前在一个锁上等待的总时间（以毫秒计）。死锁检测相对昂贵，因此服务器不会在每次等待锁时都运行这个它。我们乐观地假设在生产应用中死锁是不常出现的，并且只在开始检测死锁之前等待一会儿。增加这个值就减少了浪费在无用的死锁检测上的时间，但是减慢了报告真正死锁错误的速度。默认是 1 秒（`1s`），这可能是实际中想要的最小值。在一个高负载的服务器上，可能需要增大它。这个值的理想设置应该超过通常的事务时间，这样就可以减少在锁释放之前就开始死锁检查的机会。只有超级用户可以更改这个设置。

当 log_lock_waits 被设置时，这个参数还可以决定发出关于锁等待的日志之前等待的时长。如果想调查锁延迟，需要设置一个比正常的 `deadlock_timeout` 小的值。

`max_locks_per_transaction`

属性	描述
类型	integer
默认值	64
级别	KINGBASE

共享锁表跟踪在 `max_locks_per_transaction` * (`max_connections` + `max_prepared_transactions`) 个对象（如表）上的锁。因此，在任何一个时刻，只有不超过这么多个可区分对象能够被锁住。这个参数控制为每个事务分配的对象锁的平均数量。个体事务可以锁住更多对象，数量可以和锁表中能容纳的所有事务的锁一样多。这不是能被锁住的行数，那个值是没有限制的。默认值 64 已经被历史证明是足够的，但是如果想在一个事务中使用很多不同表的查询（例如查询一个有很多子表的父表），则需要提高这个值。这个参数只能在服务器启动时设置。

当运行一个后备服务器时，必须设置这个参数为大于等于主服务器上的值。否则，后备服务器上将不允许查询。

`max_pred_locks_per_transaction`

属性	描述
类型	integer
默认值	64
级别	KINGBASE

共享谓词锁表跟踪在 `max_pred_locks_per_transaction` * (`max_connections` + `max_prepared_transactions`) 个对象（如表）上的锁。因此，在任何一个时刻，只有不超过这么多个可区分对象能够被锁住。这个参数控制为每个事务分配的对象锁的平均数量。个体事务可以锁住更多对象，数量可以和锁表中能容纳的所有事务的锁一样多。这不是能被锁住的行数，那个值是没有限制的。默认值 64 已经在测试中被证明通常是足够的，但是如果想要在一个可序列化事务中使用很多不同表的查询（例如查询一个有很多子表的父表），则需要提高这个值。这个参数只能在服务器启动时设置。

`max_pred_locks_per_relation`

属性	描述
类型	integer
默认值	-2
级别	SIGHUP

这个参数控制在谓词锁被提升为覆盖整个关系之前，该谓词锁能够在单个关系上锁住多少页面或元组。大于等于零的值表示一种绝对限制，而负值表示用 max_pred_locks_per_transaction 除以这个设置的绝对值。默认值为-2，它将保持以往版本的 KingbaseES 中的行为。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。

`max_pred_locks_per_page`

属性	描述
类型	integer
默认值	2
级别	SIGHUP

这个参数控制在谓词锁被提升为覆盖整个页面之前，该谓词锁能在单一页面上锁住多少行。默认值是 2。这个参数只能在 `kingbase.conf` 文件中或者服务器命令行上设置。

2.14　版本和平台兼容性

2.14.1　KingbaseES 老版本

`array_nulls`

属性	描述
类型	boolean
默认值	on
级别	USER

这个参数控制数组输入解析器是否把未用引号的 NULL 识别为一个空数组元素。默认为 on，允许输入包含空值的数组值。但是 KingbaseES 之前的版本不支持数组中的空值，并且把 NULL 当作指定一个值为字符串“NULL”的正常数组元素。对于那些要求旧行为的应用的向后兼容性，这个变量可以被设置为 off。

注意即使这个变量为 off 也能够创建包含空值的数组值。

backslash_quote

属性	描述
类型	enum
默认值	safe_encoding
级别	USER

这个参数控制字符串文本中的单引号是否能够用\'来表示。首选的 SQL 标准的方法是将其双写（''），但是 KingbaseES 在历史上也接受\'。不过使用\'容易导致安全风险，因为在某些客户端字符集编码中，有多字节字符的最后一个字节在数值上等价于 ASCII 的\'。如果客户端代码没有做到正确转义，那么将会导致 SQL 注入攻击。如果服务器拒绝看起来带有被反斜线转义的单引号的查询，那么就可以避免这种风险。backslash_quote 的可用值是 on（总是允许\'）、off（总是拒绝）以及 safe_encoding（只有客户端编码不允许在多字节字符中存在 ASCII \时允许）。safe_encoding 是默认设置。

注意在符合标准的字符串文本中，\就表示\。这个参数只影响不符合标准的文本的处理，包括转义字符串语法（E'...'）。

escape_string_warning

属性	描述
类型	boolean
默认值	on
级别	USER

打开时，如果在普通字符串文本中（'...'语法）出现了一个反斜线（\）并且 standard_conforming_strings 为关闭，那么就会发出一个警告。默认值是 on。

希望使用反斜线作为转义符的应用应该被修改来使用转义字符串语法（E'...'），因为在 SQL 标准中普通字符串的默认行为是将反斜线视作一个普通字符。这个变量可以被启用来帮助定位需要被更改的代码。

locompatprivileges

属性	描述
类型	boolean
默认值	off
级别	USER

在 KingbaseES V8R2 之前，大对象不具有访问特权并且因此总是所有用户可读可写的。为了和以前的版本兼容，把这个变量设置为 `on` 可以禁用这种新的特权检查。默认是 `off`。只有超级用户可以更改这个设置。

设置这个参数不会禁用所有与大对象相关的安全检查 --除了那些在 KingbaseES V8R2 中已经修改了的默认行为。

`operator_precedence_warning`

属性	描述
类型	boolean
默认值	off
级别	USER

当开启时，对于任何从 KingbaseES V8R2 以来由于操作符优先级变化而导致含义改变的结构，解析器将发出一个警告。这有助于审计应用，已检查是否优先级变化破坏了什么东西。但是它的本意并不是希望在生产环境中保持打开，因为它会对某些完全合法、兼容标准的 SQL 代码发出警告。默认为 `off`。

更多操作符优先级信息请见 SQL 语言 中的操作符部分。

`quote_all_identifiers`

属性	描述
类型	boolean
默认值	off
级别	USER

当数据库产生 SQL 时，强制所有标识符被引号包围，即使它们（当前）不是关键字。这将影响 `EXPLAIN` 的输出以及 `sys_get_viewdef` 等函数的结果。另请参阅 sys_dump 和 sys_dumpall 的`--quote-all-identifiers` 选项。

`standard_conforming_strings`

属性	描述
类型	boolean
默认值	on
级别	USER

控制普通字符串文本（`'...'`）是否按照 SQL 标准把反斜线当普通文本。从 KingbaseES V8R2 开始，默认值为 `on`（之前的发行中默认值为 `off`）。应用可以检查这个参数来判断字符串文本如何被处理。这个参数的存在也可以被当做转义字符串语法（`E'...'`）被支持的标志。如果一个应用希望反斜线被当做转义字符，应该使用转义字符串语法（带有 C 型转义的字符串常量）。

`synchronize_seqscans`

属性	描述
类型	boolean
默认值	on
级别	USER

它允许对大型表的顺序扫描与其他扫描同步，因此并发扫描可以在几乎相同的时刻读取相同的块，这样可以分担 I/O 负载。当启用这个参数时，一个扫描可能会从表的中间开始并且之后“绕回”到开头以覆盖所有的行，这样可以与已在进行中的扫描活动同步。对于没有 `ORDER BY` 子句的查询来，这样的扫描会在返回行的顺序中造成不可预料的改变。将这个参数设置为 `off` 以保证 KingbaseES V7 之前的行为（顺序扫描总是从表的起始处开始）。默认值是 `on`。

`enable_inplace_update`

属性	描述
类型	boolean
默认值	off
级别	USER

控制本地更新表能否进行 inplace update，只有开启该参数且通过接口使用 PBE 方式才能进行本地更新，否则仍然是通过正常的 MVCC 方式更新。本地更新指在 update 操作数据时直接在元组上覆盖老数据，而不是插入新的元组，标记删除老元组。

`inplace_update_lock_type`

属性	描述
类型	integer
默认值	off
级别	KINGBASE

控制本地更新表在进行 inplace update 时开表锁的级别，该参数类型为 int 型，默认值为 7，取值范围为 3~7。

`comp_v7_program_para_def`

属性	描述
类型	bool
默认值	false
级别	USER

该参数指定了 `plsql` 中是否启用兼容 KingbaseES V7 版本支持 OUT 参数指定默认值功能。当指定为 true 时，表示支持 OUT 参数指定默认值；当指定为 false 时，表示不支持 OUT 参数指定默认值。

2.14.2　平台和客户端兼容性

`transform_null_equals`

属性	描述
类型	boolean
默认值	off
级别	USER

当打开时，形为 expr = NULL（或 NULL = expr）的表达式将被当做 expr IS NULL，也就是说，如果 expr 得出空值值则返回真，否则返回假。正确的 SQL 标准兼容的 expr = NULL 行为总是返回空（未知）。因此这个参数默认为 off。

不过，在 Microsoft Access 里的过滤表单生成的查询使用 expr = NULL 来测试空值，因此，如果希望使用这个接口访问数据库，此选项需打开。因为 expr = NULL 形式的表达式总是返回空值（使用 SQL 标准解释）。它们不是非常有用并且在普通应用中也不常见，在应用中也不常见，因此这个选项实际上没有什么危害。但是新用户常常对涉及空值的表达式的语义上感到困惑，因此这个选项默认为关闭。

请注意这个选项只影响 = NULL 形式，而不影响其它比较操作符或者其它与一些涉及等值操作符的表达式在计算上等效的其他表达式（例如 IN）。因此，这个选项不是劣质程序的一般修复。

相关信息请见 应用开发中的操作符部分。

2.14.3　一般特性兼容性

auto_increment_offset

属性	描述
类型	int
默认值	1
级别	USER

兼容 MySQL 自增列 GUC 参数，决定自增列值的起点。取值范围是 [1,65535]，当设置值小于 1，或大于 65535 将报错。

auto_increment_increment

属性	描述
类型	int
默认值	1
级别	USER

兼容 MySQL 自增列 GUC 参数，控制连续列值的间隔。取值范围是 [1,65535]，当设置值小于 1，或大于 65535 将报错。

1、当 auto_increment_offset 大于 auto_increment_increment 时，auto_increment_offset 值将被忽略，

auto_increment_offset 被视为 1。

2、auto_increment_offset 和 auto_increment_increment 影响下一序列值的计算，算法如下：自增列序列值的计算不考虑已存在的任何列值。下一序列值的计算公式如下：auto_increment_offset + N × auto_increment_increment 其中 N 是 [0, 1, 2, 3, …] 序列中的一个整数。被插入的下一个值将是大于自增列现存最大值（或者初始值）的最小序列值。

3、更新 auto_increment_offset 和 auto_increment_increment，对下一序列值或初始值没有影响，但会影响随后插入的自增列值（参见上面规则），而之后再次生成的下一序列值则根据系统变量计算得出。

`default_with_rowid`

属性	描述
类型	bool
默认值	false
级别	USER

开启后为系统上所有再创建的表增加一个 rowid 名，类型为 rowid type 的隐含列。且所有插入的数据，都会将当前行的 rowid 写入到该列之中。

该参数和 default_with_oids 的关系，两者面向不同的应用，同时当一方设置为 true 时，通过 DDL 非批量创建表 (即通过 create xxx with rowid/oids) 增加另一方参数的 id 隐含列会失败。（rowid 的参数优先级高于 oids）。

`disable_user_switching`

属性	描述
类型	bool
默认值	false
级别	SUPERUSER

打开参数后，调用 SET [SESSION | LOCAL] SESSION AUTHORIZATION 命令将报错“permission denied to set session authorization”。此参数防止切换用户时不用输入密码的行为。

`enable_autocommit_fetch`

属性	描述
类型	boolean

-续表-

属性	描述
默认值	off
级别	BACKEND

当客户端设置了 autocommit=true（自动提交模式）时，对于查询请求，在 enable_autocommit_fetch 设置为 `off`，服务端将一次性返回给客户端符合条件的所有记录。在设置为 `on` 时，服务端将按照客户端指定 fetchsize 值（JDBC 客户端应用调用 setFetchSize 方法设置）的数据量按需执行查询，并按需获取数据集返回给客户端符合条件的记录。默认值是 `off`。

`enable_autocommit_crossquery`

属性	描述
类型	boolean
默认值	off
级别	BACKEND

在 enable_autocommit_fetch 设置为 `on` 的情况下有效。在 enable_autocommit_crossquery 设置为 `on` 时，客户端语句交叉执行，在不同语句执行后，前语句按照服务端调用 HoldablePortal 方式，可以继续执行。在设置为 `off` 时，客户端语句交叉执行，在不同语句执行后，前语句不能继续执行。默认值是 `off`。

`escape`

属性	描述
类型	string
默认值	null
级别	USER

当打开时，转义符可设置为多种字符，可兼容 oracle 的特性。支持设置为 off、on、其它单字符，当设置后将与 oracle 表现基本一致（除部分容错性）。默认为 `null`，表示和老版本兼容。

例如：

```
set escape =off;
SET
select '\\' as test;
test
---------
   \\
(1 row)

set escape =on;
SET
select '\\' as test;
test
---------
   \
(1 row)
```

enable_comment_unpaired

属性	描述
类型	bool
默认值	off
级别	USER

通过该 GUC 参数来控制服务器和 ksql 客户端解析多行注释时，是按照嵌套注释解析还是非嵌套注释解析。

enable_comment_unpaired 为 on 时，支持非嵌套多行注释；enable_comment_unpaired 为 off 时，支持嵌套多行注释。

ignore_char_null_check

属性	描述
类型	boolean
默认值	off
级别	USER

当打开时，Copy 支持导出数据中包含 0x00 字节记录，导出以后 0x00 字节以二进制方式保存；也支持导入包含 0x00 字节的输入端数据，同样也要确保输入端是 0x00 字节而不是”0x00” 字符串。默认为 off。

ignore_char_converer

属性	描述
类型	boolean
默认值	off
级别	USER

当打开时，Copy 在导入文本格式的数据时，将”\0” 当做 0x00 字节，而不转换为八进制数据。默认为 off。

ignore_zero_number

属性	描述
类型	boolean
默认值	off
级别	USER

当打开时，number 类型输出时末尾忽略“0”。默认为 off。

例如：

```
select 1.00::number;
numeric
---------
  1.00
(1 row)

set ignore_zero_number =true;
SET
select 1.00::number;
numeric
---------
   1
(1 row)
```

mysql_substring_compatible

属性	描述
类型	bool
默认值	false
级别	USER

打开参数后，兼容 MySQL 的 substring、substr 函数、convert 函数。默认是 false。可参见函数 SUBSTR(MySQL)、SUBSTRING(MySQL) 、CONVERT(MySQL)。

`mysql_serial_type`

属性	描述
类型	bool
默认值	on
级别	USER

用于兼容 MySQL 的 SERIAL 类型。当 mysql_serial_type=off 时，serial 等同于 PG 模式的 serial 类型，兼容 PG 生态；当 mysql_serial_type=on 时，serial 等同于 BIGINT NOT NULL AUTO_INCREMENT UNIQUE 的别名，兼容 MySQL 生态。支持在 CREATE TABLE 和 ALTER TABLE 语句中将 serial 用作定义列类型的别名。

注意：使用 KES 的 serial 类型别名创建的自增列是 bigint 类型，而不是 bigint unsigned 类型。

`ora_forbid_func_polymorphism`

属性	描述
类型	bool
默认值	off
级别	USER

此参数设置为 on，则表示开启禁用多态, 同模式下 (不包括 sys, sys_catalog 系统内置模式) 不能创建同名的函数或存储过程；设置为 off（默认）, 则表示启用多态。

`ora_statement_level_rollback`

属性	描述
类型	bool
默认值	off
级别	USER

该参数指定了 `plsql` 中是否启用语句级回滚，取值为 on 或 off。当指定为 true 时，启动语句级回滚功能；当指定为 false 时，关闭语句级回滚功能。

`ora_func_style`

属性	描述
类型	bool
默认值	off
级别	USER

该参数指定了 `plsql` 中是否启用 PLSQL 对象状态的控制功能，取值为 on 或 off。当指定为 true 时，启动对象状态控制功能；当指定为 false 时，关闭对象状态控制功能。

`ora_style_nls_date_format`

属性	描述
类型	boolean
默认值	off
级别	USER

该变量设置为 `on` 时，日期时间的隐式输入与输出字符串格式才会使用参数 `nls_date_format`、`nls_timestamp_format` 和 `nls_timestamp_tz_format` 的配置。

`strcmp_rtrim`

属性	描述
类型	boolean
默认值	false
级别	USER

控制字符串比较是否去除右侧空格。当该变量设置为 `true` 时，字符串比较会去除右侧空格。

2.15　错误处理

`exit_on_error`

属性	描述
类型	boolean
默认值	off
级别	USER

如果为 on，任何错误将中止当前会话。默认情况下，这个值被设置为 off，这样只有 FATAL 错误（致命）将中止会话。

`ora_input_emptystr_isnull`

属性	描述
类型	bool
默认值	on
级别	USER

输入空字符串时的处理措施。默认值为 on，即将输入的空字符串作为 null 值处理。off 表示不处理。

`restart_after_crash`

属性	描述
类型	boolean
默认值	on
级别	SIGHUP

当被设置为 on（默认值）时，KingbaseES 将在一次后端崩溃后自动重新初始化。让这个值设置为 on 通常是将数据库可用性最大化的最佳方法。但是在某些环境中，例如 KingbaseES 被集群软件调用时，禁用重启可能很有用，这样集群软件可以得到控制并且采取它认为适当的行动。

`data_sync_retry`

属性	描述
类型	boolean
默认值	off
级别	KINGBASE

当设置为 off 时，这是默认值，KingbaseES 如果无法将修改后的数据文件刷新到文件系统，将引发恐慌级错误。这会导致数据库服务器崩溃。此参数只能在服务器启动时设置。

在某些操作系统上，在回写失败之后，内核页面缓存中的数据状态是未知的。在某些情况下，它可能被完全遗忘，使得重试不安全; 第二次尝试可能被报告为成功，但实际上数据已经丢失。在这种情况下，避免数据丢失的唯一方法是在报告任何故障后从 WAL 中恢复，最好是在调查故障的根本原因并更换任何有故障的硬件之后。

如果设置为 on，KingbaseES 将报告错误，但将继续运行，以便在以后的检查点重试数据刷新操作。只有在研究了操作系统在回写失败情况下对缓冲数据的处理之后，才将其设置为 on。

2.16　预置选项

下列“参数”是只读的，它们是在编译或安装 KingbaseES 时决定的。同样，它们被排除在 `kingbase.conf` 文件例子之外。这些选项报告特定应用可能感兴趣的多种 KingbaseES 行为，特别是管理前端相关的行为。

`block_size`

属性	描述
类型	integer
默认值	8KB
级别	INTERNAL

报告一个磁盘块的大小。它由编译服务器时 `BLCKSZ` 的值确定。默认值是 8192 字节。有些配置变量的含义（例如 shared_buffers ）会被 `block_size` 影响。

`data_checksums`

属性	描述
类型	boolean
默认值	off
级别	INTERNAL

报告对这个集簇是否启用了数据校验码。详见 sys_checksums 。

`data_directory_mode`

属性	描述
类型	integer
默认值	0700
级别	INTERNAL

在 Unix 系统上，这个参数报告启动时 data_directory 所定义的数据目录的权限（在 Microsoft Windows 上这个参数将总是显示 `0700`）。

`debug_assertions`

属性	描述
类型	boolean

-续表-

属性	描述
默认值	off
级别	INTERNAL

报告编译 KingbaseES 时是否启用了断言。如果 KingbaseES 被编译时定义了宏 `USE_ASSERT_CHECKING`（例如通过 `configure` 选项 `--enable-cassert` 定义），那么会报告已启用。默认情况下 KingbaseES 编译时没有用断言。

`integer_datetimes`

属性	描述
类型	boolean
默认值	on
级别	INTERNAL

报告 KingbaseES 是否在编译时打开了 64 位整数日期和时间。从 KingbaseES V8R3 起，这个值总是 `on`。

`lc_collate`

属性	描述
类型	string
默认值	zh_CN.UTF-8
级别	INTERNAL

报告文本数据排序使用的区域。详见 区域支持。该值是在数据库被创建时确定的。

`lc_ctype`

属性	描述
类型	string
默认值	zh_CN.UTF-8

-续表-

属性	描述
级别	INTERNAL

报告决定字符分类的区域。详见 区域支持。该值是在数据库被创建时决定的。通常它和 `lc_collate` 一样，但是可以为特殊应用设置成不同的值。

`max_function_args`

属性	描述
类型	integer
默认值	100
级别	INTERNAL

报告函数参数的最大数量。它由编译服务器时的 `FUNC_MAX_ARGS` 值决定的。默认值是 100 个参数。

`max_identifier_length`

属性	描述
类型	integer
默认值	64
级别	INTERNAL

报告标识符的最大长度。它由编译服务器时的 `NAMEDATALEN` 值减一决定。`NAMEDATALEN` 的默认值是 64 ；因此 `max_identifier_length` 的默认值是 63，但是在使用多字节编码时可以少于 63 个字符。

`max_index_keys`

属性	描述
类型	integer
默认值	32

-续表-

属性	描述
级别	INTERNAL

报告索引键的最大数目。它由编译服务器时的 `INDEX_MAX_KEYS` 值决定。默认值是 32 个键。

`segment_size`

属性	描述
类型	integer
默认值	1GB
级别	INTERNAL

报告一个文件段中可以存储的块（页）的数量。由编译服务器时的 `RELSEG_SIZE` 值决定。一个段文件的最大尺寸（以字节计）等于 `segment_size` 乘以 `block_size`，默认是 1GB。

`server_encoding`

属性	描述
类型	string
默认值	UTF8
级别	INTERNAL

报告数据库的编码（字符集）。这是在数据库被创建时决定的。通常，客户端只需要关心 client_encoding 的值。

`server_version`

属性	描述
类型	string
默认值	UTF8

-续表-

属性	描述
级别	INTERNAL

报告服务器版本数值。它是由编译服务器时的 `SYS_VERSION` 值决定的。

`server_version_num`

属性	描述
类型	integer
默认值	12.1
级别	INTERNAL

报告服务器版本数值的整数值。

它是由编译服务器时的 `SYS_VERSION_NUM` 值决定的。

`ssl_library`

属性	描述
类型	string
默认值	,,
级别	INTERNAL

报告此 KingbaseES 服务器构建时使用的 SSL 库的名称 (即使 SSL 当前没有配置或在此实例中使用)，例如 `OpenSSL`，或者一个空字符串 (如果没有)。

`wal_block_size`

属性	描述
类型	integer
默认值	8KB

-续表-

属性	描述
级别	INTERNAL

报告一个 WAL 磁盘块的尺寸。由编译服务器时的 `XLOG_BLCKSZ` 值决定。默认是 8192 字节。

`wal_segment_size`

属性	描述
类型	integer
默认值	16MB
级别	INTERNAL

报告 WAL 段文件的大小。默认是 16MB。

2.17　定制选项

这个特性被设计用来由附加模块向 KingbaseES 添加通常不为系统知道的参数（例如过程语言）。允许使用标准方法配制扩展模块。

自定义选项有两部分名称：一个扩展名，然后是一个句点，再然后是正确的参数名，就像 SQL 中的合格名称。例如：`plsql.variable_conflict`。

因为自定义选项可能需要在没有载入相关扩展模块的进程中设置，所以 KingbaseES 可以接收任意两部分参数名的设置。这种变量被认为是占位符，并且在定义它们的模块被载入之前不会有实际功能。当一个扩展模块被载入后，KingbaseES 会加入此模块的变量定义。根据那些定义，转换所有占位符值。如果存在以其扩展名开始的任意占位符未被识别，KingbaseES 将发出警告。

`kdb_oracle_datatype.large_lob_action`

属性	描述
类型	string
默认值	error
级别	USER

设置大对象数据类型引入长度超过 1GB-1 的时候的处理措施，默认值为 error, 即报错处理，也可以设置为 truncate, 即将长度截断为 1GB-1。

`nls_length_semantics`

属性	描述
类型	enum
默认值	char
级别	USER

该参数指定了 `varchar` 类型中 size 的默认语义，取值为 char 或 byte。当定义 char 类型未指定 size 的语义时，由该参数的值确定。显示指定 size 语义时不受 nls_length_semantics 影响。默认值为 char，也可以设置为 byte。

`nls_date_format`

属性	描述
类型	string
默认值	YYYY-MM-DD HH24:mi:ss
级别	USER

定义了 DATE 类型的默认日期格式及 `TO_CHAR` 和 `TO_DATE` 函数一起使用的默认日期格式。可以是任何有效的日期格式掩码。详见 SQL 基本元素中的日期/时间类型。

`nls_timestamp_format`

属性	描述
类型	string
默认值	YYYY-MM-DD HH24:mi:ss.FF
级别	USER

定义了 `timestamp` 类型的默认日期格式及 `to_char` 和 `to_timestamp` 函数一起使用的默认日期格式。例：

nls_timestamp_format = ‘YYYY-MM-DD HH24:mi:ss.FF’。详见 SQL 基本元素中的日期/时间类型。

nls_timestamp_tz_format

属性	描述
类型	string
默认值	YYYY-MM-DD HH24:mi:ss.FF TZH:TZM
级别	USER

该参数定义 timestamp with time zone 数据类型的默认日期格式。定义了 to_char 和 to_timestamp_tz 函数一起使用的默认日期格式。详见 SQL 基本元素中的日期/时间类型。

ora_bpchar_to_varchar_rtrim

属性	描述
类型	string
默认值	on
级别	USER

为保持和旧版本应用兼容，增加该参数 ora_bpchar_to_varchar_rtrim，默认值为 on，保持与原来行为一致，char 类型到 varchar 类型转换时删除右侧空格。如果需要兼容 Oracle 行为，则需要在 Oracle 模式下，设置 set ora_bpchar_to_varchar_rtrim=off，则 char 类型到 varchar 类型不删除右侧空格。

2.18　开发者选项

以下参数专为 KingbaseES 源代码设计，旨在某些情况下帮助恢复严重损坏的数据库。这些参数不适用于生产环境，且已从 kingbase.conf 文件中移除。注意这些参数大多数需要特殊的源代码编译标志才能生效，使用时需格外谨慎。

allow_system_table_mods

属性	描述
类型	boolean
默认值	off

-续表-

属性	描述
级别	KINGBASE

允许对系统表结构的修改。它可以被 `initdb` 使用。这个参数只能在服务器启动时设置。

`ignore_system_indexes`

属性	描述
类型	boolean
默认值	off
级别	BACKEND

读取系统表时忽略系统索引（但是修改系统表时依然同时更新索引）。这在从被破坏的系统索引中恢复数据的时有用。这个参数在会话开始之后不能被更改。

`post_auth_delay`

属性	描述
类型	integer
默认值	0
级别	BACKEND

如果为非零，那么在一个新的服务器进程派生出来之后并且在它开始认证过程之前，就会发生这么多秒的延迟。这是为了给开发者们一个机会在一个服务器进程上附加一个调试器。这个参数在会话开始之后不能被更改。

`pre_auth_delay`

属性	描述
类型	integer
默认值	0

-续表-

属性	描述
级别	SIGHUP

如果为非零，那么在一个新的服务器进程派生出来之后并且在它开始认证过程之前，就会发生这么多秒的延迟。这是为了给开发者们一个机会在一个服务器进程上附加一个调试器来跟踪认证过程中的不当行为。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

`trace_notify`

属性	描述
类型	boolean
默认值	off
级别	USER

为 `LISTEN` 和 `NOTIFY` 命令生成大量调试输出。client_min_messages 和 log_min_messages 必须是 `DEBUG1` 或者更低才能把这种输出分别发送到客户端或者服务器日志。

`trace_recovery_messages`

属性	描述
类型	enum
默认值	log
级别	SIGHUP

启用记录与恢复有关的调试输出，否则无法记录。这个参数允许用户覆盖 log_min_messages 的正常设置，但只用于指定的消息。这个参数的目的是用来调试热后备。有效值包括 `DEBUG5`、`DEBUG4`、`DEBUG3`、`DEBUG2`、`DEBUG1` 和 `LOG`。默认值 `LOG` 完全不会影响日志决定。其他值会记录相关级别或更高级别的与恢复相关的调试消息，就好像他们具有 `LOG` 优先级一样；对于 `log_min_messages` 的通用设置，这会无条件的将消息发送给服务器日志。这个参数只能在 `kingbase.conf` 文件中或在服务器命令行上设置。

`trace_sort`

属性	描述
类型	boolean
默认值	off
级别	USER

如果打开，发出在排序操作中的资源使用的相关信息。只有在编译 KingbaseES 时定义了 TRACE_SORT 宏，这个参数才可用（不过，当前在默认情况下就定义了 TRACE_SORT）。

trace_locks

属性	描述
类型	boolean
默认值	off
级别	USER

如果开启，发出锁使用情况的信息。被转储信息中包括锁操作的类型、锁的类型和被锁或被解锁对象的唯一标识符。同样包括的还有已经授予这个对象的锁类型的位掩码和等待这个对象的锁类型的位掩码。对每一种锁类型，已授权锁和等待锁的计数也会被一起转储。一个日志文件输出的例子如下：

```
LOG:  LockAcquire: new: lock(0xb7acd844) id(24688,24696,0,0,0,1)
      grantMask(0) req(0,0,0,0,0,0,0)=0 grant(0,0,0,0,0,0,0)=0
      wait(0) type(AccessShareLock)
LOG:  GrantLock: lock(0xb7acd844) id(24688,24696,0,0,0,1)
      grantMask(2) req(1,0,0,0,0,0,0)=1 grant(1,0,0,0,0,0,0)=1
      wait(0) type(AccessShareLock)
LOG:  UnGrantLock: updated: lock(0xb7acd844) id(24688,24696,0,0,0,1)
      grantMask(0) req(0,0,0,0,0,0,0)=0 grant(0,0,0,0,0,0,0)=0
      wait(0) type(AccessShareLock)
LOG:  CleanUpLock: deleting: lock(0xb7acd844) id(24688,24696,0,0,0,1)
      grantMask(0) req(0,0,0,0,0,0,0)=0 grant(0,0,0,0,0,0,0)=0
      wait(0) type(INVALID)
```

被转储结构的详细信息可以在 src/include/storage/lock.h 中找到。

只有在编译 KingbaseES 时定义了 LOCK_DEBUG 宏，这个参数才可用。

trace_lwlocks

属性	描述
类型	boolean
默认值	off
级别	USER

如果开启，发出轻量级锁的使用信息。轻量级锁主要是为了提供对共享内存数据结构的互斥访问。

只有在编译 KingbaseES 时定义了 `LOCK_DEBUG` 宏，这个参数才可用。

`trace_userlocks`

属性	描述
类型	boolean
默认值	off
级别	USER

如果开启，发出关于用户锁使用的信息。与 `trace_locks` 的输出一样，但只用于咨询锁。

只有在编译 KingbaseES 时定义了 `LOCK_DEBUG` 宏，这个参数才可用。

`trace_lock_oidmin`

属性	描述
类型	integer
默认值	0
级别	USER

如果设置，不会跟踪小于这个 OID 的锁（用于避免在系统表上的输出）。

只有在编译 KingbaseES 时定义了 `LOCK_DEBUG` 宏，这个参数才可用。

`trace_lock_table`

属性	描述
类型	integer
默认值	0
级别	USER

无条件地跟踪此表（OID）上的锁。

只有在编译 KingbaseES 时定义了 `LOCK_DEBUG` 宏，这个参数才可用。

`debug_deadlocks`

属性	描述
类型	boolean
默认值	off
级别	USER

如果设置，当死锁超时发生时，转储所有当前锁的信息。

只有在编译 KingbaseES 时定义了 `LOCK_DEBUG` 宏，这个参数才可用。

`log_btree_build_stats`

属性	描述
类型	boolean
默认值	off
级别	USER

如果设置，会记录 B 树操作上的系统资源使用情况统计（内存和 CPU）。

只有在编译 KingbaseES 时定义了 `BTREE_BUILD_STATS` 宏，这个参数才可用。

`wal_consistency_checking`

属性	描述
类型	string
默认值	,,
级别	SUPERUSER

这个参数被设计用来检查 WAL 重做例程中的缺陷。当这个参数被启用时，被修改的任何缓冲区的全页映像及其 WAL 记录都被加入到记录中。如果该记录后来被重放，系统将首先应用每个记录然后测试该记录修改的缓冲区是否符合存储的映像。在某些情况下（例如提示位），小的变动是可以接受的，并且会被忽略。任何预期之外的差别都将导致致命错误，最后中止恢复。

这个设置的默认值是空字符串，它将禁用这一特性。它可以被设置为 `all` 以检查所有记录，或者被设置为一个逗号分隔的资源管理器列表用以检查那些资源管理器产生的记录。当前，支持的资源管理器是 `heap`、`heap2`、`btree`、`hash`、`gin`、`gist`、`sequence`、`spgist`、`brin` 以及 `generic`。只有超级用户可以更改这一设置。

`trace_mdwrite_level`

属性	描述
类型	string
默认值	”non-zero”
级别	SUPERUSER

该参数的主要用途为控制页面检查的严格程度，该参数仅在 trace_mdwrite 参数打开时有效，该参数仅有三个有效值分别为 `valid`、`non-zero`、`all`，含义为追踪无效页面，追踪全零页面，追踪无效页面和全零页面。默认设置是 `non-zero`。

`wal_debug`

属性	描述
类型	boolean
默认值	,,
级别	USER

如果被打开，WAL 相关的调试输出将被发出。只有在编译 KingbaseES 时定义了 `WAL_DEBUG` 宏的情况下，这个参数才可用。

`ignore_checksum_failure`

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

只有当 data checksums 被启用时才有效。

在读取过程中检测到一次校验码失败通常会导致 KingbaseES 报告一个错误。设置 `ignore_checksum_failure` 为打开会导致系统忽略失败（但是仍然报告一个警告），并且继续执行。这种行为可能导致崩溃、传播或隐藏损坏或者其他严重的问题。但是，它允许绕过错误并且在块头部仍然健全的情况下从表中检索未损坏的元组。如果头部被损坏，即便这个选项被启用系统也将报告一个错误。默认设置是 `off`，并且只能被超级用户改变。

`zero_damaged_pages`

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

检测到一个损坏的页面头部通常会导致 KingbaseES 报告一个错误，并且中止当前事务。把 `zero_damaged_pages` 设置为打开会让系统报告一个警告、把损坏的页面填充零，然后继续处理。这种行为会毁掉数据，即被损坏页面上的所有行。但是它允许绕开错误并且从可能存在表中的任何未损坏页面中检索行。如果由于一次硬件或软件错误而发生毁坏，这种方法可用于恢复数据。通常不应该把它设置为打开，除非已经彻底放弃从表的损坏页面中恢复数据。被填充零的页面不会被强制到磁盘上，因此我们推荐在再次关闭这个参数之前先重建表或索引。默认的设置是 `off`，并且只有超级用户可以改变它。

`jit_debugging_support`

属性	描述
类型	boolean
默认值	off

-续表-

属性	描述
级别	SU_BACKEND

如果 LLVM 有所需要的功能，用 GDB 注册所生成的函数。这会让调试更加容易。默认设置是 `off`。这个参数只能在服务器启动时设置。

`jit_dump_bitcode`

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

把生成的 LLVM IR 写出到文件系统，写到 data_directory 中。只有在做 JIT 内部实现工作时，这个参数才能派上用场。默认设置是 `off`。这个参数只能由超级用户修改。

`jit_expressions`

属性	描述
类型	boolean
默认值	on
级别	USER

当 JIT 编译被激活时，确定表达式是否用 JIT 编译。默认值是 `on`。

`jit_profiling_support`

属性	描述
类型	boolean
默认值	off

-续表-

属性	描述
级别	SU_BACKEND

如果 LLVM 有所需的功能，发出需要的数据以允许 perf 对 JIT 生成的函数画像。这会写出文件到 `$HOME/.debug/jit/`中，如果需要，由用户负责对其执行清除。默认设置是 `off`。这个参数只能在服务器启动时设置。

`jit_tuple_deforming`

属性	描述
类型	boolean
默认值	on
级别	USER

当 JIT 编译被激活时，确定元组拆解是否被 JIT 编译。默认值是 `on`。

`allow_system_table_dml`

属性	描述
类型	boolean
默认值	off
级别	KINGBASE

允许对系统表进行 DML 操作。默认值是 `off`，它将禁用这一特性。这个参数不建议用户开启。

`trace_mdwrite`

属性	描述
类型	boolean
默认值	off
级别	SUPERUSER

允许进行页面校验并记录异常日志，在开启该参数后将对页面进行合法性校验，并在追踪到非法页面时记录错误日志，默认值是 `off`，当数据库出现页面异常时建议开启该功能进行调试。

2.19　短选项

为了方便起见，系统中还为一些参数提供了单字母的命令行选项开关。它们在表”短选项”中描述。其中一些选项是由于历史原因而存在，并且它们作为一个单字母选项存在并不表示它们会被大量使用。

表 2.19.1: 短选项

短选项	等效于
-B x	shared_buffers = x
-d x	log_min_messages = DEBUGx
-e	datestyle = ISO, DMY
-fb, -fh, -fi,-fm, -fn, -fo, -fs,-ft	enable_bitmapscan = off,enable_hashjoin = off, enable_indexscan = off,enable_mergejoin = off, enable_nestloop = off,enable_indexonlyscan = off,enable_seqscan = off,enable_tidscan = off
-F	fsync = off
-h x	listen_addresses = x
-i	listen_addresses = ’*’
-k x	unix_socket_directories = x
-l	ssl = on
-N x	max_connections = x
-O	allow_system_table_mods = on
-p x	port = x
-P	ignore_system_indexes = on
-s	log_statement_stats = on
-S x	work_mem = x
-tpa, -tpl, -te	log_parser_stats = on,log_planner_stats = on,log _executor_stats = on

-续表-

短选项	等效于
-W x	post_auth_delay = x

2.20　控制文件

2.20.1　控制文件多路备份

`control_file_copy`

属性	描述
类型	string
默认值	,,
级别	KINGBASE

路径和文件用来制作一个或多个 sys_control 的副本。control_file_copy 是一个空字符串（默认）。

最多设置八个副本，副本之间用“；”分隔。

这个参数只能在 kingbase.conf 文件中或在服务器启动参数上设置。如果修改了 control_file_copy 的值，服务器必须重启才能生效。

2.21　函数别名作为列名

`enable_func_colname`

属性	描述
类型	boolean
默认值	off
级别	USER

启用或禁用函数别名作为列名功能。默认此功能禁用。

此参数为 session 级参数，设置为 on 启用此功能，设置为 off 关闭此功能。

2.22　数据库审计

KingbaseES 数据库提供了一些关于的审计机制的参数，可以针对不同事件审计记录。可参见《安全指南》中的数据库审计章节。

`sysaudit.enable_encrypt`

属性	描述
类型	boolean
默认值	false
级别	SIGHUP

开启参数后，会对生成密钥、数据加密和解密事件进行审计。

`sysaudit.enable_mac`

属性	描述
类型	boolean
默认值	false
级别	SIGHUP

开启参数后，会对创建强制访问控制规则、带有读和写的强制访问控制规则事件进行审计。

`sysaudit.enable_audit_cluster_datatransfer`

属性	描述
类型	boolean
默认值	false

-续表-

属性	描述
级别	SIGHUP

开启参数后，会对集群环境下的主备机通信进行审计。

`sysaudit.enable_audit_cluster_set_guc_param`

属性	描述
类型	boolean
默认值	false
级别	SIGHUP

开启参数后，会对集群环境下的主备机之间 GUC 参数同步行为进行审计。

`sysaudit.processevent`

属性	描述
类型	boolean
默认值	false
级别	SIGHUP

开启参数后，审计后台子进程启动退出事件。

`sysaudit.all_error`

属性	描述
类型	boolean
默认值	false
级别	SIGHUP

开启参数后，审计全部错误事件，即所有的 error 以上的错误日志都记入审计日志。

`sysaudit.report_log`

属性	描述
类型	boolean
默认值	false
级别	SIGHUP

开启参数后, 将 KES 的审计日志通过操作系统标准的日志外发接口，外发到其他服务器机器上。仅审计管理员 sao 有权限对此参数进行更改，且此功能暂不支持 windows 平台上运行。

2.23　标记和强制访问控制

KingbaseES 数据库提供了一些关于的强制访问控制功能的参数，可参见《安全指南》中的 标记和强制访问控制章节。

`sysmac.enable_mac`

属性	描述
类型	boolean
默认值	false
级别	SIGHUP

强制访问控制功能的开关，取值范围为 true 或者 false ，缺省值为 false。

`sysmac.enable_col_mac`

属性	描述
类型	boolean
默认值	false
级别	SIGHUP

列级强制访问控制功能的开关，取值范围为 true 或者 false ，默认是 false。

2.24　性能调优

KingbaseES 数据库提供了一些关于性能调优的参数，具体请参见《性能调优指南》。

`bindcpulist`

属性	描述
类型	TEXT
默认值	NULL
级别	KINGBASE

功能介绍

启动 KingbaseES 服务进程时，根据此参数给出的 CPU 核号列表信息，利用操作系统的 CPU 亲和性接口函数，将当前服务进程绑定在一个固定的 CPU 核号上，以此提高性能。绑核顺序按参数列表顺序进行绑定。

语法规则

正确格式为 `start_cpunum-end_cpunum | cpunum [, ...]`。即：开始的 CPU 核号-结束的 CPU 核号，或单独给出 CPU 核号。各项之间用逗号隔开。其他非数字、非’-‘、非’,’的字符都视为非法字符。如 `bindcpulist = '0-95'`，表示将数据库进程绑定到 0 到 95 号 CPU 核心；`bindcpulist = '0-3,10,20-30,50'`，表示将数据库进程绑定到 0 到 3 号核、10 号核、20 到 30 号核、50 号核心上。

应用场景

CPU 利用率较高，且存在大量活跃连接的时，通过绑核可以减少进程在不同 CPU 核心之间切换带来的上下文开销。

第3章　存储管理

3.1　控制文件管理

您可以通过控制文件读取到 KingbaseES 内部状态的各方面信息。

- 什么是控制文件？

每个 KingbaseES 都有一个控制文件，它是一个记录数据库内部状态的重要文件。

- 控制文件准则

您可以通过指南了解控制文件的一些信息。

- 创建控制文件

控制文件在初始化数据库的时候创建。

- 备份控制文件

KingbaseES 使用基础备份的方式备份控制文件。

- 读取控制文件信息

KingbaseES 提供了 sys_controldata 工具来读取控制文件的信息。

3.1.1　什么是控制文件？

每个 KingbaseES 都有一个控制文件，它是一个记录数据库内部状态的重要文件。

控制文件中包括：

- 初始化静态信息
- WAL 及检查点的动态信息
- 一些配置信息

当 KingbaseES 启动时，控制文件必须可供数据库正确读取和写入。如果没有控制文件或控制文件不可读，数据库无法启动。

KingbaseES 的控制文件是在初始化数据库的时候创建的。

3.1.2　控制文件准则

您可以通过指南了解控制文件的一些信息。

- 控制文件路径 不能改变。
- 控制文件的大小 固定不变的。

3.1.2.1　控制文件路径

KingbaseES 控制文件路径不能改变的，控制文件放在 data/global 目录下名为 sys_control。在启动数据库的时候，KingbaseES 从这个路径下读取控制文件中的相关信息。

3.1.2.2　控制文件的大小

KingbaseES 中的控制文件的大小是固定不变的。

KingbaseES 控制文件在内存中尽量保持小于 512 个字节以使其适合一个典型的磁盘驱动的物理簇的大小。

KingbaseES 控制文件的物理大小是 8K，这样做是为了控制文件格式变化时保持物理大小不变。

3.1.3　创建控制文件

控制文件在初始化数据库的时候创建。

3.1.3.1　控制文件内部信息的创建

控制文件内部信息一部分在初始化时创建，一部分在运行时动态修改。

- 控制文件的静态信息在初始化时自动生成，运行过程中不允许修改，例如系统标识符。
- 控制文件的配置信息允许用户在初始化时一次性定制，不再允许修改，例如重做日志段尺寸。
- 控制文件的 WAL 及检查点的动态信息，在 KingbaseES 运行中动态修改。

3.1.4　备份控制文件

KingbaseES 使用基础备份的方式备份控制文件。在基础备份执行的过程中，控制文件随着数据目录一起被备份起来。

3.1.5　读取控制文件信息

KingbaseES 提供了 sys_controldata 工具来读取控制文件的信息。

```
sys_controldata -D /path/to/sysdata
```

3.2　重做日志管理

- 什么是重做日志？

KingbaseES 中的重做日志依靠预写式日志（WAL）实现的, 它是有多个重做日志段文件组成，这些段文件记录了对数据库页面所做的所有更改，任何还没有被应用到数据页面的改变可以根据其日志记录重做。

- 规划重做日志

在配置数据库实例重做日志时，您可以遵循指南。

- 手动切换重做日志

KingbaseES 允许您进行手动切换重做日志段文件。

- 验证重做日志文件中的块

您可以将数据库配置为使用一致性校验来验证重做日志文件中的块。

- 清除重做日志文件

KingbaseES 中，需要安全的删除重做日志文件，可以采用检查点的方式或者设置配置参数 archive_cleanup_command。

3.2.1　什么是重做日志？

KingbaseES 中的重做日志依靠预写式日志（WAL）实现的, 它是有多个重做日志文件组成，这些段文件记录了对数据库页面所做的所有更改，任何还没有被应用到数据页面的改变可以根据其日志记录重做。

- WalWriter 进程

会周期性地将重做日志块写入到磁盘。

- 重做日志的内容

KingbaseES 中，对数据库所有的修改通过重做日志记录的方式放在重做日志中。

- KingbaseES 数据库如何写入重做日志

KingbaseES 中，采用了共享缓存区的方式将重做日志的 I/O 操作合并在一起。

3.2.1.1　WalWriter 进程

WalWriter 进程会周期性地将重做日志块写入到磁盘。KingbaseES 中，为每个实例分配一个后台进程，所有后台进程采用共享缓存区的方式进行重做日志文件的操作。WalWriter 进程可以避免其他服务进程在事务提交时需要将重做日志同步地写入磁盘造成性能下降，事务提交记录不是在提交时同步地写入磁盘，而是在一个已知的预先设置的时间异步地写入。

3.2.1.2　重做日志的内容

KingbaseES 中，对数据库所有的修改通过重做日志记录的方式放在重做日志中。

重做日志被存放在数据目录的 sys_wal 目录里，它是作为一个文件段的集合存储的，通常每个段 16MB 大小（不过这个大小可以通过 initdb 配置选项 - wal-segsize 来修改）。每个段分割成多个页，通常每个页为 8K（该尺寸可以通过 - block-size 配置选项来修改）。

每个页中存放多个重做日志记录，重做日志记录内容取决于它记录的事件类型。段文件的名字是不断增长的数字，从 000000010000000000000000 开始。目前这些数字不能回卷，不过要把所有可用的数字都用光也需要非常非常长的时间。

3.2.1.3　KingbaseES 数据库如何写入重做日志

KingbaseES 中，采用了共享缓存区的方式将重做日志的 I/O 操作合并在一起。

共享缓存区可以理解为一个环形的共享缓存，每次缓存空间满后，会将头部的页面刷新到磁盘，同时写入新的页面，并将头部往后移动一位。具体来说，需要写入新的日志记录时:

- 当共享缓存区中有足够的空间，顺序写入到缓存区中。
- 当共享缓存区写到尾部且空间不足时，从头部刷出信息后重复利用。
- 回收和未回收的重做日志

KingbaseES 数据库一次只使用一个重做日志段文件来存储从重做日志缓冲区写入的重做记录。

- 日志切换与日志序列号

日志切换是数据库停止写入当前重做日志段文件并开始写入下一个重做日志段文件。通常，当当前重做日志文件完全填满并且必须继续写入下一个重做日志文件时，会发生日志切换。

回收和未回收的重做日志

KingbaseES 数据库一次只使用一个重做日志段文件来存储从重做日志缓冲区写入的重做记录。在执行检查点之前，从上一个检查点起产生的重做日志是不能被回收或删除的，如果开启了归档模式，重做日志可以正常归档。执行检查点之后，上一个检查点与此次检查点之间的重做日志可以被回收或删除，具体回收还是删除需要靠配置参数决定。

日志切换与日志序列号

日志切换是数据库停止写入当前重做日志段文件并开始写入下一个重做日志段文件。通常，当当前重做日志文件完全填满并且必须继续写入下一个重做日志文件时，会发生日志切换。

您也可以手动的切换日志。

当每个新的重做日志记录被写入时，重做日志记录被追加到重做日志中。插入位置由日志序列号（LSN）描述，该日志序列号是日志中的字节偏移量，随每个新记录单调递增。

3.2.2　规划重做日志

在配置数据库实例重做日志时，您可以遵循指南。

- 重做日志段文件的大小

KingbaseES 允许您配置重做日志段文件的大小。

- 重做日志文件的块大小

KingbaseES 允许您配置重做日志文件的块大小。

- 重做日志文件的数量

KingbaseES 允许您配置重做日志文件的数量。

- 重做日志记录延迟拷贝

某些情况下，一台后备服务器使用主服务器提供的重做日志记录尽快进行恢复。使用数据的延时拷贝可以提供纠正数据丢失错误的机会。

3.2.2.1　重做日志段文件的大小

KingbaseES 允许您配置重做日志段文件的大小。

- 在初始化阶段进行配置，通过 initdb 配置选项 - wal-segsize 进行选择。
- 通过 sys_resetwal 工具的选项 - wal-segsize 进行选择。

备注:

- sys_resetwal 会清除 sys_wal 目录的重做日志文件。
- sys_resetwal 方式修改 wal_segment_size 后，重启后原来的数据都不会丢失，因为修改前数据库是正常关闭的。

3.2.2.2　重做日志文件的块大小

KingbaseES 允许您配置重做日志文件的块大小。

每个重做日志段文件被分割成多个 block，默认每个 block 为 8K。该尺寸通过 initdb 配置选项 - block-size 来修改。

3.2.2.3　重做日志文件的数量

KingbaseES 允许您配置重做日志文件的数量。

KingbaseES 中，在执行检查点后会删除或回收不需要的重做日志段文件。可以通过配置参数 max_wal_size 和 min_wal_size 来限制检查点后的重做日志段文件的数量。

max_wal_size

在检查点之间允许重做日志增长到的最大尺寸。这是一个软限制，在特殊的情况下重做文件尺寸可能会超过 max_wal_size。如果指定值时没有单位，则以兆字节为单位。默认为 1 GB。增加这个参数可能导致崩溃恢复所需的时间。这个参数只能在 kingbase.conf 或者服务器命令行中设置。

min_wal_size

只要重做日志磁盘用量保持在这个设置之下，在检查点时旧的重做日志文件总是被回收以便未来使用，而不是直接被删除。这可以被用来确保有足够的重做日志空间被保留来应付重做日志使用的高峰，例如运行大型的批处理任务。如果指定值时没有单位，则以兆字节为单位。默认是 80 MB。这个参数只能在 kingbase.conf 或者服务器命令行中设置。

3.2.2.4　重做日志记录延迟拷贝

某些情况下，一台后备服务器使用主服务器提供的重做日志记录尽快进行恢复。使用数据的延时拷贝可以提供纠正数据丢失错误的机会。

通过 kingbase.conf 中的配置参数 recovery_min_apply_delay，允许备机的恢复延迟一段固定的时间，如果没有指定单位则以毫秒为单位。

备注:

- 只有在事务提交的重做日志记录上才会发生延迟，其他记录还是会被尽可能快地重放。
- 一旦恢复中的数据库已经达到一致状态，延迟就会产生，直到后备机被提升或者触发。在那之后，后备机将会结束恢复并且不再等待。
- 修改后，需要重启 standby 节点才能生效。

3.2.3　手动切换重做日志

KingbaseES 允许您进行手动切换重做日志段文件。

sys_switch_wal 是 KingbaseES 的一个系统函数，它强制 KingbaseES 切换到一个新的重做日志段文件：

```
select sys_switch_wal();
```

备注:

- 返回旧重做日志文件的结尾 LSN + 1
- 如果自上次重做日志文件切换后没有产生重做日志的活动，则不会进行切换并返回当前重做日志件的起始位置。

3.2.4 验证重做日志文件中的块

您可以将数据库配置为使用一致性校验来验证重做日志文件中的块。

在 kingbase.conf 中开启配置参数 wal_consistency_checking 时，将开启 KingbaseES 的一致性校验功能。

当这个参数被启用时，被修改的任何缓冲区的全页映像及其重做日志记录都被加入到记录中。如果该记录后来被重放，系统将首先应用每个记录然后测试该记录修改的缓冲区是否符合存储的映像。在某些情况下（例如提示位），小的变动是可以接受的，并且会被忽略。任何预期之外的差别都将导致致命错误，最后中止恢复。

这个设置的默认值是空字符串，它将禁用这一特性。它可以被设置为 all 以检查所有记录，或者被设置为一个逗号分隔的资源管理器列表用以检查那些资源管理器产生的记录。当前，支持的资源管理器是 heap、heap2、btree、bitmap、hash、gin、gist、sequence、spgist、brin 以及 generic。只有超级用户可以更改这一设置。

3.2.5 清除重做日志文件

KingbaseES 中，需要安全的删除重做日志文件，可以采用检查点的方式或者设置配置参数 archive_cleanup_command。

备注:

除特殊的情况外，不要手动删除重做日志文件。

在检查点之后，当旧的重做日志文件不再需要时，它们将被删除或回收。

备注:

- 如果使用了重做日志归档，旧的段在被归档之前不能被删除或回收，直到它们被归档。如果重做日志归档跟不上重做日志记录生成的速度，或者 archive_command 重复失败，旧的重做日志文件将在 sys_wal 中累积，直到问题解决。
- wal_keep_segments 配置参数可能需要保留一定的数据。

archive_cleanup_command 指定了一个 shell 命令，它将在每一个重启点被执行。archive_cleanup_command 的目的是提供一种清除不再被后备服务器需要的旧的已归档重做日志文件的机制。

3.3　归档重做日志文件管理

您可以在数据库启动前决定是否开启归档模式，并管理重做文件的归档。

- 什么是归档重做日志？

KingbaseES 数据库允许您将发生切换的重做日志段文件存储在其他位置，这个过程统称为归档重做日志。

- 开启和关闭归档重做日志

在启动 KingbaseES 数据库之前，必须决定是否开启归档。

- 控制归档

您可以为数据库开启归档模式。

- 指定归档目标

在归档重做日志之前，您必须确定要归档的目标。

- 查看归档重做日志状态

您可以通过状态文件了解归档重做日志状态。

3.3.1　什么是归档重做日志？

KingbaseES 数据库允许您将发生切换的重做日志段文件存储在其他位置，这个过程统称为归档重做日志。

从逻辑上来看，KingbaseES 数据库会产生一个无限长的顺序重做日志记录序列。KingbaseES 在物理上把这个重做日志记录序列分割成多个重做日志（每个重做日志为一个重做日志段），重做日志段文件被重用或删除时进行归档备份操作，把将被重用的重做日志段中的日志记录保存到其他位置。

3.3.2　开启和关闭归档重做日志

在启动 KingbaseES 数据库之前，必须决定是否开启归档。

是否启用归档的选择取决于在数据库上运行的应用程序的可用性和可靠性要求。如果由于故障发生时，您想回退到发生故障之前的位置，请开启归档重做日志文件。

- 开启归档重做日志

当开启归档重做日志文件，重做日志将在发生切换的时候进行归档。

- 关闭归档重做日志

当关闭归档重做日志，重做日志将在回收或删除的时候，没有重做日志的拷贝。

3.3.2.1　开启归档重做日志

当开启归档重做日志文件，重做日志将在发生切换的时候进行归档。

当使用物理备份方式对数据库进行备份，归档重做日志可以使数据库在启动恢复的时候恢复到之前的位置。

3.3.2.2　关闭归档重做日志

当关闭归档重做日志，重做日志将在回收或删除的时候，没有重做日志的拷贝。

在没有使用归档重做日志时，KingbaseES 通常只创建少量段文件，并且通过重命名不再使用的重做日志段文件为更高的段编号来“回收”它们。KingbaseES 假设内容位于最后一个检查点之前的重做日志段文件是无用的且可以被回收。

3.3.3　控制归档

您可以为数据库开启归档模式。

- *设置数据库归档模式*

在数据库启动之前，需要通过配置参数的设置启动归档模式。

- *更改数据库归档模式*

归档模式需要在数据库启动之前确认，更改归档模式需要重启数据库。

- *执行归档*

为了方便和高效，自动存档通常是最好的。但是，您可以手动归档。

3.3.3.1　设置数据库归档模式

数据库通过配置参数 archive_mode 指定开启/关闭归档，archive_mode 的选项：

- on：开启归档模式。
- off：禁用归档模式。
- always：在普通操作期间，与 on 参数没有区别。但是当设置为 always 时，重做日志归档进程在归档恢复或者后备模式下也会被启用，所有从归档恢复或者用流复制传来的重做日志文件将被（再次）归档。

数据库默认开启归档，配置参数 archive_mode 的默认值为 always。

除了 archive_mode 配置参数外，需要开启归档重做日志文件，还需要：

- wal_level 配置参数设置为 replica 或更高。

- 使用 archive_command 配置参数指定一个 shell 命令。

3.3.3.2　更改数据库归档模式

归档模式需要在数据库启动之前确认，更改归档模式需要重启数据库。

KingbaseES 数据库安装初始化时，默认关闭归档模式。

从关闭归档到启动归档的切换：

1. 关闭数据库。
2. 按照设置数据库归档模式 章节中进行配置参数。
3. 启动数据库。

3.3.3.3　执行归档

为了方便和高效，自动存档通常是最好的。但是，您可以手动归档。

自动触发归档

当启动归档模式时，重做日志文件写满后开始写下一个重做日志文件，此时 KingbaseES 将执行 archive_command 配置参数指定的 shell 将写满的重做日志段文件进行归档。

定时触发归档

KingbaseES 中可以通过 archive_timeout 配置参数来强制服务器定期切换到新的重做日志段文件。如果指定值时没有单位，则以秒为单位。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

当 archive_timeout 配置参数大于 0 时，只要从上次段文件切换后过了参数所设置的时间量，并且已经有过任何数据库活动（包括一个单一检查点），服务器将切换到一个新的段文件（如果没有数据库活动则会跳过检查点）。写入量很少, 只要到达 archive_timeout 会触发重做日志文件的强制切换。

如果为 0 表示不启用定时归档。

手动执行归档

在 KingbaseES 中，当使用 sys_switch_wal 系统函数时切换重做日志，这时将触发重做日志归档。

3.3.4　指定归档目标

在归档重做日志之前，您必须确定要归档的目标。

为了给数据库管理员提供最大的灵活性，KingbaseES 不对如何归档做任何假设，而是让管理员提供一个 shell 命令来拷贝一个完整的重做日志文件到备份存储位置。该命令可以就是一个 cp 命令，或者是一个复杂的 shell 脚本，所有的操作都由管理员决定。

- 设置存档目标的初始化参数

您可以在 kingbase.conf 指定归档重做日志的拷贝命令。

- 归档的状态

在正常启动归档模式后，可能出现正常归档状态和未归档状态。

3.3.4.1　设置存档目标的初始化参数

您可以在 kingbase.conf 指定归档重做日志的拷贝命令。

KingbaseES 使用 archive_command 配置参数指定本地 shell 命令，执行一个完成的重做日志文件段的归档。这个参数只能在 kingbase.conf 文件中或在服务器命令行上设置。

archive_command 中的控制符：

-
-
- 如果要在命令里嵌入一个真正的% 字符，可以使用%%。

3.3.4.2　归档的状态

在正常启动归档模式后，可能出现正常归档状态和未归档状态。

正常归档状态

archive_command 配置参数中指定的 shell 命令只在成功时返回一个零作为退出状态。

未归档状态

- 配置参数 archive_command 指定的 shell 命令执行失败，或未指定归档命令（默认为空）。重做日志归档会被临时禁用，但服务器仍会继续累计重做日志段文件，期待着一个命令被提供。
- archive_command 不设置拷贝指令。如果将 archive_command 设置为一个只返回真但不做任何事的命令（例如/bin/true 或 Windows 上的 REM）实际上会禁用归档，也会打破归档恢复所需的重做日志文件链，因此只有在极少数情况下才能用。

3.3.5　查看归档重做日志状态

您可以通过状态文件了解归档重做日志状态。

为了标识文件归档状态，KingbaseES 在数据集簇的 sys_wal/archive_status 目录下记录了每一重做日志段文件的状态文件。状态文件的前缀与段文件同名，以表示时间顺序的整数形式命名。状态文件后缀为.ready 或者.done，代表段文件的归档状态。

3.4　表空间管理

表空间是将有关的数据库逻辑数据组合在一起形成的数据库存储单元。表空间允许用户定义指定的数据库对象在文件系统中存储的位置。关系及其数据文件都存储在其所属的表空间中。

- 表空间的管理指南

  供您参考的管理表空间的建议和指南。

- 创建表空间

  创建表空间以将有关的数据库逻辑数据（如表或索引等）组合在一起。数据文件都存储在表空间中。

- 修改表空间可用性

  你可以使表空间进入脱机状态，以使它暂时不可被访问和使用，数据库的其他表空间仍然可以正常访问。你可以使已脱机的表空间重新进入联机状态，以使它可以再次被访问。

- 使用只读表空间

  你可以将表空间设置为只读模式，以防止其中存储的数据被更改。

- 修改和维护表空间

  你可以修改表空间的参数和属主等信息。

- 重命名表空间

  你可以修改表空间名称。

- 删除表空间

  当表空间不再被需要时，你可以删除表空间。在一个表空间能被删除前，其中必须没有任何数据库对象。

- 查看表空间相关信息

  KingbaseES 提供表空间信息视图，供查看表空间相关的信息。

3.4.1　表空间的管理指南

供您参考的管理表空间的建议和指南。

- 使用多个表空间

  使用多个表空间可以更灵活的执行数据库操作。

- 将表空间分配给用户

  授予用户在表空间创建对象的权限。

3.4.1.1　使用多个表空间

使用多个表空间可以更灵活地执行数据库操作。当数据库具有多个表空间时，您可以：

- 将用户数据与系统表数据分开，以减少 I/O 争用。
- 将一个应用程序的数据与另一个应用程序的数据分开，以防止在必须使表空间脱机时多个应用程序受到影响。
- 将不同表空间的数据文件存储在不同的磁盘驱动器上，以减少 I/O 争用。
- 使单个表空间脱机，而其他表空间保持联机状态，从而提供更好的整体可用性。
- 通过为特定类型的数据库使用（如高更新活动、只读活动或临时段存储）保留表空间来优化表空间使用。
- 备份单个表空间。

3.4.1.2　将表空间分配给用户

授予将要创建表、物化视图、索引和其他对象的用户在表空间中创建对象的权限。如果启用了表空间限额，则为用户在该表空间上配置合适的空间限制配额。

3.4.2　创建表空间

创建表空间以将有关的数据库逻辑数据（如表或索引等）组合在一起。数据文件都存储在表空间中。

- 关于创建表空间

要创建新的表空间，请使用 SQL 语句 CREATE TABLESPACE。

- 加密表空间

你可以加密一个表空间来保护敏感数据。

- 临时表空间

临时表空间用于在工作内存不足时存储排序或者连接运算生成的临时关系。指定合适的临时表空间可以改善这种情况下排序和连接操作的性能。

3.4.2.1　关于创建表空间

要创建新的表空间，请使用 SQL 语句 CREATE TABLESPACE。只有超级用户能创建表空间，但是它们能把表空间的拥有权赋予给非超级用户。

在创建表空间之前，必须创建一个数据库来包含它。任何新建的数据库中都有初始的两个系统表空间：sys_global 和 sys_default。这两个表空间的物理位置位于数据集簇目录下。这两个系统表空间中记录了服务器运行的基本信息，比如系统表。系统表空间可以像任何其他表空间一样进行管理，但需要更高级别的特权，并且在某些方面受到限制，例如，不能重命名或删除系统表空间，也不能使其脱机。两个系统表空间的不同之处在于，sys_global 表空间内的数据在整个数据集簇中对所有数据库共享。而 sys_default 表空间则只记录本数据库的信息。

创建表空间的步骤因操作系统而异，但第一步始终是在操作系统上创建目录结构，以为该表空间指定存储的物理位置。然后，在多数操作系统上，我们需要为该目录设置正确的所有权。一个超级用户可以执行 CREATE TABLESPACE 语句以创建表空间，我们会指定该目录作为该表空间的 LOCATION。此外选择一个合适的表空间名，该名称不能以 sys_ 开头，因为 sys_ 名称是为系统表空间保留的。

可以使用 ALTER TABLESPACE 或者 ALTER DATABASE 语句来更改表空间。同样的，您必须对该 TABLESPACE 或 DATABASE 具备操作的权限。

3.4.2.2　加密表空间

你可以对表空间中的数据进行加密，以符合安全需求。

3.4.2.3　临时表空间

临时表空间用于存储会话工作期间的临时数据。这些数据包括：

- 排序和连接操作的中间结果。
- 物化操作存储的数据。
- 临时表和临时索引（缺省状态）。

和普通表空间一样，临时表空间中的数据以临时文件和数据文件的形式存储，其中，排序等中间结果和物化结果会以临时文件存储，而临时表和临时索引则以数据文件存储。

在 KingbaseES 中，临时表空间的创建和使用和普通表空间无异，只是你需要在创建表空间之后，在参数 temp_tablespaces 中指定哪些表空间将作为临时表空间使用。参数 temp_tablespaces 的值是一个表空间名的列表，当列表中有多于一个成员时，每个临时关系会被随机的存储在其中一个临时表空间中。如果该参数未被配置，数据库将缺省的使用默认表空间，通常为 sys_default，兼作为临时表空间使用。

临时表也可以建立在用户指定的表空间中，因此临时表空间只是一个缺省的选择。

3.4.3　修改表空间可用性

你可以使表空间进入脱机状态，以使它暂时不可被访问和使用，数据库的其他表空间仍然可以正常访问。你可以使已脱机的表空间重新进入联机状态，以使它可以再次被访问。

- 使表空间脱机

  使一个表空间进入脱机状态，以使它暂时不可被访问和使用。

- 使表空间联机

  使一个表空间进入联机状态，以使它可以再次被访问。这是表空间的初始状态和通常状态。

3.4.3.1　使表空间脱机

使表空间脱机以使其中的数据不可被访问。只有表空间属主或超级用户具备执行该操作的权限。

你可能希望表空间脱机的原因有：

- 使数据库的一部分数据不可用，同时保持其余部分可以正常访问。

- 在某应用程序维护和升级期间，保护其所属的数据不被更改和访问。

使一个表空间脱机的方式：

- 执行 SQL 语句 ALTER TABLESPACE，其中包含 OFFLINE 子句。

不能使系统表空间 sys_default 和 sys_global 进入脱机模式。

在使表空间脱机之前，应小心那些已将默认表空间设置为该表空间的用户，为其变更默认表空间，否则这些用户可能会无法正常的访问数据。

例如，使用以下语句使表空间脱机：

```
ALTER TABLESPACE tablesp OFFLINE;
```

3.4.3.2　使表空间联机

使表空间联机，使其中的数据可以被正常访问。联机状态是表空间的初始状态和通常状态，以便用户正常的访问表空间中的数据。

使一个表空间联机的方式：

- 执行 SQL 语句 ALTER TABLESPACE，其中包含 ONLINE 子句。

例如，使用以下语句使表空间脱机：

```
ALTER TABLESPACE tablesp ONLINE;
```

3.4.4　使用只读表空间

你可以将表空间设置为只读模式，以防止其中存储的数据被更改。

- 关于只读表空间

只读表空间是被设置为只读模式的表空间，其中的数据不能被更改。

- 使表空间只读

使用包含 READ ONLY 子句的 ALTER TABLESPACE 命令以使表空间进入只读模式。

- 使只读表空间可读写

使用包含 READ WRITE 子句的 ALTER TABLESPACE 命令以使表空间进入读写模式。

3.4.4.1　关于只读表空间

将表空间设置为只读模式，以使表空间中的数据只能被读取，不能被更新和写入。

只读表空间的主要目的是消除对数据库大量静态数据执行备份和恢复的需要。只读表空间还提供了一种保护历史数据不被篡改的方法。将表空间设置为只读可防止用户对表空间中的所有表进行更新，而与用户的更新权限无关。

只读表空间中的对象，如表、索引等，不能被创建、删除、更改定义。目前不支持将只读表空间中的数据放在只读设备，如 CD-ROM 等上存储。

3.4.4.2　使表空间只读

通过 ALTER TABLESPACE 命令可以将表空间设置为只读模式。所有表空间被初始化时都是可读写的。只有表空间属主或超级用户具备执行该操作的权限。

使一个表空间只读的方式：

- 执行 SQL 语句 ALTER TABLESPACE，其中包含 READ ONLY 子句。

例如，使用以下语句使表空间只读：

```
ALTER TABLESPACE tablesp READ ONLY;
```

3.4.4.3　使只读表空间可读写

使只读表空间置为可读写模式，以接受对数据文件的写入操作。

使一个表空间可读写的方式：

- 执行 SQL 语句 ALTER TABLESPACE，其中包含 READ WRITE 子句。

例如，使用以下语句使表空间可读写：

```
ALTER TABLESPACE tablesp  READ WRITE;
```

3.4.5　修改和维护表空间

你可以修改表空间的参数和属主等信息。

只有表空间属主或超级用户具备执行该操作的权限。

修改表空间的参数的方式：

- 执行 SQL 语句 ALTER TABLESPACE，其中包含 SET tablespace_option = value 子句。

例如，使用以下语句修改表空间的参数：

```
ALTER TABLESPACE tablesp SET  tablespace_option = value;
```

合法的 tablespace_option 有 seq_page_cost、random_page_cost 和 effective_io_concurrency。他们功能的介绍详见：ALTER TABLESPACE(Oracle)、ALTER TABLESPACE(MySQL)、ALTER TABLESPACE(SQLServer)。

修改表空间的属主的方式：

- 执行 SQL 语句 ALTER TABLESPACE，其中包含 OWNER TO 子句。

例如，使用以下语句修改表空间的属主：

```
ALTER TABLESPACE tablesp OWNER TO user;
```

3.4.6　重命名表空间

你可以修改表空间的名称。只有表空间属主或超级用户具备执行该操作的权限。

重命名表空间的方式：

- 执行 SQL 语句 ALTER TABLESPACE，其中包含 RENAME TO 子句。

例如，使用以下语句重命名表空间：

```
ALTER TABLESPACE tablesp RENAME TO newtablesp;
```

3.4.7　删除表空间

如果不再需要表空间，你可以删除它。在一个表空间能被删除前，其中必须没有任何数据库对象。只有表空间属主或超级用户具备执行该操作的权限。

系统表空间 sys_default 和 sys_global 不能被删除。正在被用作临时表空间的表空间不能被删除，因为有临时文件存在其中。

删除表空间的方式：

- 执行 SQL 语句 DROP TABLESPACE。

例如，使用以下语句删除表空间：

```
DROP TABLESPACE tablesp;
```

3.4.8　查看表空间相关信息

以下系统表和视图提供了表空间有关的相关信息。

视图	描述
SYS_TABLESPACE	记录所有表空间信息的系统表
SYS_DATABASE	查询所有数据库使用的默认表空间

-续表-

视图	描述
DBA_TABLESPACE、USER_TABLESPACE	显示所有表空间信息的易阅读系统视图

3.5　数据文件和临时文件管理

在 KingbaseES 中，数据文件的创建、命名和删除均由数据库内部进行管理，这些操作对用户是透明的。但你仍可以依照以下建议管理数据文件和临时文件。

备注:

临时文件是一种特殊的数据文件，用于存储临时关系的数据。临时文件所属的表空间由参数 temp_tablespaces 控制。

3.5.1　数据文件的管理指南

供您参考的管理数据文件的指南，包含了管理数据文件的一些建议和注意事项。

- 关于数据文件

数据文件是一系列文件系统上的物理文件，用于存储数据库中的逻辑数据。在 KingbaseES 中的数据文件不需要显式创建，而是随着表和索引等对象操作而自动创建和维护。

- 选择适当的文件位置

数据文件的位置取决于其所属的表空间。为每个表指定合适的表空间就可以为数据文件选择合适的物理位置，合理地利用硬件资源。

- 将数据文件和重做日志文件分开存储

数据文件和重做日志文件不应该存放在同一个磁盘驱动器上。如果数据文件和重做日志文件在同一个磁盘驱动器上，那么在该磁盘驱动器出现故障时，我们将无法通过日志重做而恢复到正确的数据。

3.5.1.1　关于数据文件

数据文件是存储数据库中逻辑数据的物理文件。KingbaseES 为每个数据库对象，如表、索引或视图，生成至少一个数据文件存储其中的数据。对于普通关系，这些文件会以表或索引的 filenode 号命名。对于临时关系，文件名的形式为 tBBB_FFF，其中 BBB 是创建该文件的后台会话的后台 ID，FFF 是文件节点号。此外关系中的数据超过

1GB 之后，该关系会被划分成 1G 大小的段，每个段是一个单独的数据文件，第一个段的文件名和文件节点相同；随后的段被命名为 filenode.1、filenode.2 等等。

信息:

《系统原理》中的 存储结构。

3.5.1.2　选择适当的文件位置

数据文件的位置取决于其所属的表空间。对于普通关系，数据文件会被存放在该关系所属的表空间中。对于临时关系，临时文件会被存放在 temp_tablespaces 指定的表空间中。为每个表指定合适的表空间就可以为数据文件选择合适的物理位置，合理地利用硬件资源。

例如，如果有多个磁盘驱动器可用于存储数据，每个磁盘驱动器上创建了不同的表空间，请考虑将可能冲突访问的数据文件放在不同的磁盘上，这样用户执行查询时，两个磁盘驱动器可以同时工作，同时检索数据。

3.5.1.3　将数据文件和重做日志文件分开存储

数据文件和重做日志文件不应该存放在同一个磁盘驱动器上。如果数据文件和重做日志文件在同一个磁盘驱动器上，那么在该磁盘驱动器出现故障时，我们将无法通过日志重做而恢复到正确的数据，而只能通过备份文件进行恢复。

如果我们的重做日志文件有多份副本，那么丢失所有副本的概率很低，这种情况我们才可以将重做日志文件和数据文件放在相同的磁盘驱动器上。

第4章　内存管理

管理内存涉及维护和变更数据库实例中各项内存结构的大小，以适应数据库的相关需要。

- 关于内存管理

需要被管理的内存结构是指系统全局区（简写为 SGA）和实例进程全局区（简写为 PGA）。KingbaseES 数据库支持以配置参数的形式设置内存区域的大小。

- 内存结构概述

了解 KingbaseES 数据库基础的内存结构。

- 手动配置内存

当你希望对单个内存组件的大小进行直接的控制时，可以手动配置和调整内存组件。

4.1　关于内存管理

需要被管理的内存结构是指系统全局区（简写为 SGA）和实例进程全局区（简写为实例 PGA）。目前 KingbaseES 不支持自动内存管理，仅支持手动内存管理，数据库提供一系列配置参数，以支持对每个内存区域的大小进行直接的控制。

4.2　内存结构概述

了解 KingbaseES 数据库基础的内存结构。KingbaseES 数据库中，相关的基本内存结构包括：

- 系统全局区（简写为 SGA）SGA 是一组共享内存结构，其中包含一个 KingbaseES 数据库实例的数据和控制信息。SGA 由所有服务器和后台进程共享。存储在 SGA 中的数据包括数据块缓冲区和重做日志缓冲区等。
- 进程全局区（简写为 PGA）PGA 是包含单个服务进程的数据和控制信息的内存区域。它是在启动服务进程时创建的非共享内存。只有该服务进程可以对本 PGA 进行访问。每个服务进程都有一个 PGA，每个后台进程也会有自己的 PGA。一个实例中所有 PGA 的总和称为实例 PGA。存储在 PGA 中的数据包括排序和哈希操作可使用的工作内存等。

4.3　手动配置内存

当你希望对单个内存组件的大小进行直接的控制时，可以手动配置和调整内存组件。你可以通过配置参数指定对应区域的大小。

- 系统全局区配置

手动配置和调整系统全局区中的内存结构大小。

- 进程全局区配置

手动配置和调整进程全局区中的内存结构大小。

4.3.1　系统全局区配置

手动配置和调整系统全局区中的内存结构大小。

- 数据页面缓存

在共享内存中缓存的数据页面的大小，由 shared_buffers 参数控制其大小。该参数会影响数据库性能。配置的缓存越大，则在内存中保存的数据页面就越多，这通常会减少磁盘读取和写入的次数，并因此提高性能。然而过大的数据页面缓存也可能会占用过多的系统内存并导致内存分页或交换。

出于 IO 资源优化的角度，建议将 shared_buffers 配置为操作系统总内存的一半，根据特定需求浮动，但一般不建议设置超过操作系统总内存的 80%。

采用以下方式修改数据页面缓存大小：

```
ALTER SYSTEM SET shared_buffers = '1GB';
```

重启数据库后生效。

采用以下方式查看数据页面缓存大小：

```
SHOW shared_buffers;
```

- 日志页面缓存

数据库操作产生的重做日志内容会存放在日志页面缓存中，并在需要时由日志写进程和服务进程刷写到磁盘。参数 wal_buffers 可以设置日志页面缓存大小。如果每个事务对数据页面修改的数据量较大，占满了日志页面缓存而不得不提前刷写日志，将导致一定的 IO 资源浪费，此时可以适当增大 wal_buffers 的值，以降低日志刷写磁盘的频率。由于日志总是需要在事务提交时和检查点刷盘，因此过大的 wal_buffers 并没有实际意义。

采用以下方式修改日志页面缓存大小：

```
ALTER SYSTEM SET wal_buffers = '4MB';
```

重启数据库后生效。

采用以下方式查看日志页面缓存大小：

```
SHOW wal_buffers;
```

- 锁缓存

数据库事务所需的并发控制机制的组成部分之一是锁，而锁的控制信息则存在于系统全局区 SGA 中。锁缓存的大小由数据库内部决定，不可配置，但是参数 max_locks_per_transaction 可以影响到锁缓存的大小。参数 max_locks_per_transaction 代表了每个事务最大可持有的锁数量，缺省为 64。参数 max_locks_per_transaction 越大，锁缓存所占有的空间就越大。

通常锁缓存占用内存在系统中占比不大，不会成为系统的瓶颈，不需要手动配置。

4.3.2　进程全局区配置

手动配置和调整进程全局区中的内存结构大小。

- 临时页面缓存

在进程私有内存中用于缓存临时表的数据页面的大小，由 temp_buffers 参数控制其大小。

采用以下方式修改临时页面缓存大小：

```
ALTER SYSTEM SET temp_buffers = '8MB';
```

重启数据库后生效。

采用以下方式查看临时页面缓存大小：

```
SHOW temp_buffers;
```

- 工作内存

服务器对元组进行排序或者连接运算时，需要在 PGA 中缓存临时结果集数据，这部分内存称为工作内存。如果工作内存空间不足，数据库会转由临时文件存储这部分数据。参数 work_mem 设置每个服务进程的工作内存大小。工作内存只在正在进行排序或者连接运算的服务进程中被分配，并会随着运算结束而释放。

采用以下方式修改工作内存大小：

```
ALTER SYSTEM SET work_mem = '4MB';
```

重启数据库后生效。

采用以下方式查看工作内存大小：

```
SHOW work_mem;
```

- 维护工作内存

在维护性操作 (比如 VACUUM, CREATE INDEX, ALTER TABLE ADD FOREIGN KEY 等) 中使用的最大的内存空间。参数 maintenance_work_mem 设置维护工作内存的大小。默认是 64MB。较大的维护工作内存可以有效改进清理和恢复数据的速度。

采用以下方式修改维护工作内存大小：

```
ALTER SYSTEM SET maintenance_work_mem = '64MB';
```

重启数据库后生效。

采用以下方式查看维护工作内存大小：

```
SHOW maintenance_work_mem;
```

- 临时分配的内存

数据库在执行 SQL 过程中，用于临时存放控制信息及数据所使用的内存。此部分内存动态分配和释放，并受到内存上下文管理。此部分内存大小不可配置。

第5章　进程管理

5.1　关于独立服务进程和共享服务进程

KingbaseES 数据库使用客户端/服务器的模型，服务端创建服务进程处理客户端连接。独立服务进程指一个服务进程只处理一个客户端的请求；共享服务进程指一个服务进程处理多个客户端的请求。

5.2　管理 KingbaseES 的独立服务进程

KingbaseES 数据库使用独立服务进程模式。控制服务进程数量、资源使用的配置参数参见：

- 连接和认证
- 资源消耗

5.3　关于 KingbaseES 的后台进程

为了最大化客户端请求的吞吐量和响应时间，KingbaseES 使用后台进程完成一些工作以避免服务进程因为这些工作被拖慢。

基础的后台进程和在特定时机或使用一些特性时会有的额外后台进程参见：后台进程

5.4　终止会话

管理数据库有时需要终止会话，例如终止占用封锁或系统资源过多的会话。

5.4.1　指定需要终止的会话

终止会话需要指定会话的 pid。可以通过视图 sys_stat_activity 查找特定会话的 pid。

示例，查找用户为 kingbase 的会话 pid 和会话状态：

```
SELECT pid, state FROM sys_stat_activity WHERE usename = 'kingbase';
  pid  | state
-------+--------
 26212 | active
 26292 | active
(4 rows)
```

5.4.2　终止会话

终止会话将结束会话并断开客户端到服务端的连接。会话正在执行的事务会由于连接断开回滚。使用 sys_terminate_backend(integer) 函数终止会话，参数为会话的 pid。

示例，终止 pid 为 26212 的会话：

```
SELECT sys_terminate_backend(26212);
 sys_terminate_backend
-----------------------
 t
(1 row)
```

5.4.3　取消会话执行的 SQL 语句

某一会话执行的 SQL 语句占用过多资源影响业务时，可以通过取消会话执行的 SQL 代替终止会话。会话执行 SQL 所在的事务将回滚。使用 sys_cancel_backend(integer) 函数取消会话执行的 SQL 语句，参数为会话的 pid。

示例，取消 pid 为 26212 的会话执行的 SQL 语句：

```
SELECT sys_cancel_backend(26212);
 sys_cancel_backend
--------------------
 t
(1 row)
```

5.5　进程和会话视图

可以通过以下视图获取进程和会话信息：

`sys_catalog.sys_stat_activity`
当前系统中进程/会话的信息。

`sys.sys_session`
当前系统中的会话信息。

`sys.v$session`
兼容 Oracle 的 v*session*[*v*session](/docs/reference/database/系统视图/oracle_views/dynamic_performance/vsession)。

可以通过以上视图中 pid/sess_id/sid 作为唯一标志和其他视图关联获取进程/会话信息。

例如和 sys_locks 视图关联获取进程/会话的封锁信息。

第6章　用户管理

- 数据库在使用之前设置安全策略的重要性

制定安全策略非常重要，安全策略的建立是保护数据库免于遭受意外、恶意数据破坏或者数据库基础结构损坏的有效方法。

- 管理用户

若要连接数据库，就必须在数据库中注册有效的用户名。

- 用户的权限和角色

KingbaseES 拥有多种权限的角色，是为了控制用户对数据的访问以及区分可执行的 SQL 语句的类型。

- 审计数据库活动

您可以监控和记录包括管理员在内的选定用户的数据库操作，您也可以监控系统级的操作，以及在单个数据库对象上的操作，我们统称该类操作为数据库审计。

- 预定义的账户

KingbaseES 数据库包含多种预定义的用户帐户。

6.1　数据库在使用之前设置安全策略的重要性

为每个数据库制定安全策略是重要且有必要的。安全策略是为了保护数据库免于遭受意外、恶意数据破坏或者数据库基础结构损坏的有效方法。每个数据库都可以有一个安全管理员，负责实施和维护数据库安全策略。如果数据库系统规模很小，则数据库管理员可以承担安全管理员的职责。相反，如果数据库系统规模很大，则需要指定一个人或者一组人担任安全管理员的职责。

信息:

《安全指南》，获取有关为数据库建立安全策略的信息。

6.2　管理用户

若要连接数据库，就必须在数据库中注册有效的用户名。当您创建数据库用户（帐户）时，指定用户以下属性：

- 用户名
- 身份验证方法
- 缺省表空间

信息:

《安全指南》，了解如何创建和管理用户。

6.3　用户的权限和角色

KingbaseES 拥有多种权限的角色，是为了控制用户对数据的访问以及区分可执行的 SQL 语句的类型。

下表描述了三种类型的特权和角色：

类型	描述
系统权限	通常仅由管理员授予的系统定义特权。这些权限允许用户执行特定的数据库操作。
对象权限	一种系统定义的权限，用于控制对特定对象的访问。
角色	权限和角色的集合，存在部分系统定义的角色，但大多数由管理员创建。角色可以将权限和其他角色组合在一起，方便用户拥有多种多样权限的角色。

权限和角色可由被授予权限的用户授予其他用户。角色和特权的授予是从管理员级别开始的。在创建数据库后，用户 system 可以向其他用户授予权限和角色，还可以向用户授予向其他用户授权的权限。

信息:

《安全指南》，了获取更多关于如何管理用户权限和角色的信息。

6.4　审计数据库活动

您可以监控和记录包括管理员在内的选定用户的数据库操作，您也可以监控系统级的操作，以及在单个数据库对象上的操作，我们统称该类操作为数据库审计。

KingbaseES 数据库通过插件 sysaudit 实现审计功能，您可以通过 SQL 语句创建自定义的审计策略。

信息:

《安全指南》，了解有关数据库审计的更多信息。

6.5　预定义的账户

KingbaseES 数据库安装部署后，会默认创建多种预定义的用户，有以下三种类型：

- 数据库管理员（system）
- 安全管理员（sso）
- 审计管理员（sao）

三个用户将管理权限三权分立，在系统初始化时指定用户名和密码。

在 KingbaseES 数据库中，可通过查询系统表或系统视图查看用户账户信息。用户账户对应的视图 sys_shadow 和 sys_user，如下所示：

```
TEST=# select * from sys_user;
usename|usesysid|usecreatedb|usesuper|userepl|usebypassrls|passwd  |valuntil|
useconfig
-------+--------+-----------+--------+-------+------------+--------+--------+
system |     10 | t         | t      | t     | t          |********|        |
sso    |      8 | f         | f      | f     | f          |********|        |
sao    |      9 | f         | f      | f     | f          |********|        |
(3 行记录)
```

信息:

《安全指南》将提供有关 KingbaseES 数据库所有预定义帐户的信息。

第7章　监控管理

定期监视数据库的操作很重要。这样做不仅可以通知您尚未引起注意的错误，还可以让您更好地了解数据库的正常操作。反过来，熟悉正常行为可以帮助您识别何时出现问题。

- 监控错误和警报

  您可以监控数据库错误和警报以预防、检测和解决问题。

- 监控数据库性能

  监控性能包括监控锁和等待事件以及查询一组数据字典视图。

7.1　监控错误和警报

可以监控数据库错误和警报以预防、检测和解决问题。对于数据库运行过程中的错误，除了人工去查看数据库运行过程中产生的日志文件外，也可以使用 kbbadger 工具自动分析。

7.1.1　使用 kbbadger

kbbadger 是一个命令行工具，可以分析 KingbaseES 在运行过程中产生的日志文件，并以图形化报告的形式展示数据库性能指标。

生成日志的内容可以通过 GUC 参数比如 log_min_duration_statement 等进行控制。主要分析的统计有：连接、会话、检查点、临时文件、回收（Vacuum）、锁、查询和事件等。

支持的日志文件的格式包括：syslog，stderr，csv log 和 json log 等。生成的所有图表都是可缩放的并且可单独下载为 PNG 文件。另外，在生成的报告中 SQL 查询将突出显示。

7.1.2　基本能力

kbbadger 生成的报告中包含的信息有：

- 总体统计

- 占用时间最多的查询
- 最常见的查询
- 最常见的错误
- 查询时间直方图
- 会话时间直方图
- 顶级查询的用户
- 顶级查询的应用程序
- 最多取消的查询
- 最耗时的准备/绑定查询

以下是主要耗时的查询的示例报告：

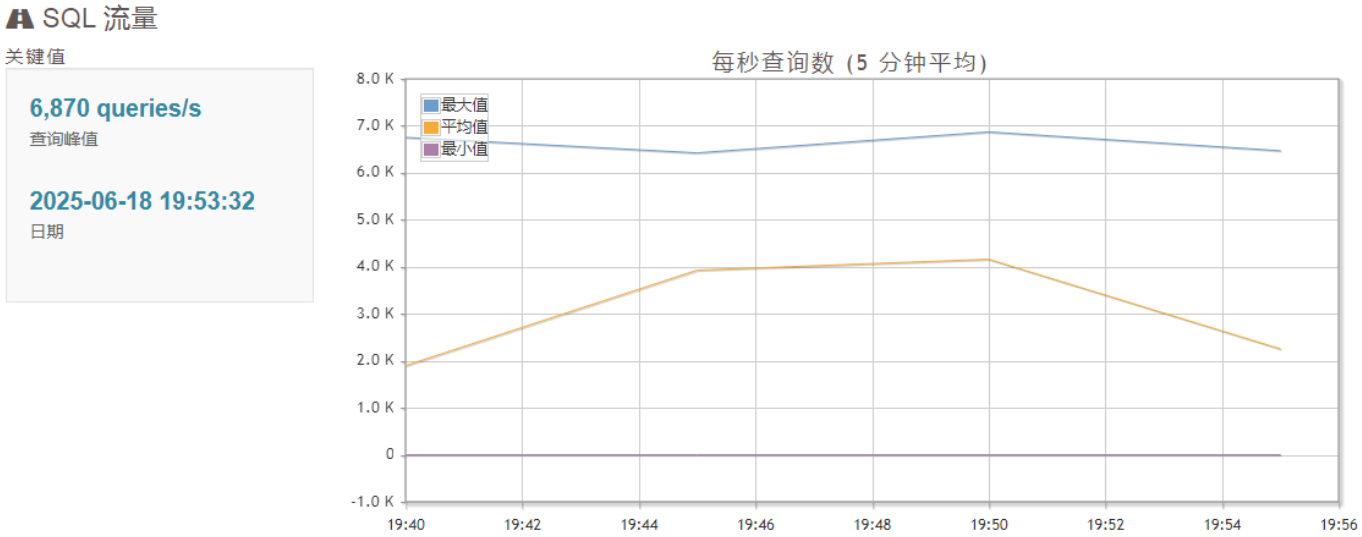
主要耗时查询

排名	总持续时间	执行次数	最小持续时间	最大持续时间	平均持续时间	查询
1	1h39m19s	69,553 细节	0ms	1s830ms	85ms	SELECT d_tax, d_next_o_id, ctid, xmin FROM bmsql_district WHERE d_w_id = '' AND d_id = '' FOR UPDATE; 示例 执行用户
2	1h36m20s	66,439 细节	0ms	2s459ms	87ms	UPDATE bmsql_district SET d_ytd = d_ytd + '' WHERE d_w_id = '' AND d_id = ''; 示例 执行用户
3	27m29s	66,439 细节	0ms	929ms	24ms	UPDATE bmsql_warehouse SET w_ytd = w_ytd + '' WHERE w_id = ''; 示例 执行用户
4	11m47s	64,963 细节	0ms	788ms	10ms	DELETE FROM bmsql_new_order WHERE no_w_id = '' AND no_d_id = '' AND no_o_id = ''; 示例 执行用户
5	5m34s	695,356 细节	0ms	886ms	0ms	SELECT s_quantity, s_data, s_dist_01, s_dist_02, s_dist_03, s_dist_04, s_dist_05, s_dist_06, s_dist_07, s_dist_08, s_dist_09, s_dist_10, ctid, xmin FROM bmsql_stock WHERE s_w_id = '' AND s_i_id = '' FOR UPDATE; 示例 执行用户

报告也会提供每小时的统计图表，其内容包括：

- SQL 查询统计信息
- 临时文件统计
- 检查点统计
- 自动 vacuum 和自动分析统计
- 已取消查询
- 错误事件（死机、致命、错误和警告）
- 错误的类分布

以下是按 5 分钟粒度分割的 SQL 流量统计的示例：



7.1.3　使用示例

kbbadger 依赖数据库日志系统，需要在 kingbase.conf 中配置好相关参数：

```
# 启用日志记录
log_min_duration_statement = 0
# 用于 kbbadger 的默认解析格式
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,remote=%h '
# 日志记录的信息
log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits = on
log_temp_files = 0
log_autovacuum_min_duration = 0
log_error_verbosity = default
```

运行 TPCC 大约 23 分钟，产生了 6.3 GB 日志：

```
$ du -sh sys_log
6.3G    sys_log
```

生成 kbbadger 报告：

```
$ kbbadger ./sys_log/kingbase* -f stderr -J 12 -j 32
[=======================>] Parsed 6704882201 bytes of 6704882201 (100.00%), queries: 7722936, events: 803
LOG: Ok, generating html report...
```

kbbadger 会在当前目录下产生 out.html 报告。

7.1.4 适用场景

kbbadger 实用工具基于数据库的日志系统，在各环境下都可以直接使用。生成的报告以网页方式呈现，图表丰富且易于阅读。

在无法使用其他性能调优工具的情况下，比如因为 KingbaseES 版本比较低没有 KWR 的场景，可以使用 kbbadger 来分析大部分性能问题。

因为日志系统要采集的信息比较大，所以 sys_log 目录磁盘空间消耗会比较多。以 TPCC 测试场景 100 数据仓库 100 并发连接为例，23 分钟内产生了 6.3 GB 的日志文件。

另外要写大量的日志到日志文件，也会对数据库服务器的性能造成较大影响，分析日志的时间也可能相对较长。在性能分析完成后，需要关闭相关日志跟踪。

7.2 监控数据库性能

监控性能包括监控锁和等待事件以及查询一组数据字典视图。

- 监控锁

锁是防止访问同一资源的事务之间的破坏性交互的机制。资源可以是用户对象，例如表和行，也可以是用户不可见的系统对象，例如内存中的共享数据结构和数据字典行。

- 监控等待事件

等待事件是由服务器进程递增的统计信息，表明它必须等待事件完成才能继续处理。会话可能出于多种原因等待，包括等待更多输入、等待操作系统完成诸如磁盘写入之类的服务，或者它可能等待锁定或闩锁。

- 性能监控视图

您可以查询一组性能视图来监控 KingbaseES 数据库实例。

7.2.1 监控锁

锁是防止访问同一资源的事务之间进行破坏性交互的机制。资源可以是用户对象，例如表和行，也可以是用户不可见的系统对象，例如内存中的共享数据结构和数据字典行。

KingbaseES 数据库在执行 SQL 语句时会自动获取并管理必要的锁，因此您无需关心这些细节。但是，数据库还允许您手动锁定数据。

当两个或更多用户正在等待彼此锁定的数据时，可能会发生死锁。死锁会阻止某些事务继续工作。KingbaseES 数据库自动检测死锁情况并通过回滚死锁中涉及的一条语句来解决它们，从而释放一组冲突的行锁。

KingbaseES 数据库旨在避免死锁，它们并不常见。大多数情况下，它们发生在事务显式覆盖数据库的默认锁定时。死锁会影响数据库的性能，因此 KingbaseES 提供了一些脚本和视图，使您能够监控锁。

7.2.1.1　使用 sys_locks

所有活动事务持有的监控锁的基本配置即为系统视图 sys_locks。这个视图为每个可加锁的对象、已请求的锁模式和相关事务包含一行记录。

```
test=# select * from sys_locks;
-[ RECORD 1 ]------+----------------
locktype           | relation
database           | 16384
relation           | 12168
page               |
tuple              |
virtualxid         |
transactionid      |
classid            |
objid              |
objsubid           |
virtualtransaction | 3/744
pid                | 224515
mode               | AccessShareLock
granted            | t
fastpath           | t
```

7.2.1.2　监控死锁

显式锁定的使用可能会增加死锁的可能性，死锁是指两个（或多个）事务相互持有对方想要的锁。

例如，如果事务 1 在表 A 上获得一个排他锁，同时试图获取一个在表 B 上的排他锁，而事务 2 已经持有表 B 的排他锁，同时却正在请求表 A 上的一个排他锁，那么两个事务就都不能进行下去。KingbaseES 能够自动检测到死锁情况并且会通过中断其中一个事务从而允许其它事务完成来解决这个问题（具体哪个事务会被中断是很难预测的，而且也不应该依靠这样的预测）。

要注意死锁也可能会作为行级锁的结果而发生（并且因此，它们即使在没有使用显式锁定的情况下也会发生)。考虑如下情况，两个并发事务在修改一个表。第一个事务执行：

```
UPDATE accounts SET balance = balance + 100.00 WHERE acctnum = 11111;
```

这样就在指定帐号的行上获得了一个行级锁。然后，第二个事务执行：

```
UPDATE accounts SET balance = balance + 100.00 WHERE acctnum = 22222;
UPDATE accounts SET balance = balance - 100.00 WHERE acctnum = 11111;
```

第一个 UPDATE 语句成功地在指定行上获得了一个行级锁，因此它成功更新了该行。但是第二个 UPDATE 语句发现它试图更新的行已经被锁住了，因此它等待持有该锁的事务结束。事务二现在就在等待事务一结束，然后再继续执行。现在，事务一执行：

```
UPDATE accounts SET balance = balance - 100.00 WHERE acctnum = 22222;
```

事务一试图在指定行上获得一个行级锁，但是它得不到：事务二已经持有了这样的锁。所以它要等待事务二完成。因此，事务一被事务二阻塞，而事务二也被事务一阻塞：一个死锁。KingbaseES 将检测这样的情况并中断其中一个事务。

7.2.2　监控等待事件

可以查看系统视图 sys_stat_activity 来进行查看每个连接上的等待事件。

备注:

当前执行的查询语句和等待事件状态需要在开启 track_activities 参数的情况下才可以查看。开启该参数会带来一定的性能损耗。

---

sys_stat_activity 视图结构如下：

表 7.2.1: 视图结构

字段名	类型	说明
datid	oid	数据库 OID
datname	name	数据库名称
pid	integer	服务于这个连接的进程 ID
usesysid	oid	用户 ID
usename	name	用户名
application_name	text	应用名称
client_addr	inet	客户端地址
client_hostname	text	客户端主机名
client_port	integer	客户端端口号
backend_start	timestamp with time zone	该连接的启动时间
xact_start	timestamp with time zone	当前事务开始时间
query_start	timestamp with time zone	当前查询开始时间
state_change	timestamp with time zone	当前状态改变时间

-续表-

字段名	类型	说明
wait_event_type	text	当前等待事件的类型
wait_event	text	当前等待事件
state	text	当前的状态
backend_xid	xid	这个后端的顶层事务标识符
backend_xmin	xid	这个进程的当前事务被启动的时间
query	text	当前查询语句
backend_type	text	后端进程的类型

其中，state 为当前连接的状态，其可能的值主要有：

- active：后端正在执行一个查询。
- idle：后端正在等待一个新的客户端命令。
- idle in transaction：后端在一个事务中，但是当前没有正在执行一个查询。
- idle in transaction (aborted)：这个状态与 idle in transaction 相似，不过在该事务中的一个语句导致了一个错误。
- fastpath function call：后端正在执行一个 fast-path 函数。
- disabled：如果在这个后端中 track_activities 被禁用，则报告这个状态。

备注:

如果一个查询或者事务执行的时间过长而一直没有结束，很有可能会拖累整个系统的性能表现，需要具体分析问题的原因并采取合适的办法。

7.2.2.1　等待事件

等待事件是数据库内部记录的一种统计信息，该信息出现表明数据库的服务进程必须在等待事件完成后才能继续处理。等待事件揭示了可能影响性能的各种问题症状，比如缓冲区中争用、锁争用等。

sys_stat_activity 里记录的等待事件是瞬时信息，没有对等待事件的时间进行累计，所以量化等待事件上存在一些问题。用户可以考虑多次人工采样来收集等待事件信息，如果在等待的连接较多，则需要等待事件对系统性能造成了较大影响，需要做具体分析。

备注:

当前执行的查询语句和等待事件状态需要在开启 track_activities 参数的情况下才可以查看。开启该参数会带来一定的性能损耗。

sys_stat_activity 视图中，wait_event_type 和 wait_event 字段记录了等待时间相关的内容。当一个会话处于等待状态时，wait_event 与 wait_event_type 两列非空，表示会话正在等待的事件和等待事件类型。

KingbaseES 的等待事件分类包括：

- LightLock：轻量级锁
- Lock：重量级锁
- BufferPin：数据缓冲区等待
- Activity：后台辅助进程活动等待
- Client：客户端等待
- Extension：扩展插件等待
- IPC：进程间通讯等待
- TimeOut：超时等待
- IO：IO 等待

每一个等待事件的名称和产生的原因，可以参考《KingbaseES 数据库参考手册》中的 等待事件。

查询等待事件示例：

```
test=# select * from sys_stat_activity;
-[ RECORD 1 ]----+--------------------------------
datid            |
datname          |
pid              | 236705
usesysid         |
usename          |
application_name |
client_addr      |
client_hostname  |
client_port      |
backend_start    | 2022-05-27 11:23:30.833022+08
xact_start       |
query_start      |
state_change     |
wait_event_type  | Activity
wait_event       | AutoVacuumMain
state            |
```

```
backend_xid      |
backend_xmin     |
query            |
backend_type     | autovacuum launcher
```

7.2.3　性能监控视图

您可以通过查询一组性能视图来监控 KingbaseES 数据库实例的运行状态。

在使用性能视图的数据时，你必须了解这些信息并非是实时更新的。每个独立的服务器进程只在进入闲置状态之前才向统计收集器传送新的统计计数；因此正在进行的查询或事务并不影响显示出来的总数。同样，收集器本身也最多每 500 毫秒发送一次新的报告。因此显示的信息总是落后于实际活动。但是由 track_activities 收集的当前查询信息总是最新的。

当一个服务器进程被要求显示任何这些统计信息时，它首先取得收集器进程最近发出的报告并且接着为所有统计视图和函数使用这个快照，直到它的当前事务的结尾。因此只要你继续当前事务，统计数据将会一直显示静态信息。相似地，当任何关于所有会话的当前查询的信息在一个事务中第一次被请求时，这样的信息将被收集。并且在整个事务期间将显示相同的信息。这是一种特性而非缺陷，因为它允许你在该统计信息上执行多个查询并且关联结果而不用担心那些数字会在你不知情的情况下改变。但是如果你希望用每个查询都看到新结果，要确保在任何事务块之外做那些查询。或者，你可以调用 sys_stat_clear_snapshot()，那将丢弃当前事务的统计快照（如果有）。下一次对统计性信息的使用将导致获取一个新的快照。

一个事务也可以在视图 sys_stat_xact_all_tables、sys_stat_xact_sys_tables、sys_stat_xact_user_tables 和 sys_stat_xact_user_functions 中看到它自己的统计信息（还没有被传送给收集器）。这些数字并不像上面所述的那样行动，相反它们在事务期间持续被更新。

关于每个性能视图的描述，可以参考《数据库参考》中的“系统视图 > KingbaseES 系统视图 > 统计视图”章节。

实时刷新的性能视图：

表 7.2.2: 实时刷新的性能视图

视图名称	描述
sys_stat_activity	每个服务器进程一行，显示与那个进程的当前活动相关的信息，例如状态和当前查询。
sys_stat_replication	每一个 WAL 发送进程一行，显示有关到该发送进程连接的后备服务器的复制的统计信息。
sys_stat_wal_receiver	只有一行，显示来自 WAL 接收器所连接服务器的有关该接收器的统计信息。
sys_stat_subscription	每个订阅至少一行，显示有关该订阅的工作者的信息。

-续表-

视图名称	描述
sys_stat_ssl	每个连接（常规的或者复制）一行，显示在这个连接上使用的 SSL 的信息。
sys_stat_gssapi	每个连接（常规和复制）有一行，显示关于 GSSAPI 验证和加密的信息。
sys_stat_progress_create_index	每个后台运行 CREATE INDEX 或 REINDEX 的后端都有一行，显示当前的进度。
sys_stat_progress_vacuum	每个运行着 VACUUM 的后端（包括 autovacuum 工作者进程）一行，显示当前的进度。
sys_stat_progress_cluster	每个运行着 CLUSTER 或 VACUUM FULL 的后端一行，显示当前进度。

非实时刷新的性能视图：

表 7.2.3: 非实时刷新的性能视图

视图名称	描述
sys_stat_archiver	只有一行，显示有关 WAL 归档进程活动的统计信息。
sys_stat_bgwriter	只有一行，显示有关后台写进程的活动的统计信息。
sys_stat_database	每个数据库一行，显示数据库范围的统计信息。
sys_stat_database_conflicts	每个数据库一行，显示数据库范围的统计信息，这些信息的内容是关于由于与后备服务器的恢复过程发生冲突而被取消的查询。
sys_stat_all_tables	当前数据库中每个表一行，显示有关访问指定表的统计信息。
sys_stat_sys_tables	和 sys_stat_all_tables 一样，但只显示系统表。
sys_stat_user_tables	和 sys_stat_all_tables 一样，但只显示用户表。
sys_stat_xact_all_tables	和 sys_stat_all_tables 相似，但计数动作只在当前事务内发生（还没有被包括在 sys_stat_all_tables 和相关视图中）。
sys_stat_xact_sys_tables	和 sys_stat_xact_all_tables 一样，但只显示系统表。
sys_stat_xact_user_tables	和 sys_stat_xact_all_tables 一样，但只显示用户表。
sys_stat_all_indexes	当前数据库中的每个索引一行，显示与访问指定索引有关的统计信息。

-续表-

视图名称	描述
sys_stat_sys_indexes	和 sys_stat_all_indexes 一样，但只显示系统表上的索引。
sys_stat_user_indexes	和 sys_stat_all_indexes 一样，但只显示用户表上的索引。
sys_statio_all_tables	当前数据库中的每个表一行，显示有关在指定表上 I/O 的统计信息。
sys_statio_sys_tables	和 sys_statio_all_tables 一样，但只显示系统表。
sys_statio_user_tables	和 sys_statio_all_tables 一样，但只显示用户表。
sys_statio_all_indexes	当前数据库中的每个索引一行，显示与指定索引上的 I/O 有关的统计信息。
sys_statio_sys_indexes	和 sys_statio_all_indexes 一样，但只显示系统表上的索引。
sys_statio_user_indexes	和 sys_statio_all_indexes 一样，但只显示用户表上的索引。
sys_statio_all_sequences	当前数据库中的每个序列一行，显示与指定序列上的 I/O 有关的统计信息。
sys_statio_sys_sequences	和 sys_statio_all_sequences 一样，但只显示系统序列（目前没有定义系统序列，因此这个视图总是为空）。
sys_statio_user_sequences	和 sys_statio_all_sequences 一样，但只显示用户序列。
sys_stat_user_functions	每一个被跟踪的函数一行，显示与执行该函数有关的统计信息。
sys_stat_xact_user_functions	和 sys_stat_user_functions 相似，但是只统计在当前事务期间的调用（还没有被包括在 sys_stat_user_functions 中）。

比如，通过 sys_stat_bgwriter 视图参考后台写和检查点的统计信息：

```
test=# select * from sys_stat_bgwriter;
-[ RECORD 1 ]---------+-----------------------------
checkpoints_timed     | 344
checkpoints_req       | 0
checkpoint_write_time | 3749
checkpoint_sync_time  | 1
buffers_checkpoint    | 37
buffers_clean         | 0
maxwritten_clean      | 0
buffers_backend       | 0
buffers_backend_fsync | 0
buffers_alloc         | 82
```

```
stats_reset           | 2022-05-30 09:35:50.464433+08
```

通过 sys_stat_database 查看每个库的事务和元组统计信息：

```
test=# select * from sys_stat_database;
-[ RECORD 1 ]---------+-----------------------------
datid                 | 16384
datname               | test
numbackends           | 1
xact_commit           | 44930
xact_rollback         | 15
blks_read             | 59142
blks_hit              | 11122493
tup_returned          | 160318834
tup_fetched           | 1491817
tup_inserted          | 1380883
tup_updated           | 7324
tup_deleted           | 35999
conflicts             | 0
temp_files            | 288
temp_bytes            | 2359296
deadlocks             | 0
checksum_failures     |
checksum_last_failure |
blk_read_time         | 8225.142
blk_write_time        | 6.066
stats_reset           | 2022-05-16 18:06:12.851563+08
```

第8章　数据库对象管理

8.1　模式对象的管理

您可以在 KingbaseES 数据库中创建和管理多种类型的模式对象。

- 关于默认模式

用户只要有权限就可以访问所连接数据库中的任何模式中的对象，在创建数据库时，KingbaseES 会默认创建两个模式：SYS_CATALOG 和 PUBLIC。

- 分析表和索引以及聚类

您可以收集模式对象的统计信息、分析统计信息以及验证模式对象。

- 清空表

您可以将一个表中的所有行清空，但表本身保留。

- 启用和禁用触发器

数据库触发器是存储在数据库中并在发生特定情况（如向表中添加行）时激活（“触发”）的过程。

- 管理完整性约束

完整性约束是限制表中一列或多列的值的规则。约束子句可以出现在或语句中，并标识受约束影响的列，并标识约束的条件。CREATE TABLEALTER TABLE

- 重命名模式对象

有多种方法可以重命名对象。

- 管理对象名称解析

SQL 语句中引用的对象名称可以由多个部分组成，以句点分隔。KingbaseES 数据库执行特定操作来解析对象名称。

- 切换到其他模式

使用 SQL 的 ALTER SESSION SET SEARCH_PATH 语句切换到其他模式。

- 显示模式对象的信息

KingbaseES 数据库提供了一个 PL/SQL 包，使您能够确定创建对象的 DDL 以及可用于显示有关模式对象的信息的视图。

8.1.1　关于默认模式

用户只要有权限就可以访问所连接数据库中的任何模式中的对象，在创建数据库时，KingbaseES 会默认创建两个模式：SYS_CATALOG 和 PUBLIC。

一个数据库可以包含一个或多个命名的模式（SCHEMA），一个模式内可以包含多个表。不同的模式中的表名可以相同，例如，schema1 和 schema2 中可以分别包含名称为 mytable 的表，但是同一个模式中的表不能同名。模式类似于操作系统级的目录，但模式不能嵌套。其中 SYS_CATALOG 模式用于存放系统表和所有内置的数据类型、函数和操作符。

当用户没有自己的模式并且其创建或使用数据库对象没有指定模式时，默认使用 PUBLIC 模式。模式通常在以下情况下使用：

- 多个用户使用同一个数据库而不会相互影响。
- 对数据库中的对象进行逻辑分组，更便于管理。
- 各个应用分别使用各自的模式，以避免命名冲突。

信息:

《安全指南》获取更多关于权限管理和模式对象相关信息

8.1.2　分析表和索引以及聚类

您可以收集模式对象的统计信息，分析统计信息，并验证模式对象。

8.1.2.1　关于分析表、索引和聚类

您可以收集关于模式对象的信息并分析这些信息。

分析一个模式对象（表、索引）：

- 收集和管理统计数据
- 验证其存储格式的有效性
- 标识表的迁移行和链接行

8.1.2.2　使用 sys_stat_statement 搜集执行语句的统计信息

sys_stat_statement 是 kingbaseES 系统的一个扩展组件，它提供了所有执行语句的统计信息，可以帮助找出哪种类型的查询很慢以及多久调用一次查询, 帮助您分析对象的 SQL 语句，选择最有效的执行计划。

使用这个模块的执行步骤：

1. 在 kingbase.conf 里添加预加载项：

```
shared_preload_libraries = 'sys_stat_statements'
```

2. 重启数据库服务器
3. 在执行的数据库里创建扩展：

```
test=# create extension sys_stat_statements;
CREATE EXTENSION
```

备注:

从 KingbaseES V8R6 版本开始，sys_stat_statements 插件已经内置化，初始化数据库实例的时候，就会被自动加载。不过该功能默认关闭，需要设置：sys_stat_statements.track = “top”才能正常使用。

信息:

《KingbaseES 数据库性能调优指南 》了解有关 sys_stat_statements 工具的详细描述

8.1.2.3　分析表、索引、聚类和物化视图

ANALYZE 收集数据库中的表的内容统计信息，并且将分析结果存储至系统表 sys_statistic。接下来，查询计划就会使用这些统计信息，以助于决定选择出最有效的查询执行计划。

以下语句分析了该表：emp

```
ANALYZE emp;
```

您可以通过包含 VERBOSE 选项来显示进度消息。语句如下:

```
ANALYZE VERBOSE emp;
```

信息:

《KingbaseES 应用开发指南 》有关 ANALYZE 语句的详细信息

8.1.3　清空表

您可以将一个表中的所有行清空，但表本身保留。例如，某单位的一个包含月度数据的表，在每个月底，必须在对其数据进行归档后清空该表 (清空所有行)。

保留表情况下清空整个表，存在多种方式：

- 使用 DELETE 清空表

  您可以使用 SQL 的 DELETE 语句删除表中的行。

- 使用 DROP 和 CREATE 清空表

  您可以先删除一个表，然后重新创建表的方式变相实现清空表的目的。

- 使用 TRUNCATE 清空表

  可以使用 TRUNCATE 语句删除表中的所有行。

8.1.3.1　使用 DELETE 清空表

您可以使用 SQL 的 DELETE 语句删除表中的行。

例如，下面的语句从 class 表中删除所有行:

```
DELETE FROM class;
```

在使用 DELETE 语句时，如果表中存在许多行，在删除这些行时将消耗大量系统资源。例如，CPU 时间、重做日志空间、撤销表和相关索引的段空间都需要资源。另外，当删除每一行时，也有可能触发触发器。以前分配给生成的空表空间仍然与该对象关联。DELETE 还可以通过 WHERE 条件子句删除满足条件的行，而 TRUNCATE 和 DROP 直接影响整个对象。

信息:

《KingbaseES 应用开发指南 》获取关于 DELETE(Oracle) 、DELETE(MySQL)、DELETE(SQLServer) 语句的信息。

8.1.3.2　使用 DROP 和 CREATE 清空表

您可以先删除一个表，然后重新创建该表的方式变相实现清空表的目的。

例如，以下语句删除并重新创建 class 表:

```
DROP TABLE class;
CREATE TABLE class ( ... );
```

8.1.3.3 使用 TRUNCATE 清空表

可以使用 TRUNCATE 语句删除表中的所有行。例如，下面的语句清空 class 表:

```
TRUNCATE emp;
```

使用 TRUNCATE 语句提供快速移除表中所有行的机制。TRUNCATE 语句不会生成任何撤销信息，会立即提交，并且它是 DDL 语句。TRUNCATE 语句不会影响与被清空的表相关的任何结构 (约束和触发器) 或授权。您可以清空自己模式中的任何表。

当 TRUNCATE 语句从表中删除行时，不会触发与表关联的触发器。此外，如果启用了审计，那么 TRUNCATE 语句不会生成与 DELETE 语句对应的任何审计信息。相反，将为发出的 TRUNCATE 语句生成单个审计记录。

信息:

《KingbaseES 应用开发指南 》获取语法和其他关于 TRUNCATE TABLE(Oracle) 、TRUNCATE TABLE(MySQL)、TRUNCATE TABLE(SQLServer) 语句的信息。

8.1.4 启用和禁用触发器

数据库触发器是存储在数据库中的由事件驱动的特殊过程，并在特定条件 (例如向表中添加一行) 发生时激活 (“触发”)。

您可以使用触发器来补充数据库的标准功能，从而实现高度定制的数据库管理系统。例如，可以创建一个触发器来限制对表的 DML 操作，比如只允许 DML 语句在正常业务时间内执行。

- 关于启用和禁用触发器

  触发器相比约束更加灵活，可以实现比外约束更为复杂的检查和操作，更精细和更强大的数据处理能力。

- 启用触发器

  您可以使用带有 ENABLE 选项的 ALTER TRIGGER 语句启用禁用状态的触发器。

- 禁用触发器

  您可以使用带有 DISABLE 选项的 ALTER TRIGGER 语句禁用触发器。

8.1.4.1 关于启用和禁用触发器

触发器相比约束更加灵活，可以实现比外约束更为复杂的检查和操作，更精细和更强大的数据处理能力。

数据库触发器可以与表、模式或数据库相关联, 它们可以在以下情况被隐式触发:

- 对关联的表执行 DML 语句 (INSERT、UPDATE、DELETE)
- 某些 DDL 语句 (例如:ALTER、CREATE、DROP) 在数据库或模式中的对象上执行

创建触发器请使用 CREATE TRIGGER 语句, 可以定义触发器函数是要在事件之前、之后被调用、还是取代该事件。下面的示例在表 class 上创建了一个 check_update_trigger 触发器。在执行任何指定 UPDATE 语句之前触发该触发器，最终会执行触发器函数 check_account_update。

```
CREATE TRIGGER class_update_trigger
    BEFORE
    UPDATE ON class
    FOR EACH ROW
    EXECUTE FUNCTION check_class_update();
```

之后可以通过执行 DROP TRIGGER 语句从数据库中删除触发器。

要使用 ALTER TABLE 语句启用或禁用触发器，您必须是这个表的拥有者或者对该表具有 ALTER 权限。如果使用 ALTER TRIGGER 语句启用或禁用单个触发器，您必须拥有该触发器。

信息:

- 《系统原理》获取更详细的触发器描述细节
- 《应用开发指南》获取关于 CREATE TRIGGER 的语法及其示例信息
- 《PL/SQL 过程语言参考》参考获取更多关于创建和使用触发器信息

8.1.4.2　启用触发器

您可以使用带有 ENABLE 选项的 ALTER TRIGGER 语句启用禁用状态的触发器。

要在 class 表中启用名为 class_trigger 的触发器 (禁用状态)，输入以下语句:

```
ALTER TRIGGER class_trigger ENABLE;
```

上述用于启动特定的触发器，如果要启用特定表的所有触发器，请使用带有 ENABLE ALL TRIGGERS 选项的 ALTER TABLE 语句。如下示例，启用 class 表定义的所有触发器，则可以输入以下语句:

```
ALTER TABLE class
    ENABLE ALL TRIGGERS;
```

信息:

《KingbaseES 应用开发指南 》获取关于 ALTER TRIGGER 语句的详细信息

8.1.4.3 禁用触发器

可以使用带有 DISABLE 选项的 ALTER TRIGGER 语句禁用触发器。

如果满足以下条件之一，请考虑暂时禁用触发器:

- 触发器引用的对象不可用。
- 您必须执行大型数据加载，并希望它在不触发触发器的情况下快速进行。
- 您正在将数据加载到触发器应用的表中。

要禁用 class 表上的 class_trigger 触发器，输入以下语句:

```
ALTER TRIGGER class_trigger DISABLE;
```

可以使用带有 DISABLE ALL TRIGGERS 选项的 ALTER TABLE 语句同时禁用与表关联的所有触发器。如下示例，禁用 class 表中定义的所有触发器，可以输入以下语句:

```
ALTER TABLE class
    DISABLE ALL TRIGGERS;
```

8.1.5 管理完整性约束

完整性约束用于保证关系型数据库中数据的精确性和一致性。对于关系型数据库来说，数据完整性由参照完整性（referential integrity，RI）来保证。完整性约束是限制表中一个或多个列的值的规则。约束子句可以出现在 CREATE TABLE 或 ALTER TABLE 语句中，并标识一个或多个列受约束影响，以及标识约束的条件。

- 完整性约束状态

完整性约束强制实施业务规则并防止将无效信息输入到表中。

- 创建约束

约束可以在创建表时同时指定，也可以在创建表之后在指定。

- 修改或删除现有的约束

可以使用 ALTER TABLE 语句启用、禁用、修改或删除约束。当数据库使用 UNIQUE 或 PRIMARY KEY 索引强制约束时，如果与该索引相关的约束被删除或禁用，则索引将被删除，除非您另行指定。

- 延迟约束检查

当数据库检查约束时，如果约束不满足，它就发出错误信号。当您发出 SET CONSTRAINTS 语句时，SET CONSTRAINTS 模式将在事务的整个过程中持续，或者直到另一个 SET CONSTRAINTS 语句重置该模式。

- 查看约束信息

KingbaseES 数据库提供了一组视图，使您能够看到表上的约束定义，并识别在约束中指定的列。

信息:

《系统原理》获取更多关于完整性约束的信息《KingbaseES 数据库开发规范》关于约束的详细信息和在应用程序中使用完整性约束的示例

8.1.5.1 完整性约束状态

完整性约束强制实施业务规则并防止将无效信息输入到表中。

- 关于完整性约束状态

您可以指定约束是启用 (ENABLE) 或禁用 (DISABLE) 状态。

- 关于禁用约束

要让表数据符合完整性约束定义的规则，则必须保证约束处于启用状态，但在某些情况下您也应考虑禁用约束。

- 关于启用约束

当启用约束时，不满足约束的数据不能插入到表中。

- 完整性约束的有效使用: 顺序

以特定顺序使用完整性约束状态是很重要的。

8.1.5.1.1 关于完整性约束状态

您可以指定约束是启用 (ENABLE) 或禁用 (DISABLE) 状态。

如果约束状态是 ENABLE，在数据库表中输入或更新数据时要进行约束检查，并阻止输入不符合约束的数据输入数据库。如果约束状态是 DISABLE，则输入或者更新数据时不会进行约束规则检查。另外，您可以指定表中的现有数据必须符合约束（VALIDATE）。相反，如果指定 NOVALIDATE 状态，则不能确保现有数据符合要求。

在表上定义的完整性约束可以处于以下状态之一:

- ENABLE, VALIDATE

将对表的现有及后续新加入的数据执行约束检查。

- ENABLE, NOVALIDATE

将仅对后续新加入的数据做强制性约束检查，而不管表中的任何现有数据。

- DISABLE, VALIDATE

约束规则关闭，但为了保证有效性不被破坏，数据库会被置为只读状态，禁止对表的 DML 操作。

- DISABLE, NOVALIDATE

约束规则关闭，可以对表的约束列进行 DML 操作。

一个 VALIDATE 状态的约束表示数据库内现有数据符合约束规则，设置或者修改约束为 ENABLE 状态时如果没指定有效性，则 VALIDATE 将是默认值。相反，NOVALIDATE 状态表示表内现有数据未做约束规则检查，如果设置或者修改约束为 DISABLE 状态时如果没指定有效性，则 NOVALIDATE 将是默认值。

8.1.5.1.2　关于禁用约束

要让表数据符合完整性约束定义的规则，则必须保证约束处于启用状态，但在某些情况下您也应考虑禁用约束。出现以下性能方面的原因，可以考虑暂时禁用表的完整性约束:

- 当将大量数据加载到表中时
- 执行对表进行大量更改的批处理操作时
- 每次导入或导出一个表

上述这三种情况下，临时禁用完整性约束可以提高操作的性能。当某个约束被禁用时，可能会输入不符合该约束的数据。因此，当我们完成上述类似的操作项后应总是重新启用约束。

8.1.5.1.3　关于启用约束

当启用约束时，不满足约束的数据不能插入到表中。

禁用约束后，违反约束的行则可以插入到表中，该行则被认为是约束异常。而且如果约束处于启用非校验状态 (ENABLE,NOVALIDATE)，则在禁用约束时插入的违反约束的数据将依然被保留。违反约束的行必须更新或者删除，才能将约束置于 VALIDATE 状态。

当表的约束处于 ENABLE,NOVALIDATE 状态时，意味着表中现有数据可以不符合约束规则，但后面新增的数据必须进行规则校验，这对于数据仓库来说特别有用。因为基本上数据仓库都是使用增量更新，在停用约束后如果采用启用校验约束状态，则需要对大量数据进行校验，影响性能。

另外说明：校验一个已经启用的约束，在校验期间不需要任何 DML 锁，因此在校验期间可以保证没有引入违反约束的数据。

8.1.5.1.4　完整性约束的有效使用: 顺序

一般可以按照以下的顺序来使用约束，以确保最佳的效益：

停用状态

执行操作 (装载、导出、导入)

启用非校验状态

启用状态

好处是：

不持有锁

所有的约束能够被并发设置为启用状态

约束启用是并行实现的

允许表进行并发操作

8.1.5.2　创建约束

约束可以在创建表时同时指定，也可以在创建表之后在指定。如果与表同时创建，那么在创建表的 CREATE 语句中通过 CONSTRAINT 关键字指定约束的名称和约束类型。

- 在创建表时指定约束

  您可以在创建表时同时定义约束。

- 在创建表后指定约束

  您也可以在创建表之后再定义约束。

8.1.5.2.1　在创建表时指定约束

您可以在创建表时同时定义约束。

创建表的格式如下：

```
CREATE TABLE 表名 (
    列 1 数据类型 CONSTRAINT 约束名 1 约束类型
    列 2 数据类型 CONSTRAINT 约束名 2 约束类型
);
```

其中约束名必须是唯一指定的名称，如果用户不使用 CONSTRAINT 指定名称，则系统会自动生成。

创建表和定义约束一种常见的方式如下：

```
CREATE TABLE student(
    stu_no number(8) PRIMARY KEY,
    stu_name char(8) NOT NULL,
    gender char(2) CHECK(gender IN ('男','女')),
    birthday date,
    class varchar(40)
);
```

这种方式就是在定义表的列时直接指定该列约束，该方式称之为列级约束。在创建 student 表时定义了三个约束，其中 stu_no 列为主键，stu_name 列约束指定姓名不能为空，gender 列约束指出该列值只能是男或者女。这些约束由于没有名字，系统将自动为它们指定各自的名字。如果想要为每个约束定义名字，可以参考如下方式：

```
CREATE TABLE student(
    stu_no number(8) CONSTRAINT con_sno PRIMARY KEY,
    stu_name char(8) CONSTRAINT con_name NOT NULL,
    gender char(2) CONSTRAINT con_gen CHECK(gender IN ('男','女')),
    birthday date,
    class varchar(40)
);
```

上述方式指定的约束都是和列一起定义的，实际上约束也可以在每列定义后再创建约束，具体的语句如下：

```
CREATE TABLE student(
    stu_no number(8),
    stu_name char(8) CONSTRAINT con_name NOT NULL,
    gender char(2),
    birthday date,
    class varchar(40),
    --定义约束，格式：CONSTRAINT 约束名 约束类型 (列名)
    CONSTRAINT con_sno PRIMARY KEY(stu_no),
    CONSTRAINT con_gen CHECK(gender IN ('男','女'))
);
```

8.1.5.2.2　在创建表后指定约束

您也可以在创建表之后再定义约束。

此方式创建约束可能会带来这样的问题：如果表中已有的数据不满足即将添加的约束，那么约束是无法添加的。因此通常还是建议在创建表时定义约束，这需要根据您的情况在创建表时充分评估。

创建表后添加约束是通过 ALTER 执行，实际上该方式是对表结构的修改，该方式添加约束不再是列级约束而是表级约束。ALTER 语句添加约束的格式为：

```
ALTER TABLE 表名 ADD CONSTRAINT 约束名 约束类型 (列名)
```

其中 CONSTRAINT 和约束名是可以省略的，省略后由系统自动生成，如果要指定多个约束，可以在 ADD 子句的括号被使用逗号分隔即可。现在假设 student 没有定义任何约束且已创建，您可以为这个

```
ALTER TABLE student
    ADD CONSTRAINT con_sno PRIMARY KEY (stu_no);

ALTER TABLE student
        ADD CONSTRAINT con_gender check(gender in ('男','女'));
```

特别注意，对于 NOT NULL 约束，你需要通过 ALTER COLUMN 子句添加。语法格式如下：

```
ALTER TABLE 表名 ALTER COLUMN 列名 SET NOT NULL;
```

往 student 表添加非空约束，具体的语句如下：

```
ALTER TABLE student ALTER COLUMN stu_name SET NOT NULL;
```

8.1.5.3　修改或删除现有的约束

可以使用 ALTER TABLE 语句启用、禁用、修改或删除约束。当数据库使用 UNIQUE 或 PRIMARY KEY 索引强制约束时，如果与该索引相关的约束被删除或禁用，则索引将被删除，除非您另行指定。当启用外键引用 PRIMARY 或 UNIQUE 键时，不能禁用或删除 PRIMARY 或 UNIQUE 键约束或索引。

- 禁用和启用约束

  您可以禁用已启用的完整性约束和启用已禁用的完整性约束。

- 重命名约束

ALTER TABLE……RENAME CONSTRAINT 语句使您能够为表重命名任何当前存在的约束。新的约束名不能与用户的任何现有约束名冲突。

- 删除约束

  如果完整性约束所执行的规则不再为真，或者不再需要该约束，则可以删除该约束。

8.1.5.3.1　禁用和启用约束

您可以禁用已启用的完整性约束和启用已禁用的完整性约束。

以下语句禁用完整性约束。

```
ALTER TABLE dept
    DISABLE CONSTRAINT dname_ukey;

ALTER TABLE dept
    MODIFY CONSTRAINT dname_ukey DISABLE VALIDATE;
```

以下语句启用 novalidate disable 状态的完整性约束:

```
ALTER TABLE dept
    ENABLE NOVALIDATE CONSTRAINT dname_ukey;

ALTER TABLE dept
    ENABLE NOVALIDATE PRIMARY KEY,
    ENABLE NOVALIDATE UNIQUE (dname, loc);
```

以下语句启用或验证禁用的完整性约束:

```
ALTER TABLE dept
    MODIFY PRIMARY KEY ENABLE NOVALIDATE;
```

以下语句启用禁用状态的完整性约束:

```
ALTER TABLE dept
    ENABLE CONSTRAINT dname_ukey;

ALTER TABLE dept
    ENABLE PRIMARY KEY,
    ENABLE UNIQUE (dname, loc);
```

8.1.5.3.2　重命名约束

ALTER TABLE……RENAME CONSTRAINT 语句使您能够为表重命名任何当前存在的约束。新的约束名不能与用户的任何现有约束名冲突。下面的语句重命名表 dept 的 dname_ukey 约束为 dname_unikey:

```
ALTER TABLE dept
    RENAME CONSTRAINT dname_ukey TO dname_unikey;
```

8.1.5.3.3　删除约束

如果完整性约束所执行的规则不再为真，或者不再需要该约束，则可以删除该约束。可以使用带有 DROP CONSTRAINT 子句的 ALTER TABLE 语句删除约束:

以下语句删除了完整性约束 dept_fkey:

```
ALTER TABLE emp
    DROP CONSTRAINT dept_fkey;
```

8.1.5.4　延迟约束检查

当数据库检查约束时，如果约束条件不满足，它将报错。当您执行 SET CONSTRAINTS 语句时，SET CONSTRAINTS 模式将在事务的整个过程中持续，延迟约束检查会在事务结束后再检查。

- 将所有约束设置为延迟

当必须为事务延迟约束时，必须在实际开始处理应用程序中用于操作数据的任何数据之前设置所有延迟约束。

- 检查提交 (可选)

在事务 COMMIT 之前，可以用 SET CONSTRAINTS ALL IMMEDIATE 语句来手工检查约束的违反情况。虽然在事务结束时会自动隐性执行这条语句，但是也是有一定的存在意义的，例如希望在错误时定义某些操作。

8.1.5.4.1　将所有约束设置为延迟

当必须为事务延迟约束时，必须在实际开始处理应用程序中用于操作数据的任何数据之前设置所有延迟约束。使用下面的 DML 语句设置所有可延期约束:

```
SET CONSTRAINTS ALL DEFERRED;
```

提示:

SET CONSTRAINTS 语句只应用于当前事务。只要约束存在，在创建约束时指定的默认值就保持不变。ALTER SESSION SET CONSTRAINTS 语句只适用于当前会话。

信息:

《KingbaseES 应用开发指南 》获取有关 SET CONSTRAINTS 语句详细语法和描述

8.1.5.4.2　检查提交 (可选)

您可以在提交之前通过发出 SET CONSTRAINTS ALL IMMEDIATE 语句检查约束违例。

如果某个约束有任何问题，则此语句失败，并识别出导致错误的约束。如果在违反约束的情况下提交，则事务将回滚，并收到一条错误消息。

8.1.5.5　查看约束信息

KingbaseES 提供了一组视图 (兼容 oracle)，使您能够看到表上的约束定义，并识别在约束中指定的列。

- DBA_CONSTRAINTS，DBA 视图描述了数据库中所有的约束定义
- ALL_CONSTRAINTS，ALL 视图描述了当前用户可访问的约束定义
- USER_CONSTRAINTS，USER 视图描述了当前用户拥有的定义
- DBA_CONS_COLUMNS，DBA 视图描述了约束中指定数据库中所有列
- ALL_CONS_COLUMNS，ALL 视图描述了约束中当前用户可访问的列
- USER_CONS_COLUMNS，USER 视图只描述了约束中指定的用户当前拥有的列

信息:

《系统原理》中 KingbaseES 兼容 Oracle 视图章节获取有关上述视图详细描述

8.1.6　重命名模式对象

重命名对象多种方法。要重命名对象，它必须在您的模式中。您可以使用以下任一种方式重命名架构对象:

- 删除并重新创建对象
- 可使用 ALTER…RENAME 语句重命名对象 (用于索引和触发器)

如果删除并重新创建对象，则将丢失授予该对象的所有权限。在重新创建对象时，必须重新授予权限。

在重命名模式对象之前，请注意以下影响:

- 所有依赖于重命名对象的视图和 PL/SQL 程序单元都将失效，在下次使用之前必须重新编译。
- 重命名对象的所有同义词在使用时都返回错误。

信息:

《KingbaseES 应用开发指南 》获取关于 RENAME 相关语法信息

8.1.7 管理对象名称解析

在 SQL 语句中引用的对象名称可以由多个部分组成，由点号分隔。KingbaseES 数据库执行特定的操作来解析对象名称。下面描述的是 KingbaseES 数据库管理系统怎样解析对象名字。

- 如果只有一个名字，而没有点号。则在当前模式下寻找是否存在相同名字的对象，如果找到，则返回；否则报错；
- 如果有点号，首先检测对象名的第一部分，如在 scott.emp 中, scott 就是第一部分。

  1）寻找哪一个模式的名字和第一部分相同，如果找到，以此模式进行步骤 2)；否则，以当前模式进行步骤 2）；

  2）在模式中需找是否有与对象名第二部分同名的对象，如果找到且待解析对象名只有两个部分，则返回；如果找到但待解析对象名多于两个部分，则转步骤 3)；否则报错；

  3）在模式中查找包含在前一个对象中并且和待解析对象名当前解析部分相同名字的对象，如果找到，循环步骤 3)，直至所有的部分检测结束后再返回；如果没有找到，则直接报错。如 scott.emp.deptno.workers，在 scott 模式下中的 emp 对象中寻找 deptno 的对象，然后在 deptno 对象中寻找 workers 对象。

信息:

《KingbaseES 应用开发指南 》获取关于" PL/SQL 名称解析" 详细信息

8.1.8 切换到其他模式

可以使用 SQL 语句 ALTER SESSION 切换到不同的模式。下面语句将当前会话模式设置为指定的模式名称。

```
ALTER SESSION SET SEARCH_PATH = <schema name>
```

在后续的 SQL 语句中，如果省略了模式名，数据库将使用该模式名作为模式限定符。

8.1.9 显示模式对象的信息

KingbaseES 提供的 PL/SQL 包 DBMS_METADATA.GET_DDL 允许您获取关于模式对象的元数据 (以用于创建对象的 DDL 的形式)。DBMS_METADATA 包是获取模式对象完整定义的强大工具。它使您能够在一次传递中获得对象的所有属性。该对象被描述为 DDL，可用于 (重新) 创建它。在下面的语句中，GET_DDL 函数用于获取当前模式中所有表的 DDL。

```
SELECT DBMS_METADATA.GET_DDL('TABLE',u.table_name)
     FROM USER_ALL_TABLES u
     WHERE u.nested='NO'
     AND (u.iot_type is null or u.iot_type='IOT');
```

信息:

《KingbaseES 插件参考手册》获取关于" dbms_metadata " 详细信息

8.2　表的管理

表的管理包括创建表、加载表、修改表和删除表等操作，以及临时表，分区表等的管理。

- 关于表

表是数据库中数据存储的基本单元，数据存储在表的行和列中。

- 表管理指南

遵循这些准则可以使表的管理更加容易，并且可以提高创建表、加载表更新以及查询表的性能。

- 创建表示例

使用 SQL 语句 CREATE TABLE 创建表。

- 加载表

有多种将数据加载到表中的技术。

- 收集数据库的统计信息

sys_stat_statement 工具允许您生成数据库的统计信息。您可以使用此工具来收集的数据库性能统计信息。

- 修改表

使用 ALTER TABLE 语句修改表。要修改一个表，表必须包含在您的模式中，或者您必须具有该表的 ALTER 对象权限或 ALTER ANY TABLE 系统权限。

- 删除表

可以使用 DROP TABLE 语句删除不再需要的表。

- 使用 FLASHBACK 删除和恢复表

删除表时，数据库不会立即删除与该表关联的空间。数据库重命名表，并将其和任何关联的对象放在回收站中，万一表被错误地删除，可以在稍后恢复。这个特性被称为 FLASHBACK DROP, FLASHBACK TABLE 语句则用于恢复表。

- 管理分区表

分区表可以将数据分割为更小、更易于管理的小单位，称为分区，甚至是更小的子分区。每个分区可以有单独的物理属性，例如物理存储设置和表空间，能有效提升大型表的访问效率。此外，可以单独管理每个分区，可以简化和减少备份和管理所需的时间。

- 表的数据视图

您可以查询一组数据字典视图，以了解有关表的信息。

8.2.1　关于表

表是数据库中数据存储的基本单位，数据存储在行和列中。表中列的个数和顺序是固定的。在创建表时，要对表中的所有列进行说明，要指明它的名称、数据类型、宽度或精度等。已经说明的列也可以使用 SQL 语句进行修改。

在逻辑结构上，一个表一定是位于某个表空间上，当创建表时，同时将创建一个表段，用于存放表中的数据。在物理结构上，表的数据都放在数据块中，数据块中存放的就是一行的数据结构。同时，一般而言，表的各行数据是以写入的先后顺序存放，而一行的各列一般按照定义表时的指定顺序存放。

列的数据类型限制了该列取值，以及说明了该列进行计算时，存储在该列上的数据的语义。例如，一个列声明为 INTEGER 类型，则其中就不能存放字符串数据，该列的数据可以进行数学运算。同样，一个列声明为字符串类型，则其中可以存放任何数据，但该列的数据不可以进行数学运算。

您可以为表的每一列指定限制规则。这些限制规则称为完整性约束。一个典型例子是 NOT NULL 完整性约束。此约束强制列在每一行中包含一个值。您可以调用透明数据加密来在存储数据之前对数据进行加密。如果用户试图绕过数据库访问控制机制，直接使用操作系统工具查看数据文件，加密会阻止这些用户查看敏感数据。

信息:

- 《KingbaseES 应用开发指南 》获取关于数据类型的描述信息
- 《安全指南》获取关于透明数据加密相关信息

8.2.2　表管理指南

遵循这些准则可以使表的管理更加容易，并且可以提高创建表、加载表更新以及查询表的性能。

- 创建表之前的设计准则

通常，应用程序开发人员负责设计应用程序的元素，包括表。数据库管理员负责建立用于保存应用程序表的底层表空间的属性。

- 指定表的存储位置

建议在 CREATE TABLE 语句中指定 TABLESPACE 子句，以标识将要存储新表的表空间。对于分区表，您可以选择标识用于存储每个分区的表空间。

- 在创建表时考虑使用 UNLOGGED

要最有效地创建表，可以使用 CREATE UNLOGGED TABLE…AS 语句。如果指定了 UNLOGGED，则创建的表将不做日志。

- 评估表格大小和相应的规划

在创建表之前估计表的大小。最好将此作为数据库规划的一部分。了解数据库表的大小和用途是数据库规划的一个重要部分。

8.2.2.1　创建表之前的设计准则

通常，应用程序开发人员负责设计应用程序的元素，包括表。数据库管理员负责建立用于保存应用程序表的底层表空间的属性。DBA 或应用程序开发人员，或者两者共同工作，可以负责实际创建表，这取决于站点的实践。与应用程序开发人员合作，在设计表时考虑以下准则:

- 对表、列、索引和聚簇使用描述性名称。
- 使用 COMMENT 命令记录每个表及其列的含义。
- 标准化每个表。
- 为每个列选择适当的数据类型。
- 考虑向某些表添加一个或多个虚拟列是否使您的应用程序从中受益。
- 定义允许最后为空的列，以节省存储空间。
- 聚簇表在任何时候都是合适的，可以实现节省存储空间并优化 SQL 语句的性能。

在创建表之前，还应该确定是否使用完整性约束。可以在表的列上定义完整性约束，将自动执行数据库的业务规则。

8.2.2.2　指定表的存储位置

建议在使用 CREATE TABLE 语句时指定 TABLESPACE 子句，可对表指定存储的表空间。对于分区表，您可以选择指定存储每个分区的表空间。如果没有指定，则该表将创建在用户的默认表空间中。在创建表时，通过指定合适的表空间，有以下优点：

- 提高数据库系统的性能，因为不同的数据库表可能对应不同的数据文件，可减少对相同文件的竞争；
- 减少数据库管理的时间，数据库表分布在不同的表空间中，即使一个表空间损坏，也不影响其他表空间上数据库表的正常访问。

8.2.2.3　在创建表时考虑使用 UNLOGGED

要最有效地创建表，可以使用 CREATE UNLOGGED TABLE…AS 语句。如果指定了 UNLOGGED，则创建的表将不做日志。

使用 UNLOGGED 子句有以下好处:

- 省去重做日志文件，节省了一定的空间。
- 创建表的时间会减少。
- 并行创建大型表性能得到提升。

8.2.2.4　评估表格大小和相应的规划

创建表之前，先进行表大小的评估，并且建议将该评估结果作为数据库规划的一部分。了解数据库表的大小和用途是数据库规划的一个重要部分。

您可以连同索引、撤销空间和重做日志文件的大小一起评估，综合评估表的空间，来确保数据库所需的磁盘空间量。根据估计结果，您就可以购买合适容量的磁盘硬件。

8.2.3　创建表

要在所属模式中创建新表，需要有 CREATE TABLE 数据库权限；而要在其他用户的模式中创建新表，则需要有 CREATE ANY TABLE 数据库权限。

创建表时，如果给定了一个模式名（例如 CREATE TABLE myschema.mytable…），那么该表被创建在指定的模式中，否则它被创建在当前模式中。

- 创建表示例

若干创建表的例子。

- 创建临时表

临时表在需要缓冲结果集的应用程序中很有用，因为它是通过运行多个 DML 操作构建的。您可以创建全局临时表或者局部临时表。

8.2.3.1　创建表示例

若干创建表的例子。

创建 distributors 表，并为 name 列定义一个唯一约束：

```
CREATE TABLE distributors (
    did     integer,
    name    varchar(40) UNIQUE
);
```

执行下列语句后，在 hr 模式中创建了一个名为 emp 的表，并将其存储在 tbs0 表空间中，并定义了主键约束，其中包含定义在不同列上的一个主键:

```
CREATE TABLE hr.emp (
    code        char(12) NOT NULL,
    title       varchar(40) NOT NULL,
    did         integer,
    date_prod   date,
    kind        varchar(10),
    len         interval hour to minute,
    CONSTRAINT code_title PRIMARY KEY(code,title)
)TABLESPACE tbs0;
```

创建 distributors 表，指定表和它的唯一索引指定 70% 的填充因子：

```
CREATE TABLE distributors (
    did     integer,
    name    varchar(40),
    UNIQUE(name) WITH (fillfactor=70)
)
WITH (fillfactor=70);
```

创建一个范围分区表：

```
CREATE TABLE measurement (
    logdate         date not null,
    peaktemp        int,
    unitsales       int
) PARTITION BY RANGE (logdate);
```

信息:

《KingbaseES 应用开发指南 》获取关于 CREATE TABLE(Oracle)、CREATE TABLE(MySQL) 、CREATE TABLE(SQLServer) 语句更多创建例子。

8.2.3.2　创建临时表

临时表在应用程序需要缓冲结果集时很有用，因为它是通过运行多个 DML 操作构建的。您可以创建全局临时表或者局部临时表。

- 临时表概述

临时表保存的数据仅在事务或会话期间存在。

- 创建临时表的注意事项

在创建临时表时，一些应该关注的注意事项。

- 临时表事务控制

临时表在一个事务块结束时的行为由 ON COMMIT 控制。

- 创建全局临时表

全局临时表是存储在磁盘上的永久数据库对象，对连接到数据库的所有会话都可见。

- 创建局部临时表

局部临时表是在事务或会话结束时删除的临时数据库对象，每个临时表只对创建它的会话可见。

信息:

《KingbaseES 应用开发指南 》中 CREATE TABLE(Oracle)、CREATE TABLE(MySQL) 、CREATE TABLE(SQLServer) 获取关于创建临时表的详细语法和描述。

8.2.3.2.1　临时表概述

临时表保存的数据仅在事务或会话期间存在。临时表中的数据是会话私有的。每个会话只能看到和修改自己的数据。简单来看全局临时表定义是持久的，而表数据是临时的，而本地临时表的表定义和表数据都是临时的，会在会话退出后被删除。

本地临时表和全局临时表具体有如下四点区别：

- 本地临时表在临时模式下，用户不可以指定；但是全局临时表创建在用户指定模式下；
- 本地临时表对象本身并不是一直存在，在会话退出后，本地临时表会被删除；全局临时表创建之后，一直存在，除非显示去删除它。
- 删除本地临时表，不受其他会话影响；但是删除全局临时表时，所有会话都不能持有全部临时表的数据。
- 全局临时表不支持外键约束也不支持其他表引用全局临时表作为外键约束，而本地临时表没有这个要求。

8.2.3.2.2　创建临时表的注意事项

在创建临时表时，一些应该关注的注意事项。

只有当当前没有会话绑定到一个现有临时表时，才允许在这个临时表上执行 DDL 操作 (TRUNCATE 除外)。如果事务回滚，则输入的数据将丢失，但表定义仍然存在。特定于事务的临时表一次只允许一个事务。如果在一个事务作用域中有多个自治事务，则每个自治事务只能在前一个事务提交之后才能使用该表。

由于临时表中的数据根据定义是临时的，所以当系统发生故障时，临时表数据的备份和恢复功能是不可用的。为了应对这种故障，应该开发时需要其他方法来保存临时表数据。

8.2.3.2.3　临时表事务控制

临时表在一个事务块结束时的行为由 ON COMMIT 控制。

三种选项具体是：

- PRESERVE ROWS

在事务结束时不采取特殊的动作。是本地临时表采用的默认行为

- DELETE ROWS

在每一个事务块结束时将删除临时表中的所有行。实质上，在每一次提交时会完成一次自动的 TRUNCATE 。在分区表上使用时，不会将其级联到其分区。这是全局临时表的默认行为。

- DROP

在当前事务块结束时将删除临时表。在分区表上使用时，此操作将删除其分区，在具有继承子级的表上使用时，将删除依赖子级。仅本地临时表支持在当前事务块结束时将删除临时表。为兼容 Oracle，全局临时表不支持 ON COMMIT DROP 选项。

8.2.3.2.4　创建全局临时表

全局临时表是存储在磁盘上的永久数据库对象，对连接到数据库的所有会话都可见。

全局临时表的定义对所有会话都是可见的，但是全局临时表中的数据只对向表中插入数据的会话可见。您使用 CREATE GLOBAL TEMPORARY TABLE 语句创建一个全局临时表。

下列示例演示了如何创建全局临时表。这条语句创建了一个特定于事务的全局临时表:

```
CREATE GLOBAL TEMPORARY TABLE admin_work_area_trans
        (startdate DATE,
         enddate DATE,
         class CHAR(20))
      ON COMMIT DELETE ROWS;
```

这条语句创建了一个特定于会话的全局临时表:

```
CREATE GLOBAL TEMPORARY TABLE admin_work_area_session
        (startdate DATE,
         enddate DATE,
         class CHAR(20))
      ON COMMIT PRESERVE ROWS;
```

8.2.3.2.5　创建局部临时表

局部临时表的元数据和内容仅在创建该临时表的会话中可见。局部临时表在以下情况下很有用:

- 当应用程序将临时数据存储在临时表中时，这些临时表只填充一次，读取几次，然后在事务或会话结束时删除
- 当一个会话被无限期地维护，并且必须为不同的事务创建不同的临时表时
- 创建临时表时，不能启动新事务或提交现有事务
- 当同一个用户的不同会话必须对一个临时表使用相同的名称时

使用 CREATE LOCAL TEMPORARY TABLE 语句创建一个局部临时表。LOCAL 可以不指定，因为默认是局部临时表。

下列示例演示如何创建局部临时表。这条语句创建了一个特定于事务的局部临时表:

```
CREATE LOCAL TEMPORARY TABLE sales_ptt_transaction
    (time_id      DATE,
     amount_sold  NUMBER(10,2))
   ON COMMIT DELETE ROWS;
```

这条语句创建了一个局部临时表，它是特定于会话的:

```
CREATE LOCAL TEMPORARY TABLE sales_ptt_session
    (time_id      DATE,
     amount_sold  NUMBER(10,2))
   ON COMMIT PRESERVE ROWS;
```

默认情况下，局部临时表中的行存储在创建它的用户的默认临时表空间中。但是，在创建临时表的过程中，可以通过 CREATE LOCAL TEMPORARY TABLE 语句的 TABLESPACE 子句将一个局部临时表分配给另一个临时表空间。

8.2.4 加载表

有几种方法可以将数据插入或初始加载到表中。

最常用的是以下几种:

- sys_bulkload 工具快速加载数据

sys_bulkload 是 KingbaseES 提供的快速加载数据的命令行工具，该工具能够把一定格式的文本数据简单、快速的加载到 KingbaseES 数据库中。

- {\hyperref[\detokenize{docs/administration/08_DB_Obj_Mgmt/02_table/03_load: 使用 CREATE TABLE AS 语句}]{\sphinxcrossref{\DUrole{std,std-ref}{使用 CREATE TABLE AS 语句}}}}

使用此 SQL 语句，您可以创建一个表，并使用从另一个现有表 (包括外部表) 选择的数据填充该表数据。

- 使用 INSERT 语句

INSERT 语句允许您通过指定列值或指定从另一个现有表选择数据的子查询向表添加行。

- 使用 MERGE 语句

通过从另一个现有表中选择行，可以使用 MERGE 语句向表中插入行或更新行。如果新数据中的一行对应于表中已经存在的项，则执行 UPDATE，否则执行 INSERT。

- 使用 COPY 语句

COPY 语句允许您将 KingbaseES 表和标准文件系统文件之间相互转移数据。可以实现将表的内容复制到文件，也可以将文件复制数据到表，甚至复制一个 SELECT 查询结果。

8.2.4.1　sys_bulkload 工具快速加载数据

sys_bulkload 是 KingbaseES 提供的快速加载数据的命令行工具，该工具能够把一定格式的文本数据简单、快速的加载到 KingbaseES 数据库中。

创建导入数据表

```
create TABLE test(id int primary key, info text, crt_time timestamp);
```

数据文件示例：将下列数据以 test.csv 为文件名保存到 KingbaseES 服务器所在目录

```
1,29b35ff06c949e7e442c929e1df86396,2017-10-08 10:52:47.746062
2,06fde814525395de5ab85f6d92b04e87,2017-10-08 10:52:47.746573
3,c93f02e8677c9cd7c906c6ad5dbd450e,2017-10-08 10:52:47.746627
4,6541700070ae3d051f965fcef43baf45,2017-10-08 10:52:47.746835
5,3d7e7246016acaa842526b6614d0edf5,2017-10-08 10:52:47.746869
6,1d1ae5a03ef0bad3bc14cd5449ba0985,2017-10-08 10:52:47.746894
7,7745c57c54b97656bec80a502ec13ec7,2017-10-08 10:52:47.746918
8,3c377131f6ef82c3284dc77a3b4ffdf7,2017-10-08 10:52:47.746942
9,5ef98d40aeeadf65eb1f0d7fd86ed585,2017-10-08 10:52:47.746968
10,312c0a0188da9e34fe45aa19d0d07427,2017-10-08 10:52:47.746993
```

导入数据

1. 以 BUFFERED 方式导入 TEXT 文件配置文件示例（以 test.ctl 为名保存到服务器所在目录，也可自行指定其他目录。）

```
TABLE = test
INPUT = /home/kingbase/test.csv
TYPE = CSV
SKIP = 2
LIMIT = 5
WRITER = BUFFERED
PROCESSOR_COUNT = 3
```

2. 使用配置文件导入命令示例

```
./sys_bulkload -h localhost -p 54321 -d TEST /home/kingbase/test.ctl -U SYSTEM -W 123456
```

3. 使用配置文件导入结果

```
导入成功提示信息：
    NOTICE: BULK LOAD START
    NOTICE: BULK LOAD END
        2 Rows skipped.
        5 Rows successfully loaded.
        0 Rows not loaded due to parse errors.
```

```
        0 Rows not loaded due to duplicate errors.
        0 Rows replaced with new rows.
    log path: /home/test/test.log
    parse error path: /home/test/test.bad
    duplicate error path: /home/test/test.dupbad
    ctrl file path: /home/test/test.ctl
    data file path: /home/test/test.csv
    Run began on 2021-03-27 22:53:26.690476+08
    Run ended on 2021-03-27 22:53:27.496344+08
```

导入结果查询（配置指定跳过前 2 行，限制导入条数 5 条）：

```
test=# select * from test;
 id |                info                |          crt_time
----+-----------------------------------+----------------------------
  3 | c93f02e8677c9cd7c906c6ad5dbd450e | 2017-10-08 10:52:47.746627
  4 | 6541700070ae3d051f965fcef43baf45 | 2017-10-08 10:52:47.746835
  5 | 3d7e7246016acaa842526b6614d0edf5 | 2017-10-08 10:52:47.746869
  6 | 1d1ae5a03ef0bad3bc14cd5449ba0985 | 2017-10-08 10:52:47.746894
  7 | 7745c57c54b97656bec80a502ec13ec7 | 2017-10-08 10:52:47.746918
(5 行记录)
```

信息:

《工具参考》获取关于 sys_bulkload 详细选项描述和示例

8.2.4.2　使用 CREATE TABLE AS 语句

CREATE TABLE AS 创建一个表，并且用由一个 SELECT 命令计算出来的数据填充该表。该表的列具有和 SELECT 的输出列相关的名称和数据类型（不过可以通过给出一个显式的新列名列表来覆盖这些列名）。

CREATE TABLE AS 和创建一个视图有些相似，但是实际上非常不同：它会创建一个新表并且只计算该查询一次用来初始填充新表。这个新表将不会跟踪该查询源表的后续变化。相反, 一个视图只要被查询, 它的定义 SELECT 语句就会被重新计算。

创建一个新表 films_new，它只由表 `films` 中最近一段时间的项组成：

```
CREATE TABLE films_new AS
  SELECT * FROM films WHERE date_prod >= '2022-01-01';
```

要完全地复制一个表，也可以使用 TABLE 命令的简短形式：

```
CREATE TABLE films_copy AS
  TABLE films;
```

用一个预备语句创建一个新的临时表 films_new，它仅由表 films 中最近的项组成。新表将在提交时被删除：

```
PREPARE recentfilms(date) AS
  SELECT * FROM films WHERE date_prod > $1;
CREATE TEMP TABLE films_new ON COMMIT DROP AS
  EXECUTE recentfilms('2002-01-01');
```

信息:

《KingbaseES 应用开发指南 》获取关于 CREATE TABLE AS 详细语法描述和示例

8.2.4.3　使用 INSERT 语句

INSERT 语句允许您通过指定列值或指定从另一个现有表选择数据的子查询向表添加行。

例如，当在表之间拷贝值时，这个 OVERRIDING USER VALUE 子句就能派上用场，下面这个例子将从 tbl1 中拷贝所有在 tbl2 中不是标识列的列，而 tbl2 中标识列的值将由与 tbl2 关联的序列产生。

```
INSERT INTO tbl2 OVERRIDING USER VALUE SELECT * FROM tbl1;
```

这个例子从表 tmp_films 中获得一些行插入到表 `films` 中，两个表具有相同的列布局：

```
INSERT INTO films SELECT * FROM tmp_films WHERE date_prod < '2022-05-07';
```

信息:

《KingbaseES 应用开发指南 》获取关于 INSERT(Oracle)、INSERT(MySQL)、INSERT(SQLServer) 语句详细语法描述和示例

8.2.4.4　使用 MERGE 语句

使用 MERGE 语法可合并 UPDATE 和 INSERT 语句功能。

通过 MERGE 语句，根据一张表（或视图）的连接条件对另外一张表（或视图）进行查询，连接条件匹配上的执行 UPDATE(可能含有 DELETE)，无法匹配的执行 INSERT。其中数据表包括：普通表、分区表。

这个例子创建了两个表 test1 和 test2，并且进行条件匹配，将结果合并到 test2 中：

```
create table test1(a int, b int);
insert into test1 values(1, 1);
insert into test1 values(2, 2);
insert into test1 values(3, 3);
```

```
create table test2(x int, y int);
insert into test2 values(1, 1);
insert into test2 values(3, 3);
insert into test2 values(5, 5);

merge into test2 using test1
on (test1.a = test2.x)
when matched then
update set y = y * -1 where test1.a > 1
when not matched then
insert values(test1.a, test1.b);
```

执行后 test2 结果：

```
x | y
---+----
 1 |  1
 5 |  5
 2 |  2
 3 | -3
(4 rows)
```

执行后 test1 结果：

```
a | b
---+----
 1 |  1
 2 |  2
 3 |  3
(3 rows)
```

8.2.4.5　使用 COPY 语句

COPY 语句允许您将 KingbaseES 表和标准文件系统文件之间相互转移数据。可以实现将表的内容复制到文件，也可以将文件复制数据到表，甚至复制一个 SELECT 查询结果。

- COPY TO 把一个表的内容复制到一个文件;
- COPY FROM 则从一个文件复制数据到一个表（把数据追加到表中原有数据）;
- COPY TO 也能复制一个 SELECT 查询的结果。

COPY TO 只能被用于纯粹的表，不能用于视图。不过您可以写 COPY (SELECT * FROM viewname) TO…来拷贝一个视图的当前内容。

COPY FROM 可以被用于普通表、外部表、分区表或者具有 INSTEAD OF INSERT 触发器的视图。

COPY 只处理提到的表，它不会从子表复制数据或者复制数据到子表中。例如 COPY table TO 会显示与 SELECT * FROM ONLY table 相同的数据。而 COPY (SELECT * FROM table) TO …可以被用来转储一个继承层次中的所有数据。

1. 下面的例子使用竖线（|）作为域定界符把 country 表复制到客户端：

```
COPY country TO STDOUT (DELIMITER '|');
```

2. 从文件 country_data 中复制数据到 country 表中：

```
COPY country FROM '/usr1/proj/bray/sql/country_data';
```

3. 只把名称以‘A’开头的国家复制到文件 a_list_countries.copy 中：

```
COPY (SELECT * FROM country WHERE country_name LIKE 'A%') TO
'/usr1/proj/bray/sql/a_list_countries.copy';
```

信息:

《KingbaseES 应用开发指南 》获取关于 COPY(Oracle)、COPY(SQLServer) 语句详细语法和示例信息

8.2.5　收集数据库的统计信息

数据库内部提供了多种性能视图和工具，将允许您生成数据库的统计信息。您可以使用诸如 sys_stat_statements 工具来收集相关的性能统计信息。数据库运行时的状态对分析某些性能问题的分析至关重要，我们可用通过对系统状态统计信息、日志信息的分析，来监控整个数据库的运行状态，并能从其中发现一些性能相关的问题。

跟数据库性能相关的信息和工具主要包括：

整体分析	sys_kwr 报告工具（基本覆盖所有优化点分析） sys_ksh 工具
CPU 优化点分析	耗时语句统计： sys_stat_statements kbbadger
会话/负载/等待事件分析	sys_stat_activity 视图 (实时) sys_ksh 报告工具 (时间区间)
锁优化点分析	sys_locks 视图
IO 优化点分析	sys_statio_user_tables 视图 sys_statio_user indexes 视图

-续表-

整体分析	sys_kwr 报告工具（基本覆盖所有优化点分析） sys_ksh 工具
应用特点分析	sys_stat_user_tables 视图 sys_stat_user_indexes 视图 sys_stat_database 视图

信息:

《KingbaseES 数据库性能调优指南 》手册的性能诊断部分获取关于数据库资源分析详细信息

8.2.6 修改表

和修改表中所包含的数据不同，有时当创建表并且填充数据后，有可能需要修改表的定义或者说结构，这一类操作可以使用 alter TABLE 语句修改表。

要修改一个表，表必须包含在您的模式中，或者您必须具有该表的 ALTER 对象权限或 ALTER ANY TABLE 系统权限。KingbaseES 提供了一族命令来对已有的表进行修改。通过更改表，用户可以对数据库中的表作如下修改：

- 增加列

  为表新增加一列。

- 移除列

  移除表中的现有列。

- 增加约束

  给表增加约束条件。

- 移除约束

  移除表中的现有约束。

- 修改列的默认值

  修改一个列的默认值。

- 修改列的数据类型

  将一个列的数据类型转换为另一种数据类型。

- 重命名列

  重命名一个现有列。

- 重命名表

  修改表的名字。

信息:

《KingbaseES 应用开发指南 》的 ALTER TABLE(Oracle)、ALTER TABLE(SQLServer)、ALTER TABLE(MySQL) 语句获取更详细的信息。

8.2.6.1 增加列

要增加一个列，可以使用这样的命令：

```
ALTER TABLE products ADD COLUMN description text;
```

新列将被默认值所填充（如果没有指定 DEFAULT 子句，则会填充空值）。

提示:

在 KingbaseES，添加具有常量默认值的列不再意味着在执行 ALTER TABLE 语句时需要更新表的每一行。相反，默认值将在下一次访问该行时返回，并在重写表时应用，这使得 ALTER TABLE 非常快，即使在大型表上也是如此。

但是，如果默认值是 volatile (e.g. clock_timestamp())，每一行都需要在执行 ALTER TABLE 时计算出的值来更新。以避免潜在的冗长的更新操作, 特别是如果你打算填列主要是默认的值, 它可能是更可取的添加没有默认的列, 使用 UPDATE 插入正确的值, 然后添加任何所需的默认如下所述。

也可以同时为列定义约束，语法：

```
ALTER TABLE products ADD COLUMN description text CHECK (description <> '');
```

事实上 CREATE TABLE 中关于一列的描述都可以应用在这里。记住不管怎样，默认值必须满足给定的约束，否则 ADD 将会失败。也可以先将新列正确地填充好，然后再增加约束 。

8.2.6.2 移除列

为了移除一个列，使用如下的命令：

```
ALTER TABLE products DROP COLUMN description;
```

列中的数据将会消失，涉及到该列的表约束也会被移除。然而，如果该列被另一个表的外键所引用，KingbaseES 不会移除该约束。我们可以通过增加 CASCADE 来授权移除任何依赖于被删除列的所有东西：

```
ALTER TABLE products DROP COLUMN description CASCADE;
```

8.2.6.3　增加约束

在现有表基础上新增加约束，可以使用表约束的语法，例如：

```
ALTER TABLE products ADD CHECK (name <> '');
ALTER TABLE products ADD CONSTRAINT some_name UNIQUE (product_no);
ALTER TABLE products ADD FOREIGN KEY (product_group_id) REFERENCES product_groups;
```

要增加一个不能写成表约束的非空约束，可使用语法：

```
ALTER TABLE products ALTER COLUMN product_no SET NOT NULL;
```

该约束会立即被检查，所以表中的数据必须在约束被增加之前就已经符合约束。

8.2.6.4　移除约束

为了移除一个约束首先需要知道它的名称。如果在创建时已经给它指定了名称，那么事情就变得很容易。否则约束的名称由系统生成，我们必须先找出这个名称。ksql 的命令 d 表名将会对此有所帮助，其他接口也会提供方法来查看表的细节。因此命令是：

```
ALTER TABLE products DROP CONSTRAINT some_name;
```

（如果处理的是自动生成的约束名称，如 $2，别忘了用双引号使它变成一个合法的标识符。）

和移除一个列相似，如果需要移除一个被某些对象依赖的约束，也需要加上 CASCADE。一个典型的例子是一个外键约束依赖于被引用列上的一个唯一主键约束。

这对除了非空约束之外的所有约束类型都一样有效。为了移除一个非空约束可以用：

```
ALTER TABLE products ALTER COLUMN product_no DROP NOT NULL;
```

（回忆一下，非空约束是没有名称的，所以不能用第一种方式。）

8.2.6.5　修改列的默认值

要为一个列设置一个新默认值，使用命令：

```
ALTER TABLE products ALTER COLUMN price SET DEFAULT 7.77;
```

注意这不会影响任何表中已经存在的行，它只是为未来的 INSERT 命令改变了默认值。要移除任何默认值，使用：

```
ALTER TABLE products ALTER COLUMN price DROP DEFAULT;
```

这等同于将默认值设置为空值。相应的，试图删除一个未被定义的默认值并不会引发错误，因为默认值已经被隐式地设置为空值。

8.2.6.6　修改列的数据类型

为了将一个列转换为一种不同的数据类型，使用如下命令：

```
ALTER TABLE products ALTER COLUMN price TYPE numeric(10,2);
```

只有当列中的每一个项都能通过一个隐式造型转换为新的类型时该操作才能成功。如果需要一种更复杂的转换，应该加上一个 USING 子句来指定应该如何把旧值转换为新值。

KingbaseES 将尝试把列的默认值转换为新类型，其他涉及到该列的任何约束也是一样。但是这些转换可能失败或者产生奇特的结果。因此最好在修改类型之前先删除该列上所有的约束，然后在修改完类型后重新加上相应修改过的约束。

8.2.6.7　重命名列

要重命名一个现有表的列，使用如下命令：

```
ALTER TABLE products RENAME COLUMN product_no TO product_number;
```

8.2.6.8　重命名表

要重命名一个现有表，使用如下命令：

```
ALTER TABLE products_table RENAME TO products_items;
```

products_table 表将被重命名为 products_items。

8.2.7　删除表

当一个表不再使用时，可以将其删除。删除表时，将产生以下结果：

- 表的结构信息从数据字典中删除，表中的数据不可访问；
- 表上的所有索引和触发器被一起清除；
- 所有建立在该表上的同义词、视图和存储过程变为无效；

一般情况下，普通用户只能删除自己模式下的表。若要删除其他模式下的表，则必须具有 DROP ANY TABLE 数据库权限。

以下语句可删除 emp 表：

```
DROP TABLE emp;
```

如果要删除的表被其他表引用，即其他表的外键引用了表的任何主键或唯一键，则需要在 DROP TABLE 语句中包含 CASCADE 选项，如：

```
DROP TABLE emp CASCADE;
```

当您删除一个表时，数据库通常不会立即释放与该表关联的空间。相反，数据库会重命名表并将其放入回收站，如果您发现错误地删除了该表，则可以使用 FLASHBACK TABLE 语句在回收站中恢复该表。如果您希望在发出 DROP TABLE 语句时立即释放与表相关的空间，请包含如下语句所示的 PURGE 子句:

```
DROP TABLE hr.emp PURGE;
```

8.2.8 使用 FLASHBACK 删除和恢复表

删除表时，数据库不会立即删除与该表关联的空间。数据库重命名表，并将其和任何关联的对象放在回收站中，万一表被错误地删除，可以在稍后恢复。这个特性被称为 FLASHBACK DROP, FLASHBACK TABLE 语句用于恢复表。

- *什么是回收站*

回收站实际上是一个数据字典表，包含关于被删除对象的信息。被删除的表和任何关联的对象 (如索引、约束、嵌套表等) 没有被删除，仍然会占用空间。

- *启用和禁用回收站*

当回收站被启用时，被删除的表及其相关对象将被放入回收站。当回收站被禁用时，被删除的表及其相关对象不会被放入回收站; 它们已被删除，您必须使用其他方法来恢复它们 (例如从备份中恢复)

- *从回收站恢复表*

使用 FLASHBACK TABLE …TO BEFORE DROP 语句从回收站中恢复表。

信息:

《KingbaseES 应用开发指南》中获取关于 FLASHBACK DROP TABLE 语句更多详细信息

8.2.8.1　什么是回收站

回收站实际上是一个数据字典表，包含关于被删除对象的信息。被删除的表和任何关联的对象 (如索引、约束、嵌套表等) 没有被删除，仍然会占用空间。这些已删除的对象会继续占用用户空间，直到从回收站中清除它们。

用户可以使用以下语句查看回收站中的对象:

```
SELECT * FROM RECYCLEBIN;
```

只有 SQL 语句，DROP TABLE 语句会将对象放入回收站。它添加表及其关联对象，以便它们可以作为一个组数据一起进行恢复。除了表本身，添加到回收站的关联对象可以包括以下类型的对象:

- 嵌套表
- LOB 数据库字段
- 索引
- 约束 (不包括外键约束)
- 触发器
- 聚簇

DROP TABLE 若是开启了 GUC 参数：kdb_flashback.db_recyclebin=on，且未指定 PURGE 选项，则删除的表及其关联的索引、规则、触发器、约束等将被放到 SYS_RECYCLEBIN 系统表中。在需要时可以将回收站中的表闪回到删除之前的状态。

8.2.8.2　启用和禁用回收站

回收站功能默认关闭，可以通过下列命令查看是否开启回收站功能：

```
show kdb_flashback.db_recyclebin;
```

开启回收站功能，在登录数据库后执行以下命令即可

```
alter system set kdb_flashback.db_recyclebin = on;
```

8.2.8.3　从回收站恢复表

开启了回收站参数后，如果不允许一个删除的表进回收站，需要在 DROP 语句后面加上 `PUGREX` 选项。flashback 支持将一个已经删除到回收站中的表及相关对象闪回到删除之前状态。

如果在当前 schema 下已经存在同名的表，则闪回失败，需要使用 rename to 选项指定一个新的表名。成功闪回一个表后，回收站视图 recyclebin 和回收站系统表 sys_recyclebin 中对象将被清除。

一个 FLASHBACK 命令：

```
FLASHBACK TABLE TABLE_name TO BEFORE DROP;
```

这个命令将在回收站中的 TABLE_name 闪回到闪回之前的状态：

```
FLASHBACK TABLE TABLE_name TO BEFORE DROP RENAME TO newname;
```

将回收站中的表闪回到一个新的表名：

提示:

要将删除的表放到回收站需要在 kingbase.conf 配置文件末尾添加 kdb_flashback.db_recyclebin=on。

8.2.9　管理分区表

分区表可以将数据分割为更小、更易于管理的小单位，称为分区，甚至是更小的子分区。每个分区可以有单独的物理属性，例如物理存储设置和表空间，能有效提升大型表的访问效率。此外，可以单独管理每个分区，可以简化和减少备份和管理所需的时间。

信息:

《系统原理》手册获取关于分区表管理的详细描述

8.2.10　表的数据视图

KingbaseES 提供了一组兼容 Oracle 的视图，您可以用来查询一组数据字典视图，获取表的信息。

视图	描述
DBA_TABLES ALL_TABLES USER_TABLES	DBA 视图描述数据库中的所有关系表。ALL 视图描述用户可访问的所有表。USER 视图仅限于用户拥有的表。
DBA_TAB_COLUMNS ALL_TAB_COLUMNS USER_TAB_COLUMNS	这些视图描述当前所在数据库中，所有的表、视图的非隐藏列信息
DBA_TAB_COMMENTS ALL_TAB_COMMENTS USER_TAB_COMMENTS	这些视图显示表和视图的注释。使用 COMMENT 语句输入注释。

-续表-

视图	描述
DBA_COL_COMMENTS ALL_COL_COMMENTS USER_COL_COMMENTS	这些视图显示表和视图列的注释。使用 COMMENT 语句输入注释。

信息:

《数据库参考手册》获取上述兼容 Oracle 视图的详细列表和信息

8.3　索引的管理

索引是一种建立在表上的数据库对象，主要作用就是加快对表的查询速度，合理的使用索引将大大减少磁盘访问次数，从而提升数据库性能。本章节索引的管理主要包括创建、修改、删除和查询索引。

- 关于索引

  索引是和表关联的可选结构，可以实现对表更快的 SQL 查询速度。

- 索引的管理指南

  可供参考的若干索引管理指南。

- 创建索引

  您可以创建多种不同类型的索引。您可以显式地创建索引，也可以创建与约束关联的索引。

- 修改索引

  若要更改索引，模式中必须包含该索引。

- 删除索引

  使用 DROP INDEX 语句删除索引。

- 索引视图

  通过查询索引视图，可以了解索引的相关信息。

8.3.1　关于索引

索引是和表关联的可选结构，可以实现对表更快的 SQL 查询速度。

就像本手册中的文章目录索引可以帮助您更快的查看到关注的章节，KingbaseES 数据库的索引也起到类似的作用，提供更快的访问表数据的方法。索引的新创建和使用无需重写任何查询语句，获取的结果与创建索引前是一样的，但您查询到结果的速度将更快了。

索引是如何加快查询速度的呢？那是因为，索引是建立在某个列或者几个列上 (即索引列)，创建索引时，数据库服务器将根据索引列进行排序，并将排序的结果存储在索引指定的存储空间中。在执行查询时，数据库服务器首先在索引中查询，然后再到表中查询，因为索引中的数据进行了排序，因此只需要很少的查询次数就可以找到指定的数据。

KingbaseES 数据库提供了几种功能互补的索引方案:

- b-tree 索引: 默认也是最常用的索引
- Hash 索引: 只能处理简单等值比较
- GiST 索引：可以实现很多不同索引策略
- SP-GiST 索引：允许实现众多不同的非平衡的基于磁盘的数据结构，例如四叉树、k-d 树和 radix 树。
- GIN 索引是“倒排索引”：它适合于包含多个组成值的数据值
- BRIN 索引（块范围索引的缩写）：存储有关存放在一个表的连续物理块范围上的值摘要信息。
- BITMAP 索引：以位图形式存储列中特定数据的出现位置信息。
- 全局索引和本地索引: 与分区表和索引相关
- 基于函数的索引: 基于函数或者表达式的索引

索引在逻辑上和物理上都是独立于关联表中的数据。作为独立的结构，它们固然需要额外的存储空间。您的创建或删除索引操作，是不会影响基表、数据库应用程序或其他索引。并且当您插入、更新和删除相关表的行时，数据库将自动维护索引。如果索引被删除，所有应用程序仍然继续正常工作，但访问数据可能会因此变慢。

8.3.2　索引的管理指南

可供参考的若干索引管理指南。

- *在插入表数据后创建索引*

一般情况下，在插入或加载数据之后，再为表创建索引会更有效率。

- *适合创建索引的表和列*

若干适合表和列创建索引的指导原则。

- *考虑排序索引列来改善性能*

CREATE INDEX 语句中的列顺序会影响查询性能。通常，首先指定最常用的列。

- *限制每个表的索引的数量*

一个表可以有任意数量的索引。但是，索引越多，修改表时产生的开销就越大，磁盘占用也越多。

- *删除不再需要的索引*

实践来看，当不再需要索引时，应及时将其删除。

- 为索引指定表空间

可以在表空间中创建索引。

- 评估索引大小和设置存储参数

在创建索引之前评估索引的占用空间的大小，将有益于磁盘空间规划和管理。

- 在禁用或删除约束之前考虑成本

因为唯一键和主键有关联的索引，所以在考虑是否禁用或删除 UNIQUE 或 PRIMARY KEY 约束时，应该考虑删除和创建索引的成本。

8.3.2.1　在插入表数据后创建索引

一般情况下，应在插入或加载数据之后，再为表创建索引会更有效率。如果您在加载数据之前创建了一个或多个索引，那么数据库必须在插入每一行数据时更新每个索引，导致插入效率降低。

8.3.2.2　适合创建索引的表和列

若干适合表和列创建索引的指导原则。可以参考下面的准则来决定何时为表创建索引：

- 如果需要经常地检索大表中的少量的行，就为查询键创建索引；
- 为了改善多个表的连接的性能，可为连接列创建索引；
- 主键和唯一键自动具有索引，在外键上很多情况下也创建索引；
- 如果查询语句花费的时间太长，那么就检查表大小。如果它发生了显著变化，则需要评估现有的索引 (如果有的话)。

选取表中的索引列时可以参考以下几点原则，越多符合越适合作为索引列：

- 列中的值相对比较唯一；
- 取值范围大，适合建立索引 (适合普通索引)；
- 列包含许多空值，但是查询通常会选择所有具有值的行。

反之，以下情况不应创建索引：

- 如果该表频繁的进行 DML 操作，不应建立索引，或者建立少量索引；
- 有很多重复值的列，一般不建议作为索引列；
- 太小的表，不用建立索引，也没有必要。

8.3.2.3　考虑排序索引列来改善性能

在 CREATE INDEX 语句中列的排序会影响查询的性能。通常，将最常用的列放在最前面。

如果查询中有多个字段组合定位，则不应为每个字段单独创建索引，而应该创建一个组合索引。当两个或多个字段都是等值查询时，组合索引中各个列的前后关系是无关紧要的。但是如果是非等值查询时，要想有效利用组合索引，则应该按等值字段在前，非等值字段在后的原则创建组合索引，查询时只能利用一个非等值的字段。

8.3.2.4　限制每个表的索引的数量

一个表可以有任意数量的索引。但是，索引越多，修改表数据的开销就越大，占用的磁盘空间也越大。

当插入或删除行时，表上的所有索引也要被更改；更改一个列时，包含该列的所有索引也要被更改。因此，在从表中检索数据的速度和更新表的速度之间需要权衡。典型的：如果一个表主要仅用于读，则索引多就有好处；如果一个表经常被更新，则索引不宜多建。

8.3.2.5　删除不再需要的索引

实践来看，当不再需要索引时，应及时将其删除。一方面能避免修改表数据的额外索引开销，另一方面也能减少磁盘空间占用。

要删除索引，您应该具有删除权限，或者该索引在您的模式中。在以下情况下应考虑将索引删除:

- 索引没有加速查询。比如，表非常小，或者表中可能有很多列，但索引项很少。
- 应用程序中的查询不使用索引，或者索引已无效。
- 重建索引之前，必须删除索引。
- 索引变得过于零散，应考虑重新创建索引，此时可以将其删除

8.3.2.6　为索引指定表空间

可以在表空间中创建索引。

如果表及其索引使用相同的表空间能更方便地对数据库进行管理（如表空间或文件备份）或保证应用的可用性，因为所有有关的数据总是在一起联机。然而，将表及其索引放在不同的表空间（在不同磁盘上）产生的性能比放在相同的表空间更好，因为这样做减少了磁盘竞争。但是将表及其索引放在不同的表空间时，如果一个表上某索引所在的表空间脱机了，则涉及这张表的 SQL 语句可能由于执行计划仍旧需要使用被脱机的索引而不能成功执行

8.3.2.7　评估索引大小和设置存储参数

在创建索引之前评估索引的占用空间的大小，将有益于磁盘空间规划和管理。

您可以使用综合评估的索引大小，结果对表、撤销表空间和重做日志文件的大小评估，以此来确定数据库所需的磁盘空间量，从而购买合适的硬件。

CREATE INDEX 语句创建索引时，可选的 WITH 子句可以为索引设置存储参数，每一种索引方法都有自己的存储参数集合。适当的存储参数可以改善使用该索引应用程序的 I/O 性能。例如，假设在创建索引之前估计索引的最大大小，之后就可以在创建该索引时设置适当的存储参数，就能很少为表的数据段分配簇。并且，所有的该索引的数据都被保存在相对连续的磁盘空间扇区中，这就减少了使用该索引的磁盘 I/O 操作所需的时间。

信息:

请参阅《KingbaseES 应用开发指南》中关于 CREATE INDEX(Oracle)、CREATE INDEX(SQLServer)、CREATE INDEX(MySQL) 语句设置存储参数更多详细的描述和信息。

8.3.2.8　在禁用或删除约束之前考虑成本

因为唯一键和主键有关联的索引，所以在考虑是否禁用或删除 UNIQUE 或 PRIMARY KEY 约束时，应该考虑删除和创建索引的成本。

如果 UNIQUE 键或 PRIMARY KEY 约束的关联索引非常大，那么您可以通过启用约束来节省时间，而不是删除并重新创建索引。

您还可以选择在删除或禁用 UNIQUE 或 PRIMARY KEY 约束时显式指定想要保留或删除索引。

8.3.3　创建索引

您可以创建多种不同类型的索引。您可以显式地创建索引，也可以创建基于函数或者表达式的索引等。要在用户自己的模式中创建索引，至少要满足如下条件之一：

- 要被索引的表是在自己的模式中；
- 在要被索引的表上有 CREATE INDEX 权限；

本节将描述如何创建不同类型的索引。

- 显式创建索引

  可以使用 SQL 语句 CREATE INDEX 显式地创建索引。

- 创建唯一索引

  索引可以是唯一的，也可以是非唯一的。

- 创建指定顺序的 B 树索引

  一个索引可能还需要将它们以指定的顺序传递。这使得查询中的 ORDER BY 不需要独立的排序步骤。

- 创建 Hash 索引

  Hash 索引只能处理简单等值比较。

- 创建基于函数的索引

基于函数的索引时将索引建立在某个函数或者表达式的基础上。

- 创建多列索引

一个索引可以定义在表的多个列上。

- 创建本地和全局索引

在分区表上创建的索引可分为全局索引和本地索引。

信息:

您也可以在《KingbaseES 应用开发指南 》中获取关于不同类型索引的详细描述

8.3.3.1　显式创建索引

可以使用 SQL 语句 CREATE INDEX 显式地创建索引。

下面的语句为 emp 表的 ename 列创建了一个名为 emp_ename_index 的索引，并存放在 users_space 表空间上:

```
CREATE INDEX emp_ename_index ON emp(ename)
      TABLESPACE users_space;
```

8.3.3.2　创建唯一索引

索引可以是唯一的，也可以是非唯一的。唯一索引保证表中没有两行在键列中有重复的值。非惟一索引则不会对列值施加此限制。使用 CREATE UNIQUE INDEX 语句创建一个唯一索引。下面的例子会为 dept 表的 ename 列创建唯一索引 dept_unique_index:

```
CREATE UNIQUE INDEX dept_unique_index ON dept (dname)
      TABLESPACE users_space;
```

备注:

当前，只有 B-tree 能够被声明为唯一。

当一个索引被声明为唯一时，索引中不允许多个表行具有相同的索引值。空值被视为不相同（除非创建索引时指定了 NULLS NOT DISTINCT 子句）。一个多列唯一索引将会拒绝在所有索引列上具有相同组合值的表行。

KingbaseES 会自动为定义了一个唯一约束或主键的表创建一个唯一索引。该索引包含组成主键或唯一约束的所有列（可能是一个多列索引），它也是用于强制这些约束的机制。

8.3.3.3　创建指定顺序的 B 树索引

一个索引可能还需要将它们以指定的顺序传递。这使得查询中的 ORDER BY 不需要独立的排序步骤。默认情况下，CREATE INDEX 命令创建适合于大部分情况的 B-tree 索引。如下列命令可以用来在表 test 上的 id 列上创建一个索引：

```
CREATE INDEX test_id_index ON test (id);
```

索引的名字 test_id_index 可以自由选择，但最好选择一个有相关性、能让用户想起该索引用途的名字。在 KingbaseES 当前支持的索引类型中，只有 B-tree 可以产生排序后的输出，其他索引类型会把行以一种没有指定的且与实现相关的顺序返回。B-tree 索引默认将表项以升序方式存储，并将空值放在最后 (在其他相等的项之间，表 TID 被视为平分符列)。我们可以在创建 B-tree 索引时通过下列选项来改变索引的排序

- ASC，指定上升排序（默认）。
- DESC，指定下降排序。
- NULLS FIRST，指定把空值排序在非空值前面。在指定 DESC 时，这是默认行为。
- NULLS LAST，指定把空值排序在非空值后面。在没有指定 DESC 时，这是默认行为。

那么典型的示例就是：

```
CREATE INDEX test2_info_nulls_low ON test2 (info NULLS FIRST);
CREATE INDEX test3_desc_index ON test3 (id DESC NULLS LAST);
```

8.3.3.4　创建 Hash 索引

Hash 索引只能处理简单等值比较。不论何时当一个索引列涉及到一个使用了 = 操作符的比较时，查询计划器将考虑使用一个 Hash 索引。下面的命令将创建一个 Hash 索引：

```
CREATE INDEX name ON table USING HASH (column);
```

8.3.3.5　创建基于函数的索引

基于函数的索引时将索引建立在某个函数或者表达式的基础上。一个索引列并不一定是底层表的一个列，也可以是从表的一列或多列计算而来的一个函数或者标量表达式。这种特性对于根据计算结果快速获取表中内容是有用的。

有一些查询，需要对表的列进行计算，此时就适合创建基于函数的索引。例如，一种进行大小写不敏感比较的常用方法是使用 lower 函数：

```
SELECT * FROM test1 WHERE lower(col1) = 'value';
```

由于 col1 没有索引，在执行语句时就需要先把列的值修改为小写，再和 value 进行匹配。为了加快速度，可以再 col1 列上创建索引，先对该列的值进行小写转换，然后再把索引存储在索引中，这样执行查询时就不再需要转换。函数结果之上的索引：

```
CREATE INDEX test1_lower_col1_idx ON test1 (lower(col1));
```

如果我们将该索引声明为 UNIQUE，它将阻止创建在 col1 值上只有大小写不同的行。

8.3.3.6　创建多列索引

一个索引可以定义在表的多个列上。

例如，我们有这样一个表：

```
CREATE TABLE test2(
  major int,
  minor int,
  name varchar
);
```

如果我们经常根据多列的相交 (通常为多个 and 操作) 或者联合 (通常为多个 or 操作) 操作做查询时，例如从表 test2 中查询 major 和 minor 均为常量 constant 的 name 信息：

```
SELECT name FROM test2 WHERE major = constant AND minor = constant;
```

那么我们可以在 major 和 minor 上定义一个多列索引：

```
CREATE INDEX test2_mm_idx ON test2 (major, minor);
```

相比于在每个列上创建索引，多列索引占用的磁盘相对更少，索引数据更新时也将更快。目前，只有 B-tree、Bitmap、GiST、GIN 和 BRIN 索引类型支持多列索引，最多可以指定 512 个列。

8.3.3.7　创建本地和全局索引

在分区表上创建的索引可分为全局索引和本地索引。

全局索引包括全局非分区索引（Global Nonpartitioned Indexes）和全局分区索引（Global Partitioned Indexes），是指与分区表有不同分区方式的索引，它是在分区表的所有分区数据基础上创建的索引。

本地索引（本地分区索引，Local Partitioned Indexes），是指在每个表分区上单独创建的索引，是一种局部索引，也是一种分区索引，某一个索引分区只能索引到一个表分区。

例如，在分区表上创建全局索引：

```
CREATE TABLE sales_table (COL1 INT, COL2 INT) PARTITION BY RANGE (COL1);
CREATE TABLE q1 partition of sales_table for values from (0) to (100);
CREATE TABLE q2 partition of sales_table for values from (101) to (1000);
CREATE unique INDEX GLOBAL_INDEX ON sales_table (COL2) GLOBAL;
```

对于非分区表而言，指定 GLOBAL/LOCAL 行为没有差别。对于分区表而言，当指定 GLOBAL 时创建全局索引，当不指定 GLOBAL 或者 LOCAL 时默认创建本地索引。

8.3.4 修改索引

若要修改索引，模式中必须包含该索引。

- 关于修改索引

可以使用 ALTER INDEX 或者 REINDEX 语句修改索引，两者的使用场景有区别。

- 修改或重建索引

通过 ALTER INDEX 或者 REINDEX 语句，可以实现修改索引的定义或者重建索引。

8.3.4.1 关于修改索引

可以使用 ALTER INDEX 或者 REINDEX 语句修改索引，两者的使用场景有区别。

ALTER INDEX 用于更改一个现有索引的属性定义，如修改索引名称，变更表空间等。而 REINDEX 则根据表重建一个索引，并且替换旧索引。如下场景应考虑使用 REINDEX：

- 一个索引已经损坏，并且不再包含合法数据。尽管理论上这不会发生，实际上索引会因为软件缺陷或硬件失效损坏。REINDEX 提供了一种恢复方法。
- 一个索引变得“臃肿”，其中包含很多空的或者近乎为空的页面。KingbaseES 中的 B-树索引在特定的非常规访问模式下可能会发生这种情况。REINDEX 提供了一种方法来减少索引的空间消耗，即制造一个新版本的索引，其中没有死亡页面。
- 修改了一个索引的存储参数（例如填充因子），并且希望确保这种修改完全生效。
- 如果 CONCURRENTLY 选项的索引构建失败，该索引被保留为“invalid”。这样的索引是无用的，但是可以方便地使用 REINDEX 来重建它们。注意，只有 REINDEX INDEX 能够在无效索引上执行并发构建。

8.3.4.2 修改或重建索引

通过 ALTER INDEX 或者 REINDEX 语句，可以实现修改索引的定义或者重建索引。

可以实现如下功能:

- 更改索引的名称
- 更改该索引的表空间为其他指定的表空间
- 把该索引标记为依赖于扩展
- 把和主表有相同表结构的主表变成该主表的一个分区
- 更改一个或者多个索引相关的存储参数

- 重置存储参数为默认值
- 为索引设置统计信息收集目标
- 重建索引

备注:

不能更改索引列结构。

具体的修改索引典型示例如下：

重命名一个现有索引：

```
ALTER INDEX distributors RENAME TO new_distributors;
```

把一个索引移动到一个不同的表空间：

```
ALTER INDEX distributors SET TABLESPACE fasttablespace;
```

更改一个索引的填充因子（假设该索引方法支持填充因子）：

```
ALTER INDEX distributors SET (fillfactor = 75);
REINDEX INDEX distributors;
```

为一个表达式索引设置统计信息收集目标：

```
CREATE INDEX coord_idx ON measured (x, y, (z + t));
ALTER INDEX coord_idx ALTER COLUMN 3 SET STATISTICS 1000;
```

重建单个索引：

```
REINDEX INDEX my_index;
```

重建表 my_table 上的所有索引：

```
REINDEX TABLE my_table;
```

KingbaseES 将重新构建索引，而不采取任何锁来防止表上的并发插入、更新或删除:

```
REINDEX TABLE CONCURRENTLY my_broken_table;
```

信息:

《KingbaseES 应用开发指南 》中获取关于 ALTER INDEX 和 REINDEX 语句的详细语法和描述信息。

8.3.5　删除索引

可以使用 DROP INDEX 语句删除索引。要执行这个命令你必须是该索引的拥有者。

删除索引的一些原因包括:

- 不再需要该索引；
- 该索引未能实现预期的性能改进。例如，表可能非常小；
- 应用程序不使用索引来查询数据；
- 索引已无效，必须在重新生成之前删除该索引；

当您删除一个索引时，该索引的在表空间上的所有索引段将得到释放。删除索引主要有两大类情况，如果使用 CREATE INDEX 语句显式创建索引，则可以用 DROP INDEX 语句删除该索引。如下面的语句删除 emp_ename 索引：

```
DROP INDEX emp_ename;
```

如果是通过在表上定义键约束隐式的创建索引。则不能直接删除与已启用的 UNIQUE KEY 键或 PRIMARY KEY 键约束相关的索引。此情况，必须停用或删除该约束本身。如下面的语句删除主键约束 pk_emp_name，同时删除其对应的索引：

```
ALTER TABLE emp DROP CONSTRAINT pk_emp_name;
```

当然，删除表也能实现删除索引，将自动删除了所有与该表相关的索引。

信息:

《KingbaseES 应用开发指南 》中获取关于 DROP INDEX 语句的详细语法和描述信息

8.3.6　索引视图

通过查询数据视图，您可以获取到索引相关的信息：

视图	描述
DBA_INDEXES ALL _INDEXES USER_INDEXES	DBA 视图会显示数据库中所有表上的索引信息。ALL 视图显示用户可访问的所有表上的索引信息。USER 视图仅限于用户拥有的索引。这些视图中的一些列包含由 sys_stat_statements 工具或 ANALYZE 语句生成的统计信息。
DBA_IND_COLUMNS ALL _IND_COLUMNS USER_IND _COLUMNS	这些视图显示表上索引列的信息。这些视图中的一些列包含由 sys_stat _statements 工具或 ANALYZE 语句生成的统计信息。

8.4　视图的管理

您可以执行创建视图、替换视图、更改视图和删除视图等任务。

- 关于视图

视图的形式类似于普通表，我们可以从视图中查询数据，实际上它是建立在表上的一种虚表，从本质上说，视图是对存储的查询。

- 创建视图和连接视图

可以使用 CREATE VIEW 语句创建视图。每个视图可以由引用表、物化视图或其他视图的查询来定义。还可以创建连接视图，即在 FROM 子句中指定多个基表或视图。

- 替换视图

可以通过删除并重新创建视图，或者通过执行带有 OR REPLACE 子句的 CREATE VIEW 语句实现替换现有视图的定义。

- 在查询中使用视图

可以像查询表一样查询视图，您还可以在视图上执行数据操作语句 (DML) 相关的操作，但有一些限制。

- 修改视图

使用 ALTER VIEW 语句只能显式地重新编译无效的视图。

- 删除视图

您可以使用 DROP VIEW 语句删除视图。

- 视图的数据字典视图

您可以查询指定数据字典视图，了解有关视图的信息。

8.4.1　关于视图

视图的形式类似于普通表，我们可以从视图中查询数据，实际上它是建立在表上的一种虚表，从本质上说，视图是对存储的查询。

视图从它所基于的表中获取数据，这些表称为基表。基表可以是实际的表，也可以是其他视图。在视图上执行的所有操作实际上都会影响视图的基表。可以像使用表一样使用视图。您可以对视图进行查询、更新、插入和删除，就像对表进行查询一样。

对视图的访问会转化成对其基表的访问，并且将所查询的结果以视图所规定的格式和顺序返回。因此当基表中的数据发生变化时，从视图中查询出的数据也随之改变了。

对用户而言通常要进行查询操作，甚至是复杂且频繁的查询，那么就可以为这个查询定义一个视图。假设用户经常执行下面的查询语句：

```
SELECT student_id, student_name
FROM student
WHERE age > 20 and gender = '男';
```

如果已经为该查询定义了一个名为 student_age_view 的视图，那么您就可以执行一条简单的语句获取您想要的结果。

```
SELECT * FROM student_age_view;
```

另外值得注意的是视图本身不保存对基表的查询结果，仅仅保存一条 SELECT 语句。从上面这个例子可以看出使用视图的目的是为了方便用户访问基表数据。实际上视图的另一个重要作用是确保用户对基表的安全访问。对于同一个表，不同的用户而言，我们可以定义不同规则的视图，从而保证用户只能进行允许的操作。

8.4.2　创建视图和连接视图

可以使用 CREATE VIEW 语句创建视图。每个视图可以由引用表、物化视图或其他视图的查询来定义。还可以创建连接视图，即在 FROM 子句中指定多个基表或视图。

- 创建视图

可以使用 CREATE VIEW 语句创建视图。

- 创建连接视图

您还可以在 CREATE VIEW 语句的 FROM 子句中指定多个基表或视图进行视图创建。

- 创建 FORCE 视图

如果 CREATE VIEW 语句带有 FORCE 关键字，意味着无论视图的依赖检查是否成功，只要语句没有语法错误，数据库也会创建该视图。

8.4.3　创建视图

可以使用 CREATE VIEW 语句创建视图。

创建视图需要满足以下要求:

- 要在模式中创建视图，必须具有 CREATE VIEW 权限。如果要在其他用户的模式中创建视图, 必须具有 CREATE ANY VIEW 系统权限。您可以显式地或通过角色获得这些特权。
- 视图的所有者 (无论是您还是其他用户) 必须被授予访问视图定义中引用的所有对象的权限。所有者不能通过角色获得这些特权。另外，视图的功能依赖于视图所有者的权限。例如，如果视图的所有者只有 test 表的 INSERT 权限，那么视图只能用于向 test 表插入新行，而不能用于 SELECT、UPDATE 或 DELETE 行。

- 如果视图的所有者打算将视图的访问权限授予其他用户，则该所有者必须通过 GRANT OPTION 获得基本对象的对象权限，或者通过 ADMIN OPTION 获得系统权限。

每个视图可以由引用表、物化视图或其他视图的查询定义。与所有子查询一样，定义视图的查询不能包含 FOR UPDATE 子句。为了说明视图，定义一张实验表，模拟学校的学生信息：

```
CREATE TABLE student (
  student_id NUMBER(4),
  student_name CHAR(8) NOT NULL,
  gender VARCHAR2(2),
  age NUMBER(2),
  grade VARCHAR2(8),
  classid NUMBER(2),
  email VARCHAR2(20),
  sdate DATE,
  CONSTRAINT pk_nmb PRIMARY KEY (student_id),
  CONSTRAINT ck_check CHECK (gender in ('男','女')),
  CONSTRAINT ck_age CHECK (age BETWEEN 12 AND 60)
);
```

下面的语句在 student 表中的数据子集上创建一个视图:

```
CREATE VIEW student_age_view AS
SELECT student_id,student_name,gender,age
FROM student
WHERE age > 20 and gender = '男'
WITH LOCAL CHECK OPTION;
```

上述例子创建了 student_age_view 视图，用于查询年龄大于 20 且性别为男的学生 ID 和姓名信息。LOCAL CHECK OPTION 子句是可选的，可以用来限定 DML 操作，并且只根据当前视图的 WHERE 条件进行限定。换种说法就是，针对该视图如果想执行 INSERT 和 UPDATE 语句，应确保新增加的数据可以被该视图重新查询到。例如，下面的 INSERT 语句根据 student_age_view 视图成功地将一行性别为男，年龄 21（大于 20）的学生数据插入到 student 表中:

```
INSERT INTO student_age_view VALUES (1726,'王猛','男',21);
```

但是，下面的 INSERT 语句将会返回一个错误，因为它试图插入一个性别为女，年龄为 19 的女学生信息，这是因为 student_age_view 视图的查询只引用性别为男且年龄大于 20 的行:

```
INSERT INTO student_age_view VALUES (1739,'王萌','女',19);
```

信息:

《KingbaseES 应用开发指南 》中关于 CREATE VIEW(Oracle)、CREATE VIEW(MySQL)、CRAETE VIEW(SQLServer) 语句详细语法描述

8.4.4　创建连接视图

您还可以在 CREATE VIEW 语句的 FROM 子句中指定多个基表或视图进行视图创建。这些基于多个表创建的视图就被称为连接视图，主要目的在于简化连接查询。

为了清楚的说明连接视图，下列是本章节视图中使用到的 user_info 和 practice_detail 两个表的详细定义：

```
CREATE TABLE user_info (
  id int NOT NULL,
  device_id int NOT NULL,
  gender varchar(14) NOT NULL,
  age int ,
  university varchar(32) NOT NULL,
  gpa float,
  active_days_within_30 int ,
  question_cnt int ,
  answer_cnt int
);

CREATE TABLE question_detail (
  id int NOT NULL,
  device_id int NOT NULL,
  question_id int NOT NULL,
  result varchar(32) NOT NULL
);
```

下面的创建视图语句创建了连接 user_info 和 practice_detail 表数据的 avg_answer_view 视图:

```
CREATE VIEW avg_answer_view AS
SELECT user_info.university,
    COUNT(question_id)/COUNT(distinct question_detail.device_id) AS avg_answer_cnt
FROM question_detail INNER JOIN user_info
ON user_info.device_id = question_detail.device_id
GROUP BY university;
```

上述视图例子根据 user_info 和 practice_detail 两表获取了指定的计算结果，但值得一提的是由于该视图中使用了 GROUP BY 子句和 DISTINCT 关键字，因此将不能通过该视图 avg_answer_view 进行 DML 相关操作。本节后面的内容将详细指出该限制条件。

8.4.5　创建 FORCE 视图

如果 CREATE VIEW 语句带有 FORCE 关键字，意味着无论视图的依赖检查是否成功，只要语句没有语法错误，数据库也会创建该视图。如果依赖检查成功则为有效视图，否则创建出来的就是无效视图。

比如视图引用了一个不存在的表或现有表中的无效列，或者视图所有者没有所需的权限时，数据库依然能创建该

视图并将其保存到数据字典中。只是该视图还不能使用，当引用的表恢复 (或者创建) 后，该视图重新编译后能恢复使用。

要创建 FORCE 视图，CREATE VIEW 语句必须包含 FORCE 关键字，具体的命令如下：

```
CREATE FORCE VIEW AS ...;
```

信息:

《KingbaseES 应用开发指南 》中关于 CREATE VIEW(Oracle)、CREATE VIEW(MySQL)、CRAETE VIEW(SQLServer) 语句中关于 FORCE 关键字的详细描述

8.4.6 替换视图

可以通过删除并重新创建视图，或者通过执行带有 OR REPLACE 子句的 CREATE VIEW 语句实现替换现有视图的定义。

要替换视图，您必须拥有删除和创建视图所需的所有权限。如果视图的 SELECT 定义必须修改，可以优先考虑使用带有 OR REPLACE 子句的 CREATE VIEW 语句执行视图替换，其他拥有关系、权限和非 SELECT 规则在内的视图属性不会被更改; 视图不能使用 ALTER VIEW 语句更改 SELECT 定义。

替换视图的方式具体如下:

- 您可以直接删除并重新创建视图。

备注:

当一个视图被删除时，所有相应的对象权限将从角色和用户中被撤销。在重新创建视图之后，必须重新授权。

- 可以使用包含 OR REPLACE 子句的 CREATE VIEW 语句重新定义视图。OR REPLACE 子句会替换当前定义的视图，并保留当前授权。

  例如，在前面小节所示创建的 student_age_view 视图，如果需要重新定义视图 WHERE 子句指定的年龄和性别以及新增查询结果。可以使用以下语句替换 student_age_view 视图为当前定义:

```
CREATE OR REPLACE VIEW student_age_view AS
  SELECT student_id, student_name, age
  FROM student
  WHERE age > 19 and gender = '女'
  WITH LOCAL CHECK OPTION;
```

在替换视图之前，请注意到可能出现的影响:

- 替换一个视图将替换数据字典中的视图定义。视图引用的所有基础对象都不受影响。
- 如果 CHECK OPTION 中不再指定 LOCAL 关键字，则将会根据当前新视图和所有基视图条件检查新数据。

8.4.7　在查询中使用视图

可以像表一样查询视图，您还可以在视图上执行数据操作语句 (DML) 相关的操作，但有一些限制。

要对视图执行查询或 INSERT、UPDATE 或 DELETE 语句，必须显式地或通过角色分别为视图拥有 SELECT、READ、INSERT、UPDATE 或 DELETE 对象权限。

类似于普通的表，可以以相同的方式进行查询。例如，要查询重新定义后的 student_age_view 视图（大于 19 岁的女生），输入引用该视图的 SELECT 语句，获取结果如下:

```
SELECT * FROM student_age_view;

student_id   student_name   gender  age
----------------------------------------
7844          宋义           男      20
7782          梅田田         女      21
7672          张华乐         男      20
7781          吉祥           男      21
7742          张丽莉         女      20
7900          黄金美         女      21
7654          倪文           女      20
7694          向玲           女      20
7698          向柯           男      20
```

可以使用视图向基表中插入、更新或从基表中删除行，但存在一些限制。下面的语句使用 student_age_view 视图向 student 表插入一个新行:

```
INSERT INTO student_age_view
    VALUES (7950, '王雯雯','女',20);
```

上述数据之所以能被成功插入到基表，是因为该行数据能被视图重新选中查询到，即满足 (只允许将年龄大于 19 且为女性)，同时不存在以下限制:

- 如果视图是包含了 DISTINCT 操作符、GROUP BY 子句、HAVING、LIMIT 或 OFFSET 的查询定义，则不能使用该视图向基表插入、更新或从基表删除行。
- 如果视图是用 WITH CHECK OPTION 定义的，则查询必须满足一定条件，如果视图不能从基表中选择行，则不能使用视图向基表插入或更新行。
- 如果不包含 DEFAULT 子句的 NOT NULL 列在视图中的新数据不满足该约束，则不能使用视图将行插入到基表中。
- 视图的选择列不能包含任何聚合、窗口函数或者集合返回函数。

8.4.8　修改视图

使用 ALTER VIEW 语句只能显式地重新编译无效的视图。

ALTER VIEW 语句允许您在运行前定位重新编译错误。要确保更改不会影响视图或依赖于它的其他对象，可以在更改视图的一个基表之后显式地重新编译视图。要使用 ALTER VIEW 语句，视图必须在您的模式中，或者您必须具有 ALTER ANY TABLE 系统权限。

信息:

《KingbaseES 应用开发指南》中关于 ALTER VIEW(Oracle)、ALTER VIEW(MySQL)、ALTER VIEW(SQLServer) 语句详细语法描述

8.4.9 删除视图

您可以使用 DROP VIEW 语句删除视图。

可以删除模式中包含的任何视图。要删除另一个用户模式中的视图，必须具有 DROP ANY VIEW 系统权限。例如，下面的语句删除了 student_age_view 视图:

```
DROP VIEW student_age_view;
```

信息:

《KingbaseES 应用开发指南》中关于 DROP VIEW(Oracle)、DROP VIEW(MySQL)、DROP VIEW(SQLServer) 语句详细语法描述

8.4.10 视图的数据字典视图

您可以查询指定的数据字典视图，了解有关视图的信息。

以下视图记录了显示关于视图的信息:

视图	描述
DBA_VIEWS, ALL_VIEWS, USER_VIEWS	DBA 视图描述数据库中的所有视图。ALL 视图被限制为当前用户可以访问的视图。USER 视图仅限于当前用户拥有的视图。

8.4.10.1 视图信息的查询

视图作为一种数据库对象，其相关信息存储在数据字典中。

与视图相关的视图有三个：

- USER_VIEWS，当前用户有关的视图信息
- DBA_VIEWS，系统中所有的视图信息
- ALL_VIEWS，当前用户可以访问的所有视图信息

例如要查询视图 stu_age_view 的相关信息，可以执行下面的 SELECT 语句：

```
SELECT text FROM user_views WHERE view_name = 'stu_age_view';
```

其中 text 中存储的是创建视图时使用的 SELECT 语句。

8.5　序列的管理

您可以执行创建序列、修改序列、使用序列和删除序列等任务。

- 关于序列

序列是一种数据库对象，用来自动生成一组具有一定规律 (增加或者减小) 变化的连续不同序列号，一般应用于表的主键列。

- 创建序列

使用 CREATE SEQUENCE 语句创建序列。

- 修改序列

使用 ALTER SEQUENCE 语句修改序列。

- 使用序列

对用户而言，序列中的可用资源就是序列号，序列号可以被多个用户访问并递增或者递减。

- 缓存序列号

使用缓存序列号可以提高访问效率。

- 删除序列

如果不再需要某个序列，可以使用 DROP SEQUENCE 语句删除该序列。

- 序列的数据字典视图

您可以查询指定数据字典视图，了解有关序列的信息。

8.5.1　关于序列

序列是一种数据库对象，用来自动生成一组具有一定规律 (增加或者减小) 变化的连续不同序列号。

序列一般应用于表的主键列，可以被多个用户使用，也可以为多个表生成主键。设想，如果没有序列，则只能编写程序生成顺序值，而在使用序列的情况下，当向表插入数据时，主键列就直接使用序列号赋值，保证了主键列没有重复值，也可以获得更可靠的主键值。

8.5.2　创建序列

使用 CREATE SEQUENCE 语句创建一个序列。

要在模式中创建序列，您必须具有 CREATE SEQUENCE 系统权限。要在其他用户的模式中创建序列，必须具有 CREATE ANY SEQUENCE 的权限。例如，下面的语句创建了一个序列，用于为 student 表的 student_id 主键列生成四位数的学生编号:

```
CREATE SEQUENCE stu_sequence
      INCREMENT BY 1
      START WITH 1000
      MINVALUE 1000
      MAXVALUE 9999
      CACHE 100
      NOCYCLE;
```

通过上述例子可以看出，您可以通过各种选项参数实现对序列的控制，指示序列是升序还是降序、序列号的起始值 (START WITH)、最小值 (MINVALUE) 和最大值 (MAXVALUE) 以及序列值之间的递增幅度 (INCREMENT BY) 等等。

其中 NOCYCLE 选项表示序列在达到最大值或最小值后不会生成更多值。如果希望序列号可以循环使用，请使用 CYCLE 选项。

CACHE 子句预先分配 100 个序列号保存在内存中。用户每使用一次序列，都需要对序列进行一次查询，当保存在内存中可以大大加快获取序列号，当缓存中的最后一个序列号被使用时，数据库将向缓存读入另一组序列号。

如果选择缓存一组序列号，则数据库可能会跳过该组序列号。例如，当实例异常关闭时 (例如，当发生实例故障或发出 SHUTDOWN ABORT 语句时)，已缓存但未使用的序列号将丢失。此外，已使用但未保存的序列号也会丢失。

信息:

《KingbaseES 应用开发指南 》中关于 CREATE SEQUENCE(Oracle)、CREATE SEQUENCE(SQLServer) 语句详细语法描述。

8.5.3　修改序列

使用 ALTER SEQUENCE 语句修改序列。

要更改序列，您的模式必须包含该序列，您必须对该序列具有 ALTER 对象权限，或者必须具有 ALTER ANY SEQUENCE 系统权限。您可以通过修改序列的生成数值的定义参数来实现序列修改。

下面的例子改变了 stu_sequence 序列的增长幅度为 2，循环使用序列号以及缓存数量变更为 20:

```
ALTER SEQUENCE stu_sequence
    INCREMENT BY 2
    CYCLE
    CACHE 20;
```

同时，您在修改序列的时候应注意到数值的合理性，比如下列语句则明显是不合理的，因为起始值 (START WITH) 是 1000，而下列修改试图将最小值修改为 2000：

```
ALTER SEQUENCE stu_sequence
  INCREMENT BY 2
    MINVALUE 2000
    MAXVALUE 8000
    CYCLE
    CACHE 20;
```

信息:

《KingbaseES 应用开发指南 》中关于 ALTER SEQUENCE(Oracle)、ALTER SEQUENCE(SQLServer) 语句详细语法描述。

8.5.4 使用序列

对用户而言，序列中的可用资源就是序列号，序列号可以被多个用户访问并递增或者递减。

要使用序列，您的模式必须包含该序列，或者您必须已被授予其他用户序列的 SELECT 对象权限。一旦序列完成创建，序列就可以被多个用户 (对包含该序列的序列拥有 SELECT 对象权限) 访问和递增，而无需等待。

- 使用 NEXTVAL 生成序列号

序列一旦生成，用户就可以在 SQL 语句中用 CURRVAL 和 NEXTVAL 伪列来存取序列的值。

- 使用 CURRVAL 序列号

要使用当前序列值，请在 SQL 语句中引用 seq_name.CURRVAL。

8.5.4.1 使用 NEXTVAL 生成序列号

序列一旦被创建，用户即可使用 SQL 语句中从 CURRVAL 和 NEXTVAL 伪列中获取序列的值。

CURRVAL 和 NEXTVAL 伪列如同序列的指针，NEXTVAL 每获得一个值，指针就向后移动一个位置，此时可以用 CURRVAL 获取当前序列号。即，其意义如下：

- CURRVAL 返回当前的序列值；
- NEXTVAL 返回下一个可用的序列号。

如上描述，NEXTVAL 和 CURRVAL 并不是保留字或关键字，可以在 SELECT、INSERT 或 UPDATE 等 SQL 语句中用作伪列名。在第一次使用序列号时，必须先访问 NEXTVAL 伪列。要生成和使用序列号，请引用 SQL 语句中的 NEXTVAL，如下示例，从伪表 dual 中查看序列号：

```
SELECT stu_sequence.NEXTVAL FROM dual;
```

这条语句的执行结果为：

```
NEXTVAL
----------
2000
```

例如，假设有新的学生注册，序列号可以在值列表中直接引用:

```
INSERT INTO student VALUES (stu_sequence.NEXTVAL, '张丽','女',18,'高中二年级',2,'105@qq.com',SYSDATE);
```

当需要更新某位同学的序列号时，可以在 UPDATE 语句的 SET 子句中引用序列号。例如:

```
UPDATE student SET student_id = stu_sequence.NEXTVAL WHERE student_id = 2022;
```

8.5.4.2　使用 CURRVAL 序列号

要使用当前序列值，请在 SQL 语句中引用 seq_name.CURRVAL。

使用 CURRVAL 之前，seq_name.NEXTVAL 必须在用户会话中已被引用 (在当前或以前的事务中) 至少一次。可以根据需要多次引用 CURRVAL，包括在同一条语句中多次引用。在引用 NEXTVAL 之前，不会生成下一个序列号。继续前面的示例，您将通过插入新学生数据，进行注册:

```
INSERT INTO student VALUES (stu_sequence.CURRVAL, '刘意','女',17,'高中一年级',4,'106@qq.com',SYSDATE);
INSERT INTO student VALUES (stu_sequence.CURRVAL, '赵文','男',19,'高中一年级',5,'107@qq.com',SYSDATE);
```

假设上一节给出的 INSERT 语句生成了一个新的序号 2022，那么本节中的语句插入的两行都插入了序号为 2022 的行。当然，如果引用 CURRVAL 的列定义了主键约束，此方法的引用会约束报错。

8.5.5　缓存序列号

缓存序列号可以提高访问效率。

- 关于缓存序列号

在序列缓存中访问序列号比从磁盘读取序列号更快。

- 序列缓存中的条目数

当应用程序访问序列缓存中的序列时，将快速读取序列号。

- 缓存中的序列号数量

当序列被读入缓存时，指定数量的序列号就已经被生成并存储在缓存中。

8.5.5.1　关于缓存序列号

在序列缓存中访问序列号比从磁盘读取序列号更快。序列缓存由条目组成，每个条目可以保存单个序列的多个序列号。

遵循这些指南快速访问所有序列号:

- 请确保序列缓存了足够的序列号，保证应用程序并发使用所有序列。
- 高并发情况应增加缓存中序列号的数量。

8.5.5.2　序列缓存中的条目数

当应用程序访问序列缓存中的序列时，将快速读取缓存中的序列号。

如果应用程序访问的序列不在缓存中，那么在使用序列号之前，必须将序列先从磁盘读取到缓存中。如果应用程序并发地使用许多序列，那么序列缓存可能不够大，不能容纳所有序列。在这种情况下，访问序列号可能经常需要读取磁盘。为了快速访问所有序列，请确保缓存中有足够数量的序列号，从而保证应用程序并发使用所有序列。

8.5.5.3　缓存中的序列号数量

当序列被读入缓存时，指定数量的序列号就已经被生成并存储在缓存中。

然后可以快速地访问这些值。存储在缓存中的序列号的数量由 CREATE SEQUENCE 语句中的 CACHE 参数决定，该参数的默认值为 1。CREATE SEQUENCE 语句创建 seq_test2 序列，该序列的 50 个值就存储在 SEQUENCE 缓存中:

```
CREATE SEQUENCE seq_test2
    CACHE 50;
```

8.5.6　删除序列

如果不再需要某个序列，可以使用 DROP SEQUENCE 语句删除该序列。

用户可以删除自己创建的任何序列。要在删除其他用户的序列，必须具有 DROP ANY SEQUENCE 系统权限。序列删除后，数据字典中不再保留相关信息。

例如，下面的语句删除了前面章节定义的 stu_sequence 序列:

```
DROP SEQUENCE stu_sequence;
```

信息:

《KingbaseES 应用开发指南 》中关于 DROP SEQUENCE(Oracle) 、DROP SEQUENCE(SQLServer) 语句详细语法描述。

8.5.7　序列的数据字典视图

您可以查询指定的数据字典视图，了解有关序列的信息。

以下视图记录了显示关于序列的信息:

视图	描述
DBA_SEQUENCES, ALL _SEQUENCES, USER _SEQUENCES	描述序列的视图。

8.5.7.1　序列信息的查询

序列作为一种数据库对象，其相关信息存储在数据字典中。

与序列相关的视图有三个：

- USER_SEQUENCES，当前用户有关的序列信息
- DBA_SEQUENCES，系统中所有的序列信息
- ALL_SEQUENCES，当前用户可以访问的所有序列信息

例如如果需要了解 stu_sequence 序列的信息，可以执行下面的 SELECT 语句：

```
SELECT min_value,max_value,increment_by,last_number,cycle_flag
FROM dba_sequences
WHERE sequence_name = 'stu_sequence';
--或者
SELECT *
FROM dba_sequences
WHERE sequence_name = 'stu_sequence';
```

8.6 同义词的管理

您可以执行创建同义词、使用同义词和删除同义词等任务。

- 关于同义词

可以为表、视图、存储过程，序列等数据库对象建立同义词，也可以为同义词再次建立同义词，甚至可以为一个不存在的对象建立同义词。

- 创建同义词

使用 CREATE SYNONYM 语句创建同义词。

- 在 DML 语句中使用同义词

在 DML 语句中，同义词的使用方式和基对象的使用方式完全相同。

- 删除同义词

您可以通过 DROP SYNONYM 语句来删除一个不再需要的同义词。

- 同义词的数据字典视图

您可以查询指定数据字典视图，了解有关同义词的信息。

8.6.1 关于同义词

可以为表、视图、存储过程，序列等数据库对象建立同义词，也可以为同义词再次建立同义词，甚至可以为一个不存在的对象建立同义词。

同义词本身并不包含原对象中的数据或者代码，它仅仅充当一个指针，是数据库对象的别名。此外，同义词易于使用，并能降低了数据库用户 SQL 语句的复杂性。同义词允许底层对象被重命名或移动，但必须重定义同义词，基于同义词的应用程序无需修改即可继续工作。您可以创建公共和私有的同义词。

公共同义词由名为 PUBLIC 的特殊用户组拥有，一般由 DBA 创建，数据库中的每个用户都可以访问它。私有同义词包含在特定用户的模式中 (非 public 模式)，仅对该用户和底层对象的被授予者可用。同义词本身不涉及安全，当您赋予一个同义词对象权限时，您实质上是在给同义词的基对象赋予权限。

8.6.2 创建同义词

使用 CREATE SYNONYM 语句创建同义词。

创建同义词时，底层模式对象不需要存在，也不需要访问对象的权限，也能创建同义词。下面的语句在包含在 schm 模式中的 tab1 表上创建一个名为 public_syn_tab1 的公共同义词:

```
CREATE PUBLIC SYNONYM public_syn_tab1 FOR schm.tab1;
```

您也可以使用带有 OR REPLACE 子句的 CREATE SYNONYM 将一个现有同义词定义替换掉：

```
CREATE OR REPLACE PUBLIC SYNONYM public_syn_tab1 FOR schm.tab1;
```

在创建同义词应注意的是：

- 同义词不能和同一模式中其他同义词同名，但不同模式下可以有同名的同义词
- 创建公有同义词不能指定模式
- 创建私有同义词不能指定 public 模式
- 创建私有同义词必须指定模式 (在当前模式路径为 public 时)

信息:

《KingbaseES 应用开发指南 》中关于 CREATE SYNONYM 语句详细语法描述

8.6.3　在 DML 语句中使用同义词

在 DML 语句中，同义词的使用方式和基对象的使用方式完全相同。

只要您被赋予了基对象权限，您就可以使用其他用户的私有同义词。例如，您只拥有 student 表的 SELECT 权限，student 表有一个同义词 stu，这时，您就只能对 stu 同义词进行查询操作，而不能使用它进行插入数据。同理，如果拥有基对象的插入权限，则可以使用其同义词进行插入操作。

例如，一个名称为 student 的同义词，下面语句是有效的：

```
INSERT INTO stu VALUES (stu_sequence.CURRVAL, '刘恩斯',18,'高中二年级',4,'108@qq.com',SYSDATE);
```

8.6.4　删除同义词

您可以通过 DROP SYNONYM 语句来删除一个不再需要的同义词。

如果您要删除公共同义词，您必须拥有 DROP PUBLIC SYNONYM 系统权限。当删除私有同义词时，省略 PUBLIC 关键字；当删除公共同义词时，需要包含 PUBLIC 关键字。

例如，下面语句删除名为 syn_test 的同义词：

```
DROP SYNONYM syn_test;
```

下面的语句删除名为 public_syn 的公共同义词：

```
DROP PUBLIC SYNONYM public_syn;
```

信息:

《KingbaseES 应用开发指南 》中关于 DROP SYNONYM 语句详细语法描述

8.6.5 同义词的数据字典视图

您可以查询指定的数据字典视图，了解有关同义词的信息。

以下视图记录了显示关于同义词的信息:

视图	描述
DBA_SYNONYMS, ALL _SYNONYMS, USER _SYNONYMS	描述同义词的视图。

8.6.5.1 同义词信息的查询

同义词作为一种数据库对象，其相关信息存储在数据字典中。

与同义词相关的视图有三个：

- USER_SYNONYMS，当前用户有关的同义词信息
- DBA_SYNONYMS，系统中所有的同义词信息, 包括每个用户创建的同义词
- ALL_SYNONYMS，当前用户可以访问的所有同义词信息，包括私有和共有同义词

要查询当前用户创建了那些同义词，各代表那个用户的对象，可以执行下列 SELECT 语句：

```
SELECT synonym_name,table_name,table_owner
FROM user_synonyms;
```

如果要在整个数据库范围内查询 system 用户创建的同义词，可以执行下面的 SELECT 语句：

```
SELECT synonym_name,table_name,table_owner
FROM dba_synonyms
WHERE owner='system';
```

```
SELECT synonym_name,table_name,table_owner
FROM dba_synonyms
WHERE owner='PUBLIC';
```

第9章　查询优化与任务管理

9.1　全文索引

本章节包含以下内容：

- 介绍
- 表和索引
- 控制文本搜索
- 额外特性
- 解析器
- 词典
- 配置例子
- 中文分词
- 使用 ftutilx 插件
- 测试和调试文本搜索
- GIN 和 GiST 索引类型
- RUM 索引类型
- ksql 支持
- 限制

9.1.1　介绍

全文搜索是指计算机索引程序通过扫描文档中的每一个词，对每一个词建立一个索引，并指明该词在文档中出现的次数和位置，当用户查询时，检索程序就根据事先建立的索引进行查找，并将查找的结果反馈给用户的检索方式。

全文搜索提供了满足查询一个自然语言文档的能力，并可以选择将它们按照与查询的相关度排序。最常用的搜索类型是找到所有包含给定查询词的文档并按照它们与查询的相似性顺序返回它们。查询和相似性的概念非常灵活并且依赖于特定的应用。最简单的搜索认为查询是一组词而相似性是查询词在文档中的频度。

文本搜索操作符已经在数据库中存在很多年了。KingbaseES 对文本数据类型提供了 `~`、`~*`、`LIKE` 和 `ILIKE` 操作符，但是它们缺少现代信息系统所要求的很多基本属性：

- 缺乏英语语言的支持。正则表达式是不够的，因为它们不易处理派生词，例如 `satisfies` 和 `satisfy`。因此可能会错过包含 `satisfies` 的文档，尽管你可能想要在对于 `satisfy` 的搜索中找到它们。可以使用 `OR` 来搜索多个派生形式，但此操作麻烦且容易出错（有些词可能有数千种派生）。
- 不提供对搜索结果的排序（排名），这使得在面对数以千计被找到的文档时变得无效。
- 因为没有索引支持它们很慢，因此必须为每次搜索处理所有的文档。

全文索引允许文档被预处理，并且保存一个索引用于以后快速的搜索。预处理包括：

- 将文档解析成记号。标识出多种类型的记号是有所帮助的，例如数字、词、复杂的词、电子邮件地址，这样它们可以被以不同的方式处理。原则上记号分类取决于相关的应用，但是对于大部分目的都可以使用一套预定义的分类。KingbaseES 使用一个解析器来执行这个步骤。其中提供了一个标准的解析器，并且为特定的需要也可以创建定制的解析器。
- 将记号转换成词位。和一个记号一样，一个词位是一个字符串，但是它已经被正规化，这样同一个词的不同形式被变成一样。例如，正规化几乎总是包括将大写字母转换成小写形式，并且经常涉及移除后缀（例如英语中的 `s` 或 `es`）。这允许搜索找到同一个词的变体形式，而不需要冗长地输入所有可能的变体。此外，这个步骤通常会消除停用词，它们是那些太普通的词，它们对于搜索是无用的（简而言之，记号是文档文本的原始片段，而词位是那些被认为对索引和搜索有用的词）。KingbaseES 使用词典来执行这个步骤。已经提供了多种标准词典，并且为特定的需要也可以创建定制的词典。
- 为搜索优化存储预处理好的文档。例如，每一个文档可以被表示为正规化的词位的一个有序数组。与词位一起，通常还想要存储用于近似排名的位置信息，这样一个包含查询词更“密集”区域的文档要比那些包含分散的查询词的文档有更高的排名。

词典允许对记号如何被正规化进行细粒度的控制。使用合适的词典，你可以：

- 定义不应该被索引的停用词。
- 使用 Ispell 把同义词映射到一个单一词。
- 使用一个分类词典把短语映射到一个单一词。
- 使用一个 Ispell 词典把一个词的不同变体映射到一种规范的形式。
- 使用 Snowball 词干分析器规则将一个词的不同变体映射到一种规范的形式。

KingbaseES 提供了一种数据类型 `tsvector` 来存储预处理后的文档，还提供了一种类型 `tsquery` 来表示处理过的查询（文本搜索类型）。有很多函数和操作符可以用于这些数据类型（文本搜索函数和操作符），其中最重要的是匹配操作符 `@@`，它在*基本文本匹配* 中介绍。全文搜索可以使用索引来加速（*GIN 和 GiST 索引类型* ）。

9.1.1.1　什么是一个文档

文档是在一个全文搜索系统中进行搜索的单元，例如，一篇杂志文章或电子邮件消息。文本搜索引擎必须能够解析文档并存储词位（关键词）与它们的父文档之间的关联。随后，这些关联会被用来搜索包含查询词的文档。

对于 KingbaseES 中的搜索，一个文档通常是一个数据库表中一行内的一个文本形式的域，或者可能是这类域的一个组合（连接），这些域可能存储在多个表或者是动态获取。换句话说，一个文档可能从用于索引的不同部分构建，并且它可能被作为一个整体存储在某个地方。例如：

```
CREATE table messages (
        mid serial primary key,
        title text,
        author text,
        abstract text,
        body text
);
INSERT INTO messages VALUES (1, 'kingbase introduce', 'kingbase', 'KES database', 'body describes kingbase
');
INSERT INTO messages VALUES (2, 'fly bird', 'mark', 'introduce fly bird story', 'body describes flyling bird
');
INSERT INTO messages VALUES (3, 'artificial intelligience', 'ai', 'introduce ai','body describes ai message
');

CREATE table docs (
        did serial primary key,
        title varchar(100),
        author varchar(100),
        abstract varchar(100),
        body varchar(100)
);
INSERT INTO docs VALUES (1, 'kingbase introduce', 'kingbase', 'KES database', 'body describes kingbase');
INSERT INTO docs VALUES (2, 'fly bird', 'mark', 'introduce fly bird story', 'body describes flyling bird');
INSERT INTO docs VALUES (3, 'artificial intelligience', 'ai', 'introduce ai','body describes ai message');

SELECT title || ' ' ||  author || ' ' ||  abstract || ' ' ||  body AS document
FROM messages
WHERE mid = 2;

SELECT m.title || ' ' || m.author || ' ' || m.abstract || ' ' || d.body AS document
FROM messages m, docs d
WHERE mid = did AND mid = 2;
```

备注:

实际上在这些例子查询中，coalesce 应该被用来防止一个单一 NULL 属性导致整个文档的一个 NULL 结果。

另一种存储文档的可能性是作为文件系统中的简单文本文件。在这种情况下，数据库可以被用来存储全文索引并执行搜索，并且某些唯一标识符可以被用来从文件系统检索文档。但是，从数据库的外面检索文件要求超级用户权限或者特殊函数支持，因此这种方法通常不如把所有数据放在 KingbaseES 内部方便。另外，把所有东西放在数据库内部允许方便地访问文档元数据来协助索引和现实。

对于文本搜索目的，每一个文档必须被缩减成预处理后的 tsvector 格式。搜索和排名被整个在一个文档的 tsvector 表示上执行—只有当文档被选择来显示给用户时才需要检索原始文本。我们因此经常把 tsvector 说成是文档，但是当然它只是完整文档的一种紧凑表示。

9.1.1.2　基本文本匹配

KingbaseES 中的全文搜索基于匹配操作符 @@，它在一个 tsvector（文档）匹配一个 tsquery（查询）时返回 true。哪种数据类型写在前面没有影响：

```
SELECT 'a fat cat sat on a mat and ate a fat rat'::tsvector @@ 'cat & rat'
::tsquery;
 ?column?
----------
 t

SELECT 'fat & cow'::tsquery @@ 'a fat cat sat on a mat and ate a fat rat'
::tsvector;
 ?column?
----------
 f
```

正如以上例子所建议的，一个 tsquery 并不只是一个未经处理的文本，顶多一个 tsvector 是这样。一个 tsquery 包含搜索术语，它们必须是已经正规化的词位，并且可以使用 AND 、OR、NOT 以及 FOLLOWED BY 操作符结合多个术语。有几个函数 to_tsquery、plainto_tsquery 以及 phraseto_tsquery 可用于将用户书写的文本转换为正确的 tsquery，它们会主要采用正则化出现在文本中的词的方法。相似地，to_tsvector 被用来解析和正规化一个文档字符串。因此在实际上一个文本搜索匹配可能看起来更像：

```
SELECT to_tsvector('fat cats ate fat rats') @@ to_tsquery('fat & rat');
 ?column?
----------
 f
```

注意如果这个匹配被写成下面这样它将不会成功：

```
SELECT 'fat cats ate fat rats'::tsvector @@ to_tsquery('fat & rat');
 ?column?
```

```
----------
 f
```

因为这里不会发生词 rats 的正规化。一个 tsvector 的元素是词位，它被假定为已经正规化好，因此 rats 不匹配 rat。

@@ 操作符也支持 text 输出，它允许在简单情况下跳过从文本字符串到 tsvector 或 tsquery 的显式转换。可用的变体是：

```
tsvector @@ tsquery
tsquery  @@ tsvector
text @@ tsquery
text @@ text
```

前两种我们已经见过。形式 text @@ tsquery 等价于 to_tsvector(x) @@ y。形式 text @@ text 等价于 to_tsvector(x) @@ plainto_tsquery(y)。

在 tsquery 中，&（AND）操作符指定它的两个参数都必须出现在文档中才表示匹配。类似地，|（OR）操作符指定至少一个参数必须出现，而!（NOT）操作符指定它的参数不出现才能匹配。例如，查询 fat & ! rat 匹配包含 fat 但不包含 rat 的文档。

在<->（FOLLOWED BY）tsquery 操作符的帮助下搜索可能的短语，只有该操作符的参数的匹配是相邻的并且符合给定顺序时，该操作符才算是匹配。例如：

```
SELECT to_tsvector('fatal error') @@ to_tsquery('fatal <-> error');
 ?column?
----------
 t

SELECT to_tsvector('error is not fatal') @@ to_tsquery('fatal <-> error');
 ?column?
----------
 f
```

FOLLOWED BY 操作符还有一种更一般的版本，形式是<N>，其中 N 是一个表示匹配词位位置之间的差。<1>和<-> 相同，而<2> 允许刚好一个其他词位出现在匹配之间，以此类推。当有些词是停用词时，phraseto_tsquery 函数利用这个操作符来构造一个能够匹配多词短语的 tsquery。例如：

```
SELECT phraseto_tsquery('cats ate rats');
       phraseto_tsquery
-------------------------------
 'cat' <-> 'ate' <-> 'rat'

SELECT phraseto_tsquery('the cats ate the rats');
       phraseto_tsquery
-------------------------------
```

```
'the' <-> 'cats' <-> 'ate' <-> 'the' <-> 'rats'
```

某些时候存在特殊情况：<0> 可以被用来要求两个匹配同一个词的模式。

圆括号可以被用来控制 tsquery 操作符的嵌套。如果没有圆括号，| 的计算优先级最低，然后从低到高依次是 &、<->、!。

值得注意的是，当 AND/OR/NOT 操作符在一个 FOLLOWED BY 操作符的参数中时，它们表示与不在那些参数中时不同的含义，因为在 FOLLOWED BY 中匹配的准确位置是有意义的。例如，通常!x 仅匹配在任何地方都不包含 x 的文档。但如果 y 不是紧接在一个 x 后面，!x <-> y 就会匹配那个 y，在文档中其他位置出现的 x 不会阻止匹配。另一个例子是，x & y 通常仅要求 x 和 y 均出现在文档中的某处，但是 (x & y) <-> z 要求 x 和 y 在紧挨着 z 之前的同一个位置匹配。因此这个查询的行为会不同于 x <-> z & y <-> z，它将匹配一个含有两个单独序列 x z 以及 y z 的文档（这个特定的查询一点用都没有，因为 x 和 y 不可能在同一个位置匹配，但是对于前缀匹配模式之类的更复杂的情况，这种形式的查询就会有用武之地）。

9.1.1.3　配置

前述的都是简单的文本搜索例子。正如前面所提到的，全文搜索功能包括做更多事情的能力：跳过索引特定词（停用词）、处理同义词并使用更高级的解析，例如基于空白之外的解析。这个功能由文本搜索配置控制。KingbaseES 中有多种语言的预定义配置，并且你可以很容易地创建你自己的配置（ksql 的\dF 命令显示所有可用的配置）。

在安装期间一个合适的配置将被选择并且 default_text_search_config 也被相应地设置在 kingbase.conf 中。如果你正在对整个集簇使用相同的文本搜索配置，你可以使用在 kingbase.conf 中使用该值。要在集簇中使用不同的配置但是在任何一个数据库内部使用同一种配置，使用 ALTER DATABASE ... SET。否则，可以在每个会话中设置 default_text_search_config。

依赖一个配置的每一个文本搜索函数都有一个可选的 regconfig 参数，因此要使用的配置可以被显式指定。只有当这个参数被忽略时，default_text_search_config 才被使用。

为了让建立自定义文本搜索配置更容易，一个配置可以从更简单的数据库对象来建立。KingbaseES 的文本搜索功能提供了四类配置相关的数据库对象：

- 文本搜索解析器将文档拆分成记号并分类每个记号（例如，作为词或者数字）。
- 文本搜索词典将记号转变成正规化的形式并拒绝停用词。
- 文本搜索模板提供位于词典底层的函数（一个词典简单地指定一个模板和一组用于模板的参数）。
- 文本搜索配置选择一个解析器和一组用于将解析器产生的记号正规化的词典。

文本搜索解析器和模板是从低层 C 函数构建而来，因此它要求 C 编程能力来开发新的解析器和模板，并且还需要超级用户权限来把它们安装到一个数据库中（在 KingbaseES 发布的 contrib/区域中有一些附加的解析器和模板的例子）。由于词典和配置只是对底层解析器和模板的参数化和连接，不需要特殊的权限来创建一个新词典或配置。创建定制词典和配置的例子将在本章稍后的部分给出。

9.1.2　表和索引

在前一节中的例子演示了使用简单常数字符串进行全文匹配。本节展示如何搜索表数据，以及可选择地使用索引。

9.1.2.1　搜索一个表

可以在没有一个索引的情况下做一次全文搜索。一个简单的查询将打印每一个行的 `title`，这些行在其 `body` 域中包含词 `friend`：

```
CREATE TABLE kbweb (
        kid serial primary key,
        title text,
        author text,
        abstract text,
        body text,
        config_name regconfig
);
INSERT INTO kbweb VALUES (1, 'friend and I', 'kingbase', 'KES database', 'body describes friendly kingbase',
'english');
INSERT INTO kbweb VALUES (2, 'friendly fly bird', 'mark', 'introduce bird', 'body describes flyling bird and
its friend', 'simple');
INSERT INTO kbweb VALUES (3, 'create table introduce', 'kingbase', 'abstract query', 'body describes create
query', 'english');
INSERT INTO kbweb VALUES (4, 'table create introduce', 'kingbase', 'abstract query', 'create mistake query',
'simple');

SELECT title
FROM kbweb
WHERE to_tsvector('english', body) @@ to_tsquery('english', 'friend');
```

这将还会找到相关的词例如 `friends` 和 `friendly`，因为这些都被约减到同一个正规化的词位。

以上的查询指定要使用 `english` 配置来解析和正规化字符串。我们也可以忽略配置参数：

```
SELECT title
FROM kbweb
WHERE to_tsvector(body) @@ to_tsquery('friend');
```

这个查询将使用由 default_text_search_config 设置的配置。

一个更复杂的例子是选择 10 个最近的文档，要求它们在 `title` 或 `body` 中包含 `create` 和 `table`：

```
SELECT title
FROM kbweb
```

```
WHERE to_tsvector(title || ' ' || body) @@ to_tsquery('create & table')
LIMIT 10;
```

为了清晰，我们忽略 coalesce 函数调用，它可能需要被用来查找在这两个域之中包含 NULL 的行。

尽管这些查询可以在没有索引的情况下工作，大部分应用会发现这种方法太慢了，除了偶尔的临时搜索。实际使用文本搜索通常要求创建一个索引。

9.1.2.2　创建索引

可以创建一个 GIN 索引（GIN 和 GiST 索引类型 ）来加速文本搜索：

```
CREATE INDEX kbweb_idx ON kbweb USING GIN(to_tsvector('english', body));
```

注意这里使用了 to_tsvector 的双参数版本。只有指定了一个配置名称的文本搜索函数可以被用在表达式索引中。这是因为索引内容必须是没有被 default_text_search_config 影响的。如果它们被影响，索引内容可能会不一致因为不同的项可能包含被使用不同文本搜索配置创建的 tsvector，并且没有办法猜测哪个是哪个。也没有可能正确地转储和恢复这样的一个索引。

由于 to_tsvector 的双参数版本被使用在上述的索引中，只有一个使用了带有相同配置名的双参数版 to_tsvector 的查询引用才能使用该索引。即，WHERE to_tsvector('english',) @@ ‘a & b’可以使用该索引，但 WHERE to_tsvector( ) @@ ’a & b’不能。这保证一个索引只能和创建索引项时所用的相同配置一起使用。可以建立更复杂的表达式索引，在其中配置名被另一个列指定，例如：

```
CREATE INDEX kbweb_idx ON kbweb USING GIN(to_tsvector(config_name, body));
```

这里 config_name 是 kbweb 表中的一个列。这允许在同一个索引中有混合配置，同时记录哪个配置被用于每一个索引项。例如，如果文档集合包含不同语言的文档，这就可能会有用。同样，要使用索引的查询必须被措辞成匹配，例如 ‘WHERE to_tsvector(config_name, body) @@ ‘a & b’ 。索引甚至可以连接列：

```
CREATE INDEX kbweb_idx ON kbweb USING GIN(to_tsvector('english', title || ' ' || body));
```

另一种方法是创建一个单独的 tsvector 列来保存 to_tsvector 的输出。若要使此列自动与源数据保持最新，请使用已存储的生成列。这个例子是 title 和 “ 的连接，使用 coalesce 来保证当其他域为 NULL 时一个域仍然能留在索引中：

```
ALTER TABLE kbweb
    ADD COLUMN textsearchable_index_col tsvector
               GENERATED ALWAYS AS (to_tsvector('english', coalesce(title, '') || ' ' || coalesce(body, '
')) STORED;
```

然后创建一个 GIN 索引来加速搜索：

```
CREATE INDEX textsearch_idx ON kbweb USING GIN(textsearchable_index_col);
```

执行一个快速的全文搜索了：

```
SELECT title
FROM kbweb
WHERE textsearchable_index_col @@ to_tsquery('create & table')
LIMIT 10;
```

在使用一个单独的列来存储 tsvector 表示时，有必要创建一个触发器在 title 或“ 改变时保证 tsvector 列为当前值。用于自动更新的触发器 解释了怎样去做。单独列方法相对于表达式索引的一个优势在于，它不必为了利用索引而在查询中显式地指定文本搜索配置。如上述例子所示，查询可以依赖 default_text_search_config。另一个优势是搜索将会更快，因为它不必重做 to_tsvector 调用来验证索引匹配（在使用 GiST 索引时这一点比使用 GIN 索引时更重要；见GIN 和 GiST 索引类型 ）。表达式索引方法更容易建立，但是它要求更少的磁盘空间，因为 tsvector 表示没有被显式地存储下来。

9.1.3　控制文本搜索

要实现全文搜索必须要有一个从文档创建 tsvector 以及从用户查询创建 tsquery 的函数。而且我们需要一种有用的顺序返回结果，因此我们需要一个函数能够根据文档与查询的相关性比较文档。还有一点重要的是要能够很好地显示结果。KingbaseES 对所有这些函数都提供了支持。

9.1.3.1　解析文档

KingbaseES 提供了函数 to_tsvector 将一个文档转换成 tsvector 数据类型。

```
to_tsvector([ config regconfig, ] document text) returns tsvector
```

to_tsvector 把一个文本文档解析成记号，把记号缩减成词位，并且返回一个 tsvector，它列出了词位以及词位在文档中的位置。文档被根据指定的或默认的文本搜索配置来处理。下面是一个简单例子：

```
SELECT to_tsvector('english', 'a fat  cat sat on a mat - it ate a fat rats');
                  to_tsvector
-----------------------------------------------------
 'ate':9 'cat':3 'fat':2,11 'mat':7 'rat':12 'sat':4
```

在上面这个例子中我们看到，作为结果的 tsvector 不包含词 a、on 或 it，词 rats 变成了 rat，并且标点符号-被忽略了。

to_tsvector 函数在内部调用了一个解析器，它把文档文本分解成记号并且为每一种记号分配一个类型。对于每一个记号，会去查询一个词典列表（词典 ），该列表会根据记号的类型而变化。第一个识别记号的词典产生一个或多个正规化的词位来表示该记号。例如，rats 变成 rat 是因为一个词典识别到该词 rats 是 rat 的复数形式。一些词会被识别为停用词，这将导致它们被忽略，因为它们出现得太频繁以至于在搜索中起不到作用。在我们的例子中有

a、on 和 it 是停用词。如果在列表中没有词典能识别该记号，那它将也会被忽略。在这个例子中标点符号-就属于这种情况，因为事实上没有词典会给它分配记号类型（空间符号），即空间记号不会被索引。对于解析器、词典以及要索引哪些记号类型是由所选择的文本搜索配置（配置例子 ）决定的。可以在同一个数据库中有多种不同的配置，并且有用于很多种语言的预定义配置。在我们的例子中，我们使用用于英语的默认配置 english。

函数 setweight 可以被用来对 tsvector 中的项标注一个给定的权重，这里一个权重可以是四个字母之一：A、B、C 或 D。这通常被用来标记来自文档不同部分的项，例如标题对正文。稍后，这种信息可以被用来排名搜索结果。

因为 to_tsvector(NULL) 将返回 NULL，不论何时一个域可能为空时，我们推荐使用 coalesce。下面是我们推荐的从一个结构化文档创建一个 tsvector 的方法：

```
UPDATE tt SET ti =
    setweight(to_tsvector(coalesce(title,'')), 'A')    ||
    setweight(to_tsvector(coalesce(keyword,'')), 'B')  ||
    setweight(to_tsvector(coalesce(abstract,'')), 'C') ||
    setweight(to_tsvector(coalesce(field_name,'')), 'D');
```

这里我们已经使用了 setweight 在完成的 tsvector 标注每一个词位的来源，并且接着将标注过的 tsvector 值用 tsvector 连接操作符 || 合并在一起（操纵文档给出了关于这些操作符的细节）。

9.1.3.2　解析查询

KingbaseES 提供了函数 to_tsquery、plainto_tsquery、phraseto_tsquery 以及 websearch_to_tsquery 用来把一个查询转换成 tsquery 数据类型。to_tsquery 提供了比 plainto_tsquery 和 phraseto_tsquery 更多的特性，但是它对其输入要求更加严格。websearch_to_tsquery 是 to_tsquery 的一个简化版本，它使用一种可选择的语法，类似于 Web 搜索引擎使用的语法。

```
to_tsquery([ config regconfig, ] querytext text) returns tsquery
```

to_tsquery 从 querytext 创建一个 tsquery 值，该值由被 tsquery 操作符 &（AND）、|（OR）、!（NOT）和<->（FOLLOWED BY）分隔的单个记号组成。这些操作符可以使用圆括号分组。换句话说，to_tsquery 的输入必须已经遵循 tsquery 输入的一般规则，区别在于基本的 tsquery 输入把记号当作表面值，而 to_tsquery 会使用指定的或者默认的配置把每一个记号正规化成一个词位，并且丢弃掉任何根据配置是停用词的记号。例如：

```
SELECT to_tsquery('english', 'The & Fat & Rats');
  to_tsquery
---------------
 'fat' & 'rat'
```

和在基本 tsquery 输入中一样，权重可以被附加到每一个词位来限制它只匹配属于那些权重的 tsvector 词位。例如：

```
SELECT to_tsquery('english', 'Fat | Rats:AB');
    to_tsquery
------------------
 'fat' | 'rat':AB
```

同样，* 可以被附加到一个词位来指定前缀匹配：

```
SELECT to_tsquery('supern:*A & star:A*B');
        to_tsquery
--------------------------
 'supern':*A & 'star':*AB
```

这样一个词位将匹配一个 tsvector 中的任意以给定字符串开头的词。

to_tsquery 也能够接受单引号短语。当配置包括一个会在这种短语上触发的分类词典时就是它的主要用处。在下面的例子中，一个分类词典含规则 supernovae stars : sn：

```
SELECT to_tsquery('''supernovae stars'' & !crab');
  to_tsquery
---------------
 'sn' & !'crab'
```

在没有引号时，to_tsquery 将为那些没有被 AND、OR 或者 FOLLOWED BY 操作符分隔的记号产生一个语法错误。

```
plainto_tsquery([ config regconfig, ] querytext text) returns tsquery
```

plainto_tsquery 将未格式化的文本 querytext 转换成一个 tsquery 值。该文本被解析并被正规化，很像 to_tsvector，然后 &（AND）布尔操作符被插入到留下来的词之间。

例子：

```
SELECT plainto_tsquery('english', 'The Fat Rats');
 plainto_tsquery
-----------------
 'fat' & 'rat'
```

注意 plainto_tsquery 不会识其输入中的 tsquery 操作符、权重标签或前缀匹配标签：

```
SELECT plainto_tsquery('english', 'The Fat & Rats:C');
   plainto_tsquery
---------------------
 'fat' & 'rat' & 'c'
```

这里，所有输入的标点都被作为空间符号并且丢弃。

```
phraseto_tsquery([ config regconfig, ] querytext text) returns tsquery
```

phraseto_tsquery 的行为很像 plainto_tsquery，不过前者会在留下来的词之间插入<->（FOLLOWED BY）操作符而不是 &（AND）操作符。还有，停用词也不是简单地丢弃掉，而是通过插入<N> 操作符（而不是<-> 操作符）来解释。在搜索准确的词位序列时这个函数很有用，因为 FOLLOWED BY 操作符不只是检查所有词位的存在性，还会检查词位的顺序。

例子：

```
SELECT phraseto_tsquery('english', 'The Fat Rats');
 phraseto_tsquery
------------------
 'fat' <-> 'rat'
```

和 plainto_tsquery 相似，phraseto_tsquery 函数不会识别其输入中的 tsquery 操作符、权重标签或者前缀匹配标签：

```
SELECT phraseto_tsquery('english', 'The Fat & Rats:C');
      phraseto_tsquery
-----------------------------
 'fat' <-> 'rat' <-> 'c'
```

```
websearch_to_tsquery([ config regconfig, ] querytext text) returns tsquery
```

websearch_to_tsquery 使用一种可供选择的语法从 querytext 创建一个 tsquery 值，这种语法中简单的未格式化文本是一个有效的查询。和 plainto_tsquery 以及 phraseto_tsquery 不同，它还识别特定的操作符。此外，这个函数绝不会报出语法错误，这就可以把原始的用户提供的输入用于搜索。支持下列语法：

- 无引号文本：不在引号中的文本将被转换成由 & 操作符分隔的词，就像被 plainto_tsquery 处理过那样。
- " 引号文本"：在引号中的文本将被转换成由<-> 操作符分隔的词，就像被 phraseto_tsquery 处理过那样。
- OR：逻辑或将被转换成 | 操作符。
- -：逻辑非操作符，被转换成! 操作符。

示例：

```
SELECT websearch_to_tsquery('english', 'The fat rats');
 websearch_to_tsquery
----------------------
 'fat' & 'rat'
(1 row)

SELECT websearch_to_tsquery('english', '"supernovae stars" -crab');
       websearch_to_tsquery
----------------------------------
 'supernova' <-> 'star' & !'crab'
```

```
(1 row)

SELECT websearch_to_tsquery('english', '"sad cat" or "fat rat"');
       websearch_to_tsquery
-----------------------------------
 'sad' <-> 'cat' | 'fat' <-> 'rat'
(1 row)

SELECT websearch_to_tsquery('english', 'signal -"segmentation fault"');
         websearch_to_tsquery
---------------------------------------
 'signal' & !( 'segment' <-> 'fault' )
(1 row)

SELECT websearch_to_tsquery('english', '""" )( dummy \\ query <->');
 websearch_to_tsquery
----------------------
 'dummi' & 'queri'
(1 row)
```

9.1.3.3　排名搜索结果

排名处理尝试度量文档和一个特定查询的接近程度，这样当有很多匹配时最相关的那些可以被先显示。KingbaseES 提供了两种预定义的排名函数，它们考虑词法、临近性和结构信息；即，它们考虑查询词在文档中出现得有多频繁，文档中的词有多接近，以及词出现的文档部分有多重要。不过，相关性的概念是模糊的并且与应用非常相关。不同的应用可能要求额外的信息用于排名，例如，文档修改时间。内建的排名函数只是例子。你可以编写你自己的排名函数和/或把它们的结果与附加因素整合在一起来适应你的特定需求。

目前可用的两种排名函数是：

`ts_rank([weights float4[],]*vector* tsvector, *query* tsquery[, normalization integer]) returns float4` 基于向量的匹配词位的频率来排名向量。

`ts_rank_cd([weights float4[],]*vector* tsvector, *query* tsquery[, normalization integer]) returns float4` 这个函数为给定文档向量和查询计算覆盖密度排名，该方法在 Clarke、Cormack 和 Tudhope 于 1999 年在期刊《Information Processing & Management》上的文章"Relevance Ranking for One to Three Term Queries" 中有描述。覆盖密度类似于 `ts_rank` 排名，不过它会考虑匹配词位相互之间的接近度。

> 这个函数要求词位的位置信息来执行其计算。因此它会忽略 `tsvector` 中任何"被剥离的"词位。如果在输入中有未被剥离的词位，结果将会是零（`strip` 函数和 `tsvector` 中的位置信息的更多内容请见操纵文档）。

对这两个函数，可选的权重参数提供了为词实例赋予更多或更少权重的能力，这种能力是依据它们被标注的情况的。权重数组指定每一类词应该得到多重的权重，按照如下的顺序：

```
{D-权重，C-权重，B-权重，A-权重}
```

如果没有提供权重，那么将使用这些默认值：

```
{0.1, 0.2, 0.4, 1.0}
```

通常权重被用来标记来自文档特别区域的词，如标题或一个初始的摘要，这样它们可以被认为比来自文档正文的词更重要或更不重要。

由于一个较长的文档有更多的机会包含一个查询术语，因此考虑文档的尺寸是合理的，例如一个一百个词的文档中有一个搜索词的五个实例而零一个一千个词的文档中有该搜索词的五个实例，则前者比后者更相关。两种排名函数都采用一个整数正规化选项，它指定文档长度是否影响其排名以及如何影响。该整数选项控制多个行为，因此它是一个位掩码：你可以使用 `|` 指定一个或多个行为（例如，`2|4`）。

- 0（默认值）忽略文档长度
- 1 用 1 + 文档长度的对数除排名
- 2 用文档长度除排名
- 4 用长度之间的平均调和距离除排名（只被 `ts_rank_cd` 实现）
- 8 用文档中唯一词的数量除排名
- 16 用 1 + 文档中唯一词数量的对数除排名
- 32 用排名 + 1 除排名

如果多于一个标志位被指定，转换将根据列出的顺序被应用。

值得注意的是排名函数并不使用任何全局信息，因此它不可能按照某些时候期望地产生一个公平的正规化，从 1% 或 100%。正规化选项 32（`rank/(rank+1)`）可以被应用来缩放所有的排名到范围零到一，但是当然这只是一个外观上的改变；它不会影响搜索结果的顺序。

这里是一个例子，它只选择十个最高排名的匹配：

```
CREATE TABLE apod (
        aid serial primary key,
        title text,
        textsearch tsvector
);
INSERT INTO apod VALUES (1, 'Neutrinos in the Sun', to_tsvector('Neutrinos in the Sun'));
INSERT INTO apod VALUES (2, 'The Sudbury Neutrino Detector', to_tsvector('The Sudbury Neutrino Detector'));
INSERT INTO apod VALUES (3, 'A MACHO View of Galactic Dark Matter', to_tsvector('A MACHO View of Galactic
Dark Matter'));
INSERT INTO apod VALUES (4, 'Hot Gas and Dark Matter', to_tsvector('Hot Gas and Dark Matter'));
INSERT INTO apod VALUES (5, 'The Virgo Cluster: Hot Plasma and Dark Matter', to_tsvector('The Virgo Cluster
Hot Plasma and Dark Matter'));
INSERT INTO apod VALUES (6, 'Rafting for Solar Neutrinos', to_tsvector('Rafting for Solar Neutrinos'));
INSERT INTO apod VALUES (7, 'NGC 4650A: Strange Galaxy and Dark Matter', to_tsvector('NGC 4650A Strange
```

```
Galaxy and Dark Matter'));
INSERT INTO apod VALUES (8, 'Hot Gas and Dark Matter', to_tsvector('Hot Gas and Dark Matter'));
INSERT INTO apod VALUES (9, 'Ice Fishing for Cosmic Neutrinos', to_tsvector('Ice Fishing for Cosmic
Neutrinos'));
INSERT INTO apod VALUES (10, 'Weak Lensing Distorts the Universe', to_tsvector('Weak Lensing Neutrinos the
Universe'));
INSERT INTO apod VALUES (11, 'Weak for Distorts the galaxy', to_tsvector('Weak for Distorts the galaxy'));
INSERT INTO apod VALUES (12, 'NGC 4240A: storage has fulled', to_tsvector('NGC 4240A storage has fulled'));

SELECT title, ts_rank_cd(textsearch, query) AS rank
FROM apod, to_tsquery('neutrino|(dark & matter)') query
WHERE query @@ textsearch
ORDER BY rank DESC
LIMIT 10;
                     title                     | rank
-----------------------------------------------+------
 Neutrinos in the Sun                          |  0.1
 The Sudbury Neutrino Detector                 |  0.1
 A MACHO View of Galactic Dark Matter          |  0.1
 Hot Gas and Dark Matter                       |  0.1
 The Virgo Cluster: Hot Plasma and Dark Matter |  0.1
 Rafting for Solar Neutrinos                   |  0.1
 NGC 4650A: Strange Galaxy and Dark Matter     |  0.1
 Hot Gas and Dark Matter                       |  0.1
 Ice Fishing for Cosmic Neutrinos              |  0.1
 Weak Lensing Distorts the Universe            |  0.1
(10 rows)
```

这是相同的例子使用正规化的排名：

```
SELECT title, ts_rank_cd(textsearch, query, 32 /* rank/(rank+1) */ ) AS rank
FROM apod, to_tsquery('neutrino|(dark & matter)') query
WHERE  query @@ textsearch
ORDER BY rank DESC
LIMIT 10;
                     title                     |    rank
-----------------------------------------------+------------
 Neutrinos in the Sun                          | 0.09090909
 The Sudbury Neutrino Detector                 | 0.09090909
 A MACHO View of Galactic Dark Matter          | 0.09090909
 Hot Gas and Dark Matter                       | 0.09090909
 The Virgo Cluster: Hot Plasma and Dark Matter | 0.09090909
 Rafting for Solar Neutrinos                   | 0.09090909
 NGC 4650A: Strange Galaxy and Dark Matter     | 0.09090909
 Hot Gas and Dark Matter                       | 0.09090909
```

```
 Ice Fishing for Cosmic Neutrinos              | 0.09090909
 Weak Lensing Distorts the Universe            | 0.09090909
(10 rows)
```

排名可能会非常昂贵，因为它要求查询每一个匹配文档的 tsvector，这可能会涉及很多 I/O 因而很慢。不幸的是，这几乎不可能避免，因为实际查询常常导致巨大数目的匹配。

9.1.3.4 加亮结果

要表示搜索结果，理想的方式是显示每一个文档的一个部分并且显示它是怎样与查询相关的。通常，搜索引擎显示文档片段时会对其中的搜索术语进行标记。KingbaseES 提供了一个函数 ts_headline 来实现这个功能。

```
ts_headline([ config regconfig, ] document text, query tsquery [, options text ]) returns text
```

ts_headline 接受一个文档和一个查询，并且从该文档返回一个引用，在其中来自查询的术语会被加亮。被用来解析该文档的配置可以用 config 指定；如果 config 被忽略，将会使用 default_text_search_config 配置。

如果一个 options 字符串被指定，它必须由一个逗号分隔的列表组成，列表中是一个或多个 option=value 对。可用的选项是：

- StartSel、StopSel：用来定界文档中出现的查询词的字符串，这用来把它们与其他被引用的词区分开。如果这些字符串包含空格或逗号，你必须把它们加上双引号。
- MaxWords、MinWords：这些数字决定要输出的最长和最短 headline。
- ShortWord：长度小于等于这个值的词将被从一个 headline 的开头或结尾处丢掉。默认值三消除普通英语文章。
- HighlightAll：布尔标志，如果为 true 整个文档将被用作 headline，并忽略前面的三个参数。
- MaxFragments：要显示的文本引用或片段的最大数量。默认值零选择一种非片段倾向的 headline 生成方法。一个大于零的值选择基于片段的 headline 生成。这种方法找到有尽可能多查询词的文本片段并且展开查询词周围的那些片段。结果是查询词会靠近每个片段的中间并且在其两侧都有词。每一个片段将是最多 MaxWords 并且长度小于等于 ShortWord 的词被从每个片段的开头或结尾丢弃。如果不是所有的查询词都在该文档中找到，文档中第一个 MinWords 的单一片段将被显示。
- FragmentDelimiter：当多于一个片段被显示时，片段将被这个字符串所分隔。

这些选项名称不区分大小写。任何未指定的选项将收到这些默认值：

```
StartSel=<b>, StopSel=</b>,
MaxWords=35, MinWords=15, ShortWord=3, HighlightAll=FALSE,
MaxFragments=0, FragmentDelimiter=" ... "
```

例如：

```
SELECT ts_headline('english',
  'The most common type of search
```

```
is to find all documents containing given query terms
and return them in order of their similarity to the
query.',
  to_tsquery('query & similarity'));
                        ts_headline
------------------------------------------------------------
 containing given <b>query</b> terms
 and return them in order of their <b>similarity</b> to the
 <b>query</b>.

SELECT ts_headline('english',
  'The most common type of search
is to find all documents containing given query terms
and return them in order of their similarity to the
query.',
  to_tsquery('query & similarity'),
  'StartSel = <, StopSel = >');
                      ts_headline
-------------------------------------------------------
 containing given <query> terms
 and return them in order of their <similarity> to the
 <query>.
```

ts_headline 使用原始文档，而不是一个 tsvector 摘要，因此它可能很慢并且应该被小心使用。

9.1.4　额外特性

这一节描述在文本搜索中有用的一些额外的函数和操作符。

9.1.4.1　操纵文档

解析文档 展示了未经处理的文本文档如何被转换成 tsvector 值。KingbaseES 也提供了用于操纵已经为 tsvector 形式的文档的函数和操作符。

tsvector || tsvector

tsvector 连接操作符返回一个向量，它结合了作为参数给出的两个向量的词位和位置信息。位置和权重标签在连接期间被保留。出现在右手向量中的位置被使用左手向量中提到的最大位置进行偏移，这样结果几乎等于在两个原始文档字符串的连接上执行 to_tsvector 的结果（这种等价不是完全的，因为从左手参数的尾端移除的任何停用词将会影响结果，而如果文本连接被使用它们就影响了右手参数中的词位位置）。

使用向量形式的连接而不是在应用 to_tsvector 之前连接文本的一个优点是你可以使用不同配置来解析文档的不同小节。此外，因为 setweight 函数按照相同的方式标记给定向量的所有词位，如果你想把文档的不同部分标注不同的权重，你就有必要解析文本并且在连接之前做 setweight。

setweight(vector tsvector, weight "char") returns tsvector
setweight 返回输入向量的一个拷贝，其中每一个位置都被标注为给定的权重：A、B、C 或 D（D 是新向量的默认值并且并不会被显示在输出上）。向量被连接时会保留这些标签，允许来自文档的不同部分的词被排名函数给予不同的权重。

注意权重标签是应用到位置而不是词位。如果输入向量已经被剥离了位置，则 setweight 什么也不会做。

length(vector tsvector) returns integer
返回存储在向量中的词位数。

strip(vector tsvector) returns tsvector
返回一个向量，其中列出了和给定向量相同的词位，不过没有任何位置或者权重信息。其结果通常比未被剥离的向量小很多，但是用处也小很多。和未被剥离的向量一样，相关度排名在已剥离的向量上也不起作用。此外，<->（FOLLOWED BY）tsquery 操作符不会匹配已剥离的输入，因为它无法确定词位之间的距离。

9.1.4.2　操纵查询

解析查询 展示了未经处理的文本形式的查询如何被转换成 tsquery 值。KingbaseES 也提供了用于操纵已经是 tsquery 形式的查询的函数和操作符。

tsquery tsquery
返回用 AND 结合的两个给定查询。

tsquery tsquery
返回用 OR 结合的两个给定查询。

!! tsquery
返回一个给定查询的反（NOT）。

tsquery <-> tsquery
返回一个查询，它用<->（FOLLOWED BY）tsquery 操作符搜索两个紧跟的匹配，第一个匹配符合第一个给定的查询而第二个匹配符合第二个给定的查询。例如：

```
SELECT to_tsquery('fat') <-> to_tsquery('cat | rat');
             ?column?
-----------------------------------
 'fat' <-> 'cat' | 'fat' <-> 'rat'
```

tsquery_phrase(query1 tsquery, query2 tsquery[, distance integer]) returns tsquery
返回一个查询，它使用<N> tsquery 操作符搜索两个距离为 distance 个词位的匹配，第一个匹配符合第一个给定的查询而第二个匹配符合第二个给定的查询。例如：

```
SELECT tsquery_phrase(to_tsquery('fat'), to_tsquery('cat'), 10);
  tsquery_phrase
------------------
 'fat' <10> 'cat'
```

numnode(query tsquery) returns integer

返回一个 tsquery 中的结点数（词位外加操作符）。要确定查询是否有意义或者是否只包含停用词时，这个函数有用，在前一种情况它返回 > 0，后一种情况返回 0。例子：

```
SELECT numnode(plainto_tsquery('english', 'the any'));
NOTICE:  text-search query contains only stop words or doesn't contain lexemes, ignored
ignored
 numnode
---------
       0

SELECT numnode('foo & bar'::tsquery);
 numnode
---------
       3
```

querytree(query tsquery) returns text

返回一个 tsquery 中可以被用来搜索一个索引的部分。这个函数可用来检测不可被索引的查询，例如那些只包含停用词或者只有否定术语的查询。例如：

```
SELECT querytree(to_tsquery('!defined'));
 querytree
-----------
 T
```

查询重写

ts_rewrite 函数族在一个给定的 tsquery 中搜索一个目标子查询的出现，并且将每一次出现替换成一个替补子查询。本质上这个操作就是一个 tsquery 版本的子串替换。一个目标和替补的组合可以被看成是一个查询重写规则。一个这类重写规则的集合可以是一个强大的搜索助手。例如，你可以使用同义词扩展搜索（如，new york、big apple、nyc、gotham），或者收缩搜索来将用户导向某些特点主题。在这个特性和分类词典（分类词典 ）有些功能重叠。但是，你可以随时修改一组重写规则而无需重索引，而更新一个分类词典则要求进行重索引才能生效。

ts_rewrite (query tsquery, target tsquery, substitute tsquery) returns tsquery

这种形式的 ts_rewrite 简单地应用一个单一重写规则：不管 target 出现在 query 中的那个地方，它都被 substitute 替代。例如：

```
SELECT ts_rewrite('a & b'::tsquery, 'a'::tsquery, 'c'::tsquery);
 ts_rewrite
------------
 'b' & 'c'
```

ts_rewrite (query tsquery, select text) returns tsquery

这种形式的 ts_rewrite 接受一个开始 query 和一个 SQL*select* 命令，它们以一个文本字符串的形式给出。select 必须得到 tsquery 类型的两列。对于 select 结果的每一行，在当前 query 值中出现的第一列值（目标）被第二列值（替补）所替换。例如：

```
CREATE TABLE aliases (t tsquery PRIMARY KEY, s tsquery);
INSERT INTO aliases VALUES('a', 'c');

SELECT ts_rewrite('a & b'::tsquery, 'SELECT t,s FROM aliases');
 ts_rewrite
------------
 'b' & 'c'
```

注意当多个重写规则被以这种方式应用时，应用的顺序很重要；因此在实际中你会要求源查询按某些排序键 ORDER BY。

考虑一个现实的天文学例子。将使用表驱动的重写规则扩展查询 supernovae：

```
CREATE TABLE aliases (t tsquery primary key, s tsquery);
INSERT INTO aliases VALUES(to_tsquery('supernovae'), to_tsquery('supernovae|sn'));

SELECT ts_rewrite(to_tsquery('supernovae & crab'), 'SELECT * FROM aliases');
           ts_rewrite
---------------------------------
 'crab' & ( 'supernovae' | 'sn' )
```

可以通过只更新表来改变重写规则：

```
UPDATE aliases
SET s = to_tsquery('supernovae|sn & !nebulae')
WHERE t = to_tsquery('supernovae');

SELECT ts_rewrite(to_tsquery('supernovae & crab'), 'SELECT * FROM aliases');
                 ts_rewrite
---------------------------------------------
 'crab' & ( 'supernovae' | 'sn' & !'nebulae' )
```

当有很多重写规则时，重写可能会很慢，因为它要为为每一个可能的匹配检查每一条规则。要过滤掉明显不符合的规则，可为 tsquery 类型使用包含操作符。在下面的例子中，只选择那些可能匹配原始查询的规则：

```
SELECT ts_rewrite('a & b'::tsquery,
                  'SELECT t,s FROM aliases WHERE ''a & b''::tsquery @> t');
 ts_rewrite
------------
 'b' & 'a'
```

9.1.4.3　用于自动更新的触发器

备注:

本节中描述的方法由于使用了存储生成的列而被废弃，正如创建索引 中所描述的那样。

当使用一个单独的列来存储文档的 tsvector 表示时，有必要创建一个触发器在文档内容列改变时更新 tsvector 列。两个内建触发器函数可以用于这个目的，或者可以编写自己的触发器函数。

```
tsvector_update_trigger(tsvector_column_name, config_name, text_column_name  [, ... ])
tsvector_update_trigger_column(tsvector_column_name, config_column_name, text_column_name [, ... ])
```

这些触发器函数在 CREATE TRIGGER 命令中指定的参数控制下，自动从一个或多个文本列计算一个 tsvector 列。它们使用的一个例子是：

```
CREATE TABLE messages (
title       text,
body        text,
tsv         tsvector
);

CREATE TRIGGER tsvectorupdate BEFORE INSERT OR UPDATE
ON messages FOR EACH ROW EXECUTE FUNCTION
tsvector_update_trigger(tsv, 'pg_catalog.english', title, body);

INSERT INTO messages VALUES('title here', 'the body text is here');

SELECT * FROM messages;
   title    |          body         |            tsv
------------+-----------------------+----------------------------
 title here | the body text is here | 'bodi':4 'text':5 'titl':1

SELECT title, body FROM messages WHERE tsv @@ to_tsquery('bodi & titl');
   title    |          body
------------+-----------------------
 title here | the body text is here
```

创建触发器后，在 title 或 body 中的任何修改将自动地被反映到 tsv 中，不需要应用来操心同步的问题。

第一个触发器参数必须是要被更新的 tsvector 列的名字。第二个参数指定要被用来执行转换的文本搜索配置。对于 tsvector_update_trigger，配置名被简单地用第二个触发器参数给出。如上所示，它必须是模式限定的，因此该触发器行为不会因为 search_path 中的改变而改变。对于 tsvector_update_trigger_column，第二个触发器参数是另一个表列的名称，它必须是类型 regconfig。这允许做一种逐行的配置选择。剩下的参数是文本列的名称（类型为 text、varchar 或 char）。它们将按给定的顺序被包括在文档中。NULL 值将被跳过（但是其他列仍将被索引）。

这些内建触发器的一个限制是它们将所有输入列同样对待。要对列进行不同的处理—例如，使标题的权重和正文的不同—就需要编写一个自定义触发器。下面是用 PL/SQL 作为触发器语言的一个例子：

```
CREATE FUNCTION messages_trigger() RETURNS trigger AS $$
begin
  new.tsv :=
     setweight(to_tsvector('sys_catalog.english', coalesce(new.title,'')), 'A') ||
     setweight(to_tsvector('sys_catalog.english', coalesce(new.body,'')), 'D');
  return new;
end
$$ LANGUAGE plsql;

CREATE TRIGGER tsvectorupdate BEFORE INSERT OR UPDATE
    ON messages FOR EACH ROW EXECUTE FUNCTION messages_trigger();
```

记住当在触发器内创建 tsvector 值时，显式地指定配置名非常重要，这样列的内容才不会被 default_text_search_config 的改变所影响。如果不这样做很可能导致问题，例如在转储并重新载入之后搜索结果改变。

9.1.4.4 收集文档统计数据

ts_stat 被用于检查你的配置以及寻找候选的停用词。

```
ts_stat(sqlquery text, [ weights text, ]
        OUT word text, OUT ndoc integer,
        OUT nentry integer) returns setof record
```

sqlquery 是一个文本值，它包含一个必须返回单一 tsvector 列的 SQL 查询。ts_stat 执行该查询并返回有关包含在该 tsvector 数据中的每一个可区分词位（词）的统计数据。返回的列是：

- word text —一个词位的值
- ndoc integer —词出现过的文档（tsvector）的数量
- nentry integer —词出现的总次数

如果提供了权重，只有具有其中之一权重的出现才被计算在内。

例如，要在一个文档集合中查找十个最频繁的词：

```
SELECT * FROM ts_stat('SELECT vector FROM apod')
ORDER BY nentry DESC, ndoc DESC, word
LIMIT 10;
```

同样的要求，但是只计算以权重 A 或 B 出现的次数：

```
SELECT * FROM ts_stat('SELECT vector FROM apod', 'ab')
ORDER BY nentry DESC, ndoc DESC, word
LIMIT 10;
```

9.1.5　解析器

文本搜索解析器负责把未处理的文档文本划分成记号并且标识每一个记号的类型，而可能的类型集合由解析器本身定义。注意一个解析器完全不会修改文本—它简单地标识看似有理的词边界。因为这种有限的视野，对于应用相关的自定义解析器的需求就没有自定义字典那么强烈。目前 KingbaseES 只提供了一种内建解析器，它已经被证实对很多种应用都适用。

内建解析器被称为 `sys_catalog.default`。它识别 23 种记号类型，如下表所示。

表 9.1.1: 默认解析器的记号类型

别名	描述	例子
asciiword	单词，所有 ASCII 字母	elephant
word	单词，所有字母	ma?ana
numword	单词，字母和数字	beta1
asciihword	带连字符的单词，所有 ASCII	up-to-date
hword	带连字符的单词，所有字母	lógico-matemática
numhword	带连字符的单词，字母和数字	kingbase-beta1
hword_asciipart	带连字符的单词部分，所有 ASCII	kingbase in the contextkingbase-beta1
hword_part	带连字符的单词部分，所有字母	lógico or matemática in the contextlógico-matemática
hword_numpart	带连字符的单词部分，字母和数字	beta1 in the context kingbase-beta1
email	Email 地址	foo@example.com
protocol	协议头部	http://
url	URL	example.com/stuff/index.html
host	主机	example.com
url_path	URL 路径	/stuff/index.html , in the context of a URL
file	文件或路径名	/usr/local/foo.txt, if not within a URL
sfloat	科学记数法	-1.234e56
float	十进制记数法	-1.234

-续表-

别名	描述	例子
int	有符号整数	-1234
uint	无符号整数	1234
version	版本号	8.3.0
tag	XML 标签	
entity	XML 实体	&
blank	空格符号	（其他不识别的任意空白或标点符号）

备注:

解析器的一个“字母”的概念由数据库的区域设置决定，具体是 `lc_ctype`。只包含基本 ASCII 字母的词被报告为一个单独的记号类型，因为有时可以用来区别它们。在大部分欧洲语言中，记号类型 `word` 和 `asciiword` 应该被同样对待。`email` 不支持 RFC 5322 定义的所有合法 email 字符。具体来说，对 email 用户名被支持的非字母数字字符只有句点、破折号和下划线。

解析器有可能从同一份文本得出相互覆盖的记号。例如，一个带连字符的词可能会被报告为一整个词或者多个部分：

```
SELECT alias, description, token FROM ts_debug('foo-bar-beta1');
      alias      |               description                |     token
-----------------+------------------------------------------+---------------
 numhword        | Hyphenated word, letters and digits      | foo-bar-beta1
 hword_asciipart | Hyphenated word part, all ASCII          | foo
 blank           | Space symbols                            | -
 hword_asciipart | Hyphenated word part, all ASCII          | bar
 blank           | Space symbols                            | -
 hword_numpart   | Hyphenated word part, letters and digits | beta1
```

这种行为是值得要的，因为它允许对整个复合词和每个部分进行搜索。这里是另一个例子：

```
SELECT alias, description, token FROM ts_debug('http://example.com/stuff/
index.html');
  alias   |  description  |            token
----------+---------------+------------------------------
 protocol | Protocol head | http://
 url      | URL           | example.com/stuff/index.html
```

```
host     | Host          | example.com
url_path | URL path      | /stuff/index.html
```

9.1.6 词典

词典被用来消除不被搜索考虑的词（stop words）、并被用来正规化词这样同一个词的不同派生形式将会匹配。一个被成功地正规化的词被称为一个词位。除了提高搜索质量，正规化和移除停用词减小了文档的 `tsvector` 表示的尺寸，因而提高了性能。正规化不会总是有语言上的意义并且通常依赖于应用的语义。

一些正规化的例子：

- 语言学的 - Ispell 词典尝试将输入词缩减为一种正规化的形式；词干分析器词典移除词的结尾
- URL 位置可以被规范化来得到等效的 URL 匹配：
  - https://bbs.kingbase.com.cn/index
  - https://bbs.kingbase.com.cn/
- 颜色名可以被它们的十六进制值替换，例如 `red, green, blue, magenta -> FF0000, 00FF00, 0000FF, FF00FF`
- 如果索引数字，可以移除某些小数位来缩减可能的数字的范围，因此如果只保留小数点后两位，例如 3.14159265359、3.1415926、3.14 在正规化后会变得相同。

一个词典是一个程序，它接受一个记号作为输入，并返回：

- 如果输入的记号对词典是已知的，则返回一个词位数组（注意一个记号可能产生多于一个词位）
- 一个 `TSL_FILTER` 标志被设置的单一词位，用一个新记号来替换要被传递给后续字典的原始记号（做这件事的一个字典被称为一个过滤字典）
- 如果字典知道该记号但它是一个停用词，则返回一个空数组
- 如果字典不识别该输入记号，则返回 `NULL`

KingbaseES 为许多语言提供了预定义的字典。也有多种预定义模板可以被用于创建带自定义参数的新词典。每一种预定义词典模板在下面描述。如果没有合适的现有模板，可以创建新的；例子见 KingbaseES 发布的 `contrib/` 区域。

一个文本搜索配置把一个解析器和一组处理解析器输出记号的词典绑定在一起。对于每一中解析器能返回的记号类型，配置都指定了一个单独的词典列表。当该类型的一个记号被解析器找到时，每一个词典都被按照顺序查询，知道某个词典将其识别为一个已知词。如果它被标识为一个停用词或者没有一个词典识别它，它将被丢弃并且不会被索引和用于搜索。通常，第一个返回非 `NULL` 输出的词典决定结果，并且任何剩下的词典都不会被查找；但是一个过滤词典可以将给定词替换为一个被修改的词，它再被传递给后续的词典。

配置一个词典列表的通用规则是将最狭窄、最特定的词典放在第一位，然后是更加通用的词典，以一个非常通用的词典结尾，像一个 Snowball 词干分析器或什么都识别的 `simple`。例如，对于一个天文学相关的搜索（`astro_en` 配置）我们可以把记号类型 `asciiword`（ASCII 词）绑定到一个天文学术语的分类词典、一个通用英语词典和一个 Snowball 英语词干分析器：

```
ALTER TEXT SEARCH CONFIGURATION astro_en
    ADD MAPPING FOR asciiword WITH astrosyn, english_ispell, english_stem;
```

一个过滤词典可以被放置在列表中的任意位置，除了在最后，因为过滤词典放在最后就等于无用。过滤词典可用于部分正规化词来简化后续词典的工作。例如，一个过滤词典可以被用来从音标字母中移除重音符号。

9.1.6.1　停用词

停用词是非常常用、在几乎每一个文档中出现并且没有任何区分度的词。因此，在全文搜索的环境中它们可以被忽略。例如，每一段英语文本都包含 a 和 the 等次，因此把它们存储在一个索引中是没有用处的。但是，停用词确实会影响在 tsvector 中的位置，这进而会影响排名：

```
SELECT to_tsvector('english','in the list of stop words');
        to_tsvector
----------------------------
 'list':3 'stop':5 'word':6
```

缺失的位置 1、2、4 是因为停用词。文档的排名计算在使用和不使用停用词的情况下是很不同的：

```
SELECT ts_rank_cd (to_tsvector('english','in the list of stop words'),
to_tsquery('list & stop'));
 ts_rank_cd
------------
       0.05

SELECT ts_rank_cd (to_tsvector('english','list stop words'),
to_tsquery('list & stop'));
 ts_rank_cd
------------
        0.1
```

如何对待停用词是由指定词典决定的。例如，ispell 词典首先正规化词并且查看停用词列表，而 Snowball 词干分析器首先检查停用词的列表。这种不同行为的原因是一冲降低噪声的尝试。

9.1.6.2　简单词典

simple 词典模板的操作是将输入记号转换为小写形式并且根据一个停用词文件检查它。如果该记号在该文件中被找到，则返回一个空数组，导致该记号被丢弃。否则，该词的小写形式被返回作为正规化的词位。作为一种选择，该词典可以被配置为将非停用词报告为未识别，允许它们被传递给列表中的下一个词典。

下面是一个使用 simple 模板的词典定义的例子：

```
CREATE TEXT SEARCH DICTIONARY public.simple_dict (
    TEMPLATE = pg_catalog.simple,
```

```
    STOPWORDS = english
);
```

这里，english 是一个停用词文件的基本名称。该文件的全名将是 $SHAREDIR/tsearch_data/english.stop，其中 $SHAREDIR 表示 KingbaseES 安装的共享数据目录，通常是/usr/local/share/kingbase（如果不确定，使用 sys_config --sharedir）。该文件格式是一个词的列表，每行一个。空行和尾部的空格都被忽略，并且大写也被折叠成小写，但是没有其他对该文件内容的处理。

测试词典：

```
SELECT ts_lexize('public.simple_dict','YeS');
 ts_lexize
-----------
 {yes}

SELECT ts_lexize('public.simple_dict','The');
 ts_lexize
-----------
 {}
```

如果没有在停用词文件中找到，可以选择返回 NULL 而不是小写形式的词。这种行为可以通过设置词典的 Accept 参数为 false 来选择。继续该例子：

```
ALTER TEXT SEARCH DICTIONARY public.simple_dict ( Accept = false );

SELECT ts_lexize('public.simple_dict','YeS');
 ts_lexize
-----------



SELECT ts_lexize('public.simple_dict','The');
 ts_lexize
-----------
 {}
```

在使用默认值 Accept = true，只有把一个 simple 词典放在词典列表的尾部才有用，因为它将不会传递任何记号给后续的词典。相反，Accept = false 只有当至少有一个后续词典的情况下才有用。

Caution

大部分类型的词典依赖于配置文件，例如停用词文件。这些文件必须被存储为 UTF-8 编码。当它们被读入服务器时，如果存在不同，它们将被翻译成真实的数据库编码。

Caution

通常，当一个词典配置文件第一次在数据库会话中使用时，数据库会话将只读取它一次。如果你修改了一个配置文件并且想强迫现有的会话取得新内容，可以在该词典上发出一个 ALTER TEXT SEARCH DICTIONARY 命令。这可以

是一次“假”更新，它并不实际修改任何参数值。

9.1.6.3　同义词词典

这个词典模板被用来创建用于同义词替换的词典。不支持短语（使用分类词典模板（分类词典 _ ）可以支持）。一个同义词词典可以被用来解决语言学问题，例如，阻止一个英语词干分析器词典把词“Paris”缩减成“pari”。在同义词词典中有一行 Paris paris 并把它放在 english_stem 词典之前就足够了。例如：

```
SELECT * FROM ts_debug('english', 'Paris');
   alias   |   description   | token |  dictionaries  |  dictionary  | lexemes
-----------+-----------------+-------+----------------+--------------+---------
 asciiword | Word, all ASCII | Paris | {english_stem} | english_stem | {pari}

CREATE TEXT SEARCH DICTIONARY my_synonym (
    TEMPLATE = synonym,
    SYNONYMS = my_synonyms
);

ALTER TEXT SEARCH CONFIGURATION english
    ALTER MAPPING FOR asciiword
    WITH my_synonym, english_stem;

SELECT * FROM ts_debug('english', 'Paris');
alias    | description  |token|       dictionaries       |dictionary|lexemes
---------+--------------+-----+--------------------------+----------+--------
asciiword|Word,all ASCII|Paris|{my_synonym,english_stem}|my_synonym|{paris}
```

synonym 模板要求的唯一参数是 SYNONYMS，它是其配置文件的基本名—上例中的 my_synonyms。该文件的完整名称将是 $SHAREDIR/tsearch_data/my_synonyms.syn（其中 $SHAREDIR 表示 KingbaseES 安装的共享数据目录）。该文件格式是每行一个要被替换的词，后面跟着它的同义词，用空白分隔。空行和结尾的空格会被忽略。

synonym 模板还有一个可选的参数 CaseSensitive，其默认值为 false。当 CaseSensitive 为 false 时，同义词文件中的词被折叠成小写，这和输入记号一样。当它为 true 时，词和记号将不会被折叠成小写，但是比较时就好像被折叠过一样。

一个星号（*）可以被放置在配置文件中一个同义词的末尾。这表示该同义词是一个前缀。当项被用在 to_tsvector() 中时，星号会被忽略；当它被用在 to_tsquery() 中时，结果将是一个带有前缀匹配标记器（见解析查询 ）的查询项。例如，假设在 $SHAREDIR/tsearch_data/synonym_sample.syn 中有这些项：

```
kingbase        kingbase
kingbase      kingbase
kingbase kingbase
gogle   googl
indices index*
```

那么将得到这些结果：

```
mydb=# CREATE TEXT SEARCH DICTIONARY syn (template=synonym, synonyms=
'synonym_sample');
mydb=# SELECT ts_lexize('syn','indices');
 ts_lexize
-----------
 {index}
(1 row)

mydb=# CREATE TEXT SEARCH CONFIGURATION tst (copy=simple);
mydb=# ALTER TEXT SEARCH CONFIGURATION tst ALTER MAPPING FOR asciiword WITH syn;
mydb=# SELECT to_tsvector('tst','indices');
 to_tsvector
-------------
 'index':1
(1 row)

mydb=# SELECT to_tsquery('tst','indices');
 to_tsquery
------------
 'index':*
(1 row)

mydb=# SELECT 'indexes are very useful'::tsvector;
            tsvector
---------------------------------
 'are' 'indexes' 'useful' 'very'
(1 row)

mydb=# SELECT 'indexes are very useful'::tsvector @@ to_tsquery('tst','indices');
 ?column?
----------
 t
(1 row)
```

9.1.6.4　分类词典

一个分类词典（有时被简写成 TZ）是一个词的集合，其中包括了词与短语之间的联系，即广义词（BT）、狭义词（NT）、首选词、非首选词、相关词等。

基本上一个分类词典会用一个首选词替换所有非首选词，并且也可选择地保留原始术语用于索引。KingbaseES 的分类词典的当前实现是同义词词典的一个扩展，并增加了短语支持。一个分类词典要求一个下列格式的配置文件：

```
# this is a comment
sample word(s) : indexed word(s)
```

```
more sample word(s) : more indexed word(s)
...
```

其中冒号（:）符号扮演了一个短语及其替换之间的定界符。

一个分类词典使用一个子词典（在词典的配置中指定）在检查短语匹配之前正规化输入文本。只能选择一个子词典。如果子词典无法识别一个词，将报告一个错误。在这种情况下，你应该移除该词的使用或者让子词典学会这个词。你可以在一个被索引词的开头放上一个星号（*）来跳过在其上应用子词典，但是所有采样词必须被子词典知道。

如果有多个短语匹配输入，则分类词典选择最长的那一个，并且使用最后的定义打破连结。

由子词典识别的特定停用词不能够被指定；改用? 标记任何可以出现停用词的地方。例如，假定根据子词典 a 和 the 是停用词：

```
? one ? two : swsw
```

匹配 a one the two 和 the one a two；两者都将被 swsw 替换。

由于一个分类词典具有识别短语的能力，它必须记住它的状态并与解析器交互。一个分类词典使用这些任务来检查它是否应当处理下一个词或者停止累积。分类词典必须被小心地配置。例如，如果分类词典被分配只处理 asciiword 记号，则一个形如 one 7 的分类词典定义将不会工作，因为记号类型 uint 没有被分配给该分类词典。

Caution

在索引期间要用到分类词典，因此分类词典参数中的任何变化都要求重索引。对于大多数其他索引类型，例如增加或移除停用词等小改动都不会强制重索引。

分类词典配置

要定义一个新的分类词典，可使用 thesaurus 模板。例如：

```
CREATE TEXT SEARCH DICTIONARY thesaurus_simple (
    TEMPLATE = thesaurus,
    DictFile = mythesaurus,
    Dictionary = sys_catalog.english_stem
);
```

这里：

- thesaurus_simple 是新词典的名称
- mythesaurus 是分类词典配置文件的基础名称（它的全名将是 $SHAREDIR/tsearch_data/mythesaurus.ths，其中 $SHAREDIR 表示安装的共享数据目录）。
- sys_catalog.english_stem 是要用于分类词典正规化的子词典（这里是一个 Snowball 英语词干分析器）。注意子词典将拥有它自己的配置（例如停用词），但这里没有展示。

现在可以在配置中把分类词典 thesaurus_simple 绑定到想要的记号类型上，例如：

```
ALTER TEXT SEARCH CONFIGURATION russian
    ALTER MAPPING FOR asciiword, asciihword, hword_asciipart
    WITH thesaurus_simple;
```

分类词典例子

考虑一个简单的天文学分类词典 thesaurus_astro，它包含一些天文学词组合：

```
supernovae stars : sn
crab nebulae : crab
```

创建一个词典并绑定一些记号类型到一个天文学分类词典以及英语词干分析器：

```
CREATE TEXT SEARCH DICTIONARY thesaurus_astro (
    TEMPLATE = thesaurus,
    DictFile = thesaurus_astro,
    Dictionary = english_stem
);

ALTER TEXT SEARCH CONFIGURATION russian
    ALTER MAPPING FOR asciiword, asciihword, hword_asciipart
    WITH thesaurus_astro, english_stem;
```

ts_lexize 对于测试一个分类词典用处不大，因为它将输入看成是一个单一记号。可以用 plainto_tsquery 和 to_tsvector，它们将把其输入字符串打断成多个记号：

```
SELECT plainto_tsquery('supernova star');
 plainto_tsquery
-----------------
 'sn'

SELECT to_tsvector('supernova star');
 to_tsvector
-------------
 'sn':1
```

原则上，如果对参数加了引号，可以使用 to_tsquery：

```
SELECT to_tsquery('''supernova star''');
 to_tsquery
------------
 'sn'
```

注意在 thesaurus_astro 中 supernova star 匹配 supernovae stars，因为在分类词典定义中指定了 english_stem 词干分析器。该词干分析器移除了 e 和 s。

要和替补一样也索引原始短语，只要将它包含在定义的右手部分中：

```
supernovae stars : sn supernovae stars

SELECT plainto_tsquery('supernova star');
       plainto_tsquery
-----------------------------
 'sn' & 'supernova' & 'star'
```

9.1.6.5　Ispell 词典

Ispell 词典模板支持词法词典，它可以把一个词的很多不同语言学的形式正规化成相同的词位。例如，一个英语 Ispell 词典可以匹配搜索词 bank 的词尾变化和词形变化，例如 banking、banked、banks、banks'和 bank's。

标准的 KingbaseES 发布不包括任何 Ispell 配置文件。用于很多种语言的词典可以从 Ispell 得到。此外，也支持一些更现代的词典文件格式 MySpell （OO < 2.0.1）和 Hunspell （OO >= 2.0.2）。词典列表可以从 OpenOffice Wiki 上获取。

创建一个 Ispell 词典，执行三步：

- 下载词典配置文件。OpenOffice 扩展文件的扩展名是.oxt。有必要抽取.aff 和.dic 文件，把扩展改为.affix 和.dict。对于某些词典文件，还需要使用下面的命令把字符转换成 UTF-8 编码（例如挪威语词典）：

```
iconv -f ISO_8859-1 -t UTF-8 -o nn_no.affix nn_NO.aff
iconv -f ISO_8859-1 -t UTF-8 -o nn_no.dict nn_NO.dic
```

- 拷贝文件到 $SHAREDIR/tsearch_data 目录
- 用下面的命令把文件载入到 KingbaseES：

```
CREATE TEXT SEARCH DICTIONARY english_hunspell (
    TEMPLATE = ispell,
    DictFile = en_us,
    AffFile = en_us,
    Stopwords = english);
```

DictFile、AffFile 和 StopWords 指定词典、词缀和停用词文件的基础名称。停用词文件的格式和前面解释的 simple 词典类型相同。其他文件的格式在这里没有指定，但是也可以从上面提到的网站获得。

Ispell 词典通常识别一个有限集合的词，这样它们后面应该跟着另一个更广义的词典；例如，一个 Snowball 词典，它可以识别所有东西。

Ispell 的.affix 文件具有下面的结构：

```
prefixes
flag *A:
    .           >   RE      # As in enter > reenter
suffixes
flag T:
```

```
    E           >   ST      # As in late > latest
    [^AEIOU]Y   >   -Y,IEST # As in dirty > dirtiest
    [AEIOU]Y    >   EST     # As in gray > grayest
    [^EY]       >   EST     # As in small > smallest
```

.dict 文件具有下面的结构：

```
lapse/ADGRS
lard/DGRS
large/PRTY
lark/MRS
```

.dict 文件的格式是：

```
basic_form/affix_class_name
```

在.affix 文件中，每一个词缀标志以下面的格式描述：

```
condition > [-stripping_letters,] adding_affix
```

这里的条件具有和正则表达式相似的格式。它可以使用分组 [...] 和 [^...]。例如，[AEIOU]Y 表示词的最后一个字母是"y" 并且倒数第二个字母是"a"、"e"、"i"、"o" 或者"u"。[^EY] 表示最后一个字母既不是"e" 也不是"y"。

Ispell 词典支持划分复合词，这是一个有用的特性。注意词缀文件应该用 compoundwords controlled 语句指定一个特殊标志，它标记可以参与到复合格式中的词典词：

```
compoundwords  controlled z
```

下面是挪威语的一些例子：

```
SELECT ts_lexize('norwegian_ispell', 'overbuljongterningpakkmesterassistent');
   {over,buljong,terning,pakk,mester,assistent}
SELECT ts_lexize('norwegian_ispell', 'sjokoladefabrikk');
   {sjokoladefabrikk,sjokolade,fabrikk}
```

MySpell 格式是 Hunspell 格式的一个子集。Hunspell 的.affix 文件具有下面的结构：

```
PFX A Y 1
PFX A   0     re         .
SFX T N 4
SFX T   0     st         e
SFX T   y     iest       [^aeiou]y
SFX T   0     est        [aeiou]y
SFX T   0     est        [^ey]
```

一个词缀类的第一行是头部。头部后面列出了词缀规则的域：

- 参数名（PFX 或者 SFX）
- 标志（词缀类的名称）
- 从该词的开始（前缀）或者结尾（后缀）剥离字符
- 增加词缀
- 和正则表达式格式类似的条件。

`.dict` 文件看起来和 Ispell 的`.dict` 文件相似：

```
larder/M
lardy/RT
large/RSPMYT
largehearted
```

备注:

MySpell 不支持复合词。Hunspell 则对复合词有更好的支持。当前，KingbaseES 只实现了 Hunspell 中基本的复合词操作。

9.1.6.6 Snowball 词典

Snowball 词典模板基于 Martin Porter 的一个项目，他是流行的英语 Porter 词干分析算法的发明者。Snowball 现在对许多语言提供词干分析算法（详见 Snowball 站点 ）。每一个算法懂得按照其语言中的拼写，如何缩减词的常见变体形式为一个基础或词干。一个 Snowball 词典要求一个 `language` 参数来标识要用哪种词干分析器，并且可以选择地指定一个 `stopword` 文件名来给出一个要被消除的词列表（KingbaseES 的标准停用词列表也是由 Snowball 项目提供的）。例如，有一个内建的定义等效于

```
CREATE TEXT SEARCH DICTIONARY english_stem (
    TEMPLATE = snowball,
    Language = english,
    StopWords = english
);
```

停用词文件格式和已经解释的一样。

一个 Snowball 词典识别所有的东西，不管它能不能简化该词，因此它应当被放置在词典列表的最后。把它放在任何其他词典前面是没有用处的，因为一个记号永远不会穿过它而进入到下一个词典。

9.1.7 配置例子

一个文本搜索配置指定了将一个文档转换成一个 `tsvector` 所需的所有选项：用于把文本分解成记号的解析器，以及用于将每一个记号转换成词位的词典。每一次 `to_tsvector` 或 `to_tsquery` 的调用都需要一个文本搜索配置来

执行其处理。配置参数 default_text_search_config 指定了默认配置的名称，如果忽略了显式的配置参数，文本搜索函数将会使用它。它可以在 `kingbase.conf` 中设置，或者使用 `SET` 命令为一个单独的会话设置。

有一些预定义的文本搜索配置可用，并且你可以容易地创建自定义的配置。为了便于管理文本搜索对象，可以使用一组 SQL 命令，并且有多个 ksql 命令可以显示有关文本搜索对象（ksql 支持 ）的信息。

作为一个例子，将创建一个配置 `pg`，从复制内建的 `english` 配置开始：

```
CREATE TEXT SEARCH CONFIGURATION public.pg ( COPY = sys_catalog.english );
```

将使用一个 KingbaseES 相关的同义词列表，并将它存储在 `$SHAREDIR/tsearch_data/sys_dict.syn` 中。文件内容看起来像：

```
kingbase    pg
kingbase       pg
kingbase  pg
```

定义同义词词典如下：

```
CREATE TEXT SEARCH DICTIONARY sys_dict (
    TEMPLATE = synonym,
    SYNONYMS = sys_dict
);
```

接下来注册 Ispell 词典 `english_ispell`，它有其自己的配置文件：

```
CREATE TEXT SEARCH DICTIONARY english_ispell (
    TEMPLATE = ispell,
    DictFile = english,
    AffFile = english,
    StopWords = english
);
```

在配置 `pg` 中建立词的映射：

```
ALTER TEXT SEARCH CONFIGURATION pg
    ALTER MAPPING FOR asciiword, asciihword, hword_asciipart,
                      word, hword, hword_part
    WITH sys_dict, english_ispell, english_stem;
```

选择不索引或搜索某些内建配置确实处理的记号类型：

```
ALTER TEXT SEARCH CONFIGURATION pg
    DROP MAPPING FOR email, url, url_path, sfloat, float;
```

测试配置：

```
SELECT * FROM ts_debug('public.pg', '
KingbaseES, the highly scalable, SQL compliant, open source object-relational
database management system, is now undergoing beta testing of the next
version of our software.
');
```

下一个步骤是设置会话让它使用新配置，它被创建在 public 模式中：

```
=> \dF
   List of text search configurations
 Schema  | Name | Description
---------+------+-------------
 public  | pg   |

SET default_text_search_config = 'public.pg';
SET

SHOW default_text_search_config;
 default_text_search_config
----------------------------
 public.pg
```

9.1.8 中文分词

9.1.8.1 zhparser 中文分词插件

zhparser 中文分词插件使用说明：

```
test=# create extension zhparser;
CREATE EXTENSION
test=# CREATE TEXT SEARCH CONFIGURATION testzhcfg (PARSER = zhparser);
CREATE TEXT SEARCH CONFIGURATION
test=#
test=# ALTER TEXT SEARCH CONFIGURATION testzhcfg ADD MAPPING FOR n,v,a,i,e,l WITH simple;
ALTER TEXT SEARCH CONFIGURATION
test=# SELECT * FROM ts_parse('zhparser', 'KingbaseES 中文分词插件测试');
tokid |   token
-------+------------
  101 | KingbaseES
  110 | 中文
  118 | 分词
  110 | 插件
  118 | 测试
```

```
 (5 rows)

test=# SELECT to_tsvector('testzhcfg','KingbaseES 中文分词插件测试');
                     to_tsvector
-----------------------------------------------------
 'kingbasees':1 '中文':2 '分词':3 '插件':4 '测试':5
 (1 row)

test=# SELECT to_tsquery('testzhcfg', '分词');
   to_tsquery
------------
  '分词'
 (1 row)
```

9.1.8.2　sys_jieba 中文分词插件

sys_jieba 中文分词插件使用说明：

```
test=# CREATE EXTENSION sys_jieba;
CREATE EXTENSION
test=# SELECT * FROM ts_parse('jieba', 'KingbaseES 中文分词插件测试');
 tokid |   token
-------+------------
     2 | KingbaseES
     2 | 分词
     2 | 插件
     2 | 测试
(5 rows)

test=# SELECT to_tsvector('jiebacfg','KingbaseES 中文分词插件测试');
                    to_tsvector
-----------------------------------------------------
 'kingbasees':1 '中文':2 '分词':3 '插件':4 '测试':5
(1 row)

test=# SELECT to_tsquery('jiebacfg', '分词');
 to_tsquery
------------
 '分词'
(1 row)
```

9.1.9　使用 ftutilx 插件

9.1.9.1　抽取内容

ftutilx 插件提供了 extracttext 函数用于抽取存储在 blob 类型字段内的文件内容。extracttext 函数接受一个代表文件内容的 blob 类型参数，返回抽取的 text 类型文本内容。其中 blob 类型字段内容可以包括 pdf、doc、docx、wps、xls、xlsx、ppt 和 pptx 格式文件。ftutilx 插件不支持加密文件格式。

```
CREATE EXTENSION ftutilx;
CREATE TABLE tab (title text, body blob);
INSERT INTO tab VALUES ('test.doc', blob_import('/home/test/data.doc'));
SELECT title, length(extracttext(body)) FROM tab;
```

ftutilx 插件支持以下参数：

1) ftutilx.max_string_length：抽取结果最大长度，默认值：128M，该参数设置后立即生效。
2) ftutilx.jvm_option_string：JVM 初始化参数，默认值："-Xmx1024m,-Xms1024m,-Xmn256m,-XX:MetaspaceSize=64m,-XX:MaxMetaspaceSize=128m,-XX:CompressedClassSpaceSize=256m"，该参数只在会话进程中首次调用 extracttext 函数创建 JVM 时生效，再次设置该参数不再有效。

在数据库默认扩展加载机制下，在一个会话中创建扩展以后，在新的会话开始后并不是立即加载扩展动态库，而是直到首次调用扩展中的接口时才会加载扩展动态库，导致在这之前在新会话中设置扩展参数无效。解决方法是：修改数据库配置文件中的 shared_preload_libraries 或者 session_preload_libraries 两个参数之一，使参数值包括 ftutilx，即可在新会话开始后立即加载 ftutilx 扩展动态库，并设置扩展参数。

9.1.9.2　联合使用方案

由于电子文档内容抽取速度较慢，为提高全文检索性能，可以在表中添加存储列，用于存储内容抽取结果或者词位列表。

```
方案一：
CREATE EXTENSION zhparser;
CREATE TEXT SEARCH CONFIGURATION zhparsercfg (PARSER = zhparser);
ALTER TEXT SEARCH CONFIGURATION zhparsercfg ADD MAPPING FOR n,v,a,i,e,l WITH simple;

CREATE EXTENSION ftutilx;
CREATE TABLE tab (title text, body blob);

ALTER TABLE tab ADD COLUMN content text GENERATED ALWAYS AS (extracttext(body)) STORED;
CREATE INDEX tab_idx ON tab USING GIN (to_tsvector('zhparsercfg', content));

INSERT INTO tab VALUES ('test.doc', blob_import('/home/test/data.doc'));

SELECT title FROM tab WHERE to_tsvector('zhparsercfg', content) @@ to_tsquery('日志');
```

```
方案二：
CREATE EXTENSION zhparser;
CREATE TEXT SEARCH CONFIGURATION zhparsercfg (PARSER = zhparser);
ALTER TEXT SEARCH CONFIGURATION zhparsercfg ADD MAPPING FOR n,v,a,i,e,l WITH simple;

CREATE EXTENSION ftutilx;
CREATE TABLE tab (title text, body blob);

ALTER TABLE tab ADD COLUMN tab_idx_col tsvector GENERATED ALWAYS AS (to_tsvector('zhparsercfg',
extracttext(body))) STORED;
CREATE INDEX tab_idx ON tab USING GIN (tab_idx_col);

INSERT INTO tab VALUES ('test.doc', blob_import('/home/test/data.doc'));

SELECT title FROM tab WHERE tab_idx_col @@ to_tsquery('日志');
```

9.1.9.3　注意事项

1）ftutilx 需要依赖于 jre-1.8.0 运行时环境，部署后需要设置 LD_LIBRARY_PATH 系统环境变量包含 jre-1.8.0 的 libjvm.so 路径。

2）ftutilx.max_string_length 参数用于配置抽取结果的最大长度，但由于 tsvector 目前最大支持 (1M-1)，所以 extracttext 结合 to_tsvector 使用时，分词结果大小不能超过 (1M-1)。

3）ftutilx 需要创建 JVM，JVM 会占用较多内存。虽然调整 ftutilx.jvm_option_string 的-Xmx 可以限制 JVM 的内存占用，但过小的-Xmx 值会导致大文件解析时 JVM 发生内存不足异常。

4）基于前面的全文检索联合使用方案，在系统内存较少的环境中，需要限制并行插入数据的会话进程数量，以免系统内存被耗尽。

9.1.10　测试和调试文本搜索

一个自定义文本搜索配置的行为很容易变得混乱。本节中描述的函数对于测试文本搜索对象有用。你可以测试一个完整的配置，或者独立测试解析器和词典。

9.1.10.1　配置测试

函数 `ts_debug` 允许简单地测试一个文本搜索配置。

```
ts_debug([ config regconfig, ] document text,
         OUT alias text,
         OUT description text,
         OUT token text,
```

```
        OUT dictionaries regdictionary[],
        OUT dictionary regdictionary,
        OUT lexemes text[])
        returns setof record
```

ts_debug 显示 document 的每一个记号的信息，记号由解析器产生并由配置的词典处理过。该函数使用由 config 指定的配置，如果该参数被忽略则使用 default_text_search_config 指定的配置。

ts_debug 为解析器在文本中标识的每一个记号返回一行。被返回的列是：

- alias text —记号类型的短名称
- description text —记号类型的描述
- token text —记号的文本
- dictionaries regdictionary[] —配置为这种记号类型选择的词典
- dictionary regdictionary —识别该记号的词典，如果没有词典能识别则为 NULL
- lexemes text[] —识别该记号的词典产生的词位，如果没有词典能识别则为 NULL；一个空数组（\{\}）表示该记号被识别为一个停用词

这里是一个简单的例子：

```
SELECT * FROM ts_debug('english','a fat  cat sat on a mat - it ate a fat rats');
   alias   |   description   | token |  dictionaries  |  dictionary  | lexemes
-----------+-----------------+-------+----------------+--------------+---------
 asciiword | Word, all ASCII | a     | {english_stem} | english_stem | {}
 blank     | Space symbols   |       | {}             |              |
 asciiword | Word, all ASCII | fat   | {english_stem} | english_stem | {fat}
 blank     | Space symbols   |       | {}             |              |
 asciiword | Word, all ASCII | cat   | {english_stem} | english_stem | {cat}
 blank     | Space symbols   |       | {}             |              |
 asciiword | Word, all ASCII | sat   | {english_stem} | english_stem | {sat}
 blank     | Space symbols   |       | {}             |              |
 asciiword | Word, all ASCII | on    | {english_stem} | english_stem | {}
 blank     | Space symbols   |       | {}             |              |
 asciiword | Word, all ASCII | a     | {english_stem} | english_stem | {}
 blank     | Space symbols   |       | {}             |              |
 asciiword | Word, all ASCII | mat   | {english_stem} | english_stem | {mat}
 blank     | Space symbols   |       | {}             |              |
 blank     | Space symbols   | -     | {}             |              |
 asciiword | Word, all ASCII | it    | {english_stem} | english_stem | {}
 blank     | Space symbols   |       | {}             |              |
 asciiword | Word, all ASCII | ate   | {english_stem} | english_stem | {ate}
 blank     | Space symbols   |       | {}             |              |
 asciiword | Word, all ASCII | a     | {english_stem} | english_stem | {}
 blank     | Space symbols   |       | {}             |              |
```

```
 asciiword | Word, all ASCII | fat   | {english_stem} | english_stem | {fat}
 blank     | Space symbols   |       | {}             |              |
 asciiword | Word, all ASCII | rats  | {english_stem} | english_stem | {rat}
```

为了一个更广泛的示范，为英语语言创建一个 public.english 配置和 Ispell 词典：

```
CREATE TEXT SEARCH CONFIGURATION public.english ( COPY = sys_catalog.english );

CREATE TEXT SEARCH DICTIONARY english_ispell (
    TEMPLATE = ispell,
    DictFile = english,
    AffFile = english,
    StopWords = english
);

ALTER TEXT SEARCH CONFIGURATION public.english
   ALTER MAPPING FOR asciiword WITH english_ispell, english_stem;
```

```
SELECT * FROM ts_debug('public.english','The Brightest supernovaes');
  alias  | description    | token      |  dictionaries  |  dictionary  |lexemes
---------+---------------+-----------+--------      --+-----------  -+--------
asciiword|Word, all ASCII|The        |{english_ispell,|english_ispell| {}
                                       english_stem}
blank    |Space symbols  |           |{}              |              |
asciiword|Word, all ASCII|Brightest  |{english_ispell,|english_ispell|{bright}
                                       english_stem}
blank    |Space symbols  |           |{}              |              |
asciiword|Word, all ASCII|supernovaes|{english_ispell,|english_stem  |{supernova}
                                       english_stem}
```

在这个例子中，词 Brightest 被解析器识别为一个 ASCII 词（别名 asciiword）。对于这种记号类型，词典列表是 english_ispell 和 english_stem。该词被 english_ispell 识别，这个词典将它缩减为名词 bright。词 supernovaes 对于 english_ispell 词典是未知的，因此它被传递给下一个词典，并且幸运地是，它被识别了（实际上，english_stem 是一个 Snowball 词典，它识别所有的东西；这也是为什么它被放置在词典列表的尾部）。

词 The 被 english_ispell 词典识别为一个停用词（停用词 ）并且将不会被索引。空格也被丢弃，因为该配置没有为它们提供词典。

可以通过显式地指定想看哪些列来缩减输出的宽度：

```
SELECT alias, token, dictionary, lexemes
FROM ts_debug('public.english','The Brightest supernovaes');
   alias   |    token    |   dictionary   |   lexemes
-----------+-------------+----------------+-------------
 asciiword | The         | english_ispell | {}
 blank     |             |                |
```

```
 asciiword | Brightest   | english_ispell | {bright}
 blank     |             |                |
 asciiword | supernovaes | english_stem   | {supernova}
```

9.1.10.2　解析器测试

下列函数允许直接测试一个文本搜索解析器。

```
ts_parse(parser_name text, document text,
         OUT tokid integer, OUT token text) returns setof record
ts_parse(parser_oid oid, document text,
         OUT tokid integer, OUT token text) returns setof record
```

ts_parse 解析给定的 document 并返回一系列记录，每一个记录对应一个由解析产生的记号。每一个记录包括一个 tokid 展示分配给记号的类型以及一个 token 展示记号的文本。例如：

```
SELECT * FROM ts_parse('default', '123 - a number');
 tokid | token
-------+--------
    22 | 123
    12 |
    12 | -
     1 | a
    12 |
     1 | number
```

```
ts_token_type(parser_name text, OUT tokid integer,
              OUT alias text, OUT description text) returns setof record
ts_token_type(parser_oid oid, OUT tokid integer,
              OUT alias text, OUT description text) returns setof record
```

ts_token_type 返回一个表，该表描述指定解析器能够识别的每一种记号类型。对于每一种记号类型，该表给出了解析器用来标注该类型记号的整数 tokid，还给出了在配置命令中命名该记号类型的 alias，以及一个简短的 description。例如：

```
SELECT * FROM ts_token_type('default');
 tokid |      alias      |               description
-------+-----------------+------------------------------------------
     1 | asciiword       | Word, all ASCII
     2 | word            | Word, all letters
     3 | numword         | Word, letters and digits
     4 | email           | Email address
     5 | url             | URL
     6 | host            | Host
```

```
     7 | sfloat          | Scientific notation
     8 | version         | Version number
     9 | hword_numpart   | Hyphenated word part, letters and digits
    10 | hword_part      | Hyphenated word part, all letters
    11 | hword_asciipart | Hyphenated word part, all ASCII
    12 | blank           | Space symbols
    13 | tag             | XML tag
    14 | protocol        | Protocol head
    15 | numhword        | Hyphenated word, letters and digits
    16 | asciihword      | Hyphenated word, all ASCII
    17 | hword           | Hyphenated word, all letters
    18 | url_path        | URL path
    19 | file            | File or path name
    20 | float           | Decimal notation
    21 | int             | Signed integer
    22 | uint            | Unsigned integer
    23 | entity          | XML entity
```

9.1.10.3　词典测试

ts_lexize 函数帮助词典测试。

```
ts_lexize(dict regdictionary, token text) returns text[]
```

如果输入的 token 是该词典已知的，则 ts_lexize 返回一个词位数组；如果记号是词典已知的但是它是一个停用词，则返回一个空数组；或者如果它对词典是未知词，则返回 NULL。

例子：

```
SELECT ts_lexize('english_stem', 'stars');
 ts_lexize
-----------
 {star}

SELECT ts_lexize('english_stem', 'a');
 ts_lexize
-----------
 {}
```

备注:

ts_lexize 函数期望一个单一记号而不是文本。下面的情况会让它搞混：

```
SELECT ts_lexize('thesaurus_astro','supernovae stars') is null;
 ?column?
----------
 t
```

分类词典 thesaurus_astro 确实知道短语 supernovae stars，但是 ts_lexize 会失败，因为它无法解析输入文本而把它当做一个单一记号。可以使用 plainto_tsquery 或 to_tsvector 来测试分类词典，例如：

```
SELECT plainto_tsquery('supernovae stars');
 plainto_tsquery
-----------------
 'sn'
```

9.1.11　GIN 和 GiST 索引类型

有两种索引可以被用来加速全文搜索。注意全文搜索并非一定需要索引，但是在一个定期会被搜索的列上，通常需要有一个索引。

CREATE INDEX name ON table USING GIN(column);
创建一个基于 GIN（通用倒排索引）的索引。column 必须是 tsvector 类型。

CREATE INDEX name ON table USING GIST(column);
创建一个基于 GiST（通用搜索树）的索引。column 可以是 tsvector 或 tsquery 类型。

GIN 索引是更好的文本搜索索引类型。作为倒排索引，每个词（词位）在其中都有一个索引项，其中有压缩过的匹配位置的列表。多词搜索可以找到第一个匹配，然后使用该索引移除缺少额外词的行。GIN 索引只存储 tsvector 值的词（词位），并且不存储它们的权重标签。因此，在使用涉及权重的查询时需要一次在表行上的重新检查。

一个 GiST 索引是有损的，这表示索引可能产生假匹配，并且有必要检查真实的表行来消除这种假匹配（KingbaseES 在需要时会自动做这一步）。GiST 索引之所以是有损的，是因为每一个文档在索引中被表示为一个定长的签名。该签名通过哈希每一个词到一个 n 位串中的一个单一位来产生，通过将所有这些位 OR 在一起产生一个 n 位的文档签名。当两个词哈希到同一个位位置时就会产生假匹配。如果查询中所有词都有匹配（真或假），则必须检索表行查看匹配是否正确。

GiST 索引可以覆盖，例如，使用 INCLUDE 子句。包含的列可以有数据类型，而不需要任何 GiST 操作符类。包含的属性将不压缩存储。

有损性导致的性能下降归因于不必要的表记录（即被证实为假匹配的记录）获取。因为表记录的随机访问是较慢的，这限制了 GiST 索引的可用性。假匹配的可能性取决于几个因素，特别是唯一词的数量，因此推荐使用词典来缩减这个数量。

注意 GIN 索引的构件时间常常可以通过增加 maintenance_work_mem 来改进，而 GiST 索引的构建时间则与该参数无关。

对大集合分区并正确使用 GIN 和 GiST 索引允许实现带在线更新的快速搜索。分区可以在数据库层面上使用表

继承来完成，或者是通过将文档分布在服务器上并收集外部的搜索结果，例如通过外部数据 访问。后者是可能的，因为排名函数只使用本地信息。

9.1.12　RUM 索引类型

RUM 索引作为一个全文索引类型，是 GIN 全文索引的扩展。与内建的 GIN 和 GiST 全文索引的区别是它是以插件包的形式存在。在使用前需要通过 `CREATE EXTENSION` 语句来加载插件包。

```
CREATE EXTENSION RUM;
```

MySQL 模式下使用 kdb_mysql_rum 插件，功能与 rum 插件相同。

RUM 索引虽然基于 GIN 索引实现，但是它能够更快地执行全文索引。

GIN 索引可以基于 tsvector 和 tsquery 类型执行快速的全文搜索，但是它也存在以下的问题：

- 排序慢。需要有关词素的位置信息才能进行排序。GIN 索引不存储词素的位置信息，因此，在索引扫描之后，我们需要额外的堆扫描以检索词素位置。
- 短语搜索慢。该问题与上一个问题是一个原因，它需要位置信息来执行短语搜索。
- 时间戳排序慢。GIN 索引无法在带有词素的索引中存储相关信息，因此，要执行额外的堆扫描。

RUM 索引解决了上述问题，但是它也存在缺点，因为它要存储额外的信息和需要使用 WAL 日志，导致它比 GIN 索引构建和插入时更慢。

9.1.12.1　常用操作符和函数

rum 插件提供了如下的操作符

操作符	返回值	描述
tsvector <=> tsquery	float4	返回 tsvector 和 tsquery 的间距
timestamp <=> timestamp	float8	返回两个 timestamp 的间距
timestamp <=| timestamp	float8	仅返回左边 timestamps 的间距
timestamp |=> timestamp	float8	仅返回右边 timestamps 的间距

加载 kdb_mysql_rum 插件后，MySQL 模式下使用操作符“<==>”替换原来的“<=>”操作符功能。

后面的三个操作符同样适用于 timestamptz, int2, int4, int8, float4, float8, money 和 oid 类型。

9.1.12.2　操作符类

rum 插件提供了如下的操作符类

rum_tsvector_ops

适用类型: tsvector

这个操作符类存储 tsvector 词素的位置信息，同时支持使用操作符 <=> 进行排序及前缀搜索。如下例所示：

```
CREATE TABLE test_rum(t text, a tsvector);

CREATE TRIGGER tsvectorupdate
BEFORE UPDATE OR INSERT ON test_rum
FOR EACH ROW EXECUTE PROCEDURE tsvector_update_trigger('a', 'sys_catalog.english', 't');

INSERT INTO test_rum(t) VALUES ('The situation is most beautiful');
INSERT INTO test_rum(t) VALUES ('It is a beautiful');
INSERT INTO test_rum(t) VALUES ('It looks like a beautiful place');
```

要使用 rum 索引，需要先创建 rum 插件:

```
CREATE EXTENSION rum;
```

然后我们创建 rum 索引:

```
CREATE INDEX rumidx ON test_rum USING rum (a rum_tsvector_ops);
```

执行如下的 SQL 语句:

```
SELECT t, a <=> to_tsquery('english', 'beautiful | place') AS rank
    FROM test_rum
    WHERE a @@ to_tsquery('english', 'beautiful | place')
    ORDER BY a <=> to_tsquery('english', 'beautiful | place');
                t                 |  rank
----------------------------------+---------
 It looks like a beautiful place | 8.22467
 The situation is most beautiful | 16.4493
 It is a beautiful               | 16.4493
(3 rows)

SELECT t, a <=> to_tsquery('english', 'place | situation') AS rank
    FROM test_rum
    WHERE a @@ to_tsquery('english', 'place | situation')
    ORDER BY a <=> to_tsquery('english', 'place | situation');
                t                 |  rank
----------------------------------+---------
 The situation is most beautiful | 16.4493
```

```
 It looks like a beautiful place | 16.4493
(2 rows)
```

rum_tsvector_hash_ops

适用类型: tsvector

这个操作符类存储了 tsvector 词素位置的哈希值，支持操作符 <=> 排序，但是不支持前缀搜索。

rum_TYPE_ops

TYPE 对应类型: int2, int4, int8, float4, float8, money, oid, time, timetz, date, interval, macaddr, inet, cidr, text, varchar, char, bytea, bit, varbit, numeric, timestamp, timestamptz

支持的操作符: <, <=, =, >=, > 适用上述所有类型；'<=>', <=| 和 |=> 适用 int2, int4, int8, float4, float8, money, oid,timestamp 和 timestamptz 类型。

支持使用 <=>, <=| 和 |=> 操作符进行排序，同时也可以使用 rum_tsvector_addon_ops, rum_tsvector_hash_addon_ops 和 rum_anyarray_addon_ops 操作符类.

rum_tsvector_addon_ops

适用类型: tsvector

这个操作符支持以附加的形式保存 tsvector 词素.

```
CREATE TABLE tsts (id int, t tsvector, d timestamp);

\copy tsts from 'rum/data/tsts.data'

CREATE INDEX tsts_idx ON tsts USING rum (t rum_tsvector_addon_ops, d)
    WITH (attach = 'd', to = 't');
```

执行如下 SQL 语句：

```
EXPLAIN (costs off)
    SELECT id, d, d <=> '2016-05-16 14:21:25' FROM tsts WHERE t @@ 'wr&qh' ORDER BY d <=> '2016-05-16 14:21:
25' LIMIT 5;
                                    QUERY PLAN
-------------------------------------------------------------------------------------
 Limit
   ->  Index Scan using tsts_idx on tsts
         Index Cond: (t @@ '''wr'' & ''qh'''::tsquery)
         Order By: (d <=> 'Mon May 16 14:21:25 2016'::timestamp without time zone)
(4 rows)

SELECT id, d, d <=> '2016-05-16 14:21:25' FROM tsts WHERE t @@ 'wr&qh' ORDER BY d <=> '2016-05-16 14:21:25'
LIMIT 5;
 id  |                d                 |   ?column?
-----+----------------------------------+---------------
```

```
 355 | Mon May 16 14:21:22.326724 2016 |      2.673276
 354 | Mon May 16 13:21:22.326724 2016 |   3602.673276
 371 | Tue May 17 06:21:22.326724 2016 |  57597.326724
 406 | Wed May 18 17:21:22.326724 2016 | 183597.326724
 415 | Thu May 19 02:21:22.326724 2016 | 215997.326724
(5 rows)
```

rum_tsvector_hash_addon_ops

适用类型: tsvector

这个操作符以附加的信息保存 tsvector 词素的哈希值。它不支持前缀搜索。

rum_tsquery_ops

适用类型: tsquery

在附加信息里保存查询树的分支:

```
CREATE TABLE query (q tsquery, tag text);

INSERT INTO query VALUES ('supernova & star', 'sn'),
    ('black', 'color'),
    ('big & bang & black & hole', 'bang'),
    ('spiral & galaxy', 'shape'),
    ('black & hole', 'color');

CREATE INDEX query_idx ON query USING rum(q);
```

执行如下 SQL 语句：

```
SELECT * FROM query
    WHERE to_tsvector('black holes never exists before we think about them') @@ q;
        q         |  tag
------------------+-------
 'black'          | color
 'black' & 'hole' | color
(2 rows)
```

rum_anyarray_ops

适用类型: anyarray

这个操作符类保存 anyarrray 元素的长度，支持 &&, @>, <@, =, % 操作符，支持使用 <=> 操作进行排序。

```
CREATE TABLE test_array (i int2[]);

INSERT INTO test_array VALUES ('{}'), ('{0}'), ('{1,2,3,4}'), ('{1,2,3}'), ('{1,2}'), ('{1}');
```

```
CREATE INDEX idx_array ON test_array USING rum (i rum_anyarray_ops);
```

执行如下 SQL 语句：

```
SET enable_seqscan TO off;

EXPLAIN (COSTS OFF) SELECT * FROM test_array WHERE i && '{1}' ORDER BY i <=> '{1}' ASC;
                QUERY PLAN
-----------------------------------------
 Index Scan using idx_array on test_array
   Index Cond: (i && '{1}'::smallint[])
   Order By: (i <=> '{1}'::smallint[])
(3 rows

SELECT * FROM test_array WHERE i && '{1}' ORDER BY i <=> '{1}' ASC;
     i
-----------
 {1}
 {1,2}
 {1,2,3}
 {1,2,3,4}
(4 rows)
```

rum_anyarray_addon_ops

使用类型: anyarray

这个操作符类支持在任何支持附加属性的地方保存 anyarrray 元素。

9.1.13　ksql 支持

关于文本搜索配置对象的信息可以在 ksql 中使用一组命令获得：

```
\dF{d,p,t}[+] [PATTERN]
```

可选的 + 能产生更多细节。

可选参数 PATTERN 可以是一个文本搜索对象的名称，可以是模式限定的。如果 PATTERN 被忽略，则所有可见对象的信息都将被显示。PATTERN 可以是一个正则表达式并且可以为模式和对象名称提供独立的模式。下面的例子展示了这些特性：

```
=> \dF *fulltext*
       List of text search configurations
 Schema |  Name        | Description
```

```
--------+--------------+-------------
 public | fulltext_cfg |
```

```
=> \dF *.fulltext*
       List of text search configurations
 Schema   |  Name        | Description
----------+----------------------------
 fulltext | fulltext_cfg |
 public   | fulltext_cfg |
```

可用的命令是：

`\dF[+] [PATTERN]`
列出文本搜索配置（加上 + 得到更多细节）。

`\dFd[+] [PATTERN]`
列出文本搜索词典（加上 + 得到更多细节）。

`\dFp[+] [PATTERN]`
列出文本搜索解析器（加上 + 得到更多细节）。

`\dFt[+] [PATTERN]`
列出文本搜索模板（加上 + 得到更多细节）。

9.1.14　限制

KingbaseES 的文本搜索特性的当前限制是：

- 每一个词位的长度必须小于 2K 字节
- 一个 `tsvector`（词位 + 位置）的长度必须小于 1 兆字节
- 词位的数量必须小于 264
- `tsvector` 中的位置值必须大于 0 并且小于 16,383
- `<N>`（FOLLOWED BY）`tsquery` 操作符中的匹配距离不能超过 16,384
- 每个词位不超过 256 个位置
- 一个 `tsquery` 中结点（词位 + 操作符）的个数必须小于 32,768

为了对比，KingbaseES V7 的文档包含 10,441 个唯一词，总数 335,420 个词，并且最频繁的词“kingbase”在 655 个文档中被提到 6,127 次。

另一个例子—KingbaseES 的邮件列表归档在 461,020 条消息的 57,491,343 个词位中包含 910,989 个唯一词。

9.2　规则系统

本文档讨论 KingbaseES 中的规则系统。

规则系统（查询重写规则系统）与存储过程和触发器完全不同。它将查询修改为需要考虑规则，并且把修改过的查询传递给查询计划器进行规划和执行。规则系统非常强大，可以被应用于查询语言过程、视图和版本等多领域。

本章节包含以下内容：

- 查询树
- 视图和规则系统
- 物化视图
- INSERT、UPDATE 和 DELETE 上的规则
- 规则和权限
- 规则和命令状态
- 规则 vs 触发器

9.2.1　查询树

了解规则系统是如何工作的，必须知道它在什么时候被调用以及其输入和结果是什么。

规则系统位于解析器和规划器之间。它采用解析器的输出（即一个查询树）和用户定义的重写规则（也是查询树，不过带有一些额外信息），并且常见零个或者更多个查询树作为结果。因此它的输入和输出总是那些规划器自身就能产生的东西，并且因此它看到的任何东西都可以被表示成一个 SQL 语句。

那么什么是一个查询树？它是一个 SQL 语句的一种内部表示，其中用于创建它的每一个单独的部分都被独立存储。如果你设置了配置参数 `debug_print_parse`、`debug_print_rewritten` 或 `debug_print_plan`，这些查询树可以被显示在服务器日志中。规则动作也被做为查询树存储在系统目录 `sys_rewrite` 中。它们没有被格式化为日志输出的形式，但是它们包含完全相同的信息。

阅读一棵未加工的查询树需要要一些经验。但是由于查询树的 SQL 表示形式足以用来理解规则系统，本章将不会教授如何阅读查询树。

在阅读本章中查询树的 SQL 表现形式时，读者需要能够知道语句被分解成了哪些部分并且能在查询树结构中标识它们。一棵查询树的部分有：

- 命令类型

  这是一个简单的值来说明是哪一种命令（`SELECT`、`INSERT`、`UPDATE`、`DELETE`）产生了该查询树。

- 范围表

  范围表是被使用在该查询中的关系的列表。在一个 `SELECT` 语句中，范围表是在关键词 `FROM` 后面给出的关系。

每一个范围表项标识一个表或视图，并且说明在该查询的其他部分要以哪个名称调用它。在查询树中，范围表项被使用编号而不是名称来引用，因此在一个 SQL 语句中出现重复的名字也没有关系。在规则的范围表被合并以后可能会发生这种情况。本章中的例子将不会有这种情况。

- 结果关系

  这是一个指向范围表的索引，它标识了该查询的结果应该去哪个关系。

  SELECT 查询没有结果关系（特殊情况 SELECT INTO 几乎等于 CREATE TABLE 后面跟上 INSERT ... SELECT，并且不在这里单独讨论）。

  对于 INSERT、UPDATE 和 DELETE 命令，结果关系是修改要进行的表（或视图！）。

- 目标列表

  目标列表是一个表达式的列表，它定义了查询的结果。在一个 SELECT 的情况下，这些表达式会构建出该查询最终的输出。它们对应于关键字 SELECT 和 FROM 之间的表达式（* 是一个关系所有列名的缩写。解析器会把它扩展成独立的列，因此规则系统永远见不到它）。

  DELETE 命令不需要一个目标列表，因为它们不产生任何结果。相反，规划器会向空的目标列表中加入一个特殊的 CTID 项来允许执行器找到要被删除的行（当结果关系是一个普通表时才加入 CTID。如果结果关系是一个视图，则会被规则系统加入一个整行变量，如更新一个视图 所述）。

  对于 INSERT 命令，目标列表描述了将要进入到结果关系中的新行。它由 VALUES 子句中的表达式或来自 INSERT ... SELECT 中 SELECT 子句的表达式构成。重写处理的第一步会为那些没有被原始命令赋值但有默认值的列增加目标列项。任何剩余的列（既没有给定值也没有默认值）将被规划器用一个常量空值表达式填充。

  对于 UPDATE 命令，目标列表描述要替换旧行的新行。在规则系统中，它只包含来自命令的 SET column = expression 部分的表达式。规划器将处理缺失的列，做法是为它们插入表达式，这种表达式会把旧行的值复制到新行。正如 DELETE 一样，会增加一个 CTID 或整行变量，这样执行器能够标识要被更新的旧行。

  目标列表中的每一个项所包含的表达式可以是一个常量值、一个指向范围表中关系的列的变量、一个参数或一个由函数调用、常量、变量、操作符等构成的表达式树。

- 条件

  查询的条件是一个表达式，它很像包含在目标列表项中的表达式。这个表达式的结果值是一个布尔值，它说明对最终结果行的操作（INSERT、UPDATE、DELETE 或 SELECT）是否应该被执行。它对应于一个 SQL 语句的 WHERE 子句。

- 连接树

  查询的连接树展示了 FROM 子句的结构。对于一个 SELECT ... FROM a, b, c 这样的简单查询，连接树就是 FROM 项的一个列表，因为我们被允许以任何顺序连接它们。但是当 JOIN 表达式（特别是外连接）被使用时，我们必须按照连接显示的顺序来连接。在这种情况下，连接树展示了 JOIN 表达式的结构。与特定 JOIN 子句（来自 ON 或 USING）相关的限制被存储为附加到那些连接树节点的条件表达式。我们发现把顶层 WHERE 表达式存储为附加到顶层连接树项的一个条件也很方便。这样实际上连接树表达了一个 SELECT 的 FROM 和 WHERE 子句。

- 其他

查询树的其他部分（如 ORDER BY 子句）在这里并不受到关注。规则系统在应用规则时会替换这里的某些项，但是这些与规则系统的基础没有什么关系。

9.2.2　视图和规则系统

KingbaseES 中的视图是通过规则系统来实现的。事实上，下面的命令

```
CREATE TABLE mytab(
    id INT PRIMARY KEY,
    col1 VARCHAR(50),
    col2 INT,
    col3 DATE
);
CREATE VIEW myview AS SELECT * FROM mytab;
```

与下面两个命令相比没有不同：

```
CREATE TABLE myview (same column list as mytab);
CREATE RULE "_RETURN" AS ON SELECT TO myview DO INSTEAD
    SELECT * FROM mytab;
```

因为这就是 CREATE VIEW 命令在内部所作的。这样做有一些副作用。其中之一就是在 KingbaseES 系统目录中的视图信息与表的信息完全一样。所以对于解析器来说，表和视图之间完全没有区别。它们是同样的事务：关系。

9.2.2.1　SELECT 规则如何工作

规则 ON SELECT 被应用于所有查询作为最后一步，即使给出的是一条 INSERT、UPDATE 或 DELETE 命令。而且它们与其他命令类型上的规则有着不同的语义，它们会就地修改查询树而不是创建一个新的查询树。因此我们首先描述 SELECT 规则。

目前，一个 ON SELECT 规则中只能有一个动作，而且它必须是一个无条件的 INSTEAD 的 SELECT 动作。这个限制是为了令规则足够安全，以便普通用户也可以打开它们，并且它限制 ON SELECT 规则使之行为类似视图。

本章的例子是两个连接视图，它们做一些运算并且某些更多视图会轮流使用它们。最前面的两个视图之一后面将利用对 INSERT、UPDATE 和 DELETE 操作增加规则的方法被自定义，这样最终结果将是一个视图，它表现得像一个具有魔力的真正的表。这个例子不适合于作为简单易懂的例子，它可能会让本章更难懂。但是用一个覆盖所有关键点的例子来一步一步讨论要比举很多例子搞乱思维好。

例如，我们需要一个小巧的 min 函数用于返回两个整数值中较小的那个。我们这样创建它：

```
CREATE FUNCTION min(integer, integer) RETURNS integer AS $$
    SELECT CASE WHEN $1 < $2 THEN $1 ELSE $2 END
$$ LANGUAGE SQL STRICT;
```

在前两个规则系统描述中我们需要真实表是：

```
CREATE TABLE shoe_data (
    shoename   text,          -- 主键
    sh_avail   integer,       -- 可用的双数
    slcolor    text,          -- 首选的鞋带颜色
    slminlen   real,          -- 最小鞋带长度
    slmaxlen   real,          -- 最大鞋带长度
    slunit     text           -- 长度单位
);

CREATE TABLE shoelace_data (
    sl_name    text,          -- 主键
    sl_avail   integer,       -- 可用的双数
    sl_color   text,          -- 鞋带颜色
    sl_len     real,          -- 鞋带长度
    sl_unit    text           -- 长度单位
);

CREATE TABLE unit (
    un_name    text,          -- 主键
    un_fact    real           -- 转换到厘米的参数
);
```

如你所见，它们表示鞋店的数据。

视图被创建为：

```
CREATE VIEW shoe AS
    SELECT sh.shoename,
           sh.sh_avail,
           sh.slcolor,
           sh.slminlen,
           sh.slminlen * un.un_fact AS slminlen_cm,
           sh.slmaxlen,
           sh.slmaxlen * un.un_fact AS slmaxlen_cm,
           sh.slunit
      FROM shoe_data sh, unit un
     WHERE sh.slunit = un.un_name;

CREATE VIEW shoelace AS
    SELECT s.sl_name,
           s.sl_avail,
           s.sl_color,
           s.sl_len,
           s.sl_unit,
           s.sl_len * u.un_fact AS sl_len_cm
      FROM shoelace_data s, unit u
```

```
     WHERE s.sl_unit = u.un_name;

CREATE VIEW shoe_ready AS
    SELECT rsh.shoename,
           rsh.sh_avail,
           rsl.sl_name,
           rsl.sl_avail,
           min(rsh.sh_avail, rsl.sl_avail) AS total_avail
      FROM shoe rsh, shoelace rsl
     WHERE rsl.sl_color = rsh.slcolor
       AND rsl.sl_len_cm >= rsh.slminlen_cm
       AND rsl.sl_len_cm <= rsh.slmaxlen_cm;
```

创建 shoelace 视图的 CREATE VIEW 命令（也是最简单的一个）将创建一个 shoelace 关系和一个 sys_rewrite 项，这个 sys_rewrite 项说明有一个重写规则，只要一个查询的范围表中引用了关系 shoelace，就必须应用它。该规则没有规则条件（稍后和非 SELECT 规则一起讨论，因为目前的 SELECT 规则不能有规则条件）并且它是 INSTEAD 规则。要注意规则条件与查询条件不一样。我们的规则的动作有一个查询条件。该规则的动作是一个查询树，这个查询是视图创建命令中的 SELECT 语句的一个拷贝。

备注:

你在 sys_rewrite 项中看到的两个额外的用于 NEW 和 OLD 的范围表项不是 SELECT 规则感兴趣的东西。

现在我们填充 unit、shoe_data 和 shoelace_data，并且在视图上运行一个简单的查询：

```
INSERT INTO unit VALUES ('cm', 1.0);
INSERT INTO unit VALUES ('m', 100.0);
INSERT INTO unit VALUES ('inch', 2.54);

INSERT INTO shoe_data VALUES ('sh1', 2, 'black', 70.0, 90.0, 'cm');
INSERT INTO shoe_data VALUES ('sh1', 0, 'black', 30.0, 40.0, 'inch');
INSERT INTO shoe_data VALUES ('sh2', 4, 'brown', 50.0, 65.0, 'cm');
INSERT INTO shoe_data VALUES ('sh3', 3, 'brown', 40.0, 50.0, 'inch');

INSERT INTO shoelace_data VALUES ('sl1', 5, 'black', 80.0, 'cm');
INSERT INTO shoelace_data VALUES ('sl2', 6, 'black', 100.0, 'cm');
INSERT INTO shoelace_data VALUES ('sl3', 0, 'black', 35.0 , 'inch');
INSERT INTO shoelace_data VALUES ('sl4', 8, 'black', 40.0 , 'inch');
INSERT INTO shoelace_data VALUES ('sl5', 4, 'brown', 1.0 , 'm');
INSERT INTO shoelace_data VALUES ('sl6', 0, 'brown', 0.9 , 'm');
INSERT INTO shoelace_data VALUES ('sl7', 7, 'brown', 60 , 'cm');
INSERT INTO shoelace_data VALUES ('sl8', 1, 'brown', 40 , 'inch');

SELECT * FROM shoelace;
```

```
 sl_name   | sl_avail | sl_color | sl_len | sl_unit | sl_len_cm
-----------+----------+----------+--------+---------+-----------
 sl1       |        5 | black    |     80 | cm      |        80
 sl2       |        6 | black    |    100 | cm      |       100
 sl7       |        7 | brown    |     60 | cm      |        60
 sl3       |        0 | black    |     35 | inch    |      88.9
 sl4       |        8 | black    |     40 | inch    |     101.6
 sl8       |        1 | brown    |     40 | inch    |     101.6
 sl5       |        4 | brown    |      1 | m       |       100
 sl6       |        0 | brown    |    0.9 | m       |        90
(8 rows)
```

这是你可以在我们的视图上做的最简单的 SELECT，所以我们用这次机会来解释视图规则的基本要素。SELECT * FROM shoelace 会被解析器解释并生成下面的查询树：

```
SELECT shoelace.sl_name, shoelace.sl_avail,
       shoelace.sl_color, shoelace.sl_len,
       shoelace.sl_unit, shoelace.sl_len_cm
  FROM shoelace shoelace;
```

然后这将被交给规则系统。规则系统遍历范围表，检查有没有可用于任何关系的规则。在为 shoelace（到目前为止的唯一一个）处理范围表时，它会发现查询树里有 _RETURN 规则：

```
SELECT s.sl_name, s.sl_avail,
       s.sl_color, s.sl_len, s.sl_unit,
       s.sl_len * u.un_fact AS sl_len_cm
  FROM shoelace old, shoelace new,
       shoelace_data s, unit u
 WHERE s.sl_unit = u.un_name;
```

要扩展该视图，重写器简单地创建一个子查询范围表项，它包含规则的动作的查询树，然后用这个范围表记录取代原来引用视图的那个。作为结果的重写后的查询树几乎与你键入的那个一样：

```
SELECT shoelace.sl_name, shoelace.sl_avail,
       shoelace.sl_color, shoelace.sl_len,
       shoelace.sl_unit, shoelace.sl_len_cm
  FROM (SELECT s.sl_name,
               s.sl_avail,
               s.sl_color,
               s.sl_len,
               s.sl_unit,
               s.sl_len * u.un_fact AS sl_len_cm
          FROM shoelace_data s, unit u
         WHERE s.sl_unit = u.un_name) shoelace;
```

不过有一个区别：子查询的范围表有两个额外的项 shoelace old 和 shoelace new。这些项并不直接参与到查询中，因为它们没有被子查询的连接树或者目标列表引用。重写器用它们存储最初出现在引用视图的范围表项中表达的访问权限检查信息。以这种方式，执行器仍然会检查该用户是否有访问视图的正确权限，尽管在重写后的查询中没有对视图的直接使用。

这是被应用的第一个规则。规则系统将继续检查顶层查询里剩下的范围表项（本例中没有了），并且它将递归的检查增加的子查询中的范围表项，看看其中有没有引用视图的（不过这样不会扩展 old 或 new —否则我们会得到无限递归！）。在这个例子中，没有用于 shoelace_data 或 unit 的重写规则，所以重写结束并且上面得到的就是给规划器的最终结果。

现在我们想写一个查询，它找出目前在店里哪些鞋子有匹配的（颜色和长度）鞋带并且完全匹配的鞋带双数大于等于二。

```
SELECT * FROM shoe_ready WHERE total_avail >= 2;

 shoename | sh_avail | sl_name | sl_avail | total_avail
----------+----------+---------+----------+-------------
 sh1      |        2 | sl1     |        5 |           2
 sh2      |        4 | sl7     |        7 |           4
(2 rows)
```

这词解析器的输出是查询树：

```
SELECT shoe_ready.shoename, shoe_ready.sh_avail,
       shoe_ready.sl_name, shoe_ready.sl_avail,
       shoe_ready.total_avail
  FROM shoe_ready shoe_ready
 WHERE shoe_ready.total_avail >= 2;
```

第一个被应用的规则将是用于 shoe_ready 的规则并且它会导致查询树：

```
SELECT shoe_ready.shoename, shoe_ready.sh_avail,
       shoe_ready.sl_name, shoe_ready.sl_avail,
       shoe_ready.total_avail
  FROM (SELECT rsh.shoename,
               rsh.sh_avail,
               rsl.sl_name,
               rsl.sl_avail,
               min(rsh.sh_avail, rsl.sl_avail) AS total_avail
          FROM shoe rsh, shoelace rsl
         WHERE rsl.sl_color = rsh.slcolor
           AND rsl.sl_len_cm >= rsh.slminlen_cm
           AND rsl.sl_len_cm <= rsh.slmaxlen_cm) shoe_ready
 WHERE shoe_ready.total_avail >= 2;
```

相似地，用于 shoe 和 shoelace 的规则被替换到子查询的范围表中，得到一个三层的最终查询树：

```
SELECT shoe_ready.shoename, shoe_ready.sh_avail,
       shoe_ready.sl_name, shoe_ready.sl_avail,
       shoe_ready.total_avail
  FROM (SELECT rsh.shoename,
               rsh.sh_avail,
               rsl.sl_name,
               rsl.sl_avail,
               min(rsh.sh_avail, rsl.sl_avail) AS total_avail
         FROM (SELECT sh.shoename,
                      sh.sh_avail,
                      sh.slcolor,
                      sh.slminlen,
                      sh.slminlen * un.un_fact AS slminlen_cm,
                      sh.slmaxlen,
                      sh.slmaxlen * un.un_fact AS slmaxlen_cm,
                      sh.slunit
                 FROM shoe_data sh, unit un
                WHERE sh.slunit = un.un_name) rsh,
               (SELECT s.sl_name,
                      s.sl_avail,
                      s.sl_color,
                      s.sl_len,
                      s.sl_unit,
                      s.sl_len * u.un_fact AS sl_len_cm
                 FROM shoelace_data s, unit u
                WHERE s.sl_unit = u.un_name) rsl
        WHERE rsl.sl_color = rsh.slcolor
          AND rsl.sl_len_cm >= rsh.slminlen_cm
          AND rsl.sl_len_cm <= rsh.slmaxlen_cm) shoe_ready
 WHERE shoe_ready.total_avail > 2;
```

最后规划器会把这个树折叠成一个两层查询树：最下层的 SELECT 命令将被“提升”到中间的 SELECT 中，因为没有必要分别处理它们。但是中间的 SELECT 仍然和顶层的分开，因为它包含聚集函数。如果我们把它们也提升，它将改变顶层 SELECT 的行为，这不是我们想要的。不过，折叠查询树是一种优化，重写系统不需要关心它。

9.2.2.2　非 SELECT 语句中的视图规则

有两个查询树的细节在上面的视图规则的描述中没有涉及。它们是命令类型和结果关系。实际上，视图规则不需要命令类型，但是结果关系可能会影响查询重写器工作的方式，因为如果结果关系是一个视图，我们需要采取特殊的措施。

一个 SELECT 的查询树和其它命令的查询树之间很少的几处不同。显然，它们有不同的命令类型并且对于 SELECT 之外的命令，结果关系指向结果将进入的范围表项。其它所有东西都完全相同。所以如果有两个表 t1 和 t2 分别有列 a 和 b，下面两个语句的查询树：

```
CREATE TABLE t1(a INT, b INT);
CREATE TABLE t2(a INT, b INT);

SELECT t2.b FROM t1, t2 WHERE t1.a = t2.a;

UPDATE t1 SET b = t2.b FROM t2 WHERE t1.a = t2.a;
```

几乎是一样的。特别是：

- 范围表包含表 t1 和 t2 的项。
- 目标列表包含一个变量，该变量指向表 t2 的范围表项的列 b。
- 条件表达式比较两个范围表项的列 a 以寻找相等。
- 连接树展示了 t1 和 t2 之间的一次简单连接。

结果是，两个查询树生成相似的执行计划：它们都是两个表的连接。对于 UPDATE 语句，规划器把 t1 缺失的列加到目标列并且最终查询树读起来是：

```
UPDATE t1 SET a = t1.a, b = t2.b FROM t2 WHERE t1.a = t2.a;
```

因此在连接上运行的执行器将产生完全相同的结果集：

```
SELECT t1.a, t2.b FROM t1, t2 WHERE t1.a = t2.a;
```

但是在 UPDATE 中有个小问题：执行器计划中执行连接的部分不关心连接的结果的含义。它只是产生一个行的结果集。一个是 SELECT 命令而另一个是由执行器中的更高层处理的 UPDATE 命令，在那里执行器知道这是一个 UPDATE，并且它知道这个结果应该进入表 t1。但是这里的哪些行必须被新行替换呢？

要解决这个问题，在 UPDATE 和 DELETE 语句的目标列表里面增加了另外一个项：当前元组 ID（CTID）。这是一个系统列，它包含行所在的文件块编号和在块中的位置。在已知表的情况下，CTID 可以被用来检索要被更新的 t1 的原始行。在添加 CTID 到目标列之后，该查询实际看起来像：

```
SELECT t1.a, t2.b, t1.ctid FROM t1, t2 WHERE t1.a = t2.a;
```

现在，另一个 KingbaseES 的细节进入到这个阶段了。表中的旧行还没有被覆盖，这就是为什么 ROLLBACK 很快的原因。在一个 UPDATE 中，新的结果行被插入到表中（在剥除 CTID 之后），并且把 CTID 指向的旧行的行头部中的 cmax 和 xmax 项设置为当前命令计数器和当前事务 ID 。这样旧的行就被隐藏起来，并且在事务提交之后 vacuum 清理器就可以最终移除死亡的行。

知道了所有这些，我们就可以用完全相同的方式简单地把视图规则应用到任意命令中。没有任何区别。

9.2.2.3　KingbaseES 中视图的能力

上文演示了规则系统如何把视图定义整合到原始的查询树中。在第二个例子中，一个来自于一个视图的简单 SELECT 创建了一个四表连接（unit 以不同的名字被用了两次）的最终查询树。

用规则系统实现视图的好处是，规划器拥有关于哪些表必须被扫描、这些表之间的联系、来自于视图的限制性条件、一个单一查询树中原始查询的条件等所有信息。当原始查询已经是一个视图上的连接时仍然是这样。规划器必须决定执行查询的最优路径，而且规划器拥有越多信息，该决定就越好。并且 KingbaseES 中实现的规则系统保证这些信息是此时能获得的有关该查询的所有信息。

9.2.2.4　更新一个视图

如果视图是 INSERT、UPDATE 或 DELETE 的目标关系会怎样？使用上文所述的替换将给出一个查询树，其中的结果关系指向一个子查询范围表项，这样无法工作。不过，KingbaseES 中有几种方法来支持更新视图。

如果子查询从一个单一基本关系选择并且该关系足够简单，重写器会自动地把该子查询替换成底层的基本关系，这样 INSERT、UPDATE 或 DELETE 会被以适当的方式应用到该基本关系。其中“足够简单”的视图被称为 * 自动可更新 *。有关这种可以被自动更新的视图类别的详细信息，请见 CREATE VIEW(Oracle)、CREATE VIEW(MySQL)、CRAETE VIEW(SQLServer)。

或者，该操作可以被定义在视图上的一个用户提供的 INSTEAD OF 触发器处理。在这种情况下重写工作有一点点不同。对于 INSERT，重写器对视图什么也不做，让它作为查询的结果关系。对于 UPDATE 和 DELETE，仍有必要扩展该视图查询来产生命令将尝试更新或删除的“旧”行。因此该视图被按照通常的方式扩展，但是另一个未被扩展的范围表项会被增加到查询来表示该视图会尽其所能作为结果关系。

现在出现的问题是如何标识在视图中要被更新的行。回忆一下，当结果关系是一个表时，一个特殊的 CTID 项会被加入到目标列表来标识要被更新的行的物理位置。如果结果关系是一个视图这就行不通，因为一个视图根本就没有 CTID，它的行没有实际的物理位置。对于一个 UPDATE 或 DELETE 操作，一个特殊的 wholerow 项会被增加到目标列表中，它会扩展来包括来自该视图的所有列。执行器使用这个值来提供“旧”行给 INSTEAD OF 触发器。现在就轮到触发器来基于新旧行值来找出要更新什么了。

另外一种可能性是让用户定义 INSTEAD 规则，这种规则指定对视图上的 INSERT、UPDATE 和 DELETE 命令的替代动作。这些规则将重写该命令，通常是重写成一个更新一个或多个表（而不是视图）的命令。这是INSERT、UPDATE 和 DELETE 上的规则 的主题。

注意规则会首先被计算，然后在原始查询被规划和执行之前重写它。因此，如果一个视图上同时有 INSTEAD OF 触发器和 INSERT、UPDATE 或 DELETE 规则，那么首先会计算规则，然后根据其结果决定是否执行触发器，触发器可能完全都不会被使用。

在一个简单视图上的 INSERT、UPDATE 或 DELETE 查询的自动重写总是在最后尝试。因此，如果一个视图有规则或触发器，它们将重载自动可更新视图的默认行为。

如果对该视图没有 INSTEAD 规则或 INSTEAD OF 触发器，并且重写器不能自动地把该查询重写成一个底层基本关系上的更新，将会抛出一个错误，因为执行器不能更新一个这样的视图。

9.2.3　物化视图

KingbaseES 中的物化视图像视图一样使用了规则系统，但是以一种类表的形式保留了结果。在物化视图：

```
CREATE MATERIALIZED VIEW mymatview AS SELECT * FROM mytab;
```

和视图：

```
CREATE TABLE mymatview AS SELECT * FROM mytab;
```

之间的主要区别是物化视图不能直接被更新，并且用于创建物化视图的查询的存储方式和视图查询的存储方式完全相同，因此要为物化视图生成新鲜的数据：

```
REFRESH MATERIALIZED VIEW mymatview;
```

有关一个 KingbaseES 系统目录中的物化视图的信息和一个表或视图的信息完全相同。因此对于解析器，一个物化视图就是一个关系，就像一个表或一个视图。当一个物化视图被一个查询引用时，数据直接从物化视图中返回，如同表一样；规则只被用来填充物化视图。

虽然对物化视图中存储的数据的访问常常要快于直接访问底层表或通过一个视图访问，但是数据并不总是最新的；但是某些时候并不需要当前数据。考虑一个记录销售情况的表：

```
CREATE TABLE invoice (
    invoice_no    integer        PRIMARY KEY,
    seller_no     integer,       -- 销售员的 ID
    invoice_date  date,          -- 销售日期
    invoice_amt   numeric(13,2)  -- 销售量
);
```

如果人们想快速绘制历史销售数据，他们可能希望汇总，并且他们可能并不关心当前日期的不完整数据：

```
CREATE MATERIALIZED VIEW sales_summary AS
  SELECT
      seller_no,
      invoice_date,
      sum(invoice_amt)::numeric(13,2) as sales_amt
    FROM invoice
    WHERE invoice_date < CURRENT_DATE
    GROUP BY
      seller_no,
      invoice_date
    ORDER BY
      seller_no,
      invoice_date;

CREATE UNIQUE INDEX sales_summary_seller
  ON sales_summary (seller_no, invoice_date);
```

这个物化视图可能对在为销售员创建的控制面板上显示一个图表非常有用。可以用一个计划任务在每晚使用这个 SQL 语句更新该统计信息：

```
REFRESH MATERIALIZED VIEW sales_summary;
```

物化视图的另一种使用是允许通过一个外部数据包装器对来自一个远程系统的数据进行更快的访问。下面有一个使用 file_fdw 的简单例子，但是由于本地系统上可以使用高速缓存，因此比起访问一个远程系统的性能差异可能会比这里所展示的更大。注意鉴于 file_fdw 不支持索引，我们也使用这种能力来在物化视图上放置索引。这种优势可能不适用于其他种类的外部数据访问。

建立：

```
CREATE EXTENSION file_fdw;
CREATE SERVER local_file FOREIGN DATA WRAPPER file_fdw;
CREATE FOREIGN TABLE words (word text NOT NULL)
  SERVER local_file
  OPTIONS (filename '/usr/share/dict/words');
CREATE MATERIALIZED VIEW wrd AS SELECT * FROM words;
CREATE UNIQUE INDEX wrd_word ON wrd (word);
CREATE EXTENSION sys_trgm;
CREATE INDEX wrd_trgm ON wrd USING gist (word gist_trgm_ops);
VACUUM ANALYZE wrd;
```

现在让我们对一个词进行拼写检查。直接使用 file_fdw：

```
SELECT count(*) FROM words WHERE word = 'caterpiler';

 count
-------
     0
(1 row)
```

通过 EXPLAIN ANALYZE，我们可以看到：

```
Aggregate  (cost=21763.99..21764.00 rows=1 width=0) (actual time=188.180..188.181
rows=1 loops=1)
   ->  Foreign Scan on words  (cost=0.00..21761.41 rows=1032 width=0) (actual
       time=188.177..188.177 rows=0 loops=1)
         Filter: (word = 'caterpiler'::text)
         Rows Removed by Filter: 479829
         Foreign File: /usr/share/dict/words
         Foreign File Size: 4953699
 Planning time: 0.118 ms
 Execution time: 188.273 ms
```

如果使用物化视图，该查询会快很多：

```
Aggregate  (cost=4.44..4.45 rows=1 width=0) (actual time=0.042..0.042 rows=1
loops=1)
```

```
   ->  Index Only Scan using wrd_word on wrd  (cost=0.42..4.44 rows=1 width=0)
       (actual time=0.039..0.039 rows=0 loops=1)
         Index Cond: (word = 'caterpiler'::text)
         Heap Fetches: 0
 Planning time: 0.164 ms
 Execution time: 0.117 ms
```

不管哪种方式，单词都是被拼错的，因此让我们看看什么是我们可能想要的。再次使用 `file_fdw`：

```
SELECT word FROM words ORDER BY word <-> 'caterpiler' LIMIT 10;

     word
---------------
 cater
 caterpillar
 Caterpillar
 caterpillars
 caterpillar's
 Caterpillar's
 caterer
 caterer's
 caters
 catered
(10 rows)
```

```
Limit  (cost=11583.61..11583.64 rows=10 width=32) (actual time=1431.591..1431.594
rows=10 loops=1)
   ->  Sort  (cost=11583.61..11804.76 rows=88459 width=32) (actual
      time=1431.589..1431.591 rows=10 loops=1)
         Sort Key: ((word <-> 'caterpiler'::text))
         Sort Method: top-N heapsort  Memory: 25kB
         ->  Foreign Scan on words  (cost=0.00..9672.05 rows=88459 width=32)
            (actual time=0.057..1286.455 rows=479829 loops=1)
               Foreign File: /usr/share/dict/words
               Foreign File Size: 4953699
 Planning time: 0.128 ms
 Execution time: 1431.679 ms
```

使用物化视图：

```
Limit  (cost=0.29..1.06 rows=10 width=10) (actual time=187.222..188.257
rows=10 loops=1)
   ->  Index Scan using wrd_trgm on wrd  (cost=0.29..37020.87 rows=479829
       width=10) (actual time=187.219..188.252 rows=10 loops=1)
         Order By: (word <-> 'caterpiler'::text)
```

```
Planning time: 0.196 ms
Execution time: 198.640 ms
```

如果你能够忍受定期把远程数据更新到本地数据库，其性能收益可能是巨大的。

9.2.4　INSERT、UPDATE 和 DELETE 上的规则

定义在 INSERT、UPDATE 和 DELETE 上的规则与前一节描述的视图规则有明显的不同。

首先，它们的 CREATE RULE 命令允许更多：

- 它们可以没有动作。
- 它们可以有多个动作。
- 它们可以是 INSTEAD 或 ALSO（缺省）。
- 伪关系 NEW 和 OLD 变得有用了。
- 它们可以有规则条件。

第二，它们不是就地修改查询树，而是创建零个或多个新查询树并且可能把原始的那个查询树扔掉。

注意:

在很多情况下，由 INSERT/UPDATE/DELETE 上的规则执行的任务用触发器能做得更好。触发器在记法上要更复杂些，但是它们的语义理解起来更简单些。当原始查询包含不稳定函数时，规则容易产生令人惊讶的结果：在执行规则的过程中不稳定函数的执行次数可能比语气中的更多。

还有，有些情况根本无法用这些类型的规则支持，典型的是在原始查询中包括 WITH 子句以及在 UPDATE 查询的 SET 列表中包括多个赋值的子 SELECT。这是因为把这些结构复制到一个规则查询中可能导致子查询的多次计算，这与查询作者表达的意图相悖。

9.2.4.1　更新规则如何工作

记住以下语法：

```
CREATE [ OR REPLACE ] RULE name AS ON event
    TO table [ WHERE condition ]
    DO [ ALSO | INSTEAD ] { NOTHING | command | ( command ; command ... ) }
```

在随后的内容中，* 更新规则 * 表示定义在 INSERT、UPDATE 或 DELETE 上的规则。

如果查询树的结果关系和命令类型等于 CREATE RULE 命令中给出的对象和事件，规则系统就会应用更新规则。对于更新规则，规则系统会创建一个查询树列表。一开始该查询树列表是空的。更新规则中可以有零个（NOTHING 关

键字）、一个或多个动作。为简单起见，我们先看一个只有一个动作的规则。这个规则可以有条件或者没有条件，并且它可以是 INSTEAD 或 ALSO（缺省）。

什么是规则条件？它是一个限制，告诉规则动作什么时候做、什么时候不做。这个条件只能引用 NEW 和/或 OLD 伪关系，它们基本上代表作为对象给定的关系（但是有着特殊含义）。

所以，对这个单动作的规则生成下面的查询树，我们有三种情况。

- 没有条件，有 ALSO 或 INSTEAD

  来自规则动作的查询树，在其上增加原始查询树的条件

- 给出了条件，有 ALSO

  来自规则动作的查询树，在其上加入规则条件和原始查询树的条件

- 给出了条件，有 INSTEAD

  来自规则动作的查询树，在其上加入规则条件和原始查询树的条件；以及带有反规则条件的原始查询树

最后，如果规则是 ALSO，那么未修改的原始查询树也被加入到列表。因为只有合格的 INSTEAD 规则已经被加入到原始查询树中，对于单动作的规则，我们将结束于一个或两个输出查询树。

对于 ON INSERT 规则，原始查询（如果没有被 INSTEAD 取代）是在任何规则增加的动作之前完成的。这样就允许动作看到被插入的行。但是对 ON UPDATE 和 ON DELETE 规则，原始查询是在规则增加的动作之后完成的。这样就确保动作可以看到将要更新或者将要删除的行；否则，动作可能什么也不做，因为它们无法发现符合它们要求的行。

从规则动作生成的查询树会被再次丢给重写系统，并且可能有更多规则被应用而得到更多或更少的查询树。所以一个规则的动作必须有一种不同的命令类型或者和规则所在的关系不同的另一个结果关系。否则这样的递归处理就会没完没了（规则的递规展开会被检测到，并当作一个错误报告）。

在 sys_rewrite 系统目录中的动作中的查询树只是模板。因为它们可以引用 NEW 和 OLD 的范围表项，在使用它们之前必须做一些替换。对于任何 NEW 的引用，都要先在原始查询的目标列表中搜索对应的项。如果找到，该项的表达式将会替换该引用。否则 NEW 和 OLD 的含义一样（对于 UPDATE）或者被替换成一个空值（对于 INSERT）。任何对 OLD 的引用都用结果关系的范围表项的引用替换。

在系统完成应用更新规则后，它再应用视图规则到生成的查询树上。视图无法插入新的更新动作，所以没有必要向视图重写的输出应用更新规则。

9.2.4.1.1　第一个规则循序渐进

假设我们想要跟踪 shoelace_data 关系中的 sl_avail 列。所以我们建立一个日志表和一条规则，这条规则每次在 shoelace_data 上执行 UPDATE 时有条件地写入一个日志项。

```
CREATE TABLE shoelace_log (
    sl_name    text,          -- 改变的鞋带
    sl_avail   integer,       -- 新的可用值
    log_who    text,          -- 谁做的
    log_when   timestamp      -- 何时做的
);
```

```
CREATE RULE log_shoelace AS ON UPDATE TO shoelace_data
    WHERE NEW.sl_avail <> OLD.sl_avail
    DO INSERT INTO shoelace_log VALUES (
                                    NEW.sl_name,
                                    NEW.sl_avail,
                                    current_user,
                                    current_timestamp
                                );
```

现在有人做：

```
UPDATE shoelace_data SET sl_avail = 6 WHERE sl_name = 'sl7';
```

然后看看日志表：

```
SELECT * FROM shoelace_log;

 sl_name | sl_avail | log_who | log_when
---------+----------+---------+----------------------------------
 sl7     |        6 | Al      | Tue Oct 20 16:14:45 1998 MET DST
(1 row)
```

这就是我们所期望的。在后台发生的的事情如下。解析器创建查询树：

```
UPDATE shoelace_data SET sl_avail = 6
  FROM shoelace_data shoelace_data
 WHERE shoelace_data.sl_name = 'sl7';
```

这是一个带有规则条件表达式的 ON UPDATE 规则 log_shoelace ，条件是：

```
NEW.sl_avail <> OLD.sl_avail
```

它的动作是：

```
INSERT INTO shoelace_log VALUES (
       new.sl_name, new.sl_avail,
       current_user, current_timestamp )
  FROM shoelace_data new, shoelace_data old;
```

（这看起来有点奇怪，因为你通常不能写 INSERT ... VALUES ... FROM。这里的 FROM 子句只是表示查询树里有用于 new 和 old 的范围表项。这些东西是必需的，这样它们就可以被 INSERT 命令的查询树中的变量引用）。

该规则是一个有条件的 ALSO 规则，所以规则系统必须返回两个查询树：更改过的规则动作和原始查询树。在第 1 步里，原始查询的范围表被集成到规则动作的查询树中。得到：

```
INSERT INTO shoelace_log VALUES (
       new.sl_name, new.sl_avail,
       current_user, current_timestamp )
  FROM shoelace_data new, shoelace_data old,
       shoelace_data shoelace_data;
```

第 2 步把规则条件增加进去，所以结果集被限制为 sl_avail 改变了的行：

```
INSERT INTO shoelace_log VALUES (
       new.sl_name, new.sl_avail,
       current_user, current_timestamp )
  FROM shoelace_data new, shoelace_data old,
       shoelace_data shoelace_data
 WHERE new.sl_avail <> old.sl_avail;
```

（这看起来更奇怪，因为 INSERT ... VALUES 也没有 WHERE 子句，但是规划器和执行器处理它没有任何难度。不管怎样，它们需要为 INSERT ... SELECT 支持这种相同功能）。

第 3 步把原始查询树的条件加进去，把结果集进一步限制成只有被初始查询树改变的行：

```
INSERT INTO shoelace_log VALUES (
       new.sl_name, new.sl_avail,
       current_user, current_timestamp )
  FROM shoelace_data new, shoelace_data old,
       shoelace_data shoelace_data
 WHERE new.sl_avail <> old.sl_avail
   AND shoelace_data.sl_name = 'sl7';
```

第 4 步把 NEW 引用替换为来自原始查询树的目标列表项或来自结果关系的相匹配的变量引用：

```
INSERT INTO shoelace_log VALUES (
       shoelace_data.sl_name, 6,
       current_user, current_timestamp )
  FROM shoelace_data new, shoelace_data old,
       shoelace_data shoelace_data
 WHERE 6 <> old.sl_avail
   AND shoelace_data.sl_name = 'sl7';
```

第 5 步，用结果关系引用把 OLD 引用替换掉：

```
INSERT INTO shoelace_log VALUES (
       shoelace_data.sl_name, 6,
       current_user, current_timestamp )
  FROM shoelace_data new, shoelace_data old,
       shoelace_data shoelace_data
 WHERE 6 <> shoelace_data.sl_avail
```

```
   AND shoelace_data.sl_name = 'sl7';
```

这就完成了。因为规则是 ALSO，我们还要输出原始查询树。简而言之，从规则系统输出的是一个包含两个查询树的列表，它们与下面语句相对应：

```
INSERT INTO shoelace_log VALUES (
       shoelace_data.sl_name, 6,
       current_user, current_timestamp )
  FROM shoelace_data
 WHERE 6 <> shoelace_data.sl_avail
   AND shoelace_data.sl_name = 'sl7';

UPDATE shoelace_data SET sl_avail = 6
 WHERE sl_name = 'sl7';
```

这些会按照这个顺序被执行，并且这也正是规则要做的事情。

做的替换和追加的条件用于确保对于下面这样的原始查询不会有日志记录被写入：

```
UPDATE shoelace_data SET sl_color = 'green'
 WHERE sl_name = 'sl7';
```

在这种情况下，原始查询树不包含 sl_avail 的目标列表项，因此 NEW.sl_avail 将被 shoelace_data.sl_avail 代替。所以，规则生成的额外命令是：

```
INSERT INTO shoelace_log VALUES (
       shoelace_data.sl_name, shoelace_data.sl_avail,
       current_user, current_timestamp )
  FROM shoelace_data
 WHERE shoelace_data.sl_avail <> shoelace_data.sl_avail
   AND shoelace_data.sl_name = 'sl7';
```

并且条件将永远不可能为真。

如果原始查询修改多个行，这也能争产工作。所以如果某人发出命令：

```
UPDATE shoelace_data SET sl_avail = 0
 WHERE sl_color = 'black';
```

实际上有四行（sl1、sl2、sl3 和 sl4）被更新。但 sl3 已经是 sl_avail = 0。在这种情况下，原始查询树的条件不同并且导致规则产生额外的查询树：

```
INSERT INTO shoelace_log
SELECT shoelace_data.sl_name, 0,
       current_user, current_timestamp
  FROM shoelace_data
```

```
 WHERE 0 <> shoelace_data.sl_avail
   AND shoelace_data.sl_color = 'black';
```

这个查询树将肯定插入三个新的日志项。这也是完全正确的。

到这里我们就能明白为什么原始查询树最后执行非常重要。如果 UPDATE 先被执行，则所有的行都已经被设为零，所以记日志的 INSERT 将无法找到任何符合 0 <> shoelace_data.sl_avail 的行。

9.2.4.2　与视图合作

要保护一个视图关系不被 INSERT、UPDATE 或 DELETE，一种简单的方法是让那些查询树被丢掉。因此我们可以创建规则：

```
CREATE RULE shoe_ins_protect AS ON INSERT TO shoe
    DO INSTEAD NOTHING;
CREATE RULE shoe_upd_protect AS ON UPDATE TO shoe
    DO INSTEAD NOTHING;
CREATE RULE shoe_del_protect AS ON DELETE TO shoe
    DO INSTEAD NOTHING;
```

如果现在某人尝试对视图关系 shoe 做任何这些操作，规则系统将应用这些规则。因为这些规则没有动作而且是 INSTEAD，作为的查询树列表将是空的并且整个查询将变得什么也不做，因为经过规则系统处理后没有什么东西剩下来被优化或执行了。

一个更好的使用规则系统的方法是创建一些规则，这些规则把查询树重写成一个在真实表上进行正确的操作的查询树。要在视图 shoelace 上做这件事，我们创建下列规则：

```
CREATE RULE shoelace_ins AS ON INSERT TO shoelace
    DO INSTEAD
    INSERT INTO shoelace_data VALUES (
           NEW.sl_name,
           NEW.sl_avail,
           NEW.sl_color,
           NEW.sl_len,
           NEW.sl_unit
    );

CREATE RULE shoelace_upd AS ON UPDATE TO shoelace
    DO INSTEAD
    UPDATE shoelace_data
       SET sl_name = NEW.sl_name,
           sl_avail = NEW.sl_avail,
           sl_color = NEW.sl_color,
           sl_len = NEW.sl_len,
           sl_unit = NEW.sl_unit
```

```
     WHERE sl_name = OLD.sl_name;

CREATE RULE shoelace_del AS ON DELETE TO shoelace
    DO INSTEAD
    DELETE FROM shoelace_data
     WHERE sl_name = OLD.sl_name;
```

如果你要在视图上支持 RETURNING 查询，你需要让规则包含 RETURNING 子句来计算视图行。这对于基于单个表的视图来说通常非常简单，但是对于连接视图（如 shoelace）就有点冗长了。对于插入的一个例子：

```
CREATE RULE shoelace_ins AS ON INSERT TO shoelace
    DO INSTEAD
    INSERT INTO shoelace_data VALUES (
           NEW.sl_name,
           NEW.sl_avail,
           NEW.sl_color,
           NEW.sl_len,
           NEW.sl_unit
    )
    RETURNING
           shoelace_data.*,
           (SELECT shoelace_data.sl_len * u.un_fact
            FROM unit u WHERE shoelace_data.sl_unit = u.un_name);
```

注意，这个规则同时支持该视图上的 INSERT 和 INSERT RETURNING 查询—对于 INSERT 会简单地忽略 RETURNING 子句。

现在假设有时一包鞋带抵达了商店，并且随着它有一个大的清单。但是你不想每次都手工更新 shoelace 视图。取而代之的是我们建立两个小表：一个用来从清单向其中插入东西，另一个则用了一个特殊的技巧。这些东西的创建命令如下：

```
CREATE TABLE shoelace_arrive (
    arr_name    text,
    arr_quant   integer
);

CREATE TABLE shoelace_ok (
    ok_name     text,
    ok_quant    integer
);

CREATE RULE shoelace_ok_ins AS ON INSERT TO shoelace_ok
    DO INSTEAD
    UPDATE shoelace
       SET sl_avail = sl_avail + NEW.ok_quant
     WHERE sl_name = NEW.ok_name;
```

现在你可以用来自清单的数据填充表 shoelace_arrive：

```
SELECT * FROM shoelace_arrive;

 arr_name | arr_quant
----------+-----------
 sl3      |        10
 sl6      |        20
 sl8      |        20
(3 rows)
```

快速地看一看当前的数据：

```
SELECT * FROM shoelace;

 sl_name  | sl_avail | sl_color | sl_len | sl_unit | sl_len_cm
----------+----------+----------+--------+---------+-----------
 sl1      |        5 | black    |     80 | cm      |        80
 sl2      |        6 | black    |    100 | cm      |       100
 sl7      |        6 | brown    |     60 | cm      |        60
 sl3      |        0 | black    |     35 | inch    |      88.9
 sl4      |        8 | black    |     40 | inch    |     101.6
 sl8      |        1 | brown    |     40 | inch    |     101.6
 sl5      |        4 | brown    |      1 | m       |       100
 sl6      |        0 | brown    |    0.9 | m       |        90
(8 rows)
```

现在把到的货鞋带移到：

```
INSERT INTO shoelace_ok SELECT * FROM shoelace_arrive;
```

并检查结果：

```
SELECT * FROM shoelace ORDER BY sl_name;

 sl_name  | sl_avail | sl_color | sl_len | sl_unit | sl_len_cm
----------+----------+----------+--------+---------+-----------
 sl1      |        5 | black    |     80 | cm      |        80
 sl2      |        6 | black    |    100 | cm      |       100
 sl7      |        6 | brown    |     60 | cm      |        60
 sl4      |        8 | black    |     40 | inch    |     101.6
 sl3      |       10 | black    |     35 | inch    |      88.9
 sl8      |       21 | brown    |     40 | inch    |     101.6
 sl5      |        4 | brown    |      1 | m       |       100
 sl6      |       20 | brown    |    0.9 | m       |        90
```

```
(8 rows)

SELECT * FROM shoelace_log;

 sl_name | sl_avail | log_who| log_when
---------+----------+--------+----------------------------------
 sl7     |        6 | Al     | Tue Oct 20 19:14:45 1998 MET DST
 sl3     |       10 | Al     | Tue Oct 20 19:25:16 1998 MET DST
 sl6     |       20 | Al     | Tue Oct 20 19:25:16 1998 MET DST
 sl8     |       21 | Al     | Tue Oct 20 19:25:16 1998 MET DST
(4 rows)
```

从一个 INSERT ... SELECT 到这些结果经过了很长的过程。并且该查询树转换的描述将出现在本章的最后。首先，这里是解析器的输出：

```
INSERT INTO shoelace_ok
SELECT shoelace_arrive.arr_name, shoelace_arrive.arr_quant
  FROM shoelace_arrive shoelace_arrive, shoelace_ok shoelace_ok;
```

现在应用第一条规则 shoelace_ok_ins 被应用并且把这个输出转换成：

```
UPDATE shoelace
   SET sl_avail = shoelace.sl_avail + shoelace_arrive.arr_quant
  FROM shoelace_arrive shoelace_arrive, shoelace_ok shoelace_ok,
       shoelace_ok old, shoelace_ok new,
       shoelace shoelace
 WHERE shoelace.sl_name = shoelace_arrive.arr_name;
```

并且丢掉 shoelace_ok 上的 INSERT。这个被重写后的查询被再次传递给规则系统，并且第二个被应用的规则 shoelace_upd 会产生：

```
UPDATE shoelace_data
   SET sl_name = shoelace.sl_name,
       sl_avail = shoelace.sl_avail + shoelace_arrive.arr_quant,
       sl_color = shoelace.sl_color,
       sl_len = shoelace.sl_len,
       sl_unit = shoelace.sl_unit
  FROM shoelace_arrive shoelace_arrive, shoelace_ok shoelace_ok,
       shoelace_ok old, shoelace_ok new,
       shoelace shoelace, shoelace old,
       shoelace new, shoelace_data shoelace_data
 WHERE shoelace.sl_name = shoelace_arrive.arr_name
   AND shoelace_data.sl_name = shoelace.sl_name;
```

同样这是一个 INSTEAD 规则并且前一个查询树会被丢弃掉。注意这个查询仍然使用视图 shoelace。但是规则系统还没有完成这一步，所以它会继续并在其上应用 _RETURN 规则，并且我们得到：

```
UPDATE shoelace_data
   SET sl_name = s.sl_name,
       sl_avail = s.sl_avail + shoelace_arrive.arr_quant,
       sl_color = s.sl_color,
       sl_len = s.sl_len,
       sl_unit = s.sl_unit
  FROM shoelace_arrive shoelace_arrive, shoelace_ok shoelace_ok,
       shoelace_ok old, shoelace_ok new,
       shoelace shoelace, shoelace old,
       shoelace new, shoelace_data shoelace_data,
       shoelace old, shoelace new,
       shoelace_data s, unit u
 WHERE s.sl_name = shoelace_arrive.arr_name
   AND shoelace_data.sl_name = s.sl_name;
```

最后，规则 log_shoelace 被应用，生成额外的查询树：

```
INSERT INTO shoelace_log
SELECT s.sl_name,
       s.sl_avail + shoelace_arrive.arr_quant,
       current_user,
       current_timestamp
  FROM shoelace_arrive shoelace_arrive, shoelace_ok shoelace_ok,
       shoelace_ok old, shoelace_ok new,
       shoelace shoelace, shoelace old,
       shoelace new, shoelace_data shoelace_data,
       shoelace old, shoelace new,
       shoelace_data s, unit u,
       shoelace_data old, shoelace_data new
       shoelace_log shoelace_log
 WHERE s.sl_name = shoelace_arrive.arr_name
   AND shoelace_data.sl_name = s.sl_name
   AND (s.sl_avail + shoelace_arrive.arr_quant) <> s.sl_avail;
```

完成这些之后，规则系统用完了所有的规则并且返回生成的查询树。

所以我们结束于两个最终查询树，它们等效于 SQL 语句：

```
INSERT INTO shoelace_log
SELECT s.sl_name,
       s.sl_avail + shoelace_arrive.arr_quant,
       current_user,
       current_timestamp
  FROM shoelace_arrive shoelace_arrive, shoelace_data shoelace_data,
       shoelace_data s
 WHERE s.sl_name = shoelace_arrive.arr_name
```

```
   AND shoelace_data.sl_name = s.sl_name
   AND s.sl_avail + shoelace_arrive.arr_quant <> s.sl_avail;

UPDATE shoelace_data
   SET sl_avail = shoelace_data.sl_avail + shoelace_arrive.arr_quant
  FROM shoelace_arrive shoelace_arrive,
       shoelace_data shoelace_data,
       shoelace_data s
 WHERE s.sl_name = shoelace_arrive.sl_name
   AND shoelace_data.sl_name = s.sl_name;
```

结果是从一个关系来的数据插入了到另一个中，改变成第三个上的更新，改变成更新第四个外加做日志，在第五个中的最后更新缩减为两个查询。

有一个小细节有点丑陋。看看那两个查询，我们会发现 shoelace_data 关系在范围表中出现了两次而实际上绝对可以缩为出现一次。规划器不会处理它，因此 INSERT 的规则系统输出的执行规划会是

```
Nested Loop
  ->  Merge Join
        ->  Seq Scan
              ->  Sort
                    ->  Seq Scan on s
        ->  Seq Scan
              ->  Sort
                    ->  Seq Scan on shoelace_arrive
  ->  Seq Scan on shoelace_data
```

在省略额外的范围表项后会得到

```
Merge Join
  ->  Seq Scan
        ->  Sort
              ->  Seq Scan on s
  ->  Seq Scan
        ->  Sort
              ->  Seq Scan on shoelace_arrive
```

这在日志表中生成完全一样的项。因此，规则系统导致了 shoelace_data 表上的一次绝对不必要的扫描。并且同样的冗余扫描会在 UPDATE 中进行。但是要把这些全部实现实在是一项很困难的工作。

现在我们对 KingbaseES 规则系统及其能力做最后一个演示。假设你向你的数据库中添加一些有特别颜色的鞋带：

```
INSERT INTO shoelace VALUES ('sl9', 0, 'pink', 35.0, 'inch', 0.0);
INSERT INTO shoelace VALUES ('sl10', 1000, 'magenta', 40.0, 'inch', 0.0);
```

我们想要建立一个视图来检查哪些 shoelace 项在颜色上不配任何鞋子。适用的视图是：

```
CREATE VIEW shoelace_mismatch AS
    SELECT * FROM shoelace WHERE NOT EXISTS
        (SELECT shoename FROM shoe WHERE slcolor = sl_color);
```

它的输出是：

```
SELECT * FROM shoelace_mismatch;

 sl_name | sl_avail | sl_color | sl_len | sl_unit | sl_len_cm
---------+----------+----------+--------+---------+-----------
 sl9     |        0 | pink     |     35 | inch    |      88.9
 sl10    |     1000 | magenta  |     40 | inch    |     101.6
```

现在我们想建立它，这样没有库存的不匹配的鞋带都会被从数据库中删除。为了对 KingbaseES 有点难度，我们不直接删除它们。而是我们再创建一个视图：

```
CREATE VIEW shoelace_can_delete AS
    SELECT * FROM shoelace_mismatch WHERE sl_avail = 0;
```

然后用下面方法：

```
DELETE FROM shoelace WHERE EXISTS
    (SELECT * FROM shoelace_can_delete
             WHERE sl_name = shoelace.sl_name);
```

输出：

```
SELECT * FROM shoelace;

 sl_name | sl_avail | sl_color | sl_len | sl_unit | sl_len_cm
---------+----------+----------+--------+---------+-----------
 sl1     |        5 | black    |     80 | cm      |        80
 sl2     |        6 | black    |    100 | cm      |       100
 sl7     |        6 | brown    |     60 | cm      |        60
 sl4     |        8 | black    |     40 | inch    |     101.6
 sl3     |       10 | black    |     35 | inch    |      88.9
 sl8     |       21 | brown    |     40 | inch    |     101.6
 sl10    |     1000 | magenta  |     40 | inch    |     101.6
 sl5     |        4 | brown    |      1 | m       |       100
 sl6     |       20 | brown    |    0.9 | m       |        90
(9 rows)
```

对一个视图上的 DELETE，这个命令带有一个总共使用了四个嵌套/连接视图的子查询条件，这四个视图之一本身有一个包含一个视图的子查询条件，该条件计算使用的视图列；这个命令被重写成了一个查询树，该查询树从一个真正的表里面把需要删除的数据删除。

在现实世界里只有很少的情况需要上面的这样的构造。但这些东西能运转肯定让你感觉不错。

9.2.5　规则和权限

由于 KingbaseES 规则系统对查询的重写，会访问没有在原始查询中指定的表/视图。使用更新规则时，这可能包括对表的写权限。

重写规则并不拥有一个独立的所有者。关系（表或视图）的所有者自动成为为其所定义的重写规则的所有者。KingbaseES 规则系统改变了默认的访问控制系统的行为。由于规则被使用的关系会按照规则所有者的权限来检查，而不是调用规则的用户。这表示用户只需要在其查询中显式指定的表/视图上的所需权限。

例如：某用户有一个电话号码列表，其中一些是私人的，另外的一些是办公室助理需要的。该用户可以构建下面的东西：

```
CREATE TABLE phone_data (person text, phone text, private boolean);
CREATE VIEW phone_number AS
    SELECT person, CASE WHEN NOT private THEN phone END AS phone
    FROM phone_data;
GRANT SELECT ON phone_number TO assistant;
```

除了该用户以外（还有数据库超级用户）没有人可以访问 phone_data 表。但因为 GRANT 的原因，助理可以在 phone_number 视图上运行 SELECT。规则系统将把 phone_number 上的 SELECT 重写为 phone_data 上的 SELECT。因为该用户是 phone_number 的所有者，因此也是规则的所有者，对 phone_data 的读访问现在被根据该用户的权限检查，并且该查询被允许。同时也要检查访问 phone_number 的权限，但这是针对调用用户进行的，所以除了用户自己和助理外没有人可以使用它。

权限检查是按规则逐条进行的。所以此时助理是唯一的一个可以看到公共电话号码的人。但助理可以建立另一个视图并且赋予该视图公共权限。这样，任何人都可以通过助理的视图看到 phone_number 数据。助理不能做的事情是创建一个直接访问 phone_data 的视图（实际上助理是可以的，但没有任何作用，因为每次访问都会因通不过权限检查而被否定）。而且该用户一旦注意到助理开放了他的 phone_number 视图，该用户还可以收回助理的访问权限。立刻，所有对助理视图的访问将会失败。

有人可能会认为这种逐条规则的检查是一个安全漏洞，但事实上不是。如果这样做不能奏效，助理将必须建立一个与 phone_number 有相同列的表并且每天拷贝一次数据进去。那么这是助理自己的数据因而助理可以为每一个想要访问的人授权。一个 GRANT 意味着“我信任你”。如果某个你信任的人做了上面的事情，那么是时候认为信任已经结束并且要使用 REVOKE。

需要注意的是，虽然视图可以用前文展示的技术来隐藏特定列的内容，它们不能可靠地在不可见行上隐藏数据，除非标志被设置。例如，下面的视图是不安全的：

```
CREATE VIEW phone_number AS
    SELECT person, phone FROM phone_data WHERE phone NOT LIKE '412%';
```

这个视图看起来是安全的，因为规则系统会把任何 phone_number 上的 SELECT 重写成 phone_data 上的 SELECT，并且增加限制使得只有 phone 不以 412 开头的项才被处理。但是如果用户可以创建自己的函数，那就不难让规划器在 NOT LIKE 表达式之前先执行用户自定义函数。例如：

```
CREATE FUNCTION tricky(text, text) RETURNS bool AS $$
BEGIN
    RAISE NOTICE '% => %', $1, $2;
    RETURN true;
END
$$ LANGUAGE plsql COST 0.0000000000000000000001;

SELECT * FROM phone_number WHERE tricky(person, phone);
```

phone_data 表中的每一个人和电话号码会被打印成一个 NOTICE，因为规划器会选择在执行 NOT LIKE 之前先执行 tricky，因为前者的开销大。即使禁止用户自定义一个新函数，内置函数也可以用在类似的攻击中（例如，大部分造型函数会在它们产生的错误信息中包含它们的输入值）。

类似的考虑应用于更新规则。在前一节的例子中，例子数据库中表的所有者可以把 shoelace 视图上的 SELECT、INSERT、UPDATE 和 DELETE 权限授予其他人，但对 shoelace_log 只有 SELECT 权限。写日志项的规则动作将仍然可以被成功地执行，并且其它用户可以看到日志项。但他们不能创建伪造的项，并且他们也不能操纵或移除现有的项。在这种情况下，不可能通过让规划器改变操作的顺序来推翻规则，因为引用 shoelace_log 的唯一规则是无限制的 INSERT。在更复杂的情景中，这可能不正确。

当需要对一个视图提供行级安全时，security_barrier 属性应该被应用到该视图。这会阻止恶意选择的函数和操作符通过行被传递，直到视图完成其工作。例如，如果前文所示的视图被创建成这样，它就是安全的：

```
CREATE VIEW phone_number WITH (security_barrier) AS
    SELECT person, phone FROM phone_data WHERE phone NOT LIKE '412%';
```

使用 security_barrier 创建的视图的性能会远差于没有使用该选项的视图。通常，没有办法来避免这种现状：如果最快的候选计划可能在安全性上折衷，它就必须被拒绝。出于该原因，这个选在在默认情况下是没有启用的。

当处理没有副作用的函数时，查询计划器有更多的灵活性。这类函数被称为 LEAKPROOF，并且包括很多简单常用的操作符，例如很多等于操作符。查询计划器可以安全地允许这类函数在查询执行过程中的任何点被计算，因为在用户不可见的行上调用它们将不会泄露关于不可见行的任何信息。更进一步，不接收参数或者不从安全屏障视图得到任何参数的函数不必被标记为 LEAKPROOF 以便被下推，因为它们从来不会从该视图接收数据。相反，一个可能会基于接收到的参数值抛出错误的函数（例如在溢出或被零除事件中抛出错误的函数）不是防泄漏的，并且如果它被应用在安全性视图的行过滤器之前，它可能会提供有关不可见行的有效信息。

有一点很重要的是理解：即使一个视图使用 security_barrier 选项创建，它也只在不可见元组不会被传递给可能不安全的函数的前提下才是安全的。用户可能也有其他方式来推断不可见数据；例如，他们可以使用 EXPLAIN 看到查询计划，或者针对视图来测量查询的运行时间。一个恶意攻击者可能有能力推断有关不可见数据的总量，或者甚至得到有关数据分布的某些信息或最常用值（因为这些东西可以影响计划的运行时间；或者甚至计划的选择，因为它们也被反映在优化器的统计数据中）。如果这类“隐通道”攻击很重要，那么授予任何到该数据的访问都可能是不明智的。

9.2.6　规则和命令状态

KingbaseES 服务器为它收到的每个命令返回一个命令状态字符串，例如 INSERT 149592 1。没有涉及规则时这很简单，但是查询被规则重写时会发生什么呢？

规则对命令状态的影响如下：

- 如果没有查询的无条件 INSTEAD 规则，那么原始给出的查询将会被执行，并且它的命令状态将像平常一样被返回（但是请注意如果存在任何有条件 INSTEAD 规则，那么它们的反条件将被加到原始查询中。这样可能会减少它处理的行数，并且报告的状态将受影响）。
- 如果有查询的任何无条件 INSTEAD 规则，那么原始查询将完全不被执行。在这种情况下，服务器将返回由服务器将返回由 INSTEAD 规则（有条件的或无条件的）插入的最后一条和原始查询命令类型（INSERT、UPDATE 或 DELETE）相同的查询的命令状态。如果任何规则添加的查询都不符合这些要求，那么返回的命令状态显示原始查询类型并且行计数和 OID 域为零。

通过为任何想要的 INSTEAD 规则指定在活动规则中排名最后的规则名，程序员可以确保该规则都是在第二种情况里设置命令状态的规则，因为它会被最后一个应用。

9.2.7　规则 vs 触发器

许多触发器可以干的事情同样也可以用 KingbaseES 规则系统来实现。目前不能用规则来实现的东西之一是某些约束，特别是外键。可以放置一个合格的规则在一列上，这个规则在列的值没有出现在另一个表中时把命令重写成 NOTHING。但是这样做数据就会被不声不响地丢弃，因此也不是一个好主意。如果要求检查值的有效性，并且在出现无效值的情况下应该生成一个错误消息，这种需求就必须要用触发器来完成。

在本章中，我们关注于使用规则来更新视图。本章中所有的更新规则的例子都可以使用视图上的 INSTEAD OF 触发器来实现。编写这类触发器通常比编写规则要容易，特别是在要求使用复杂逻辑来执行更新的情况下。

对于两者都可实现的情况，哪个更好取决于对数据库的使用。触发器为每一个受影响的行都执行一次。规则修改查询树或生成一个额外的查询。所以如果在一个语句中影响到很多行，一个发出额外查询的规则通常可能会比一个触发器快，因为触发器对每一个行都要被调用，并且每次被调用时都需要重新判断要做什么样的操作。不过，触发器方法从概念上要远比规则方法简单，并且很容易让新人上手。

下面我们展示一个例子，该例子说明了在同种情况下两种选择的比较。这里有两个表：

```
CREATE TABLE computer (
    hostname        text,    -- 被索引
    manufacturer    text     -- 被索引
);

CREATE TABLE software (
    software        text,    -- 被索引
    hostname        text     -- 被索引
);
```

两个表都有数千行，并且在 hostname 上的索引是唯一的。规则或触发器应该实现一个约束，该约束从 software 中删除引用已删除计算机的行。触发器可以用下面这条命令：

```
DELETE FROM software WHERE hostname = $1;
```

因为触发器会为每一个从 computer 中删除的独立行调用一次，那么它可以准备并且保存这个命令的规划，把 hostname 作为参数传入。规则应该被写为：

```
CREATE RULE computer_del AS ON DELETE TO computer
    DO DELETE FROM software WHERE hostname = OLD.hostname;
```

现在看看不同类型的删除。在这种情况：

```
DELETE FROM computer WHERE hostname = 'mypc.local.net';
```

表 computer 被使用索引（快速）扫描，并且由触发器发出的命令也将使用一个索引扫描（同样快速）。来自规则的额外查询应该是：

```
DELETE FROM software WHERE computer.hostname = 'mypc.local.net'
                       AND software.hostname = computer.hostname;
```

由于已经建立了合适的索引，规划器将创建一个规划

```
Nestloop
  ->  Index Scan using comp_hostidx on computer
  ->  Index Scan using soft_hostidx on software
```

所以在触发器和规则的实现之间没有太多的速度差别。

在接下来的删除中，我们想要去掉所有 2000 个 hostname 以 old 开头的计算机。有两个命令可以来做这件事。一个是：

```
DELETE FROM computer WHERE hostname >= 'old'
                       AND hostname <  'ole'
```

被规则增加的命令将是：

```
DELETE FROM software WHERE computer.hostname >= 'old' AND computer.hostname
< 'ole'
                       AND software.hostname = computer.hostname;
```

计划是：

```
Hash Join
  ->  Seq Scan on software
  ->  Hash
    ->  Index Scan using comp_hostidx on computer
```

另一个可能的命令是：

```
DELETE FROM computer WHERE hostname ~ '^old';
```

它会为规划增加的命令产生下面的执行计划：

```
Nestloop
  ->  Index Scan using comp_hostidx on computer
  ->  Index Scan using soft_hostidx on software
```

这表明，当有多个条件表达式被使用 AND 组合在一起时，规划器不能认识到表 computer 中 hostname 上的条件也可以被用于一个 software 上的索引扫描，而在该命令的正则表达式版本中正是这样做的。触发器将为要被删除的 2000 个旧计算机中的每一个调用，并且会导致在 computer 上的一次索引扫描和 software 上的 2000 次索引扫描。采用规则的实现将会使用两个使用索引的命令来完成。并且在顺序扫描情况下规则是否仍将更快是取决于 software 表的总体大小的。即使所有的索引块都将很快地进入高速缓存，通过 SPI 管理器执行来自触发器的 2000 个命令也要花不少时间。

我们要看的最后一个命令是：

```
DELETE FROM computer WHERE manufacturer = 'bim';
```

同样，这也会导致很多行被从 computer 中删除。所以触发器同样会通过执行器运行很多命令。规则生成的命令将会是：

```
DELETE FROM software WHERE computer.manufacturer = 'bim'
                       AND software.hostname = computer.hostname;
```

这个命令的计划又将是在两个索引扫描上的嵌套循环，只不过使用了 computer 上的另一个索引：

```
Nestloop
  ->  Index Scan using comp_manufidx on computer
  ->  Index Scan using soft_hostidx on software
```

在任何这些情况之一，来自规则系统的额外命令都或多或少与命令中影响的行数无关。

概括来说，规则只有在其动作导致了大而且糟糕的条件连接时才会明显地慢于触发器，这种情况下规划器将没有什么办法。

9.3　调度管理

9.3.1　数据库调度概念

- 调度器概述

KingbaseES 数据库拥有作业调度功能，可以帮助简化数百甚至数千个任务的调度。KingbaseES 调度器 (Scheduler) 依赖于 PL/SQL 提供的 DBMS_SCHEDULER 相关函数和存储过程。

- 作业和支持的调度对象

可以使用作业和其他调度对象进行任务调度。

- 关于作业的更多描述

作业存在多种类型，一个作业实例表示一个具体运行的作业。

9.3.1.1 调度器概述

KingbaseES 数据库拥有作业调度功能，可以帮助简化数百甚至数千个任务的调度。KingbaseES 调度器 (Scheduler) 依赖于 PL/SQL 提供的 DBMS_SCHEDULER 相关函数和存储过程。

可以使用调度器在生产环境中自主控制在何时生成何种计算任务，同时可以帮助有效地管理这些任务。无需人工干预的自动任务，可以有效的降低人工操作成本，同时可靠的例行程序，可以最大限度地减少人为错误。

作业调度器提供了复杂、灵活的作业调度功能，可以使用它来:

- 运行数据库程序单元

可以在本地数据库上运行程序单元，即 PL/SQL 匿名块、PL/SQL 存储过程。

- 运行外部可执行文件 (数据库外部的可执行文件)

可以在本地系统上运行外部可执行文件，例如应用程序、shell 脚本。

- 基于时间的调度:

可以将作业在特定的日期和时间运行，可以配置为只运行一次，也可以配置为规律性的重复运行。

9.3.1.2 作业和支持的调度对象

可以使用作业和其他调度对象进行任务调度。

关于作业和支持调度对象

要使用调度器，则需要创建调度对象。调度对象定义了作业调度的内容、时间。调度对象支持以模块化的方式管理任务。模块化方法的一个优点是，当创建与现有任务类似的新任务时，可以复用对象。

主要的调度对象是作业。作业定义了要执行的动作、动作的时间表。大多数其他调度对象创建来是为了支持作业。

KingbaseES 的自动作业功能是通过 kdb_schedule 插件提供的 DBMS_JOB 和 DBMS_SCHEDULER 包实现，其中 DBMS_JOB 模式提供调度和管理调度任务的功能，该包功能可以由 DBMS_SCHEDULER 替代。

这些调度对象将在本节后面进行描述。

程序

程序对象描述调度器要运行的内容。

程序包括:

- 操作: 例如，存储过程的名称、操作系统文件系统中可执行文件的名称 (“外部可执行文件”) 或 PL/SQL 匿名块。
- 类 型: PLSQL_BLOCK、STORED_PROCEDURE、SQL_SCRIPT、EXECUTABLE、EXTERNAL_SCRIPT、BACKUP_SCRIPT 或可执行文件，其中可执行文件表示外部可执行文件。
- 参数数量: 存储过程或外部可执行文件接受的参数数量。

程序是一个独立于作业的实体。作业则在某个确定的时间运行，并调用某个程序。可以创建指向现有程序对象的作业，这意味着不同的作业可以使用相同的程序，并在不同的时间以不同的设置运行该程序。有了正确的权限，不同的用户可以使用相同的程序，而不必重新定义它。因此，可以创建程序库，用户可以从现有程序列表中进行选择。

调度计划

调度计划对象 (schedule) 指定作业何时运行以及运行多少次。

调度计划可以被多个作业共享。例如，对于许多作业来说，一个商业季度的结束可能是一个常见的时间框架。与每次季度末定义新作业不同，作业创建者可以指向一个已命名的调度计划。KingbaseES 支持基于时间的调度，可以让作业立即运行或稍后运行。时间调度包括开始日期及时间、结束日期及时间、重复间隔。

作业

作业就是用户定义的任务。

- 关于作业

作业对象是描述用户定义任务的元数据集合。它定义了必须执行什么 (动作)、何时 (一次性或重复执行的调度或触发事件)。作业有一个所有者，即创建作业的模式。

运行数据库程序的作业称为数据库作业。运行外部可执行文件的作业称为外部作业。

- 指定作业操作

可以通过指定数据库程序或外部可执行文件或现有程序对象 (程序) 的名称来指定作业的操作。

可以通过以下方式之一指定作业操作:

- 通过将数据库程序单元或要运行的外部可执行文件指定为作业属性。这称为内联指定作业操作。
- 通过将现有程序的名称指定为作业属性，该程序指定要运行的数据库程序单元或外部可执行文件。
- 指定作业调度

可以通过设置作业对象的属性或现有调度对象 (schedule) 的名称来指定作业调度。

可以通过以下方式之一指定作业调度:

- 通过设置作业对象的属性来定义开始和结束日期以及重复间隔，或定义触发作业的事件。
- 通过将现有调度的名称指定为作业属性，该名称定义了开始和结束日期、重复时间间隔或定义了事件。

9.3.1.3　关于作业的更多描述

作业存在多种类型。一个作业实例表示一个具体运行的作业。可以自定义作业参数来覆盖默认的参数值。

- 作业类别

KingbaseES 数据库的 Scheduler 支持多种类型的作业。

- 作业实例

一个作业实例表示一个具体运行的作业。一个只调度运行一次的作业只有一个实例。具有重复调度或每次事件发生时运行的作业有多个实例，作业的每次运行代表一个实例。

- 作业参数

当作业引用程序对象 (程序) 时，可以提供作业参数来覆盖默认的程序参数值，或者为没有默认值的程序参数提供值。

9.3.1.3.1　作业类别

KingbaseES 支持多种类型的作业。

- 数据库作业
- 外部作业

数据库作业

数据库作业运行数据库程序单元。可以运行本地和远程数据库作业。

- 关于数据库作业

数据库作业运行的数据库程序单元，包括了 PL/SQL 匿名块、PL/SQL 存储过程。对于内联指定操作的数据库作业，job_type 被设置为‘PLSQL_BLOCK’或‘STORED_PROCEDURE’，并且 job_action 包含 PL/SQL 匿名块的文本或存储过程的名称。(如果程序是一个命名程序对象，而不是内联指定的程序操作，则必须设置相应的 program_type 和 program_action。) 在原始数据库上运行的数据库作业称为本地数据库作业。在初始数据库以外的位置数据库上运行的数据库作业称为远程数据库作业。可以在原始数据库上的作业日志视图中查看本地数据库和远程数据库作业的运行结果。

- 本地数据库的作业

本地数据库作业以数据库用户 (即作业所有者) 的身份在原始数据库上运行。作业所有者是创建作业的模式的名称。

- 远程数据库作业

远程数据库作业的位置数据库可以是远程主机上的 KingbaseES 数据库，也可以是原始数据库所在主机上的另一个数据库实例。

外部作业

外部作业在数据库外部运行可执行程序。可以运行本地外部作业。

- 关于外部作业

外部作业运行外部可执行文件。外部可执行文件是运行在数据库外部的操作系统的可执行文件。对于外部作业，job_type 指定为‘EXECUTABLE’。(如果使用命名程序，对应的 program_type 将是‘EXECUTABLE’。)job_action(或对应的 program_action) 是所需的外部可执行文件的完全依赖于操作系统的路径，不包括任何命令行参数。例如: /usr/local/bin/perl。

请注意，Windows 批处理文件不是直接可执行的，必须在命令提示符 (cmd.exe) 中运行。

外部可执行文件必须作为某个操作系统用户运行。因此，Scheduler 使能够将操作系统凭据分配给创建的任何外部作业。与远程数据库作业一样，可以使用凭据对象 (凭据) 指定这些凭据，并将凭据分配给外部作业。

本地外部作业与调度该作业的数据库在同一台计算机上运行其外部可执行文件。

- 关于本地外部作业

本地外部作业与调度该作业的 KingbaseES 数据库在同一台计算机上运行其外部可执行文件。

9.3.1.3.2　作业实例

一个作业实例就表示一个特定运行的作业。只调度运行一次的作业只有一个实例。具有重复调度或每次事件发生时运行的作业有多个实例，作业的每次运行代表一个实例。

例如, 一个只调度在 2009 年 10 月 8 日运行的作业只有一个实例, 调度一周的每天中午运行的作业有 7 个实例。

9.3.1.3.3　作业参数

当作业引用程序对象 (程序) 时，可以设置作业参数来覆盖默认的程序参数值，或者为没有默认值的程序参数提供值。还可以为作业指定的内联操作 (例如，存储过程) 提供参数值。

在定义了所有必需的程序参数值 (作为引用的程序对象中的默认值或作为作业参数) 之前，不能启用作业。

作业的一个常见示例是运行一组夜间报告。如果不同部门需要不同的报表，可以为该任务创建程序，供不同部门的不同用户共享。程序操作运行一个报告脚本，程序有一个参数: 部门编号。然后，每个用户都可以创建指向该程序的作业，并可以将部门编号指定为作业参数。

9.3.2　数据库作业调度

可以使用 DBMS_SCHEDULER 创建、运行和管理作业。

本章描述了如何使用 DBMS_SCHEDULER 包来处理 Scheduler 对象。

- 作业调度及其子程序

可以使用 DBMS_SCHEDULER 包中的子程序来管理常见任务和调度计划。

9.3.2.1　作业调度及其子程序

可以使用 DBMS_SCHEDULER 包中的子程序来管理常见任务和调度计划。

下图展示了常见任务及其 DBMS_SCHEDULER 子程序：

表 9.3.1: DBMS_SCHEDULER 软件包子程序

子程序	描述
CREATE_PROGRAM Procedure	创建一个程序
DROP_PROGRAM Procedure	删除一个程序
CREATE_SCHEDULE Procedure	创建一个调度计划
DROP_SCHEDULE Procedure	删除一个调度计划
EVALUTE_CALENDAR_STRING Procedure	计算符合规则的下一个日期
CREATE_JOB Procedure	创建一个作业
DROP_JOB Procedure	删除一个作业
RUN_JOB Procedure	运行一个作业
DISABLE Procedure	将一个作业状态设置为 DISABLE
ENABLE Procedure	将一个作业状态设置为 ENABLE

DBMS_SCHEDULER 具有如下限制：

1. 需要创建 kdb_schedule 扩展。
2. 间隔时间采用日历表示法。
3. 使用该功能需要必要的 GUC 参数。

9.3.2.2　DBMS_SCHEDULER 包使用

- 创建程序

  创建程序为了告诉调度器要运行什么。

- 删除程序

  可以通过 DBMS_SCHEDULER 的 DROP_PROGRAM 删除一个或多个程序。

- 创建调度计划

  可以通过 DBMS_SCHEDULER 的 CREATE_SCHEDULE 创建调度计划。

- 删除调度计划

  可以通过 DBMS_SCHEDULER 的 DROP_SCHEDULE 删除调度计划。

- 创建作业

当完成了程序和调度计划的创建，意味着可以基于两者创建一个有计划的作业（JOB）。

- 删除作业

当我们不再需要作业时，可以使用 DROP_JOB 删除一个指定的作业。

- 运行作业

通过调用 DBMS_SCHEDULER.RUN_JOB，可以使用 RUN_JOB 程序来测试作业或者在调度计划之外运行它。

- 禁用作业

可以通过 DBMS_SCHEDULER 的 DISABLE 禁用作业。

- 启用作业

可以通过 DBMS_SCHEDULER 的 ENABLE 禁用作业。

- 任务的日历表示法

创建的任务中会涉及使用日历表示法进行时间定义，日历表示法是用来定义重复计划的实现方法，能实现一些复杂的时间计划。

- 计算符合规则的下一个日期

可以通过 DBMS_SCHEDULER 的 EVALUATE_CALENDAR_STRING 计算符合规则的下一次计划运行时间。

创建程序

创建程序为了告诉调度器要运行什么。

默认情况下，程序在创建者的模式中创建。要在另一个用户的模式中创建程序，必须使用模式名称对程序名进行限定。要让其他用户使用的程序，他们必须具有执行权限，因此一旦创建了程序，必须授予该程序执行权限。

下面的例子创建一个计划任务的程序，名字为 program1，程序的类型为 PLSQL_BLOCK，程序执行创建表 t2。

```
call DBMS_SCHEDULER.CREATE_PROGRAM('program1', 'PLSQL_BLOCK', 'create table t2(A int)','user=system
dbname=test port=54321 password=123456', 'test', 0, true, 'this is test program');
```

创建后可以禁用或者启用该任务，后面将描述该内容。

删除程序

可以通过 DBMS_SCHEDULER 的 DROP_PROGRAM 删除一个或多个程序。

当程序被删除时，与之相关的任何参数都会被删除，如下示例，可以在一次调用中通过逗号删除多个程序，也可以只删除一个程序。

```
call DBMS_SCHEDULER.DROP_PROGRAM('program1');
```

创建调度计划

可以通过 DBMS_SCHEDULER 的 CREATE_SCHEDULE 创建调度计划。

调度计划是在创建该任务时的用户模式下创建的，并且第一次创建时即启动。可以在其他用户模式下创建计划任务，一旦被创建，它就可以被其他用户使用。

下列示例创建一个名为 schedule1 的计划任务，规划在每天的早上 10 点 10 分 10 秒时。

```
call DBMS_SCHEDULER.CREATE_SCHEDULE('schedule1', now(), 'FREQ=MINUTELY;BYSECOND=10', NULL, 'this is test
schedule');
```

删除调度计划

可以通过 DBMS_SCHEDULER 的 DROP_SCHEDULE 删除调度计划。

```
call DBMS_SCHEDULER.DROP_SCHEDULE('schedule1');
```

创建作业

当完成了程序和调度计划的创建，意味着可以基于两者创建一个有计划的作业（job）。

如下示例，schedule1 定义了调度计划，program1 定义了要执行的什么内容（程序），创建的作业将两者“组合”形成一个有调度计划的作业。

```
call DBMS_SCHEDULER.CREATE_JOB('job1', 'program1', 'schedule1', 'Routine Maintenance', true, true, 'this is
test job');
```

删除作业

当我们不再需要作业时，可以使用 DROP_JOB 删除一个指定的作业。

如下示例，删除了一个名为 job1 的作业。

```
call DBMS_SCHEDULER.DROP_JOB('job1', false, false, 'STOP_ON_FIRST_ERROR');
```

运行作业

通过调用 DBMS_SCHEDULER.RUN_JOB，可以使用 RUN_JOB 程序来测试作业或者在调度计划之外运行它。

RUN_JOB 的 use_current_session 确定的是作业时异步还是同步，默认为 true，目前暂时不支持。

下列示例展示了，运行一个名为 job1 的任务。

```
call DBMS_SCHEDULER.RUN_JOB('job1', true);
```

禁用作业

可以通过 DBMS_SCHEDULER 的 DISABLE 禁用作业。

作业也可以通过其他方式失去功能效果，比如，删除作业指向的程序或者调度计划将形成失去功能，调度器尝试运行时将会报错。禁用作业意味着，尽管作业的元数据存在，但作业调度器不会对其进行处理。当作业被禁用时，作业在作业表中的状态为已禁用。

当一个正在运行的作业被禁用时，并且 force（第二个参数）为 false（默认值），将返回一个错误。目前 force 暂不支持 true。如果 commit_semantics 被设置为 STOP_ON_FIRST_ERROR（默认值），将在出现第一个错误时返回调用。（该参数暂不支持其他设置）

下列示例展示了禁用一个作业。

```
call DBMS_SCHEDULER.DISABLE('job1', false, 'STOP_ON_FIRST_ERROR');
```

启用作业

可以通过 DBMS_SCHEDULER 包的 ENABLE 启用作业。

该子程序的作用是，禁用状态的作业被调度器重新捡起来进行处理，默认情况下，创建的作业时禁用状态的，因此需要启用它们才能正常工作。当作业启动时，将执行有效性检查，如果检查不通过将不会启动作业。

启用作业后会立刻根据调度计划运行作业。如果 commit_semantics 被设置为 STOP_ON_FIRST_ERROR（默认值），将在出现第一个错误时返回调用。

下列示例展示了启用一个名为 job1 的计划任务。

```
call DBMS_SCHEDULER.ENABLE('job1', 'STOP_ON_FIRST_ERROR');
```

任务的日历表示法

创建的任务中会涉及使用日历表示法进行时间定义，日历表示法是用来定义重复计划的实现方法，能实现一些复杂的时间计划。

下列是一些可行的日历表示法示例：

示例：

每天早上 10 点

```
FREQ=DAILY;BYHOUR=10;BYMINUTE=0;BYSECOND=0;
```

每隔 7 天的早上 10 点

```
FREQ=DAILY;INTERVAL=7;BYHOUR=10;BYMINUTE=0;BYSECOND=0;
```

每年 10 月 1 号早上 10 点

```
FREQ=YEARLY;BYMONTH=10;BYMONTHDAY=1;BYHOUR=10;BYMINUTE=0;BYSECOND=0;
```

计算符合规则的下一个日期

可以通过 DBMS_SCHEDULER 的 EVALUATE_CALENDAR_STRING 计算符合规则的下一次计划运行时间。计算结果保存在参数 next_run_date 中。

下列例子展示了，计算’freq=yearly;interval=5;bymonth=10;bymonthday=10;byhour=10;byminute=10;bysecond=10’这个较复杂的日历规则下一次的计划运行时间，并将其打印显示出来。

示例：

```
declare
next_run_date timestamp;
begin DBMS_SCHEDULER.EVALUATE_CALENDAR_STRING('freq=yearly;interval=5;bymonth=10;
bymonthday=10;byhour=10;byminute=10;bysecond=10','','',next_run_date);
DBMS_OUTPUT.PUT_LINE(next_run_date);
end;
```

9.3.3　管理数据库调度器

可以配置、管理、监控和排查数据库调度器故障。

备注:

本章描述了如何使用 DBMS_SCHEDULER 包管理数据库调度器。

- 配置调度器

KingbaseES 通过 kdb_schedule 插件提供了 DBMS_JOB 和 DBMS_SCHEDULER 包，其中定义了自动作业功能的相关函数。

- 监控和管理调度程序

可以查看有关当前运行的作业的信息，监控和管理窗口和作业日志，并管理调度器安全性。

- 导入/导出计划任务

可以使用 sys_dump 和 sys_restore 导入/导出计划任务。

9.3.3.1　配置调度器

KingbaseES 通过 kdb_schedule 插件提供了 DBMS_JOB 和 DBMS_SCHEDULER 包，其中定义了自动作业功能的相关函数。在使用 kdb_shcedule 之前，我们需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中，并重启数据库。

9.3.3.2　监控和管理调度程序

可以查看有关当前运行的作业的信息，监控和管理作业日志。

查找当前运行的作业信息

可以通过查询 kdb_schedule.DBA_SCHEDULER_JOBS 视图来检查作业的状态。

例如，执行以下语句:

```
SELECT DISTINCT job_name, state FROM kdb_schedule.dba_scheduler_jobs
WHERE job_name = 'job1';

jobname   | jastatus
----------+----------
 job1     | failed
 job1     | success
(2 rows)
```

该例子中，可以使用 enable 子程序启用作业。作业状态值如下表所示：running, success, failed, internal failure, aborted

作业状态	描述
running	作业当前正在运行。
aborted	作业运行终止了。
failed	作业计划运行一次，但失败了。
internal failure	该作业至少失败过一次，并且已计划执行重试。
success	作业被调度运行一次并成功完成。

你可以通过执行以下语句，来了解某个作业的相关信息:

```
SELECT * FROM kdb_job WHERE jobname='job1';
```

监控和管理作业日志

可以在作业日志中查看作业运行、作业状态变化和作业失败的信息。作业日志实现如下两个数据字典视图:

- KDB_SCHEDULE.*_SCHEDULER_JOB_LOG
- KDB_SCHEDULE.*_SCHEDULER_JOB_RUN_DETAILS

你可以通过执行以下语句，来了解某个作业的日志信息:

```
SELECT * FROM kdb_schedule.all_scheduler_job_log WHERE job_name = 'job1';
SELECT * FROM kdb_schedule.all_scheduler_job_run_details WHERE job_name = 'job1';
```

9.3.3.3　导入/导出计划任务

sys_dump 和 sys_restore 是一种用于导入/导出 KingbaseES 数据库的工具。即使数据库正在并发使用，也能执行导出计划任务。sys_dump 不阻塞其他用户访问数据库（读取或写入）。当一种归档文件格式与 sys_restore 结合使用时，sys_dump 提供了一种灵活的归档和传输机制。

sys_dump 可用于导出整个数据库，而 sys_restore 可用于检查归档并/或选择要恢复数据库的哪些部分。最灵活的输出文件格式是“自定义”格式（-Fc）和“目录”格式（-Fd）。它们允许选择和重排序所有已归档项，支持并行恢复，并且默认是压缩的。“目录”格式是唯一支持并行转储的格式。

9.3.4　管理数据库自动维护任务

KingbaseES 基于后台进程来实现自动作业功能。可以创建、修改和删除自动作业，以及启用或禁用某个自动作业。

- *关于自动维护任务*

自动维护任务是指自动定期的对数据库进行维护操作的任务。

- *使用自动作业功能*

KingbaseES 通过 kdb_schedule 插件提供了 DBMS_JOB 和 DBMS_SCHEDULER 包，其中定义了自动作业功能的相关函数，在使用 kdb_shcedule 之前，我们需要将它添加到 kingbase.conf 文件的 shared_preload_libraries 中。

- *启用和禁用自动维护任务*

如果使用 DBMS_JOB 包创建的自动维护任务，则可以使用 BROKEN 子程序将 job 任务设置为中止，job 将不会再运行但仍然存在，后续可以重新启用。

- *修改自动作业间隔时间*

创建自动作业任务后，可以通过 DBMS_JOB 的 INTERVAL 子程序更改 job 的作业间隔时间（涉及日历表示法）。

9.3.4.1　关于自动维护任务

自动维护任务是指自动定期的对数据库进行维护操作的任务。可以启用和禁用某个自动作业，创建、修改和删除自动作业。

KingbaseES 的自动作业功能是通过 kdb_schedule 插件提供的 DBMS_JOB 和 DBMS_SCHEDULER 包实现，其中 DBMS_JOB 模式提供调度和管理计划任务的功能，该包功能可以由 DBMS_SCHEDULE 替代，本章节主要结合 DBMS_JOB 说明自动作业功能。

9.3.4.2　使用自动作业功能

KingbaseES 通过 kdb_schedule 插件提供了 DBMS_JOB 和 DBMS_SCHEDULER 包，其中定义了自动作业功能的相关函数，在使用 kdb_shcedule 之前，我们需要将他添加到 kingbase.conf 文件的 shared_preload_libraries 中。配置命令如下：

```
shared_preload_libraries = 'kdb_schedule' # (change requires restart)
```

并重启数据库, 并在数据库中通过下列 create extension 命令创建该插件：

```
create extension kdb_schedule;
```

以下三个配置选项可以在配置文件中指定：

- job_queue_processes：允许用户启动的最大并发数，当其值设置为 0 时，表示不启动自动作业功能, 默认为 0，不开启自动作业。
- sys_job.log_level：用于设置 JOB 后台进程的日志级别，更改后需要重新加载配置文件，可选项：LOG_ERROR，LOG_WARNING，LOG_DEBUG，默认为 LOG_ERROR。
- sys_job.poll_time：用于设置轮询系统表间隔时间, 单位秒，默认值为 10 秒。

9.3.4.3　启用和禁用自动维护任务

如果使用 DBMS_JOB 包创建的自动维护任务，则可以使用 BROKEN 子程序将 job 任务设置为中止，job 将不会再运行但仍然存在，后续可以重新启用。

示例如下，将 job ID 为 1 的作业任务立刻中止：

```
call dbms_job.broken(1,TRUE,now());
```

如果想恢复使用（true 为禁用，false 为启用），示例如下：

```
call dbms_job.broken(1,FALSE,now());
```

如果使用 DBMS_SCHEDULER 包创建的维护任务，则可以使用 DISABLE 子程序将 job 任务设置为禁用，job 将不会再运行但仍然存在。

示例如下，将禁用名为 myjob 的维护任务

```
call DBMS_SCHEDULER.DISABLE('myjob', false, 'STOP_ON_FIRST_ERROR');
```

9.3.4.4　修改自动作业间隔时间

采用 DBMS_JOB 创建自动作业任务后，可以通过 DBMS_JOB 的 INTERVAL 子程序更改 job 的作业间隔时间涉及日历表示法。

示例如下，将 job ID 为 1 的作业，更改间隔时间属性为每天晚上 23 点：

```
call dbms_job.interval(1, 'Freq=daily;BYHOUR=23;BYMINUTE=0;BYSECOND=0');
```

第10章　问题排查

10.1　服务连续性

服务连续性运维的目标是保障需求制定的 RTO 的达成，主要工作是识别并处置风险、问题。本文档描述如无特别说明，均为运行维护阶段的内容。服务连续性运维文档使用维护过程的各个活动作为目录组织，可能导致停机的原因和这些原因的监控、识别、溯因、处理是多对多的关系。

服务连续性运维文档包括以下内容：

1. 影响服务连续性的典型问题及原因概览

   描述了可能导致停机的各类原因。

   备注:

   造成停机的原因分为计划外停机和计划内停机（概念请参考《高可用概述》），本文档只覆盖计划外停机的原因和相关维护工作。

2. 风险问题监控数据采集

   针对可能导致停机的原因，监控和收集相关信息用于后续的识别、溯因和处理。

   本节的监控项覆盖可能造成风险/问题的原因，监控项和原因为多对多关系。

3. 风险问题识别

   根据获取的信息识别风险/问题。

4. 风险问题溯因

   对已经识别的风险/问题进行分析，分析形成的原因，分析其影响。

   可以将收集到的信息传递给数据库厂商服务人员，由数据库厂商服务人员对风险/问题进行分析。

   为了更好的对风险/问题进行分析，需要收集监控信息、识别风险/问题后针对性测试的结果。

   备注:

和概览中的原因对应，文档中不考虑计划内维护操作作为引发风险/问题的原因。

5. 风险问题处理

处理风险/问题，包括：

对风险的影响评估

紧急情况下避免风险/问题影响扩大的处理，明确原因后的完整处理。

备注:

本节和【风险/问题溯因】一节的行为可能在同时发生，没有必然的时间先后顺序。

10.1.1 影响服务连续性的典型问题及原因概览

造成停机的计划外原因包括：站点故障、集群故障、节点故障、网络故障、存储故障、数据损坏、人为错误、资源耗尽。

备注:

以上原因中存储故障、数据损坏两类原因和其他原因中会造成数据丢失并影响服务连续性的问题将在数据防丢失运维 一节中覆盖。

站点故障

站点的定义在本地部署和云部署中不同，可能是：数据中心（data center）、地域（region）或可用区（available zone）。

站点故障会导致站点内全部服务不可用，典型原因包括：站点范围的断电、网络中断、自然灾害导致的站点无法运作。

集群故障

集群范围的服务不可用，可能包括：

- 集群节点故障数量超过容错的范围
- 集群管理软件失效导致集群停机
- 特定失效导致集群同步阻塞

节点故障

- 节点硬件故障
- 节点操作系统中数据库依赖的子系统故障
- KingbaseES 实例故障

网络故障

网络故障的原因非常复杂，可能由硬件或软件引起，表现也不尽相同。具体包括：

- 网络中断
- 端口被屏蔽
- DNS 解析失败
- 网络延迟（高）
- 丢包
- 网络分割

网络故障的原因有：

- 网卡故障
- 网线脱落
- 交换机、路由器故障
- 防火墙
- 网络拥塞

网络问题可能导致数据库无法提供服务、数据同步失败、双主等问题。

人为错误

从已有的案例总结，典型原因包括：

- 误操作
  - 错误的关闭/重启集群、数据库、集群守护进程
  - 错误的关闭/重启服务器、操作系统、各类硬件设备
- 配置错误
  - 集群参数配置错误
  - 数据库参数配置错误
  - 操作系统参数配置错误或遗漏
  - 应用、接口、中间件配置错误
- 恶意数据修改
  - 篡改数据

资源耗尽

资源耗尽可能导致响应超时、访问挂起、连接失败等影响服务连续性的问题。

1. 环境资源耗尽

数据库所在环境的资源使用耗尽导致访问失败或超时，常见的包括：

- 存储容量
- 存储处理能力
- 内存容量
- CPU 处理能力
- 网络带宽
- 其他
  - 文件句柄
  - 信号量
  - ……

2. 数据库资源耗尽

资源使用达到了数据库配置限制或软件限制，导致相关访问失败或超时。常见的包括：

- 连接数耗尽
- 事务号耗尽
- 对象封锁冲突
- license 过期
- ……

10.1.2　风险/问题监控、数据采集

- 监控

  针对风险/问题的原因做监控。注意监控项中描述的监控频率是建议值，在条件不允许时可以按实际情况执行。例如在没有监控工具或脚本的情况下，可以按巡检频率来获取监控数据。

  日常监控是非常重要的，一方面意外总是会非预期而至，良好的监控可以在意外来临时及时的预警，通知 DBA 等专业人员处理，避免故障进一步蔓延；另一方面是持续的监控可以尽早的发现可能出现的问题，预测事故未来发展的严重程度及可能出现的危机结果，以防范于未然。

- 数据采集

  采集内容可能不与风险/问题的原因直接对应，但在识别、溯源过程中可能需要这些信息。这些信息需要触发式或周期性的采集。后续识别、溯源需要的此类数据在这一节覆盖。

10.1.2.1　监控

10.1.2.1.1　集群状态类问题

集群故障、节点故障、网络故障和部分人为错误、资源耗尽问题都会表现出集群状态异常，集群状态信息是监控中重要的监控项。

1. 节点状态

   监控指标：节点状态

   监控频率：每 1 分钟

   告警值：

   status 列不是 running 状态

   upstream 列，不是级联复制情况下不是 primary 的 node_name

   不同节点获得的数据不同

```
# 在存活的数据库节点执行（当前数据库故障则无法查看状态）
repmgr cluster show
```

   保留时间：至少一周

2. 守护进程状态

   监控指标：守护进程状态

   监控频率：每 1 分钟

   告警值：

   repmgrd 列，不是 running 状态

```
# 在存活的数据库节点执行（当前数据库故障则无法查看状态）
repmgr service status
```

   保留时间：至少一周

3. 流复制状态

   监控指标：流复制状态

   监控频率：每 1 分钟

   告警值：

   state 列值不为 streaming

   sync_state 列和预期不符（配置的同步方式）

   复制差距超过预设值

```
#ksql 连接数据库执行（连接主库查询）
# 查询流复制信息
SELECT * FROM sys_stat_replication;

# 查询主备流复制差距
select sys_wal_lsn_diff(sys_current_wal_flush_lsn(), replay_lsn) as lsn_lag, * from sys_stat_replication;
```

4. 复制槽

   监控指标：复制槽状态

   监控频率：每 10 分钟

   告警值：

   主库复制槽个数、slot_type 列值或 active 列值为预期外

```
#ksql 连接数据库执行（连接主库或备库查询）
# 复制槽信息
SELECT * FROM sys_replication_slots;
```

10.1.2.1.2　资源耗尽类问题

1. 存储容量

   监控指标：

   以下存储用量百分比：

   数据（包括各表空间）、WAL、归档、备份

   监控频率：每 5 分钟

   告警值：用量超过根据业务需要预设的比例

   保留时间：至少一周

2. CPU、内存、网络、IO 用量

   监控指标：CPU、内存、网络、IO 用量

   监控频率：每 1 分钟

   告警值：用量超过根据业务需要预设的比例

   保留时间：至少一周

3. 连接数

   监控指标：当前连接数

   监控频率：每 5 分钟

   告警值：conns 超过 max_connections 的 80%

```
SELECT count(*) as conns FROM sys_stat_activity WHERE backend_type IN ('client backend', 'walsender');
```

保留时间：至少一周

4. 长事务

监控指标：当前实例中可见的最老事务

监控频率：每小时

告警值：age 超过 2^32 - 30000000

```
SELECT pid, datname, query, age(backend_xmin) as age FROM sys_stat_activity WHERE backend_xmin IS NOT NULL
ORDER BY age(backend_xmin) DESC LIMIT 1;
```

保留时间：至少一周

5. 事务号使用

监控指标：当前各库的事务号、多事务号使用

监控频率：每小时

告警值：age 或 mxid_age 超过 2^32 – 30000000

```
SELECT datname, age(datfrozenxid), mxid_age(datminmxid) FROM sys_database;
```

保留时间：至少一周

6. 封锁情况

监控指标：当前等待封锁的进程数量

监控频率：每分钟

告警值：无固定值，根据业务情况设置

```
SELECT count(*) FROM sys_locks WHERE granted = false;
```

保留时间：至少一周

7. license

监控指标：license 剩余有效期

监控频率：每天

告警值：> 0 且 < 30

```
SELECT get_license_validdays() AS valid_days;
```

10.1.2.1.3　日志

监控指标：数据库、高可用组件（包括配置的一键数据库状态检查）、OS 日志

监控频率：每 5 分钟

告警值：出现警告及以上级别的日志且日志内容不在白名单中

保留时间：至少一周

10.1.2.2　数据采集

10.1.2.2.1　日志

内容：数据库、高可用组件（包括配置的一键数据库状态检查）、OS 日志

数据库日志内容参数：

```
log_statement = ‘ddl’
log_autovacuum_min_duration = 600000
```

保留时长：至少一周。

10.1.2.2.2　健康检查

内容：配置周期性的一键数据库状态检查并输出结果。频率至少每天。

保留时长：至少一周。

状态检查项中支持监控项的检查，在没有其他监控工具时可以配置检查替代监控工具。

一键数据库状态检查的结果监控和告警值已纳入“日志”一节。

10.1.2.2.3　集群事件

内容：集群事件

通过以下命令行命令获取的数据：

```
# 在存活的数据库节点执行（当前数据库故障则无法查看）
repmgr cluster event
```

保留时长：至少一周。

示例输出：

```
[kingbase@nn02 data]$ repmgr cluster event
 Node ID | Name     | Event                      | OK | Timestamp           | Details
---------+----------+----------------------------+----+---------------------+-------------------------------------------------------------------
 2       | node102  | child_node_new_connect     | t  | 2020-10-05 13:58:38 | new standby "node103" (ID: 3) has connected
 3       | node103  | repmgrd_failover_follow    | t  | 2020-10-05 13:58:37 | node 3 now following new upstream node 2
 3       | node103  | standby_follow             | t  | 2020-10-05 13:58:37 | standby attached to upstream node "node102" (ID: 2)
 2       | node102  | repmgrd_reload             | t  | 2020-10-05 13:58:32 | monitoring cluster primary "node102" (ID: 2)
 2       | node102  | repmgrd_failover_promote   | t  | 2020-10-05 13:58:32 | node 2 promoted to primary; old primary 1 marked as failed
 2       | node102  | standby_promote            | t  | 2020-10-05 13:58:32 | server "node102" (ID: 2) was successfully promoted to primary
 1       | node101  | child_node_reconnect       | t  | 2020-10-05 13:56:36 | standby node "node103" (ID: 3) has reconnected after 6 seconds
 1       | node101  | child_node_reconnect       | t  | 2020-10-05 13:56:36 | standby node "node102" (ID: 2) has reconnected after 6 seconds
 1       | node101  | child_node_disconnect      | t  | 2020-10-05 13:56:30 | standby node "node103" (ID: 3) has disconnected
 1       | node101  | child_node_disconnect      | t  | 2020-10-05 13:56:30 | standby node "node102" (ID: 2) has disconnected
 1       | node101  | repmgrd_local_reconnect    | t  | 2020-10-05 13:56:27 | reconnected to primary node after 36 seconds, resuming monitori
 2       | node102  | repmgrd_upstream_reconnect | t  | 2020-10-05 13:56:27 | reconnected to upstream node 1 after 36 seconds, resuming monit
 3       | node103  | repmgrd_upstream_reconnect | t  | 2020-10-05 13:56:27 | reconnected to upstream node 1 after 36 seconds, resuming monit
 3       | node103  | repmgrd_start              | t  | 2020-10-05 12:09:54 | monitoring connection to upstream node "node101" (ID: 1)
 1       | node101  | child_node_new_connect     | t  | 2020-10-05 12:09:35 | new standby "node103" (ID: 3) has connected
 3       | node103  | standby_register           | t  | 2020-10-05 12:09:30 | standby registration succeeded; upstream node ID is 1
 3       | node103  | standby_clone              | t  | 2020-10-05 12:09:29 | cloned from host "10.10.10.101", port 54321; backup method: sys
 2       | node102  | repmgrd_start              | t  | 2020-10-05 12:07:41 | monitoring connection to upstream node "node101" (ID: 1)
 1       | node101  | child_node_new_connect     | t  | 2020-10-05 12:07:28 | new standby "node102" (ID: 2) has connected
 2       | node102  | standby_register           | t  | 2020-10-05 12:07:22 | standby registration succeeded; upstream node ID is 1
```

10.1.3　风险/问题识别

风险/问题识别基于监控和数据采集的输入，主要通过以下方式：

1. 阈值告警

   监控过程中监控项达到了告警值，例如数据库日志中出现白名单外的 ERROR 信息。

2. 趋势分析

   对比一定时间内的多次数据，变化趋势超过预期，例如存储容量在过去一周内增长超过 30%。

3. 经验

   历史上接受的风险，例如触发式或周期性的业务高峰即将到来。

10.1.3.1　识别

10.1.3.1.1　集群状态类问题

适用阈值告警方法，不适用趋势分析方法。

10.1.3.1.2　资源耗尽类问题

适用的识别方法如下：

	阈值告警	趋势分析
存储容量	适用	应在一定范围内周期性波动或按一定幅度稳定增长
CPU、内存、网络、IO 用量	适用	应在一定范围内周期性波动
连接数	适用	应在一定范围内周期性波动
长事务	适用	应在一定范围内周期性波动

-续表-

	阈值告警	趋势分析
事务号使用	适用	应在一定范围内周期性波动
封锁情况	适用	应在一定范围内周期性波动
License	适用	不适用

10.1.4　风险/问题溯因

10.1.4.1　信息收集

10.1.4.1.1　日志

操作系统日志、KingbaseES 日志、高可用组件日志是风险/问题溯因的重要信息来源。

集群环境中分析集群状态相关问题时，建议使用 WalMiner 日志逻辑解码工具降低分析难度和时间。

WalMiner 日志逻辑解码工具可对以上日志进行收集、加工、多机日志整合，以自定义匹配的关键内容呈现输出，去除无用信息，减少分析难度。

具体使用方式参见《WalMiner 日志逻辑解码工具》。

10.1.4.1.2　数据库状态检查结果

在发生涉及集群状态的风险/问题时，建议在分析前运行一次一键检查工具作为辅助信息。

具体请参见《工具参考》的一键检查数据库状态工具 KBchk.sh 。

10.1.4.1.3　其他信息

1. core dump file

KingbaseES 相关组件产生的 core dump file 是溯因的重要信息来源，默认情况下文件生成在数据目录中。

2. 数据库进程调用栈

风险/问题发生时，有可能数据库无法响应客户端的请求，此时各类数据库内的监控和采集都有可能失效，通过 os 层面输出的进程调用栈可以帮助定位此类问题。

通过以下命令获得全部 kingbase 进程的调用栈

```
(for i in `ps -ef | grep kingbase | grep -v grep | awk '{print $2}'`; do pstack $i; done) > pstack.kingbase
```

备注:

需要在数据库服务器上安装 gdb。

没有安装的情况下可以通过 cat /proc/<PID>/stack 获取进程的调用栈

3. 当前活动连接

当 sql 执行超长时间未结束时，需要收集当前活动连接信息：

```
select * from sys_stat_activity;
```

10.1.4.2　溯因

风险溯因是指在感知到风险存在时，找到造成风险的直接原因、间接原因和根本原因。

10.1.4.2.1　集群状态类问题

1. **节点状态异常**

- 网关故障关库

在故障库的 kbha.log 中出现以下错误：

```
[NOTICE] PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.

--- 10.10.0.1 ping statistics ---
3 packets tansmitted, 0 received, 100% packet loss, time 2000ms

[WARNING] ping host"10.10.0.1" failed
[DETAIL] average RTT value is not greater than zero
[INFO] stop database …
```

报错原因：

【简单说明】kbha 进程连续多次 ping 信任网关失败后会关闭当前节点的数据库

【详细说明】kbha 进程会不断 ping 信任网关，ping 失败会继续重试，一旦连续 reconnect_attempts 次都失败后，认为当前节点网络异常，会关闭当前节点数据库（2021 年 7 月之前的版本）。

出现场景：

网络异常，导致 kbha 检测信任网关失败。（2021 年 7 月之前的版本）

- 磁盘故障关库（use_check_disk=on）

  在故障库的 kbha.log 中出现以下错误：

```
[ERROR] [thread xx] Failed to check the mount_point_list 6 / 6, all failed
[NOTICE] [thread xx] the use_check_disk = on, will stop the database
[INFO] stop database …
```

  报错原因：

  【简单说明】kbha 连续多次检测数据目录的读写失败，关闭数据库

  【详细说明】kbha 进程每隔 60 秒会检测 mount_point_list（默认为空，检测数据目录）目录下的 I/O 状况，如果失败则间隔 10 秒再次检测，连续 mount_check_max_retries（默认 6）次都失败后，关闭数据库。

  检测方式：查看目录的读写权限，创建文件，写入 8k 数据、读取 8k 数据，删除文件。任何一项失败则认为检测失败。

  出现场景：

  – 磁盘故障
  – 业务压力大 I/O 波动，短时间内 kbha 检测失败

- 数据库 coredump

  在故障库的 `sys_log/*` 中出现以下错误：

```
[PANIC] XXXX
Failed process was running XXXX
```

  或 data 目录中出现 core 文件

  出现场景：

  软件失效或特定故障

- 检查点（checkpoint）错误

  无法启动数据库的 `sys_log/*` 中出现以下错误：

```
LOG: invalid primary checkpoint record
PANIC: could not locate a valid checkpoint record
```

  报错原因：

  【简单说明】控制文件和 wal 日志对应不上，其中一个被篡改了。

  【详细说明】数据库启动时，需要从控制文件 sys_control 中记录的 checkpoint 开始 redo。出现此报错的原因是数据库根据 checkpoint 记录的 LSN 从本地的 WAL 文件中找不到 WAL 日志或者找到的 WAL 日志不是 checkpoint。

  出现场景：

无（正常情况下不应该出现此问题）

- 时间线（timeline）错误

无法启动数据库的 `sys_log/*` 中出现以下错误：

```
FATAL: requested timeline 6 is not a child of this server's history
    Latest checkpoint is at 0/C3000058 on timeline 5, but in the history of the requested
timeline, the server forked off from the timeline at 0/C2003018
```

报错原因：

【简单说明】归档或本地 sys_wal 目录下多了一个或多个 history 文件。

【详细说明】数据库启动时会读取 sys_wal 目录下的所有 X.history 文件，并取最高的来判断控制文件中的 checkpoint 是否合理。X.history 会记录每个时间线 timeline 的结束 LSN，当发现 checkpoint 的 LSN 为 A，而 history 文件中记录对应时间线的结束 LSN 为 B，且 A>B 时，就会认为 checkpoint 是非法的，就会出现此报错。

出现场景：

– 连续多次 PITR 或物理还原（生成了多个 history 文件）

– 集群切换后，新主（生成的 history 被归档）重新恢复为备机，之后节点故障恢复

- 配置错误

无法启动数据库的 `sys_log/*` 中出现以下错误：

```
端口被占用
内存超限
不识别的配置项或配置项的值不合法
```

报错原因：

【简单说明】参数配置不合法或参数配置不合理

出现场景：

– 错误的手动修改参数

- 存在双主（不自动恢复故障节点）

主库 hamgr.log 中出现以下错误：

```
[ERROR] [thread pid: xx] there may be multiple nodes running as primary, can not do recovery for
failed node
```

报错原因：

【简单说明】出现双主或多主，集群不能执行自动恢复

【详细说明】集群是由主库的 repmgrd 来检测并恢复故障节点。一旦集群中出现多个主库，如果不加限制，会有多个主库的 repmgrd 对同一个故障节点进行恢复，就会造成集群状态异常，并且可能会损坏故障节点的数据。所以，当出现多主后，集群不能执行自动恢复。

出现场景：

– 网络分隔后恢复，出现双主（最新版本中新主库会关闭老主库）

– 手动启动了原主库，出现双主

- 无法判断节点是备库（recovery=standby）

主库 hamgr.log 中出现以下错误：

```
[INFO] unable to connect via ES to host "xxxx", can't confirm if it's a standby node
[INFO] [thread pid: xx] node (ID: x; host "xxxx") is not attached
[INFO] [thread pid: xx] can not do auto-recovery because not confirmed a standby node
```

报错原因：

【简单说明】recovery=standby 只恢复备库，如果无法确认是否为备库则不恢复。

【详细说明】主库的 repmgrd 会判断 repmgr.nodes 中为 standby 的节点是否需要恢复，如果需要恢复，就会远程查看该数据库是否真正为备库。会通过 ssh 或 sys_securecmd 远程查看故障数据库是否存在 data/standby.signal 文件，只有确定故障数据库存在该文件才会恢复故障节点。如果远程失败，或者没有此文件，则不会恢复故障节点。

出现场景：

– ssh 或 sys_securecmd 连接失败（日志中有打印：unable to connect via ES to）

– 故障节点没有 data/standby.signal 文件（人为删除，或者原来为主库？）

- 故障节点 TimeLine 更高

主库 hamgr.log 中出现以下错误或者手动执行命令的命令行打印：

```
ERROR: this node's timeline is ahead of the rejoin target node's timeline
DETAIL: this node's timeline is 8, rejoin target node's timeline is 7
```

报错原因：

【简单说明】故障节点的时间线高于当前主库，为了保护数据不允许继续恢复。

【详细说明】自动恢复或手动恢复（kbha -A rejoin 或 repmgr node rejoin）时，程序会自动判断故障节点与当前主节点的时间线（TimeLine）和 LSN，如果发现故障节点的 TimeLine 高于当前主库，则认为发生了数据分歧，继续执行有可能丢失数据，所以会报错退出。

出现场景：

– 脑裂后，手动恢复新主库为备库

– 集群所有节点故障，且有多个主库，启动老主库，将其他数据库恢复为备库

- 集群中存在多个主节点

  集群状态 cluster show 结果中 Role 列显示多个 primary：

```
[kingbase@nn02 ~]$ repmgr cluster show
 ID | Name     | Role    | Status    | Upstream | Location | Priority | Timeline |
----+----------+---------+-----------+----------+----------+----------+----------+
 1  | node101  | primary | ! running |          | default  | 100      | 21       |
 2  | node102  | primary | * running |          | default  | 100      | 22       |
```

  其他请参考“存在双主（不自动恢复故障节点）”.

2. 守护进程异常

- 节点未注册

  在 hamgr.log 中出现以下错误：

```
[ERROR] no metadata record found for this node - terminating
[HINT] Check that "repmgr (primary|standby) register" was executed for this node
[INFO] repmgrd terminating...
```

  报错原因：

  【简单说明】当前节点未注册到集群中。

  【详细说明】守护进程 repmgrd 启动时会根据配置文件 repmgr.conf 中记录的节点 ID 从本地数据库中获取当前节点的信息，如果数据库中的 repmgr.nodes 表中没有当前节点信息，则会报错退出。

  出现场景：

  – 物理备份恢复到了一个比较老的数据版本，当前节点还未注册到集群中
  – 本节点被用户手动操作从集群中注销了（使用部署工具删除节点，或命令行执行了 repmgr primary/standby/witness unregister）
  – 本节点的配置文件 repmgr.conf 中的 node_id 参数被手动修改为了其他值
  – 当前节点是被扩容的新节点，执行了 repmgr standby/witness register 后，新数据还未同步到此节点

3. 流复制状态异常

- 缺少 WAL 文件

  备库 `sys_log/*` 中出现以下错误：

```
FATAL: could not receive data from WAL stream: ERROR: requested WAL segment
00000001000003AB00000026 has already been removed
```

  报错原因：

  【简单说明】备库需要的 WAL 日志在主库上不存在。

【详细说明】备库启动时会根据 backup_label 文件或者 sys_control 文件开始 redo，本地 WAL 日志 redo 完成后会连接主库拉取 wal 日志。如果主库没有备库需要的 WAL 日志就会返回此报错信息。

出现场景：

- 原主库恢复为备库
- 备库 rewind 自动恢复（wal_keep_segments+max_wal_size 配置不合理、长时间未恢复）

4. 复制槽异常
   - 复制槽数量、类型异常

     建议与数据库管理员（DBA）及相关权限用户沟通确认此行为是否符合预期设置。
   - 复制槽活动状态异常

     通常由备机故障或复制状态异常引起。
5. 数据库进程无响应
   - 数据库进程依赖的 OS 子系统无响应

     可能表现为无法连接、连接后查询或写入无响应

     可能原因常见为 I/O 子系统无响应：

     1. 通过 top 获取无响应进程的进程号。使用-c 可以获得进程的 process 描述帮助定位。
     2. 确认 top 输出的 S 列值，如果是 D 状态，说明请求在 OS 内核中没有响应。
     3. 通过 cat /proc/<PID>/stack 获取进程的调用栈，协调 OS 工程师进一步做溯因。
     4. 非 D 状态的无响应考虑排查资源耗尽类问题。

10.1.4.2.2　资源耗尽类问题

1. 环境资源
   - 存储容量

     存储容量超过预期的常见原因包括：

     1. 数据（包括各表空间）存储容量不足
        1. 残留的临时文件

           确认 syssql_tmp 目录容量是否是引起容量不足的原因
        2. 表膨胀

           参见《处理表索引膨胀 》
        3. 过多的性能优化辅助存储结构，例如索引、物化视图

           通过存储容量函数确认容量的分布情况，确认排名靠前对象的类型。

4. 业务预期外的增长

   通过存储容量函数确认容量的分布情况，和应用接口人共同排查确认是否符合预期。

2. 在线 WAL 存储容量不足

   1. 备机故障未及时恢复

      通过查看集群状态和复制槽中的 LSN 信息可以确认是否是此类原因。

   2. 归档失败导致 WAL 无法被正常回收

      数据库日志中会有对应的 archive command failed 日志和执行的具体 command，可以手动执行排查失败的具体问题。

   3. 检查点预期外延迟导致 WAL 无法被正常回收

      通过 sys_control_checkpoint() 确认上一次检查点时间，通过 sys_stat_activity 中 checkpointer 的 wait_event 确认目前检查点的等待阶段。通常由 I/O 资源不足引起。

   4. 业务峰值或发生不常见的业务操作

      通过 waldump 对比预期外增长 wal 和正常 wal 的差异或是 kwr 中的 top sql 对比确认引发 wal 增长的操作，和应用接口人共同排查确认操作是否符合预期。

3. 归档存储容量不足

   1. 备份管理任务没有按预期清理旧备份

      备份工具会在备份后清理旧备份和归档，查看备份的保留配置参数是否符合预期；查看备份执行日志确认清理过程是否报错。

   2. 业务峰值或发生不常见的业务操作

      同在线 WAL 的排查。

4. 备份存储容量不足

   1. 备份管理任务没有按预期清理旧备份

      同归档的排查方式。

   2. 数据或 WAL 造成

      同数据或 WAL 的排查方式。

- CPU、内存、网络、IO 用量

  资源用量的溯因请参考《风险问题溯因 》。

2. 数据库资源

   - 连接数

     连接数增加常见的原因包括：

     1. 业务变化

        1. 新增业务模块带来的压力增加

2. 原有业务的规模/用量增加或变化

   需要和业务方面接口人确认是否有此情况。

2. 性能或资源原因造成处理能力下降

   依赖性能或资源风险/问题的溯因和处理

- 长事务

  引起长事务常见的原因是

  1. 各类客户端工具在非自动提交模式下人为操作后未关闭

     可以根据 sys_stat_activity 中的 client_addr 和 application_name 判断该事务是否来自客户端工具

  2. 应用的特定实现方式或缺陷造成事务未提交

     提供 sys_stat_activity 中的 query 和事务中其他语句（如果有日志）给应用接口人排查是否是预期的行为

  3. 未结束的两阶段提交事务

     查询 sys_prepared_xacts 确认是否是未结束的两阶段提交事务，通过事务名称和应用接口人排查是否是预期的行为

  4. 性能或资源原因造成事务阻塞

     依赖性能或资源风险/问题的溯因和处理

- 事务号使用

  事务号耗尽常见的原因是：

  1. 清理回收没有按预期启动

     1. 错误的 threshold 相关参数配置

        确认自动清理相关参数是否开启

     2. 清理速度慢，autovacuum worker 被占用

        通过 sys_class 中对象的事务 age 查找到最长 age 的对象，通过 sys_stat_user_tables 确认该对象的自动清理情况，通过日志中 autovacuum 信息确认是否有对象长时间占用自动清理工作进程导致其他对象没有得到自动清理

  2. 清理没达到预期效果

     1. 存在长事务

        溯源方式参见“长事务”。

     2. 存在残留的复制槽事务号

        查询 sys_replication_slots 确认是否有未预期的 backend_xmin，可能有故障且未恢复的备机或是残留的 slot

- 封锁情况

  封锁超预期的常见原因：

  1. 有长时间持有高级别封锁的操作，常见为对象的 DDL 操作

     查看数据库日志，确认对应时间内的 DDL 日志。和应用接口人确认是否是预期的行为。

  2. 长事务导致事务内封锁未及时释放，常见为阻塞并发的数据更新

     溯源方式参见“长事务”。

- license

  license 到期常见原因：

  1. 试用版 license 到期
  2. 未使用正确的 license

  需要确认使用的 license 的预期授权，通过查看 license.dat 文件确认是否和预期相符

- 事务状态访问异常

  常见报错：

```
could not access status of transaction xxxx
could not open file "sys_xact/xxxx": No such file or directory
```

  信息查看与梳理：

  在 ksql 中 show block_size; 查看页面大小。

  1. 单机环境

     查找 kingbase 日志，以及逻辑备份日志，找到最早报该错误的时间点；

  2. 集群环境

     查找集群中所有节点的 kingbase 日志，以及逻辑备份日志，找到最早报该错误的时间点；

     根据集群中所有节点的 kbha.log 日志、hamgr.log 日志梳理出问题前后集群状态变化情况；

  常见原因：

  1. xact 不正常情况

     a). 新 xact 文件丢失

        排查思路:

        (1). sys_controldata data 目录 | grep NextXID 查看下一个事务 id；

        (2). 报错的事务 id 与 NextXID 对比，如果相差很大，则不是新 xact 文件丢失问题；

        (3). 如果相差比较小的话，如果小于 128*block_size，则报错的事务 id 在当前正在写入的 xact 日志段中，需要排查操作系统日志，是否有存储或内存方面的报错；

     b). 旧 xact 文件被异常清理

排查思路：

(1). 丢失 xact 文件的前后文件均存在，可以查找存储是否有异常信息；

(2). 如若存储没有异常，则将相关情况反馈给研发定位分析；

2. xact 正常情况

a). 被 frozen 的数据元组使用 xid 判断可见性

排查思路：

(1). 定位报错时正在操作的表对象

1). 查看 kingbase 或逻辑备份日志，找最早报错执行的 sql 并找出 sql 中所涉及的表对象；

2). set synchronize_seqscans to off;

3). 通过 select xmin, xmax, ctid, *from 的查询方式来定位具体是哪个表的问题；

(2). 系统表查询

```
select relfrozenxid from sys_class where relname = '定位的表对象';
```

(3). 查看具体 page 页中的元组头信息

1). 创建 pageinspcet 插件；

2). 通过 (1) 中知道具体是在查询哪一行后报了错，来确定报错的这一行的 ctid 值；(比如 (1) 中最后能查到的一行元组的 ctid 为 (X,Y)，则报错的元组的 ctid 的页面号可能为 X 或者 X+1)

3). select t_infomask, * from heap_page_items(get_raw_page(‘报错的表对象’, 页面号)) where t_xmin = xxxx;

4). select 查询到的 t_infomask & 768;

如果为 768，则该 tuple 有被设置 frozen 标志；否则，没有。

5). 如果是集群环境，排查所有节点的 hamgr.log 日志，查看主库是否对最早出问题的节点执行过多次故障恢复，是否有多次 rewind 成功，但起库失败的情况；

将以上收集的信息，提供给研发，以便接着分析；

b). 数据损坏导致 xid 异常

排查思路：

(1). 使用 for i in {1···N}; do printf ‘125’; done > xxxx 造丢失的 xact 文件；(N 为 32*block_size)

(2). 定位到的出错表的数据可以正常读取，则不是数据损坏的问题；如果不能正常读取，则需要借助备份重做数据；

10.1.5　风险/问题处理

处理的目标是在分析评估风险/问题后选择适合的处理方案来避免、减轻或接受风险/问题。不考虑识别后无需处理的情况，处理方案大致分为两类：

1. 紧急处理：风险/问题经过影响分析后认为可能引起重大问题情况下，在可能不确认原因甚至可能引起其他风险/问题的情况下对风险/问题的临时处置，避免引起更严重或更大范围的问题。
2. 完整处理：明确风险/问题原因后针对根因的处置方式。

10.1.5.1　影响分析

在资源完全耗尽的情况下，大部分会造成服务中断，影响分析的重点是估算资源消耗趋势和时间，时间允许情况下使用完整处理方式避免对业务的影响。

10.1.5.2　处理

以上人为操作引起的后果无法预料且很难追溯，所以尽可能通过工具或脚本进行日常操作，人工操作有条件下加入复核或记录操作步骤。影响未知的操作前最好进行备份、使用相关安全功能避免和追溯恶意操作。

10.1.5.2.1　集群状态类问题

1. 集群中数据库意外关闭
2. 网关故障关库（老版本）

   关键错误（kbha.log）：

   处理步骤：【恢复网络，恢复节点】

   请先和客户确认，是否存在网络波动，后续是否还会出现。

   - 如果集群中无主库存活：

     1. 查看集群每个节点状态。

     ```
     ls -l data              # 查看 standby.signal 文件，如果有则是备库
     sys_controldata -D data  # 查看控制文件中的 checkpoint，判断数据量
     ```

     2. 确认主库（数据量最多），直接启动。

     ```
     sys_ctl -D data start
     ```

     3. 如果有其他故障主库，手动恢复（恢复前先备份，防止数据丢失）

     ```
     kbha -A rejoin -h ${主库 IP}
     ```

     4. 等待集群自动恢复

```
repmgr service status  # 查看集群状态，等待恢复
```

- 如果集群中有主库存活：
  1. 查看故障节点状态。

```
ls -l data              # 查看 standby.signal 文件，如果有则是备库

sys_controldata -D data  # 查看控制文件中的 checkpoint，判断数据量
```

  2. 如果故障节点和主库无数据差异，可以手动恢复（先备份，防止数据丢失）
  3. 如果有其他故障主库，手动恢复（恢复前先备份，防止数据丢失）

```
kbha -A rejoin -h ${主库 IP}
```

  4. 等待集群自动恢复

```
repmgr service status  # 查看集群状态，等待恢复
```

3. 磁盘故障关库（use_check_disk=on）

   处理步骤：【调整参数】

   和客户确认是否磁盘出现问题（可以查看操作系统 message），如果是，请修复磁盘问题。

   调整参数：

   方法 1：设置 `use_check_disk=off`，仅检测，检测磁盘故障后不关闭数据库。

   方法 2：增大磁盘检查次数，设置 `mount_check_max_retries=20`，防止短时间磁盘 IO 问题导致数据库关闭。

4. 数据库 coredump

   处理步骤：

```
请联系电科金仓技术服务人员处理。
```

5. 数据库无法启动（主库或备库）
   1. 检查点（checkpoint）错误

      处理步骤：【重置控制文件中的 checkpoint】

      1. 获取控制文件中记录的 WAL file 名称

```
sys_controldata -D data
# 找到第 8 行，记录的 wal file 名称，例如：0000000100000000A0000002D
```

      2. 使用 sys_resetwal 重置控制文件（重置后可能会丢失数据）

```
sys_resetwal -l ${WAL file} data
# 会自动根据 sys_wal 目录下已有的 wal 文件以及-l 指定的文件，找到一个最大的作为新的 checkpoint 和 wal 文件
# 在集群环境下使用 sys_resetwal，最好只修改主库，修改主库后备库需要重新克隆，否则可能无法建立流复制。
```

3. 启动数据库。

2. 时间线（timeline）错误

   处理步骤：【删除多余的 history 文件】

   1. 获取控制文件中记录的 timeline

   ```
   sys_controldata -D data
   # 找到第 9 行，记录的 TimeLineID
   ```

   2. 删除以下目录中高于控制文件中 TimeLineID 的 history 文件（0000000X.history）
      - 备库和主库：`data/sys_wal` 目录。
      - 归档目录：如果是 sys_rman 备份，则位于 `repo/archive/xxxx` 目录。
   3. 启动数据库。

3. 配置错误

处理步骤：【更正配置】

```
更正不合法或不合理的配置
```

6. 集群故障节点无法自动恢复

   1. 存在双主（不自动恢复故障节点）

      处理步骤：【消除双主、关闭原主库】

      1. 确认多个主库的数据情况，保证要关闭的主库数据较少。

         备份所有主节点的数据目录，连接到主数据库中，人为判断业务数据。

         ```
         select sys_current_wal_lsn();
         ```

         备注:

         切换时间线时执行 checkpoint 会导致较高时间线的数据库 LSN 大，但是数据并不一定最新。

      2. 如果 recovery=automatic，请先暂停集群。

```
repmgr service pause
repmgr service status   # 查看是否成功暂停：'Pause?' 列为 yes
```

3. 关闭其中一个主库（有条件，关闭后可以备份 data）

```
sys_ctl -D data stop
```

4. 取消集群暂停

```
repmgr service unpause
repmgr service status   # 查看是否取消暂停，集群恢复正常：'Pause?' 列为 no
```

5. （可选）恢复原主库为备库

```
kbha -A rejoin -h ${新主库 IP}
# 如果新主库时间线较低，上面命令可能会失败，请执行以下命令恢复
repmgr node rejoin --force-rewind --no-check-wal -h ${新主库 IP} -U esrep -d esrep -p ${新主库
端口}
```

2. 无法判断节点是备库（recovery=standby）

   处理步骤：【解决判断备库失败问题】

   - 如果 ssh 或 sys_securecmd 连接失败：
     1. 在主库手动执行 `ssh kingbase@[故障节点 IP] date` 。
     2. 在主库手动执行 `sys_securecmd kingbase@[故障节点 IP] date` 。
     3. 查看是否有报错，解决报错问题。
   - 如果故障节点没有 data/standby.signal 文件：
     1. 确认该节点是否可以直接恢复，是否需要备份（防止数据丢失）。
     2. 在故障节点创建 data/standby.signal 空文件。

     ```
     touch data/standby.signal
     ```

3. 故障节点 TimeLine 更高

   处理步骤：【恢复为备库】

   备注:

   请确认故障节点无数据需要找回后再执行操作。

   在故障数据库执行以下命令进行恢复

```
repmgr node rejoin -h ${主库 IP} -U esrep -d esrep -p ${主库端口} --force-rewind --no-check-wal
```

7. 集群中存在多个主节点

   处理步骤参考“存在双主（不自动恢复故障节点”。

8. 守护进程异常

   1. 节点未注册

   处理步骤：【强制注册备节点】

```
repmgr standby register -F
```

9. 流复制状态异常

   1. 缺少 WAL 文件

      关键错误（备库 sys_log/* ）：

```
FATAL: could not receive data from WAL stream: ERROR: requested WAL segment
00000001000003AB00000026 has already been removed
```

      报错原因：

      【简单说明】备库需要的 WAL 日志在主库上不存在。

      【详细说明】备库启动时会根据 backup_label 文件或者 sys_control 文件开始 redo，本地 WAL 日志 redo 完成后会连接主库拉取 wal 日志。如果主库没有备库需要的 WAL 日志就会返回此报错信息。

      出现场景：

      - 原主库恢复为备库
      - 备库 rewind 自动恢复（wal_keep_segments+max_wal_size 配置不合理、长时间未恢复）

      处理步骤：【重做备库】

      1. 在主库执行命令暂停集群功能（防止重做过程中集群操作此节点）

```
repmgr service pause
repmgr service status   # 查看是否成功暂停：'Pause?' 列为 yes
```

      2. 在备库执行命令重做数据库

```
repmgr standby clone -F -h ${主库 IP} -U esrep -d esrep -p ${主库端口} --fast-checkpoint
```

      3. 启动数据库并重新注册为备库

```
sys_ctl -D data start
repmgr standby register -F
```

4. 取消集群暂停

```
repmgr service unpause
repmgr service status   # 查看是否取消暂停，集群恢复正常：'Pause?' 列为 no
```

10. 复制槽异常

    1. 复制槽数量、类型异常

       预期行为情况下：无需处理，但需要注意，手动增加的复制槽在数据库节点切换后需要重新手动创建。

       非预期行为情况下：手动恢复复制槽、删除复制槽

       恢复集群应有复制槽，注意集群应有的复制槽会在切换后自动重建。

       ```
       select sys_create_physical_replication_slot('名称');
       ```

       删除预期外复制槽

       ```
       select sys_drop_replication_slot('名称');
       ```

    2. 复制槽活动状态异常

       修复备机故障或复制状态异常即可自动恢复

11. 数据库进程无响应

12. 数据库进程依赖的 OS 子系统无响应

    OS 内核无响应的进程无法在用户态控制，在确认原因后需要重启 OS 恢复服务

10.1.5.2.2　资源耗尽类问题

1. 存储容量

   存储容量超过预期的处理方式：

   1. 数据（包括各表空间）存储容量不足

      1. 残留的临时文件

         紧急处理：R6 旧版本可能在数据库 crash 时残留临时文件，根据停机时间和文件的修改时间手动删除残留的文件。紧急处理不会影响使用，但在下次 crash 时仍可能残留临时文件。

         完整处理：更新到 R6 最新版本，数据库在恢复时会自动删除临时文件。

      2. 表膨胀

         参见《处理表索引膨胀 》

      3. 过多的性能优化辅助存储结构，例如索引、物化视图

紧急处理、完整处理：通过视图 sys_stat_user_indexes 的 idx_scan 列确认容量大的索引的使用情况，删除使用不多的索引；确认物化视图的使用必要性。如果性能优化需要这些辅助存储结构，参照”业务预期外的增长”处理方式。

紧急处理可能影响使用对象业务的性能。

4. 业务预期外的增长

紧急处理：在可用的其他存储上创建表空间并移动部分表到新表空间；根据业务使用情况，导出部分历史数据到列存表中压缩存储。

紧急处理可能涉及大量 I/O 操作和对表的封锁影响线上业务。

完整处理：扩容存储容量。

2. 在线 WAL 存储容量不足

1. 备机故障未及时恢复

紧急处理：短时间无法恢复备节点的情况下，手动删除复制槽。

紧急处理会造成备节点需要通过重建方式恢复。

完整处理：恢复备节点后检查点回收。

2. 归档失败导致 WAL 无法被正常回收

紧急处理：修改 archvie_command 为空并重载配置，WAL 将归档成功并被检查点回收。

紧急处理会造成归档不连续，影响基于 PITR 的备份能力。

完整处理：修复归档失败的问题，自动归档 WAL 并回收。

3. 检查点预期外延迟导致 WAL 无法被正常回收

紧急处理：cancel 部分造成 I/O 压力的操作确保检查点得到足够的资源。

紧急处理会导致对应的业务请求失败。

完整处理：优化产生意外 I/O 压力的操作。

4. 业务峰值或发生不常见的业务操作

紧急处理：无

完整处理：根据新场景重新评估调整检查点参数或扩容存储。

3. 归档存储容量不足

1. 备份管理任务没有按预期清理旧备份

紧急处理：手动通过 sys_rman 清理备份和归档。

完整处理：修复备份的定时任务。

2. 业务峰值或发生不常见的业务操作

同在线 WAL 的处理

4. 备份存储容量不足
    1. 备份管理任务没有按预期清理旧备份

       同归档的处理
    2. 数据或 WAL 造成

       同数据或 WAL 的处理

2. CPU、内存、网络、IO 用量

   资源用量的处理请参考《风险问题处理 》。

3. 连接数

   连接数增加的处理方式：

   1. 业务变化
      1. 新增业务模块带来的压力增加
      2. 原有业务的规模/用量增加或变化

      紧急处理：如果有，释放部分业务连接池占用但没有实际使用的连接。否则需要调整最大连接数参数。

      紧急调整最大连接数需要重启数据库服务。

      完整处理：根据新业务模式调整连接数总量或各业务的连接池参数。
   2. 性能或资源原因造成处理能力下降

      依赖性能或资源风险/问题的溯因和处理

4. 长事务

   长事务处理方式

   1. 各类客户端工具在非自动提交模式下人为操作后未关闭

      紧急处理：终止对应的连接。

      完整处理：调整空闲连接自动断开参数避免此类问题发生。
   2. 应用的特定实现方式或缺陷造成事务未提交

      紧急处理：取消或终止对应的连接。

      紧急处理可能造成应用异常。

      完整处理：修复缺陷后更新应用版本。
   3. 未结束的两阶段提交事务

      紧急处理：回滚两阶段提交事务。

      紧急处理可能导致业务逻辑的错误。

      完整处理：确认业务的逻辑后处理遗留事务，修复遗留两阶段事务的业务缺陷。

4. 性能或资源原因造成事务阻塞

   依赖性能或资源风险/问题的溯因和处理

5. 事务号使用

   事务号耗尽处理方式：

   1. 清理回收没有按预期启动

      1. 错误的 threshold 相关参数配置
      2. 清理速度慢，autovacuum worker 被占用

      紧急处理：确认 age 最长的对象

      如果不是临时表，手动对对象执行 VACUUM FREEZE VERBOSE，如果还不生效，排除可能的封锁进程后 VACUUM FULL。VACUUM FULL 可能阻塞业务。

      如果是临时表，取消或终止使用临时表的进程。操作可能造成被取消/终止的业务受影响。

      完整处理：参照《处理表索引膨胀 》调整参数，确认长期使用临时表的业务，增加 session 内对临时表的清理操作。

   2. 清理没达到预期效果

      1. 存在长事务

         处理方式参见“长事务”。

      2. 存在残留的复制槽事务号

         紧急处理：短时间无法恢复备节点的情况下，手动删除复制槽。

         紧急处理会造成备节点需要通过重建方式恢复。

         完整处理：恢复备节点后检查点回收。

6. 封锁情况

   封锁超预期处理方式：

   1. 有长时间持有高级别封锁的操作，常见为对象的 DDL 操作

      紧急处理：取消或终止对应的操作。

      紧急处理可能造成应用异常。

      完整处理：确认对应操作的合理性，变更操作方式或时间段。

   2. 长事务导致事务内封锁未及时释放，常见为阻塞并发的数据更新

      处理方式参见“长事务”。

7. license

   license 到期处理方式：

1. 试用版 license 到期

   紧急处理：一般已处于无法启动状态，找到其他可使用的试用 license 替换后启动。

   完整处理：申请新 license 替换使用。

2. 未使用正确的 license

   紧急处理：一般已处于无法启动状态，找到正确的 license 或其他可使用的试用 license 替换后启动。

   完整处理：使用正确的 license 替换。

8. 事务状态访问异常

   正常处理：如果有备份，则使用备份进行恢复；

   紧急处理：如果没有备份，则可以创建一个事务状态都是" 已提交" 的日志文件，具体的创建方法：for i in {1⋯N}; do printf '125'; done > xxxx (xxxx 为报错找不到的日志文件名，N 为 32*block_size)

10.2　数据防丢失

10.2.1　概述

数据防丢失运维的目的是保障项目的 RPO 目标达成，大部分情况下目标是保证数据和故障灾难前保持一致，即 RPO 0。

本部分包含以下主题：

- 影响数据丢失的典型问题及原因概览
- 风险/问题监控、数据采集
- 风险/问题识别
- 风险/问题溯因
- 风险/问题处理

10.2.2　影响数据丢失的典型问题及原因概览

数据丢失问题通常由硬件引发，硬件设备自身的缺陷或者硬件老化都可能会引发数据丢失的问题；另外，软件功能欠缺或缺陷也会引发数据丢失，因此需要关注操作系统、数据库等关键软件厂商发布的产品加固补丁，及时安装对应的软件补丁；最后，不可控的自然灾害也会导致数据丢失。因此，周期性的对数据进行备份是预防数据丢失最重要的手段。

数据丢失类问题有时比较难以察觉，因此掌握一定的运维手段和运维步骤便于快速的定位和解决问题/隐患。

10.2.2.1　经典问题

10.2.2.1.1　基础数据丢失

主要指数据库基本数据丢失或损坏，无法找回。

数据丢失或损坏从物理层面可以分为：

- Data 目录/表空间
- 文件
  - 控制文件
  - 数据文件
  - 密钥文件
  - Redo 文件
- 数据块
  - 表页面
  - 索引页面

数据丢失或损坏从逻辑对象可以分为：

- database 数据库
- 表

当我们的生产数据库因为各种原因导致基础数据丢失后，可以通过后备数据库来接管并提供完整数据。如果没有备数据库，那么只有通过备份数据进行恢复。

10.2.2.1.2　备份数据丢失

备份数据是数据安全最后的保证，一旦生产数据库、备数据库都无法提供完整数据之后，只能通过备份数据来尝试恢复完整数据。备份数据分为：

- 逻辑备份
- 物理备份

逻辑备份比较灵活，但很难保证能够恢复完整数据，主要原因有备份时间长，需挑选备份对象，只能恢复到备份点等因素。

一般我们通过物理备份 + 归档 WAL 日志 + 在线 WAL 日志来恢复完整数据，所以，一旦备份数据丢失，或者 WAL 日志丢失，那么无法恢复完整数据。

10.2.2.2　典型原因

10.2.2.2.1　存储故障

存储故障主要考虑：

- 磁盘或闪存驱动器故障
- 磁盘控制器故障
- 存储阵列故障
- 磁盘物理损坏

由于数据是持久化在存储上的，存储故障基本上都会导致数据大范围的丢失。

10.2.2.2.2　数据损坏

此类场景一般存在于数据已经进行了持久化，但是后期发生了数据篡改，与存储也有一定的关系。

数据损坏主要考虑：

- 已修改的块，与 kingbase 期望找到的不同
  1. 物理
     1. 无法识别
     2. 校验错误
     3. 全零
  2. 逻辑
     1. 通过了物理块校验，但逻辑上可能不一致
- 写丢失或误写
  1. 写入 IO 未在持久存储上发生，但返回完成，后续读取返回旧版本，更新数据库其他块
  2. 写入 IO 已完成，但写入其他地方，后续读取返回旧版本

10.2.2.2.3　人为损坏

1. **误删除**

误删除场景有

- 逻辑删除
  1. 通过 SQL 等业务操作逻辑删除或更新了数据，导致数据丢失或损坏。
- 物理删除
  1. 通过命令行例如 rm -rf 物理操作误删除了数据文件或目录。

2. **配置错误**

配置错误场景有

- 配置了错误的数据存储路径。
- 配置了错误的数据库参数导致数据库无法保证数据可靠。
- 配置了错误的操作系统参数导致数据丢失。
- 未配置备份，导致无备份数据。

10.2.2.2.4　软件缺陷

1. **数据库软件缺陷**

数据库软件缺陷一般会由官方定期的发布一些产品补丁，在用户所使用的版本上打入 patch。

已知修复过的的数据库软件缺陷有：

- 执行计划缺陷导致数据丢失。
- 事务隔离性缺陷导致数据损坏或丢失。
- 并发执行缺陷导致临界区问题。
- 重复多次恢复导致数据不一致。
- 未选择同步节点进行故障自动转移。
- 脑裂后自动恢复将数据丢弃。

2. **操作系统软件缺陷**

操作系统软件缺陷，一般也会由 OS 厂商定期发布一补丁，在用户所使用的版本上打入 patch。

已知的一些国内 OS 软件缺陷有：

- Fsync 数据丢失。
- 数据落盘出错。
- 读数据异常
- 网卡传输队列异常。

10.2.2.2.5　不可预防的损坏或灾难

自然或不可抗力因素：

- 断电
- 地震、火灾等

10.2.2.2.6　其他错误

- 系统服务异常终止。
- 自动备份被异常终止。
- 磁盘满。

10.2.3　风险/问题监控、数据采集

日常监控是非常重要的，一方面意外总是会非预期而至，良好的监控可以在意外来临时及时的预警，通知 DBA 等专业人员处理，避免故障进一步蔓延；另一方面是持续的监控可以尽早的发现可能出现的问题，预测事故未来发展的严重程度及可能出现的危机结果，以防范于未然。

10.2.3.1　监控思路

通过两个方面去监控：

- 基础数据
  - 数据库各目录是否完整
  - 数据库核心文件是否缺失
  - 数据库文件块是否异常
  - 数据库运行状态
  - 数据库配置参数
  - 服务器 CPU 状态
  - 服务器内存状态
  - 服务器磁盘状态
- 备份数据
  - 操作系统配置
  - 自动备份任务
  - 备份参数
  - 备份服务器 CPU 状态
  - 备份服务器内存状态
  - 备份磁盘状态

10.2.3.2　具体监控内容

- 监控丢失内容
- 监控服务器硬件指标

10.2.3.3　数据采集

基础数据采集

名称	描述
单机/集群	数据库运行于单机环境还是集群环境
节点物理资源	CPU 架构/内存大小/磁盘剩余空间/磁盘最大读写速度/网络带宽
CPU 型号	CPU 型号以及具体型号名称
操作系统版本	操作系统类型以及系统版本
是否专用机	数据库是否部署在专用机上
是否线上环境	
是否涉密	
数据库版本	数据库版本号
大小写敏感	数据库是否大小写敏感
数据库数据页大小	block_size
通信工具	ssh/sys_securecmdd
是否开启归档	是否启用 wal 归档 (archive_mode 为 on)
是否定期逻辑备份	
是否定期物理备份	
数据总量	数据目录的整体大小
数据日增量	数据目录每日增量的大小
Crontab 服务	操作系统是否安装了 crontab 工具
防火墙状态	防火墙是否处于开启状态

备份数据采集

名称	描述
单机/集群	数据库运行于单机环境还是集群环境
本地备份/远端备份	备份数据存储在本地还是远端服务器
备份节点物理资源	CPU/内存大小/磁盘剩余空间/磁盘最大读写速度/网络带宽
物理备份策略	全量备份周期/增量备份周期/备份集保留策略
是否独立备份服务器	备份磁盘是网络挂载的还是本地硬盘

确定配置项的生效值

配置项	配置项说明	查看方式
wal_level	需要 replica 或 logical 级别，重启生效	ksql 执行“show wal_level;”获取或者查看 kingbase.conf、es_rep.conf 文件确定
archive_mode	需要 on/always 状态，重启生效	ksql 执行“show archive_mode;”获取或者查看 kingbase.conf 文件、es_rep.conf 文件确定
archive_command	sys_backup.sh init 时自动设置	ksql 执行“show archive_command;”获取或者查看 kingbase.conf 文件、es_rep.conf 文件确定
data_directory	数据库 data 目录	ksql 执行“show data_directory;”获取

1. 收集所有节点（包括主备节点、存储节点）的 kbbr_repo/ sys_rman.conf 配置信息，以及存储节点内 kbbr_repo/backup/kingbase/backup.info、kbbr_repo/archive/kingbase/archive.info 文件内容信息
2. 收集 sys_rman info 命令（参考：/home/kingbase/cluster/project/custer/kingbase/bin/sys_rman –config=/home/kingbase/kbbr_repo/sys_rman.conf --stanza=kingbase info）的输出，下图为 info 命令正常的输出示例

```
[kingbase@kingbase ]$ /home/kingbase/opt/Kingbase/ES/ /Server/bin/sys_rman --config /home/kingbase/opt/Kingbase/ES/ /kbbr_repo
/sys_rman.conf --stanza=kingbase info
stanza: kingbase
    status: ok
    cipher: none

    db (current)
        wal archive min/max (V ): 000000010000000000000005/000000010000000000000007

        full backup: 20220613-175740F
            timestamp start/stop: 2022-06-13 17:57:40 / 2022-06-13 17:57:56
            wal start/stop: 000000010000000000000006 / 000000010000000000000006
            database size: 107.8MB, database backup size: 107.8MB
            repo1: backup set size: 13.4MB, backup size: 13.4MB
```

3. 收集 sys_rman check 命令（参考：/home/kingbase/cluster/project/custer/kingbase/bin/sys_rman –config=/home/kingbase/kbbr_repo/sys_rman.conf --stanza=kingbase check）的输出，下图为 check 命令正常的输出示

例

```
[kingbase@kingbase ]$ /home/kingbase/opt/Kingbase/ES/Server/bin/sys_rman --config /home/kingbase/opt/Kingbase/ES/kbbr_repo
/sys_rman.conf --stanza=kingbase check
2022-06-13 17:58:33.843 P00   INFO: check command begin 2.27: --config=/home/kingbase/opt/Kingbase/ES/kbbr_repo/sys_rman.conf -
-exec-id=16727-b3b68628 --kb1-path=/home/kingbase/opt/Kingbase/ES/data --kb1-port=54321 --kb1-user=system --log-level-console=i
nfo --log-level-file=info --log-path=/home/kingbase/opt/Kingbase/ES/Server/log --log-subprocess --repo1-path=/home/kingbase/opt
/Kingbase/ES/kbbr_repo --stanza=kingbase
2022-06-13 17:58:34.953 P00   INFO: check repo1 configuration (primary)
2022-06-13 17:58:35.165 P00   INFO: check repo1 archive for WAL (primary)
2022-06-13 17:58:35.367 P00   INFO: WAL segment 000000010000000000000007 successfully archived to '/home/kingbase/opt/Kingbase/ES/
/kbbr_repo/archive/kingbase/12-1/0000000100000000/000000010000000000000007-e162e32da79c71a90c35252eb647fdae91d3e1ff.gz' on repo1
2022-06-13 17:58:35.367 P00   INFO: check command end: completed successfully (1531ms)
```

4. 收集数据库服务器日志内的 archive-push 相关日志，查看归档是否成功，参考命令：grep -rn “archive-push” data/sys_log
5. 收集数据库服务器的控制信息，参考命令：sys_controldata -D data
6. 如果是 sys_rman restore 有问题，需要额外收集以下信息

配置项	说明
恢复后，启动前的 data 目录内的 backup.label	该文件指明了还原的基础数据信息
恢复后，启动前的 data 目录内的 kingbase.auto.conf	该文件指明了启动后的恢复目标点等信息
启动后的数据库服务器日志	

10.2.4　风险/问题识别

10.2.4.1　基础数据分析识别

名称	分析
单机/集群	单机出现单点故障时，无法继续提供服务，且有可能丢失数据。
节点物理资源	物理资源是否能继续维持良好的数据库服务环境
CPU 型号	不同 CPU 型号兼容性有差异
操作系统版本	不同操作系统、不同版本之间存在兼容性差异
是否专用机	
是否线上环境	
是否涉密	
数据库版本	不同的数据库版本支持的功能有差异

-续表-

名称	分析
大小写敏感	
数据库数据页大小	block_size
通信工具	ssh/sys_securecmdd
是否开启归档	物理备份必须开启归档
是否定期逻辑备份	减小数据丢失风险
是否定期物理备份	减小数据丢失风险
数据总量	全量数据大小影响物理全量备份时间、备份集保留策略
数据日增量	增量数据大小影响物理增量备份时间、增量备份周期等
crontab 服务	物理备份策略依赖 crontab 服务
防火墙状态	独立备份节点和数据节点之间需要互联

10.2.4.2　备份数据分析识别

sys_rman check 命令（参考：/home/kingbase/cluster/project/custer/kingbase/bin/sys_rman －config=/home/kingbase/kbbr_repo/sys_rman.conf --stanza=kingbase check）的输出没有报错信息，那么 sys_rman 的配置和归档是正常的，如下图所示：

```
[kingbase@kingbase ]$ /home/kingbase/opt/Kingbase/ES/ Server/bin/sys_rman --config /home/kingbase/opt/Kingbase/ES/ kbbr_repo
/sys_rman.conf --stanza=kingbase check
2022-06-13 17:58:33.843 P00   INFO: check command begin 2.27: --config=/home/kingbase/opt/Kingbase/ES/ kbbr_repo/sys_rman.conf -
-exec-id=16727-b3b68628 --kb1-path=/home/kingbase/opt/Kingbase/ES/ data --kb1-port=54321 --kb1-user=system --log-level-console=i
nfo --log-level-file=info --log-path=/home/kingbase/opt/Kingbase/ES/ Server/log --log-subprocess --repo1-path=/home/kingbase/opt
/Kingbase/ES/ kbbr_repo --stanza=kingbase
2022-06-13 17:58:34.953 P00   INFO: check repo1 configuration (primary)
2022-06-13 17:58:35.165 P00   INFO: check repo1 archive for WAL (primary)
2022-06-13 17:58:35.367 P00   INFO: WAL segment 000000010000000000000007 successfully archived to '/home/kingbase/opt/Kingbase/ES/
/kbbr_repo/archive/kingbase/12-1/0000000100000000/000000010000000000000007-e162e32da79c71a90c35252eb647fdae91d3e1ff.gz' on repo1
2022-06-13 17:58:35.367 P00   INFO: check command end: completed successfully (1531ms)
```

10.2.5　风险/问题溯因

10.2.5.1　基础数据访问异常

用户某张表的访问报错，提示 block 块出现问题。

直接原因：数据库访问特定对象故障。

间接原因：是否仅有此对象故障，是否出现大面积坏块。

根本原因: 可能由于操作系统层面、或磁盘问题导致写入磁盘异常，数据丢失或损坏，需要根据现象排查多重日志，最后发现根本原因是硬盘出现坏道或其他损坏，需要进行返厂维修。

10.2.5.2　物理备份未配置

对下图的错误进行风险溯源

```
check sys_rman configuration      ERROR      sys_rman is not configured
```

直接原因：没有配置 sys_rman

间接原因：没有配置 sys_rman

根本原因：没有配置 sys_rman，配置 sys_rman 物理备份，具体操作方式参考《备份与恢复手册》

10.2.5.3　未归档 WAL 累积过多

```
ERROR RMAN-00500: Non archived WAL files more than 1024 M , Please check the user-guide
```

直接原因：物理备份工具 sys_rman 在开启每一次物理备份之前，检查数据库中的未归档 WAL 日志的容量，如果超过了设置的阈值，报错退出。

间接原因：物理备份开始时，将执行一次 sys_switch_wal() 来强制产生新的 WAL 文件，并在备份结束时等待此 WAL 文件实际归档到备份目录中。如果未发现此 WAL 文件，备份过程报错并终止。

根本原因：因为物理备份工具支持 PITR，需要持续的 WAL 归档文件，因此工具预先对未归档的 WAL 文件的总容量进行检查，避免在异常情况下开启不完整的物理备份。

10.2.5.4　WAL 日志归档失败

```
ERROR: [082]: WAL segment 000000010000000000000001 was not archived before the 60000ms timeout
HINT: check the archive_command to ensure that all options are correct (especially --stanza).
HINT: check the Kingbase server log for errors.
```

直接原因：WAL 文件没有在设置的时间范围内归档到备份工具指定的归档目录中。

间接原因：

1. kingbase 的 archive_mode 或 archive_command 配置有误
2. kingbase 处于业务繁忙阶段，未能及时归档 WAL 日志

根本原因：

因为备份工具 sys_rman 支持 PITR 功能，需要持续的、不间断的 WAL 日志文件。

kingbase 在归档 WAL 时，顺序性的、串行的逐个文件进行归档。

确保 archive_mode 为 on/always，确保 archive_command 由 sys_rman 工具的 archive_push 命令构成且全路径指定工具 sys_rman 和配置文件 sys_rman.conf；

不在业务繁忙阶段进行物理备份。

10.2.5.5　data 目录设置不当导致备份失败

```
ERROR: [058]: version '12' and path '/home/test 0916-bufix/bin/./data'
queried from cluster do not match version '12' and
'/home/test/ '0916-bufix/bin/data' read from
'/home/test/ '0916-bufix/bin/data/global/sys_control'
```

直接原因：如图所示，kingbase 启动时指定了相对目录，导致物理备份不能正常启动。

间接原因：物理备份工具 sys_rman 对比从配置文件获得 kingbase 的 DATA 目录和从数据库的 SQL 接口获取到的 DATA 目录，如果不一致，就报错退出。

根本原因：因为物理备份具有特殊性，备份产生的备份集作为存档，被使用的场景和主机都具有不确定性。物理备份集支持在不同的物理主机或者不同的原始目录进行还原，因此不支持使用相对目录启动数据库实例。

10.2.5.6　使用索引时出现 index …contains unexpected zero page at block …故障

直接原因：

在使用索引时，读入内存中的页面为全零页面。

间接原因：

由于硬件系统的原因导致写入外存中的页面或者是读入内存中的页面为全零页面，下面将通过实际举例展示如何查看全零页面。

1. 查看目标索引是否成功创建

```
test=# \di
```

2. 创建并使用 pageinspect 插件（插件的具体使用方法详见 KingbaseES 插件参考手册 pageinspect）查看内存 buff 中读入的页面是否为全零页面

```
test=# CREATE EXTENSION pageinspect;

SELECT get_raw_page::text FROM get_raw_page('index', 1);
```

其中’index’为目标索引，1 为要查看的内存中的页号，页面号的值可以在错误信息中找到。

3. 如果上述步骤读入内存中的页面是全零页面，那么记录下页面号，找到目标索引的 relfilenode 再通过 relfilenode 找到目标索引在 data 目录下的位置，然后使用 Linux 命令 hexdump 查看外存中索引文件中的目标页面并确认目标页面是否在写入外存时就已经发生损坏变成全零页面

以下将分步展示查看外存页面的步骤：

3.1. 计算段号 segNo

segNo = pageNo * pageSize / segSize（注：结果向下取整）

在上述公式中 segNo 代表着段号，在 KingbaseES 中数据是存储在一个或多个文件中的，每一个保存数据的文件被称为段，段号就代表了存放页面的文件的序号。

pageNo 代表了页面号，页面号在报错信息中记录。

pageSize 代表了页面大小，可以通过下面的命令获取它的值：

```
test=# show block_size;

block_size

------------

8192
```

segSize 代表的是段容量，每个段的容量大小是固定的 1048576KB。

3.2. 计算起始地址 start

start = pageNo * pageSize - segNo * segSize

由于在外存中数据是通过文件的形式存储的，所以你需要使用 hexdump 命令查看目标文件，但是在一个文件中又存储了大量的页面，为了只看你关心的目标页面，你需要计算该页面的起始地址。

3.3. 找到目标文件

```
test=# SELECT relfilenode,relname FROM pg_class;
```

在得到必要的参数后下一步就是需要找到你要查看的文件，文件在路径 bin/base/目标数据库 oid/ 下，使用以上示例命令，通过观察 relname 可以找到你要查看的索引的 relfilenode。

文件的命名方式为 relfilenode.segNo，但是有一种特殊情况就是当段号为 0 时文件仅以 relfilenode 命名。

例如文件 13521 就是 relfilenode 为 13521 的第 0 段文件，而文件 13521.2 就是 relfilenode 为 13521 的第 2 段文件。

3.4. 查看目标文件

查看文件需要使用 Linux 的 hexdump 命令，命令的格式为：hexdump -s start -n pageSize filename

在上述命令中 <-s start> 代表了文件的起始地址。

<-n pageSize> 代表了输出的内容的大小，由于你只需要看一个页面所以只需要输出一个页面的大小就可以。

示例：

```
hexdump -s 8192 -n 8192 16426
```

上述示例的含义为我要查看文件 16426 从 8192 地址开始往后 8KB 的内容。

4. 最后确认上述结果后在 kingbase.conf 中配置 trace_mdwrite 参数为 on，trace_mdwrite_level 参数为 all，重新创建索引再查看 sys_log 日志

trace_mdwrite_level 参数有三个级别分别为 valid、non-zero、all，代表含义分别为追踪无效页面，追踪全零页面，追踪无效页面和全零页面。

将 trace_mdwrite_level 参数置为 all 则可以保证创建索引时系统写入的页面的页头既有效又不为全零，如果二者之中有一个条件不符合，那么在 sys_log 文件中将存在相应的日志记录。

在查看日志发现没有相应日志记录之后再次执行上述步骤，确定是在写入外存还是在读入内存时发生页面出现全零现象。

根本原因：

存储数据的磁盘受损，应当考虑更换磁盘重启数据库服务器，重新创建索引，再次尝试使用。

为避免页面受损出现全零页面应当保证做出以下要求：

1. 使用可靠的数据存储磁盘。
2. 完善灾备系统对数据做好充分地备份，以减少甚至避免硬件故障产生的损失。

10.2.6 风险/问题处理

10.2.6.1 影响分析

在数据丢失后首先要做的是第一时间找回丢失的数据，降低应用访问异常时间。

具体参见《备份与恢复手册 》。

其次是保留可能的现场，为后续追溯原因提供环境。

10.2.6.2 常见数据故障及问题处理

sys_rman 常见备份还原问题处理，请参见公司官网手册：https://help.kingbase.com.cn

10.3　数据库性能

本部分包含以下主题：

- 风险问题监控数据采集
- 风险问题识别
- 风险问题溯因
- 风险问题处理
- 常用的性能诊断工具

10.3.1　风险/问题监控、数据采集

监控指标：CPU、内存、网络、IO 用量、磁盘

监控频率：每 1 分钟

保留时间：至少一周

建议工具：nmon

监控指标：KWR、KSH 中包括的数据库性能数据

监控频率：每小时一个快照（KSH 不用设置快照时间）

保留时间：至少一周

数据库监控工具：KWR、KSH

操作系统监控方法参见《KingbaseES 数据库性能调优指南 》中“操作系统资源分析”章节，nmon 工具的使用方法可以直接参见本文nmon 章节。

数据库监控方法参见《KingbaseES 数据库性能调优指南 》中“自动负载信息库 SYS_KWR”以及“活跃会话历史报告 SYS_KSH”章节。

10.3.2　风险/问题识别

针对上文中收集到的数据，使用分析工具生成趋势、报告：

- 操作系统资源数据分析：nmon analyzer
- 数据库资源数据分析：KWR、KSH 报告

操作系统数据分析方法参考《KingbaseES 数据库性能调优指南》中“CPU 资源分析”和“内存资源分析”、“IO 资源分析”、“网络资源分析”章节。

数据库资源数据分析参考《KingbaseES 数据库性能调优指南》中“KWR 报告”、“KSH 报告”章节。

1. 可以通过监控项值的阈值和生成的报告识别风险/问题，具体如下表：

分类	子类	风险/问题识别
操作系统	CPU	利用率超过 70% 利用率快速上涨 % System 过高
	IO	利用率超过 50%
	网络	利用率超过 50% 网络存在丢包和报错情况
	内存	内存空闲少于 10% 或存在 swap 情况
数据库	等待事件	等待时间超过 5%
	事务	回滚比例超过 20%
	事务	事务执行时间过长
	SQL	SQL 执行时间过长

10.3.3　风险/问题溯因

10.3.3.1　信息收集

收集已有的性能数据或执行收集操作获得最新数据并生成报告，包括：

1. CPU、内存、网络、IO 用量、磁盘监控信息
2. KWR、KSH 中包括的数据库性能数据

在问题发生过程中或复现过程中收集：

1. 火焰图热点函数信息

火焰图的收集方法参见火焰图 ；分析方法请咨询 KingbaseES 的支持人员。

10.3.3.2　溯因

针对风险/问题的溯因方式如下表：

分类	子类	风险/问题识别	溯因
操作系统	CPU	利用率超过 70% 利用率快速上涨 % System	重点关注火焰图和 KWR 报告中的 TOP SQL 部分
	IO	利用率超过 50%	pidstat/iotop 查看操作系统 io 分布 KWR 报告，重点关注 IO Profile
	网络	利用率超过 50% 网络存在丢包和报错情况	通过 nethogs/iftop 查看操作系统网络分布 返回行数与网络资源消耗存在直接关系，可通过查看 KWR 报告中（根据返回行数的 TOP SQL）报告排查问题
	内存	内存空闲少于 10% 或存在 swap 情况	top 查看内存使用以及数据库内存配置
数据库	等待事件	等待事件超过 5%	查看 KWR、KDDM 报告中等待事件相关部分
	事务	回滚比例超过 20%	分析运行日志与应用
	事务	事务执行时间过长	分析运行日志确认事务中执行时间长的 SQL，参考《KingbaseES 数据库 SQL 调优指南》优化 SQL
	SQL	SQL 执行时间过长	分析运行日志确认 SQL，参考《KingbaseES 数据库 SQL 调优指南》优化 SQL

具体分析方法参见《KingbaseES 数据库性能调优指南 》的“第二部分性能诊断”章节以及《KingbaseES 数据库 SQL 调优指南 》的“4.3 分析执行计划”章节。

10.3.4　风险/问题处理

10.3.4.1　处理方法

处理方法参见《KingbaseES 数据库性能调优指南 》的“第三部分性能优化”章节以及《KingbaseES 数据库 SQL 调优指南 》的“第 5 章 SQL 优化手段”章节。

在性能调优到最佳状态，风险/问题仍然存在时，可以通过扩展瓶颈资源方式处理，例如：

1. 纵向扩展：例如扩容存储、增加内存容量、增加更高效的存储、增加处理器核数
2. 横向扩展：例如增加只读副本并均衡负载到副本

10.3.4.2　常见风险问题处理

慢 SQL 优化

执行计划分析 + 火焰图，参考《KingbaseES 数据库性能调优指南 》

并发场景吞吐量低

收集问题时间段的 nmon、KWR 报告、火焰图，参考《KingbaseES 数据库性能调优指南 》

响应时间持续上升

在响应时间指标对比明显的两个时刻下，分别收集 nmon、KWR 报告、火焰图，分析关键指标的差异，尤其是时间模型的数据差异。

10.3.5　常用的性能诊断工具

10.3.5.1　nmon

nmon 是一种在 AIX 与各种 Linux 操作系统上广泛使用的监控与分析工具。它能在系统运行过程中实时地捕捉系统资源（CPU、IO、内存、网络等）的使用情况，记录的信息比较全面。

在通过 nmon 完成采样后，可以通过 nmon 的 spreadsheet analyze 来找出当时的性能监控数据，并下载 nmon analyzer，在 excel 中离线查看当时的性能监控数据。

工具获取

nmon：nmon for Linux | Site / Download (sourceforge.net)

nmon analyzer：nmon for Linux | Site / Nmon-Analyser (sourceforge.net)

交互模式

直接通过./nmon 启动。

```
nmon-16k──────────────────────Hostname=DESKTOP-MBR3DRefresh= 2secs ───11:46.19──────────────

   ------------------------------
    _ __   _ __ ___    ___   _ __
   | '_ \| '_ ` _ \ / _ \| '_ \          For help type H or ...
   | | | | | | | | (_) | | | |           nmon -?  - hint
   |_| |_|_| |_| |_|\___/|_| |_|           nmon -h  - full details
                                          To stop nmon type q to Quit
   ------------------------------




   Use these keys to toggle statistics on/off:
     c = CPU         l = CPU Long-term     - = Faster screen updates
     C = " WideView  U = Utilisation      + = Slower screen updates
     m = Memory      V = Virtual memory    j = File Systems
     d = Disks       n = Network           . = only busy disks/procs
     r = Resource    N = NFS              h = more options
     k = Kernel      t = Top-processes     q = Quit
```

通过 help 中的快捷键选择需要实时显示的信息

```
nmon-16k──────────────────────Hostname=DESKTOP-MBR3DRefresh= 2secs ───11:46.46──────
 CPU Utilisation ──────────────────────────────────────────────────────────────────
---------------------------+-------------------------------------------------+
CPU User%  Sys% Wait%  Idle|0          |25         |50         |75       100|
  1   0.0   0.0   0.0 100.0|>                                                  |
  2   0.0   0.0   0.0 100.0|>                                                  |
  3   0.0   0.0   0.0 100.0|>                                                  |
  4   0.0   0.0   0.0 100.0|>                                                  |
  5   0.0   0.0   0.0 100.0|>                                                  |
  6   0.0   0.0   0.0 100.0|>                                                  |
  7   0.0   0.0   0.0 100.0|>                                                  |
  8   0.0   0.0   0.0 100.0|>                                                  |
---------------------------+-------------------------------------------------+
Avg   0.0   0.0   0.0 100.0|>                                                  |
---------------------------+-------------------------------------------------+
 Network I/O ──────────────────────────────────────────────────────────────────────
I/F Name Recv=KB/s Trans=KB/s packin packout insize outsize Peak->Recv Trans
      lo      0.4       0.4       1.0    1.0   380.0  380.0       59.4     59.4
   bond0      0.0       0.0       0.0    0.0     0.0    0.0        0.0      0.0
  dummy0      0.0       0.0       0.0    0.0     0.0    0.0        0.0      0.0
   tunl0      0.0       0.0       0.0    0.0     0.0    0.0        0.0      0.0
    sit0      0.0       0.0       0.0    0.0     0.0    0.0        0.0      0.0
    eth0      0.0       0.0       0.0    0.0     0.0    0.0        0.4      0.2
 Network Error Counters ───────────────────────────────────────────────────────────
I/F Name iErrors iDrop iOverrun iFrame oErrors   oDrop oOverrun oCarrier oColls
      lo       0      0       0      0       0      0       0       0      0
   bond0       0      0       0      0       0      0       0       0      0
  dummy0       0      0       0      0       0      0       0       0      0
   tunl0       0      0       0      0       0      0       0       0      0
    sit0       0      0       0      0       0      0       0       0      0
    eth0       0      0       0      0       0      0       0       0      0
 Disk I/O ──/proc/diskstats────mostly in KB/s─────Warning:contains duplicates────────
DiskName Busy  Read WriteKB|0          |25         |50         |75       100|
sda        0%    0.0    0.0|>                                                  |
sdb        0%    0.0    0.0|>                                                  |
Totals Read-MB/s=0.0      Writes-MB/s=0.0      Transfers/sec=0.0
──────────────────────────────────────────────────────────────────────────────────
```

后台收集模式

后台收集模式是通过采样的方式收集一段时间内的操作系统资源使用情况，并输出一个特定的文件，并可利用 excel 分析工具（nmon analyser）进行数据的统计分析。

```
Hint for nmon version 16k
        Full Help Info : nmon -h

        On-screen Stats: nmon
        Data Collection: nmon -f [-s <seconds>] [-c <count>] [-t|-T]
        Capacity Plan  : nmon -x
Interactive-Mode:
        Read the Welcome screen & at any time type: "h" for more help
        Type "q" to exit nmon

For Data-Collect-Mode
        -f            Must be the first option on the line (switches off interactive mode)
                      Saves data to a CSV Spreadsheet format .nmon file in then local directory
                      Note: -f sets a defaults -s300 -c288    which you can then modify
        Further Data Collection Options:
        -s <seconds>  time between data snapshots
        -c <count>    of snapshots before exiting
        -t            Includes Top Processes stats (-T also collects command arguments)
        -x            Capacity Planning=15 min snapshots for 1 day. (nmon -ft -s 900 -c 96)
---- End of Hints
```

使用示例：

- 信息收集：./nmon -f -t -s 10 -c 100 -m /home/kingbase/nmon/
- 可视化数据：通过 nmon analyzer 导入 nmon 文件即可生成可视化报告

10.3.5.2　KWR、KSH 报告

参考《KingbaseES 数据库性能调优指南 》中 KWR 报告章节以及 KSH 报告章节。

10.3.5.3　sys_stat_activity 视图

参考 sys_stat_activity 视图。

10.3.5.4　sys_stat_statement 插件

参考 sys_stat_statements 插件手册。

10.3.5.5　火焰图

火焰图使用步骤：

1）下载工具，上传到数据库服务器

```
git clone https://github.com/brendangregg/FlameGraph.git # or download it from github
```

2）用 root 用户，进入到工具文件夹

```
cd FlameGraph
```

3）选择想要采集的时间启动 perf 记录数据，

```
perf record -F 99 -a -g
```

4）等到想要采集的数据足够多时（如 1 分钟左右），ctrl+c 结束 perf record 命令，可以看到 perf.data 就是生成的记录结果

5）生成火焰图

```
perf script | ./stackcollapse-perf.pl > out.perf-folded
./flamegraph.pl out.perf-folded > perf-kernel.svg
```

把火焰图拍照或导出。

10.4　常见命令

本部分包含以下主题：

- 数据库启停
- 常用集群相关操作
- 常用 SQL 命令
- 数据库常用命令工具
- 操作系统常用命令
- 软件包安装及升级管理

10.4.1　数据库启停

10.4.1.1　单机环境

命令介绍

- 启动数据库，通过 sys_ctl 命令启动数据库

  ```
  sys_ctl start -D /home/kingbase/KingbaseES/V9/data
  ```

- 关闭数据库

  通过 sys_ctl -D data 路径 stop 命令关闭数据库，如下：

```
sys_ctl  -D  /home/kingbase/KingbaseES/V9/data  stop
```

若关闭成功，“waiting for server to shut down …. Down”的提示。通过 ps -ef|grep kingbase 不能查看到 kingbase 对应的进程。

- 重启数据库

通过 sys_ctl -D data 路径 restart 重启数据库，如下：

```
sys_ctl  -D  /home/kingbase/KingbaseES/V9/data  restart
```

若重启成功，有“waiting for server to shut down …done”以及“server started”的提示。

sys_controldata -D data 路径，查看 Time of latest checkpoint: 行最新时间。

数据库启动后，可通过 ps -ef | grep kingbase 查看启动数据库启动是否成功，如下图：

```
[kingbase@test11 pkg]$ ps -ef | grep kingbase
root     17779 22710  0 19:55 ?        00:00:00 sshd: kingbase [priv]
root     17782 22710  0 19:55 ?        00:00:00 sshd: kingbase [priv]
kingbase 17783 17779  0 19:55 ?        00:00:00 sshd: kingbase@pts/2
kingbase 17795 17782  0 19:55 ?        00:00:00 sshd: kingbase@notty
kingbase 17798 17795  0 19:55 ?        00:00:00 /usr/libexec/openssh/sftp-server
kingbase 17810 17783  0 19:55 pts/2    00:00:00 -bash
kingbase 22687     1  0 20:02 ?        00:00:00 /home/kingbase/cluster/test1/kingbase/bin/kingbase -D /home/kingbase/cluster/test1/kingbase/data
kingbase 22688 22687  0 20:02 ?        00:00:00 kingbase: logger
kingbase 23223     1  0 20:02 ?        00:00:00 /home/kingbase/cluster/test1/kingbase/bin/repmgrd -d -v -f /home/kingbase/cluster/test1/kingbase/bin/../etc/repmgr.conf
kingbase 23400     1  0 20:02 ?        00:00:00 /home/kingbase/cluster/test1/kingbase/bin/kbha -A daemon -f /home/kingbase/cluster/test1/kingbase/bin/../etc/repmgr.conf
kingbase 23875 22687  0 20:04 ?        00:00:00 kingbase: checkpointer
kingbase 23876 22687  0 20:04 ?        00:00:00 kingbase: background writer
kingbase 23877 22687  0 20:04 ?        00:00:00 kingbase: walwriter
kingbase 23878 22687  0 20:04 ?        00:00:00 kingbase: autovacuum launcher
kingbase 23879 22687  0 20:04 ?        00:00:00 kingbase: archiver
kingbase 23880 22687  0 20:04 ?        00:00:00 kingbase: stats collector
kingbase 23881 22687  0 20:04 ?        00:00:00 kingbase: kwr collector
kingbase 23882 22687  0 20:04 ?        00:00:00 kingbase: ksh writer
kingbase 23883 22687  0 20:04 ?        00:00:00 kingbase: ksh collector
kingbase 23884 22687  0 20:04 ?        00:00:00 kingbase: logical replication launcher
kingbase 23887 22687  0 20:04 ?        00:00:00 kingbase: esrep esrep 10.10.8.11(21634) idle
kingbase 23888 17810  0 20:04 pts/2    00:00:00 ps -ef
kingbase 23889 17810  0 20:04 pts/2    00:00:00 grep --color=auto kingbase
```

若启动成功，能够看到有“xxx/bin/kingbase -D …”，该进程是必不可少的，还有类似“kingbase: logger”的辅助进程。以及如图所示 checkpointer 检查点进程 walwriter，background writer 后台进程等，这些后台进程是数据库正常活动必不可少的。这里一定要体现出来。

其中启动命令示例中的/home/kingbase/KingbaseES/V9/data 为数据库 data 路径，每个数据库的 data 路径可能不一致。data 路径的查看方法：

- 通过进程（已启动情况下）

```
ps -ef | grep kingbase
```

备注:

含 xxx/Server/bin/kingbase -D …一行，-D 后为 data 所在绝对路径或者相对于启动路径的相对路径

- 通过参数（已启动情况下）

```
ksql -USYSTEM -d TEST -c "show data_directory;"
```

- 通过文件（未启动情况下）

```
find / -name kingbase.conf
```

备注:

查找含 data 路径的，如有多个，可根据控制文件确定

启动过程常见故障及问题处理

在关闭数据库的过程中，一直在提示“wait for server to shutdown…”

原因：

1. 数据库此时有可能在将 buffer 中的脏数据库刷盘，此时是正常现象。“wait for server to shutdown…”提示一段时间后，数据库成功关闭
2. 若“wait for server to shutdown…”提示后，一直没有关闭数据库，此时数据库可能有异常（如一个事务一直没有结束等），请联系运维人员。

10.4.1.2　集群环境

具体命令介绍

KingbaseES 集群的启停管理通过 sys_monitor.sh 脚本实现，此脚本可以集群任意节点执行。

1. sys_monitor.sh 脚本使用

```
[kingbase@node101 bin]$ pwd
/home/kingbase/cluster/R6HA/kha/kingbase/bin

[kingbase@node101 bin]$ ./sys_monitor.sh --help
Usage: ./sys_monitor.sh {start|stop|restart|stoplocal|set [--restart]|node_name sync\potential\async|sync_
nodes\potential_nodes\async_nodes "nodes name"|switch_synchronous [sync\quorum\async\all\custom]|change_
password user password}
```

2. 启动集群

```
[kingbase@node101 bin]$ ./sys_monitor.sh start
```

3. 停止集群

```
[kingbase@node101 bin]$ ./sys_monitor.sh stop
```

4. 停止本节点的服务（包括集群管理和数据库服务，一般用于备库）

```
[kingbase@node101 bin]$ ./sys_monitor.sh stoplocal
```

5. 重新启动集群

```
[kingbase@node101 bin]$ ./sys_monitor.sh restart
```

6. 配置单个节点的同步类型，该方法会将新节点追加到对应的同步类型参数末尾。

```
[kingbase@node101 bin]$ ./sys_monitor.sh node1 sync
```

7. 配置多个节点的同步类型，该方法将覆盖原同步类型参数中已经存在的节点。

```
[kingbase@node101 bin]$ ./sys_monitor.sh sync_nodes "node1,node2,node3"
```

8. 切换 synchronous 模式，可在 sync、quorum、async、all、custom 之间切换，当从其他模式切换为 custom 模式后，应继续配置各个节点的同步类型。（注：两地三中心集群不支持配置为 custom 模式）

```
[kingbase@node101 bin]$ ./sys_monitor.sh switch_synchronous custom
```

启动过程常见故障及问题处理

1. **集群启动**“incorrect command permissions for the virtual ip”**故障**

故障现象：

在使用 sys_monitor.sh start 启动集群时出现以下故障

```
server started
2021-03-01 13:30:12 execute to start DB on "[192.168.7.243]" success, connect to check it.
2021-03-01 13:30:13 DB on "[192.168.7.243]" start success.
2021-03-01 13:30:13 Try to ping trusted_servers on host 192.168.7.248 ...
2021-03-01 13:30:15 Try to ping trusted_servers on host 192.168.7.243 ...
2021-03-01 13:30:18 begin to start DB on "[192.168.7.248]".
incorrect command permissions for the virtual ip.
waiting for server to start.... done
server started
2021-03-01 13:30:19 execute to start DB on "[192.168.7.248]" success, connect to check it.
2021-03-01 13:30:20 DB on "[192.168.7.248]" start success.
ERROR: No execute permission for "/home/kingbase/cluster/R6C5/R6C5R//kingbase/bin/arping"
incorrect command permissions for the virtual ip.
2021-03-01 13:30:20 There is no primary DB running, will do nothing and exit.
```

问题分析：

对于集群主库 vip 地址的加载，需要使用 arping 工具执行 arp 缓存的清理和 ip 地址占用的测试，如果 arping 工具出现版本不兼容及权限问题，将导致在主库加载 vip 地址失败。

- **查看** arping **配置** (repmgr.conf)**：**

```
arping_path='/home/kingbase/cluster/R6C5/R6C5R/kingbase/bin'
```

- **查看** arping **版本信息：**

```
[kingbase@node101 bin]$ ./arping -V
arping utility, iputils-s20210301kb
```

- **查看 arping 权限**

```
[kingbase@node1 bin]$ ls -lh arping
 -rwxr-xr-x 1 kingbase root 11K Nov 5 2021 arping
```

备注:

对于通用机环境 arping 属主应该是 root 用户，由于需要 kingbase 用户可以执行此工具，因此需要增加 setuid。

问题解决：

配置 arping 权限

```
[root@node3 ~]# cd /home/kingbase/cluster/R6C5/R6C5R//kingbase/bin
[root@node3 bin]# chown -R root.root arping
[root@node3 bin]# chmod u+s arping
[root@node3 bin]# ls -lh arping
-rwsr-xr-x 1 root root 11K Nov 5 2021 arping
```

配置 arping 权限后，重新执行 sys_monitor.sh 启动集群，问题解决。

问题总结：

对于 kingbaseES 集群使用数据库系统自带的 arping 软件包，一般不会出现版本不兼容的问题；对于 arping 工具的属主应该是 root，不是 kingbase 用户，但为了 kingbase 用户也能执行 arping，必须配置 arping 的 setuid 权限。

2. **备节点“node record is inactive”案例**

故障现象：

集群在启动后，查看集群节点状态信息，如下所示：standby 节点状态为“inactive”。

```
 ID | Name   | Role    | Status    | Upstream | repmgrd    | PID  | Paused? | Upstream last seen
----+--------+---------+-----------+----------+------------+------+---------+--------------------
 1  | node248 | standby | ! running | node243  | not running | n/a  | n/a     | n/a
 3  | node243 | primary | * running |          | running    | 6036 | no      | n/a

WARNING: following issues were detected
  - node "node248" (ID: 1) is running but the repmgr node record is inactive
```

问题分析：

对于 repmgr 集群中，节点为“inactive”状态时，需要从以下几个方面进行检查分析：

1). 查看数据库进程启动是否正常（主备库的状态）

2). 查看数据库 sys_log 日志，是否有复制槽（slot）等故障信息。

3). 查看数据库 sys_log 日志，是否有 timeline 等故障信息。

通过对 standby 节点以上信息检查，没有发现错误。

问题解决：

在备库执行“repmgr standby register --force”后，查看集群节点状态，standby 节点状态恢复正常。

问题总结：

对于 repmgr 集群“node record is inactive”问题，一般是因为 standby 节点数据库服务或流复制出现问题，节点注册集群失败，导致集群状态显示为“inactive”，根据错误日志分析，一般处理的思路可以按照以下步骤:

1). 执行‘repmgr standby register -force’，问题解决，无需执行下一步。

2). 执行‘repmgr node rejoin’，问题解决，无需执行下一步。

3). 执行‘repmgr standby clone’。

3. **集群启动后 repmgr 工具执行出现“ - data-directory-config”check 故障**

故障现象：

在执行‘repmgr standby switchover’时，出现以下故障：

```
ERROR: “data_directory” parameter in repmgr.conf on “” is incorrectly configured
```

问题分析：

1). 检查 repmgr.conf 文件中 data_directory 变量的配置。

2). 使用‘repmgr node check --data-directory-config‘检查变量。

3). 检查数据库进程启动 -D 指定数据库文件存储路径，发现数据库启动路径采用了相对路径。

问题解决：

重新启动集群，数据库服务启动时 -D 后路径应使用绝对路径，问题解决。

问题总结：

在使用 sys_ctl 手工启动集群主备节点的数据库服务时，应该在‘-D’参数后使用绝对路径，不要使用相对路径，避免在执行 repmgr 工具或 sys_rman 备份时，出现故障。

4. **集群启动出现“双主”案例分析**

适用版本：KingbaseES V9（所有版本）

故障现象：

集群在启动过程中，出现“双主”的故障，对于“双主”故障，需要人工参与，判断集群中那个节点是最新的主库，重新恢复集群。

如下图所示，集群节点状态显示：node249 为集群主节点（primary），node248 为集群备节点（standby），但是数据库启动为主库状态。

```
[kingbase@node2 bin]$ ./repmgr cluster show
 ID | Name     | Role    | Status                | Upstream | Location | Priority | Timeline | Connection string
----+---------+---------+----------------------+----------+----------+----------+--------
 1  | node248 | standby | ! running as primary |          | default  | 100      | 5        | host=192.168.7.248
user=esrep dbname=esrep port=54321 connect_timeout=10 keepalives=1 keepalives_idle=10 keepalives_interval=1
keepalives_count=3
 2  | node249 | primary | * running            |          | default  | 100      | 4        | host=192.168.7.249
user=esrep dbname=esrep port=54321 connect_timeout=10 keepalives=1 keepalives_idle=10 keepalives_interval=1
keepalives_count=3

WARNING: following issues were detected
  - node "node248" (ID: 1) is running as primary but the repmgr node record is inactive
```

问题分析：

此故障一般当集群主机掉电重启或 failover 切换后，原主库没有正常被降为备库，加入集群。对于此类问题可以采用以下步骤处理：

1). 查看主备库数据库服务进程，是否正常启动了数据库服务。

2). 通过控制文件对比两个“主库”之间的 timeline、checkpoint 的 lsn 及事务 ID，通常选择更新的时间线、lsn 记录或事务 ID 确定最新的主库。但是对于生产环境，需要通过业务数据来确定最新的主库。

```
[kingbase@node101 bin]$./sys_controldata -D data
```

3). 确定最新主库后，可以将另外的节点重新加入集群。

问题解决：

1). 确定新主库后，将备库节点重新 rejoin，作为 standby 加入集群。

2). 测试主备流复制状态及集群状态。

3). 确定主备库数据安全。

详见存在双主（不自动恢复故障节点）

问题总结

对于集群“双主”或“脑裂”问题，要人工及时发现并介入处理，首先要确定集群最新的主库节点，对于生产环境，应该以业务数据作为首选项，然后将另外的节点，作为 standby 节点重新加入集群。

当系统启动防火墙、节点间网络阻塞、备库 data 目录下缺少 standby.signal 文件等，启动集群后都有可能导致“双主”问题复现，因此在启动集群前，需要做对应的检查。

10.4.2　常用集群相关操作

集群常用集群操作参见 常用集群操作

10.4.3 常用 SQL 命令

10.4.3.1 数据库版本的查询

查询数据库版本：

```
select version();
```

10.4.3.2 数据库连接情况

查看支持的最大连接数量：

```
Show max_connections;
```

查看所有连接状态：

```
select * from sys_stat_activity;
```

关闭连接：

```
Select sys_terminate_backend(pid);
```

备注:

sys_terminate_backend(pid)：1) 需要 superuser 权限，可以关闭所有的后台进程；2) 向后台发送 SIGTERM 信号，关闭事务、关闭 Process。

10.4.3.3 查看 license 到期时间

```
select get_license_validdays();
```

10.4.3.4 查看锁进程及对象

通过系统视图 sys_locks、sys_stat_activity 去查看。

10.4.3.5 kill session

1. 通过“ps -ef|grep kingbase”命令查看 KingbaseES 的所有进程，除辅助进程外的其它进程为 session 对应的进程

2. 找出想要杀死的 session 对应的进程 ID，通过输入“kill -9 + 想要杀死的进程 ID”，如下图：

```
[kingbase@test11 pkg]$ ps -ef | grep kingbase
root     17779 22710  0 19:55 ?        00:00:00 sshd: kingbase [priv]
root     17782 22710  0 19:55 ?        00:00:00 sshd: kingbase [priv]
kingbase 17783 17779  0 19:55 ?        00:00:00 sshd: kingbase@pts/2
kingbase 17795 17782  0 19:55 ?        00:00:00 sshd: kingbase@notty
kingbase 17798 17795  0 19:55 ?        00:00:00 /usr/libexec/openssh/sftp-server
kingbase 17810 17783  0 19:55 pts/2    00:00:00 -bash
kingbase 22687     1  0 20:02 ?        00:00:00 /home/kingbase/cluster/test1/kingbase/bin/kingbase -D /home/kingbase/cluster/test1/kingbase/data
kingbase 22688 22687  0 20:02 ?        00:00:00 kingbase: logger
kingbase 22690 22687  0 20:02 ?        00:00:00 kingbase: checkpointer
kingbase 22691 22687  0 20:02 ?        00:00:00 kingbase: background writer
kingbase 22692 22687  0 20:02 ?        00:00:00 kingbase: walwriter
kingbase 22693 22687  0 20:02 ?        00:00:00 kingbase: autovacuum launcher
kingbase 22694 22687  0 20:02 ?        00:00:00 kingbase: archiver   last was 000000010000000000000003.00000028.backup
kingbase 22695 22687  0 20:02 ?        00:00:00 kingbase: stats collector
kingbase 22696 22687  0 20:02 ?        00:00:00 kingbase: kwr collector
kingbase 22697 22687  0 20:02 ?        00:00:00 kingbase: ksh writer
kingbase 22698 22687  0 20:02 ?        00:00:00 kingbase: ksh collector
kingbase 22699 22687  0 20:02 ?        00:00:00 kingbase: logical replication launcher
kingbase 22866 22687  0 20:02 ?        00:00:00 kingbase: walsender esrep 10.10.8.12(48233) streaming 0/40047C8
kingbase 23221 22687  0 20:02 ?        00:00:00 kingbase: esrep esrep 10.10.8.11(21350) idle
kingbase 23223     1  0 20:02 ?        00:00:00 /home/kingbase/cluster/test1/kingbase/bin/repmgrd -d -v -f /home/kingbase/cluster/test1/kingbase/bin/../etc/repmgr.conf
kingbase 23294 22687  0 20:02 ?        00:00:00 kingbase: esrep esrep 10.10.8.12(48247) idle
kingbase 23400     1  0 20:02 ?        00:00:00 /home/kingbase/cluster/test1/kingbase/bin/kbha -A daemon -f /home/kingbase/cluster/test1/kingbase/bin/../etc/repmgr.conf
kingbase 23478 22687  0 20:02 ?        00:00:00 kingbase: esrep esrep 10.10.8.11(21412) idle
kingbase 23592 23400  0 20:03 ?        00:00:00 ping -q -c3 -w2 10.10.0.1
kingbase 23593 17810  0 20:03 pts/2    00:00:00 ps -ef
kingbase 23594 17810  0 20:03 pts/2    00:00:00 grep --color=auto kingbase
```

3. 再次通过“ps -ef|grep kingbase”查看 session 对应的进程 ID 是否杀死，如下图：

```
[kingbase@test11 pkg]$ ps -ef | grep kingbase
root     17779 22710  0 19:55 ?        00:00:00 sshd: kingbase [priv]
root     17782 22710  0 19:55 ?        00:00:00 sshd: kingbase [priv]
kingbase 17783 17779  0 19:55 ?        00:00:00 sshd: kingbase@pts/2
kingbase 17795 17782  0 19:55 ?        00:00:00 sshd: kingbase@notty
kingbase 17798 17795  0 19:55 ?        00:00:00 /usr/libexec/openssh/sftp-server
kingbase 17810 17783  0 19:55 pts/2    00:00:00 -bash
kingbase 22687     1  0 20:02 ?        00:00:00 /home/kingbase/cluster/test1/kingbase/bin/kingbase -D /home/kingbase/cluster/test1/kingbase/data
kingbase 22688 22687  0 20:02 ?        00:00:00 kingbase: logger
kingbase 23223     1  0 20:02 ?        00:00:00 /home/kingbase/cluster/test1/kingbase/bin/repmgrd -d -v -f /home/kingbase/cluster/test1/kingbase/bin/../etc/repmgr.conf
kingbase 23400     1  0 20:02 ?        00:00:00 /home/kingbase/cluster/test1/kingbase/bin/kbha -A daemon -f /home/kingbase/cluster/test1/kingbase/bin/../etc/repmgr.conf
kingbase 23875 22687  0 20:04 ?        00:00:00 kingbase: checkpointer
kingbase 23876 22687  0 20:04 ?        00:00:00 kingbase: background writer
kingbase 23877 22687  0 20:04 ?        00:00:00 kingbase: walwriter
kingbase 23878 22687  0 20:04 ?        00:00:00 kingbase: autovacuum launcher
kingbase 23879 22687  0 20:04 ?        00:00:00 kingbase: archiver
kingbase 23880 22687  0 20:04 ?        00:00:00 kingbase: stats collector
kingbase 23881 22687  0 20:04 ?        00:00:00 kingbase: kwr collector
kingbase 23882 22687  0 20:04 ?        00:00:00 kingbase: ksh writer
kingbase 23883 22687  0 20:04 ?        00:00:00 kingbase: ksh collector
kingbase 23884 22687  0 20:04 ?        00:00:00 kingbase: logical replication launcher
kingbase 23887 22687  0 20:04 ?        00:00:00 kingbase: esrep esrep 10.10.8.11(21634) idle
kingbase 23888 17810  0 20:04 pts/2    00:00:00 ps -ef
kingbase 23889 17810  0 20:04 pts/2    00:00:00 grep --color=auto kingbase
```

10.4.3.6　查看慢 sql

- 方法一：

  1. 在 kingbase.auto.conf 配置文件中配置如下参数，如果有 SQL 执行时间超过 1000ms，会把执行的 SQL 语句记录在日志中。

```
Log_min_duration_statement=1000
Log_statement='mod'
```

  2. 重启数据库生效

- 方法二

  执行 SQL 命令（需要超级用户权限）：

```
set Log_min_duration_statement=1000;
set Log_statement='mod';
```

10.4.3.7　统计不同类型数据库对象数量

Table,view,sequence,function,procedure,package,packagebody,materialized view,type,type body 等。

- 获取表的数量，通过

```
select count(*) from sys_class where relkind='r' or relkind='t' or relkind='f' or relkind='p';
```

其中‘r’表示普通标，’t’表示 TOAST 表，‘f’表示外部表，’p’表示分区表

```
test=#
test=# select count(*) from pg_class where relkind='r' or relkind='t' or relkind='f' or relkind='p';
 count
-------
   161
(1 行记录)
```

- 获取视图数量，通过

```
select count(*) from sys_class where relkind='v';
```

获取视图数量

```
test=# select count(*) from pg_class where relkind='v';
 count
-------
   383
(1 行记录)
```

- 获取 sequence 数量，通过

```
select count(*) from sys_class where relkind='S';
```

获取 sequence 的数量

```
test=# select count(*) from pg_class where relkind='S';
 count
-------
     1
(1 行记录)
```

- 获取 function 数量，通过

```
select count(*) from sys_proc where prokind='f' or prokind='a' or prokind='w';
```

获取 function 的数量

```
test=# select count(*) from pg_proc where prokind='f' or prokind='a' or prokind='w';
 count
-------
  4105
(1 行记录)
```

- 获取 procedure 数量，通过

```
select count(*) from sys_proc where prokind='p';
```

获取 procedure 的数量

```
test=# select count(*) from pg_proc where prokind='p';
 count
-------
    35
(1 行记录)
```

- 获取 type 数量，通过

```
select count(*) from sys_type;
```

获取 type 数量。

```
test=# select count(*) from pg_type;
 count
-------
   823
(1 行记录)
```

10.4.3.8　获取对象 DDL

1. 创建插件 dbms_metadata
2. 执行 SQL 命令‘select dbms_metadata.get_ddl(‘table’,‘t1’);’

```
test=# create extension dbms_metadata;
CREATE EXTENSION
test=#
test=# select dbms_metadata.get_ddl('table','t1');
          get_ddl
----------------------------
 CREATE TABLE PUBLIC.T1( +
  A INTEGER,             +
  B TEXT                 +
 )                       +

(1 行记录)
```

** 注意：** 以上操作需要超级用户权限。

10.4.3.9　查询表及表行数

通过命令：

```
select relname, reltuples from sys_class cls left join sys_namespace N ON (N.oid = CLS.relnamespace) where
nspname NOT IN ('sys_catalog', 'information_schema') AND cls.relkind = 'r' order by reltuples desc;
```

如下图：

```
test=# select relname, reltuples from pg_class cls left join pg_namespace N ON (N.oid = CLS.relnamespace) where nspname NOT IN ('pg_catalog', 'information_schema') AND cls.relkind = 'r' order by reltuples des>
          relname          | reltuples
---------------------------+-----------
 t1                        |     1e+06
 dual                      |         1
 sysmac_policy_enforcement |         0
 sysmac_user               |         0
 sysmac_label              |         0
 sysmac_compartment        |         0
 sysmac_level              |         0
 sysmac_obj                |         0
 sysmac_policy             |         0
(9 行记录)
```

10.4.3.10　查询包含 lob 字段的表

通过如下命令查询包括 lob 字段的表

```
select relname from sys_class,sys_attribute,sys_type where sys_class.oid = sys_attribute.attrelid and sys_
attribute.atttypid=sys_type.oid and sys_type.typname like '%lob';
```

```
test=# select relname from pg_class,pg_attribute,pg_type where pg_class.oid = pg_attribute.attrelid and pg_attribute.atttypid=pg_type.oid and pg_type.typname like '%lob';
  relname
------------
 all_source
 t2
(2 行记录)
```

10.4.3.11　查询分区表及其对应的子分区

1. 通过‘d’命令查看当前数据库的所有表，通过类型一列查看该表是否为分区表

2. 对于分区表，通过 d+ 表名查看对应的子分区，如下图：

```
test=# \d
                    关联列表
 架构模式 |        名称          |  类型  | 拥有者
----------+----------------------+--------+--------
 public   | sys_stat_statements | 视图   | zlq
 public   | t1                   | 数据表 | zlq
 public   | t2                   | 数据表 | zlq
 public   | t3                   | 分区表 | zlq
 public   | t3_p1                | 数据表 | zlq
 public   | t3_p2                | 数据表 | zlq
 public   | t3_p3                | 数据表 | zlq
(7 行记录)

test=#
test=# \d+ t3
                          分区表 "public.t3"
 栏位 |  类型   | 校对规则 | 可空的 | 预设 | 存储  | 统计目标 | 描述
------+---------+----------+--------+------+-------+----------+------
 a    | integer |          |        |      | plain |          |
分区键值: RANGE (a)
分区: t3_p1 FOR VALUES FROM (MINVALUE) TO (10),
      t3_p2 FOR VALUES FROM (10) TO (20),
      t3_p3 FOR VALUES FROM (20) TO (MAXVALUE)
```

10.4.3.12　无效对象

通过命令：

```
select count(*) from sys_class where reloptions='{status=false}';
```

获取无效对象的个数

```
test=# select count(*) from pg_class where reloptions='{status=false}';
 count
-------
     0
(1 行记录)
```

10.4.4　数据库常用命令工具

10.4.4.1　ksql 命令的使用

登录数据库：ksql -U 用户名 -W 数据库名

登录数据库之后可做如下操作：

- 退出登录：`\q`
- 查看数据库列表：`\l`
- 查看默认模式下所有表的名称：`\dt`
- 查看某个表的结构：`\d table_name`
- 查看元命令帮助信息：`\?`

10.4.4.2　memstat

参见 memstat

10.4.4.3　sys_dump

参见 sys_dump

10.4.5　操作系统常用命令

10.4.5.1　查看硬件配置

1. CPU 信息

```
cat /proc/cpuinfo
lscpu | grep -E "Architecture|^CPU\(s\)|^CPU MHz|Model name"
```

```
[root@dbhost03 ~]# lscpu | grep -E "Architecture|^CPU\(s\)|^CPU MHz|Model name"
Architecture:          x86_64
CPU(s):                4
Model name:            Intel(R) Core(TM) i7-6700HQ CPU @ 2.60GHz
CPU MHz:               2592.000
```

2. MEMORY

```
cat /proc/meminfo
```

```
[root@dbhost03 ~]# cat /proc/meminfo
MemTotal:        4770676 kB
MemFree:         1994604 kB
MemAvailable:    4076664 kB
Buffers:            2224 kB
Cached:          2262668 kB
SwapCached:            0 kB
```

```
Active:           492420 kB
Inactive:        1849180 kB
Active(anon):      80428 kB
Inactive(anon):    46604 kB
Active(file):     411992 kB
Inactive(file):  1802576 kB
Unevictable:           0 kB
Mlocked:               0 kB
SwapTotal:       2097148 kB
SwapFree:        2097148 kB
Dirty:                24 kB
Writeback:             0 kB
AnonPages:         76728 kB
Mapped:            86792 kB
Shmem:             50324 kB
Slab:             229464 kB
SReclaimable:     152424 kB
SUnreclaim:        77040 kB
KernelStack:        6416 kB
PageTables:        11300 kB
NFS_Unstable:          0 kB
Bounce:                0 kB
WritebackTmp:          0 kB
CommitLimit:     4482484 kB
Committed_AS:    1024280 kB    # 当前已经申请的内存，并不表示已分配
VmallocTotal:   34359738367 kB
VmallocUsed:      198252 kB
VmallocChunk:   34359310332 kB
HardwareCorrupted:     0 kB
AnonHugePages:      4096 kB
CmaTotal:              0 kB
CmaFree:               0 kB
HugePages_Total:       0      # 大页空间，如果启用大页。
HugePages_Free:        0
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:       2048 kB
DirectMap4k:      122688 kB
DirectMap2M:     2899968 kB
DirectMap1G:     2097152 kB
```

```
free -m
```

```
[root@dbhost03 ~]# free -m
              total        used        free      shared  buff/cache   available
Mem:           4658         346        1949          49        2363        3984
Swap:          2047           0        2047
```

备注:

如果想查看内存型号、主频等信息，可以用 dmidecode | grep -P -A200 “Memorys+Device” | more 命令。

3. 存储空间

- 文件系统空间使用率

```
df -m
```

```
[root@dbhost03 scsi_disk]# df -m
Filesystem              1M-blocks  Used Available Use% Mounted on
devtmpfs                     2313     0      2313   0% /dev
tmpfs                        2330     1      2330   1% /dev/shm
tmpfs                        2330     5      2326   1% /run
tmpfs                        2330     0      2330   0% /sys/fs/cgroup
/dev/mapper/centos-root     17394  8968      8427  52% /
/dev/sdb                    20470  1053     19418   6% /data
/dev/sda1                    1014   185       830  19% /boot
tmpfs                         466     0       466   0% /run/user/0
```

- 文件系统 inode 使用率

```
df -i
```

```
[root@dbhost03 scsi_disk]# df -i
Filesystem                Inodes  IUsed    IFree IUse% Mounted on
devtmpfs                  592064    417   591647    1% /dev
tmpfs                     596334      2   596332    1% /dev/shm
tmpfs                     596334   1385   594949    1% /run
tmpfs                     596334     16   596318    1% /sys/fs/cgroup
/dev/mapper/centos-root  8910848 252374  8658474    3% /
/dev/sdb                10485760   4387 10481373    1% /data
/dev/sda1                 524288    385   523903    1% /boot
tmpfs                     596334      1   596333    1% /run/user/0
```

- 磁盘信息

```
fdisk -l | grep -i disk
```

```
[root@dbhost03 scsi_disk]# fdisk -l | grep -i disk
Disk /dev/sda: 21.5 GB, 21474836480 bytes, 41943040 sectors
Disk label type: dos
Disk identifier: 0x000da0e4
Disk /dev/sdb: 21.5 GB, 21474836480 bytes, 41943040 sectors
Disk /dev/mapper/centos-root: 18.2 GB, 18249416704 bytes, 35643392 sectors
Disk /dev/mapper/centos-swap: 2147 MB, 2147483648 bytes, 4194304 sectors
```

```
ls -l /dev/disk/by-uuid
```

```
[root@dbhost03 scsi_disk]# ls -l /dev/disk/by-uuid
total 0
lrwxrwxrwx 1 root root  9 Mar  2 16:58 2019-09-11-18-50-31-00 -> ../../sr0
lrwxrwxrwx 1 root root 10 Mar  2 16:58 5b563fbc-7b54-4738-b2e5-f4fbf2c5cdf5 -> ../../dm-0
lrwxrwxrwx 1 root root 10 Mar  2 16:58 5ba0d132-3cd0-4b15-b194-33b373de322b -> ../../dm-1
lrwxrwxrwx 1 root root  9 Mar  2 16:58 acf1311f-c906-405f-b443-1dbb9e11ec3b -> ../../sdb
lrwxrwxrwx 1 root root 10 Mar  2 16:58 b572cafe-06f7-428b-a5f5-4546ea309fb9 -> ../../sda1
```

- 多路径信息

```
multipath -ll
```

```
[root@dbhost03 scsi_disk]# multipath -ll
Mar 05 10:25:15 | DM multipath kernel driver not loaded
Mar 05 10:25:15 | /etc/multipath.conf does not exist, blacklisting all devices.
Mar 05 10:25:15 | A default multipath.conf file is located at
Mar 05 10:25:15 | /usr/share/doc/device-mapper-multipath-0.4.9/multipath.conf
Mar 05 10:25:15 | You can run /sbin/mpathconf --enable to create
Mar 05 10:25:15 | /etc/multipath.conf. See man mpathconf(8) for more details
Mar 05 10:25:15 | DM multipath kernel driver not loaded
```

- 文件系统类型

```
mount
```

```
[root@dbhost03 scsi_disk]# mount
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
devtmpfs on /dev type devtmpfs (rw,nosuid,size=2368256k,nr_inodes=592064,mode=755)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000)
...
/dev/mapper/centos-root on / type xfs (rw,relatime,attr2,inode64,noquota)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=27,pgrp=1,timeout=0,minprc
debugfs on /sys/kernel/debug type debugfs (rw,relatime)
hugetlbfs on /dev/hugepages type hugetlbfs (rw,relatime)
mqueue on /dev/mqueue type mqueue (rw,relatime)
/dev/sdb on /data type xfs (rw,relatime,attr2,inode64,noquota)
/dev/sda1 on /boot type xfs (rw,relatime,attr2,inode64,noquota)
sunrpc on /var/lib/nfs/rpc_pipefs type rpc_pipefs (rw,relatime)
tmpfs on /run/user/0 type tmpfs (rw,nosuid,nodev,relatime,size=477068k,mode=700)
```

4. 网卡信息

```
ifconfig -a
```

```
[root@dbhost03 network-scripts]# ifconfig -a
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.237.43  netmask 255.255.255.0  broadcast 192.168.237.255
        inet6 fe80::de7b:791a:dc57:9cdb  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:8b:f9:92  txqueuelen 1000  (Ethernet)
        RX packets 39227  bytes 5882017 (5.6 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 3195  bytes 629076 (614.3 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 446684  bytes 63136681 (60.2 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 446684  bytes 63136681 (60.2 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

virbr0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 192.168.122.1  netmask 255.255.255.0  broadcast 192.168.122.255
        ether 52:54:00:23:68:93  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

virbr0-nic: flags=4098<BROADCAST,MULTICAST>  mtu 1500
        ether 52:54:00:23:68:93  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

ethtool 网卡名

```
[root@dbhost03 rules.d]# ethtool ens33
Settings for ens33:
        Supported ports: [ TP ]
        Supported link modes:   10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full
        Supported pause frame use: No
        Supports auto-negotiation: Yes
        Supported FEC modes: Not reported
        Advertised link modes:  10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full
        Advertised pause frame use: No
        Advertised auto-negotiation: Yes
        Advertised FEC modes: Not reported
        Speed: 1000Mb/s
        Duplex: Full
        Port: Twisted Pair
        PHYAD: 0
        Transceiver: internal
        Auto-negotiation: on
        MDI-X: off (auto)
        Supports Wake-on: d
        Wake-on: d
        Current message level: 0x00000007 (7)
                               drv probe link
        Link detected: yes
```

5. Swapinfo

- 查看 swap 使用情况

```
swapon -s
```

```
[root@dbhost03 sbin]# swapon -s
Filename                                Type            Size    Used    Priority
/dev/dm-1                               partition       2097148 0       -2
```

```
free -m
```

```
[root@dbhost03 sbin]# free -m
              total        used        free      shared  buff/cache   available
Mem:           4658         307        4011          47         340        4064
Swap:          2047           0        2047
```

6. 其它配置

```
lspci
```

```
[root@dbhost03 sbin]# lspci
00:00.0 Host bridge: Intel Corporation 440BX/ZX/DX - 82443BX/ZX/DX Host bridge (rev 01)
00:01.0 PCI bridge: Intel Corporation 440BX/ZX/DX - 82443BX/ZX/DX AGP bridge (rev 01)
00:07.0 ISA bridge: Intel Corporation 82371AB/EB/MB PIIX4 ISA (rev 08)
00:07.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01)
00:07.3 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
00:07.7 System peripheral: VMware Virtual Machine Communication Interface (rev 10)
00:0f.0 VGA compatible controller: VMware SVGA II Adapter
00:10.0 SCSI storage controller: Broadcom / LSI 53c1030 PCI-X Fusion-MPT Dual Ultra320 SCSI (rev 01)
00:11.0 PCI bridge: VMware PCI bridge (rev 02)
....
02:00.0 USB controller: VMware USB1.1 UHCI Controller
02:01.0 Ethernet controller: Intel Corporation 82545EM Gigabit Ethernet Controller (Copper) (rev 01)
02:02.0 Multimedia audio controller: Ensoniq ES1371/ES1373 / Creative Labs CT2518 (rev 02)
02:03.0 USB controller: VMware USB2 EHCI Controller
02:05.0 SATA controller: VMware SATA AHCI controller
```

10.4.5.2　系统配置相关

1. 操作系统版本

```
cat /etc/system-release
```

```
[root@dbhost03 ~]# cat /etc/system-release
CentOS Linux release 7.7.1908 (Core)
```

2. 内核版本

```
uname -a
```

```
[root@dbhost03 ~]# uname -a
Linux dbhost03 3.10.0-1062.el7.x86_64 #1 SMP Wed Aug 7 18:08:02 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux
```

3. 系统参数

```
sysctl -a
```

```
[root@dbhost03 ~]# sysctl -a
abi.vsyscall32 = 1
crypto.fips_enabled = 0
debug.exception-trace = 1
debug.kprobes-optimization = 1
debug.panic_on_rcu_stall = 0
dev.cdrom.autoclose = 1
dev.cdrom.autoeject = 0
dev.cdrom.check_media = 0
dev.cdrom.debug = 0
......
```

4. 系统模块

列出所有已加载的模块：

```
lsmod
```

```
[root@dbhost02 ~]# lsmod
Module                  Size  Used by
vmw_vsock_vmci_transport    30577  1
vsock                  35327  2 vmw_vsock_vmci_transport
snd_seq_midi           13565  0
snd_seq_midi_event     14899  1 snd_seq_midi
ppdev                  17671  0
iosf_mbi               13523  0
crc32_pclmul           13113  0
ghash_clmulni_intel    13259  0
```

modprobe -l 列出系统已经编译的模块（redhat7 已经没有 list 选项），或者用如下命令查看

```
ls /lib/modules/`uname -r`/kernel
```

5. 资源限制

```
ulimit -Ha
```

```
[root@dbhost03 security]# ulimit -Ha
core file size          (blocks, -c) unlimited
data seg size           (kbytes, -d) unlimited
scheduling priority             (-e) 0
file size               (blocks, -f) unlimited
pending signals                 (-i) 18501
max locked memory       (kbytes, -l) 64
max memory size         (kbytes, -m) unlimited
open files                      (-n) 4096
pipe size            (512 bytes, -p) 8
POSIX message queues     (bytes, -q) 819200
real-time priority              (-r) 0
stack size              (kbytes, -s) unlimited
cpu time               (seconds, -t) unlimited
max user processes              (-u) 18501
virtual memory          (kbytes, -v) unlimited
file locks                      (-x) unlimited
```

用户的进程个数限制：

```
cat /etc/security/limits.d/*conf
```

```
[root@dbhost03 security]# cat /etc/security/limits.d/*conf
# Default limit for number of user's processes to prevent
# accidental fork bombs.
# See rhbz #432903 for reasoning.

*          soft    nproc     4096
root       soft    nproc     unlimited
```

6. Selinux

```
cat /etc/selinux/config | grep ^SELINUX
```

```
[root@dbhost03 limits.d]# cat /etc/selinux/config | grep ^SELINUX
SELINUX=disabled
SELINUXTYPE=targeted
```

7. logind.conf

```
cat /etc/systemd/logind.conf
```

```
root@dbhost03:~# cat /etc/systemd/logind.conf
#  This file is part of systemd.
#
#  systemd is free software; you can redistribute it and/or modify it under the
#  terms of the GNU Lesser General Public License as published by the Free
#  Software Foundation; either version 2.1 of the License, or (at your option)
#  any later version.
#
# Entries in this file show the compile time defaults. Local configuration
# should be created by either modifying this file, or by creating "drop-ins" in
# the logind.conf.d/ subdirectory. The latter is generally recommended.
# Defaults can be restored by simply deleting this file and all drop-ins.
#
# Use 'systemd-analyze cat-config systemd/logind.conf' to display the full config.
#
# See logind.conf(5) for details.

[Login]
#NAutoVTs=6
#ReserveVT=6
#KillUserProcesses=no
#KillOnlyUsers=
#KillExcludeUsers=root
#InhibitDelayMaxSec=5
#UserStopDelaySec=10
#HandlePowerKey=poweroff
#HandleSuspendKey=suspend
#HandleHibernateKey=hibernate
#HandleLidSwitch=suspend
#HandleLidSwitchExternalPower=suspend
#HandleLidSwitchDocked=ignore
#HandleRebootKey=reboot
#HandleRebootKeyLongPress=poweroff
#PowerKeyIgnoreInhibited=no
#SuspendKeyIgnoreInhibited=no
#HibernateKeyIgnoreInhibited=no
#LidSwitchIgnoreInhibited=yes
#RebootKeyIgnoreInhibited=no
#HoldoffTimeoutSec=30s
#IdleAction=ignore
#IdleActionSec=30min
#RuntimeDirectorySize=10%
#RuntimeDirectoryInodes=400k
#RemoveIPC=yes
#InhibitorsMax=8192
#SessionsMax=8192
root@dbhost03:~#
```

8. system.conf

```
cat /etc/systemd/system.conf
```

```
root@dbhost03:~# cat /etc/systemd/system.conf
#  This file is part of systemd.
#
#  systemd is free software; you can redistribute it and/or modify it under the
#  terms of the GNU Lesser General Public License as published by the Free
#  Software Foundation; either version 2.1 of the License, or (at your option)
#  any later version.
#
# Entries in this file show the compile time defaults. Local configuration
# should be created by either modifying this file, or by creating "drop-ins" in
# the system.conf.d/ subdirectory. The latter is generally recommended.
# Defaults can be restored by simply deleting this file and all drop-ins.
#
# Use 'systemd-analyze cat-config systemd/system.conf' to display the full config.
#
# See systemd-system.conf(5) for details.

[Manager]
#LogLevel=info
#LogTarget=journal-or-kmsg
#LogColor=yes
#LogLocation=no
#LogTime=no
#DumpCore=yes
#ShowStatus=yes
#CrashChangeVT=no
#CrashShell=no
#CrashReboot=no
#CtrlAltDelBurstAction=reboot-force
#CPUAffinity=
#NUMAPolicy=default
#NUMAMask=
#RuntimeWatchdogSec=off
#RebootWatchdogSec=10min
#KExecWatchdogSec=off
#WatchdogDevice=
#CapabilityBoundingSet=
#NoNewPrivileges=no
#SystemCallArchitectures=
#TimerSlackNSec=
#StatusUnitFormat=description
#DefaultTimerAccuracySec=1min
#DefaultStandardOutput=journal
```

9. IO 调度算法

```
ls /sys/block/|while read diskname; do fdisk -l /dev/$diskname >&
/dev/null; test $? -eq 0 && (echo -n " 磁盘 $diskname : " ; cat
/sys/block/$diskname/queue/scheduler); done
```

```
root@dbhost03:~# ls /sys/block/|while read diskname; do    fdisk  -l /dev/$diskname
>& /dev/null;   test $? -eq 0 && (echo -n "磁盘 $diskname : " ; cat /sys/block/$disk
name/queue/scheduler) ; done
磁盘 loop0 : [mq-deadline] none
磁盘 nvme0n1 : [none] mq-deadline
磁盘 sda : [mq-deadline] none
磁盘 sdb : [mq-deadline] none
root@dbhost03:~#
```

10. SSH 配置

```
cat /etc/ssh/ssh_config
```

```
root@dbhost03:~# cat /etc/ssh/ssh_config

# This is the ssh client system-wide configuration file.  See
# ssh_config(5) for more information.  This file provides defaults for
# users, and the values can be changed in per-user configuration files
# or on the command line.

# Configuration data is parsed as follows:
#  1. command line options
#  2. user-specific file
#  3. system-wide file
# Any configuration value is only changed the first time it is set.
# Thus, host-specific definitions should be at the beginning of the
# configuration file, and defaults at the end.

# Site-wide defaults for some commonly used options.  For a comprehensive
# list of available options, their meanings and defaults, please see the
# ssh_config(5) man page.

Host *
#   ForwardAgent no
#   ForwardX11 no
#   ForwardX11Trusted yes
#   PasswordAuthentication yes
#   HostbasedAuthentication no
#   GSSAPIAuthentication no
#   GSSAPIDelegateCredentials no
#   GSSAPIKeyExchange no
#   GSSAPITrustDNS no
#   BatchMode no
#   CheckHostIP yes
#   AddressFamily any
#   ConnectTimeout 0
#   StrictHostKeyChecking ask
#   IdentityFile ~/.ssh/id_rsa
#   IdentityFile ~/.ssh/id_dsa
#   IdentityFile ~/.ssh/id_ecdsa
#   IdentityFile ~/.ssh/id_ed25519
#   Port 22
#   Protocol 2
#   Ciphers aes128-ctr,aes192-ctr,aes256-ctr,aes128-cbc,3des-cbc
#   MACs hmac-md5,hmac-sha1,umac-64@openssh.com
#   EscapeChar ~
#   Tunnel no
#   TunnelDevice any:any
#   PermitLocalCommand no
#   VisualHostKey no
#   ProxyCommand ssh -q -W %h:%p gateway.example.com
#   RekeyLimit 1G 1h
    SendEnv LANG LC_*
    HashKnownHosts yes
    GSSAPIAuthentication yes

#  每50秒给SSH服务器发送请求，防止中断
ServerAliveInterval 50
#  错误次数最大3次
ServerAliveCountMax 3
root@dbhost03:~#
```

11. 防火墙

```
systemctl status firewalld.service
```

```
root@dbhost03:/home/buxs/src/Linux# systemctl status firewalld.service
● firewalld.service - firewalld - dynamic firewall daemon
     Loaded: loaded (/lib/systemd/system/firewalld.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2022-06-12 12:22:05 CST; 7min ago
       Docs: man:firewalld(1)
   Main PID: 24761 (firewalld)
      Tasks: 2 (limit: 76365)
     Memory: 20.8M
     CGroup: /system.slice/firewalld.service
             └─24761 /usr/bin/python3 /usr/sbin/firewalld --nofork --nopid

6月 12 12:22:05 dbhost03 systemd[1]: Starting firewalld - dynamic firewall daemon...
6月 12 12:22:05 dbhost03 systemd[1]: Started firewalld - dynamic firewall daemon.
root@dbhost03:/home/buxs/src/Linux# iptables -n -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0            ctstate RELATED,ESTABLISHED
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0
INPUT_direct  all  --  0.0.0.0/0            0.0.0.0/0
INPUT_ZONES_SOURCE  all  --  0.0.0.0/0            0.0.0.0/0
INPUT_ZONES  all  --  0.0.0.0/0            0.0.0.0/0
DROP       all  --  0.0.0.0/0            0.0.0.0/0            ctstate INVALID
REJECT     all  --  0.0.0.0/0            0.0.0.0/0            reject-with icmp-host-prohibited

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0            ctstate RELATED,ESTABLISHED
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0
FORWARD_direct  all  --  0.0.0.0/0            0.0.0.0/0
FORWARD_IN_ZONES_SOURCE  all  --  0.0.0.0/0            0.0.0.0/0
FORWARD_IN_ZONES  all  --  0.0.0.0/0            0.0.0.0/0
FORWARD_OUT_ZONES_SOURCE  all  --  0.0.0.0/0            0.0.0.0/0
FORWARD_OUT_ZONES  all  --  0.0.0.0/0            0.0.0.0/0
DROP       all  --  0.0.0.0/0            0.0.0.0/0            ctstate INVALID
REJECT     all  --  0.0.0.0/0            0.0.0.0/0            reject-with icmp-host-prohibited

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
OUTPUT_direct  all  --  0.0.0.0/0            0.0.0.0/0

Chain INPUT_direct (1 references)
target     prot opt source               destination

Chain INPUT_ZONES_SOURCE (1 references)
target     prot opt source               destination

Chain INPUT_ZONES (1 references)
target     prot opt source               destination
IN_public  all  --  0.0.0.0/0            0.0.0.0/0            [goto]
IN_public  all  --  0.0.0.0/0            0.0.0.0/0            [goto]
IN_public  all  --  0.0.0.0/0            0.0.0.0/0            [goto]

Chain FORWARD_direct (1 references)
target     prot opt source               destination

Chain FORWARD_IN_ZONES_SOURCE (1 references)
target     prot opt source               destination
```

12. 多路径配置

```
cat /etc/multipathd.conf|grep -v '^#'|grep -v '^$'
```

```
root@dbhost03:/etc# cat /etc/multipathd.conf|grep -v '^#'|grep -v '^$'
defaults {
        udev_dir                /dev
        polling_interval        10
        selector                "round-robin 0"
        path_grouping_policy    failover
        getuid_callout          "/sbin/scsi_id -g -u -s /block/%n"
        prio_callout            "/bin/true"
        path_checker            tur
        rr_min_io               100
        rr_weight               uniform
        failback                immediate
        no_path_retry           12
        user_friendly_names     yes
}
blacklist {
        devnode         "^(ram|raw|loop|fd|md|dm-|sr|scd|st)[0-9]*"
        devnode         "^hd[a-z][[0-9]*]"
}
devices {
device {
        vendor                  "HP|COMPAQ"
        product                 "HSV1[01]1 COMPAQ|HSV2[01]0|HSV300|HSV4[05]0"
        path_grouping_policy    group_by_prio
        getuid_callout          "/sbin/scsi_id -g -u -s /block/%n"
        path_checker            tur
        path_selector           "round-robin 0"
        prio_callout            "/sbin/mpath_prio_alua /dev/%n"
        rr_weight               uniform
        failback                immediate
        hardware_handler        "0"
        no_path_retry           18
        rr_min_io               100
}
device {
        vendor                  "HP"
        product                 "P2000 G3 FC|P2000G3 FC/iSCSI"
        path_grouping_policy    group_by_prio
        getuid_callout          "/sbin/scsi_id -g -u -s /block/%n"
        path_checker            tur
        path_selector           "round-robin 0"
        prio_callout            "/sbin/mpath_prio_alua /dev/%n"
        rr_weight               uniform
        failback                immediate
        hardware_handler        "0"
        no_path_retry           18
        rr_min_io               100
}
device {
        vendor                  "HP"
        product                 "OPEN-.*"
        path_grouping_policy    multibus
        getuid_callout          "/sbin/scsi_id -g -u -s /block/%n"
        path_selector           "round-robin 0"
        rr_weight               uniform
        path_checker            tur
        hardware_handler        "0"
        failback                immediate
```

13. LVM 配置

```
lvs


pvs
```

```
vgs
```

```
root@dbhost03:~# lvs
  LV    VG     Attr       LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lvol0 appvg  -wi-ao----  60.00g
  lvol1 appvg  -wi-ao----  20.00g
  lvol2 appvg  -wi-ao---- <99.99g
  lvol3 appvg  -wi-ao----  60.00g
  home  centos -wi-ao---- <17.39g
  root  centos -wi-ao---- <35.61g
  swap  centos -wi-ao----   6.00g
root@dbhost03:~# pvs
  PV         VG     Fmt  Attr PSize   PFree
  /dev/sda2  centos lvm2 a--  <59.00g    0
  /dev/sdb   appvg  lvm2 a--  <80.00g    0
  /dev/sdc   appvg  lvm2 a--  <80.00g    0
  /dev/sdd   appvg  lvm2 a--  <80.00g    0
root@dbhost03:~# vgs
  VG     #PV #LV #SN Attr   VSize    VFree
  appvg    3   4   0 wz--n- <239.99g    0
  centos   1   3   0 wz--n-  <59.00g    0
root@dbhost03:~#
root@dbhost03:~#
root@dbhost03:~#
root@dbhost03:~#
root@dbhost03:~#
```

14. 文件系统类型、参数

```
mount
```

```
root@dbhost03:~# mount
文件系统 加载点 类型 参数
systemd-1 /proc/sys/fs/binfmt_misc autofs (rw,relatime,fd=22,pgrp=1,timeout=0,minproto=5,maxpro
to=5,direct,pipe_ino=18345)
binfmt_misc /proc/sys/fs/binfmt_misc binfmt_misc (rw,relatime)
cgroup /sys/fs/cgroup/blkio cgroup (rw,nosuid,nodev,noexec,relatime,blkio)
cgroup /sys/fs/cgroup/cpu,cpuacct cgroup (rw,nosuid,nodev,noexec,relatime,cpuacct,cpu)
cgroup /sys/fs/cgroup/cpuset cgroup (rw,nosuid,nodev,noexec,relatime,cpuset)
cgroup /sys/fs/cgroup/devices cgroup (rw,nosuid,nodev,noexec,relatime,devices)
cgroup /sys/fs/cgroup/freezer cgroup (rw,nosuid,nodev,noexec,relatime,freezer)
cgroup /sys/fs/cgroup/hugetlb cgroup (rw,nosuid,nodev,noexec,relatime,hugetlb)
cgroup /sys/fs/cgroup/memory cgroup (rw,nosuid,nodev,noexec,relatime,memory)
cgroup /sys/fs/cgroup/net_cls,net_prio cgroup (rw,nosuid,nodev,noexec,relatime,net_prio,net_cls
)
cgroup /sys/fs/cgroup/perf_event cgroup (rw,nosuid,nodev,noexec,relatime,perf_event)
cgroup /sys/fs/cgroup/pids cgroup (rw,nosuid,nodev,noexec,relatime,pids)
cgroup /sys/fs/cgroup/systemd cgroup (rw,nosuid,nodev,noexec,relatime,xattr,release_agent=/usr/
lib/systemd/systemd-cgroups-agent,name=systemd)
configfs /sys/kernel/config configfs (rw,relatime)
debugfs /sys/kernel/debug debugfs (rw,relatime)
devpts /dev/pts devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000)
devtmpfs /dev devtmpfs (rw,nosuid,size=23286112k,nr_inodes=5821528,mode=755)
hugetlbfs /dev/hugepages hugetlbfs (rw,relatime)
mqueue /dev/mqueue mqueue (rw,relatime)
nfsd /proc/fs/nfsd nfsd (rw,relatime)
proc /proc proc (rw,nosuid,nodev,noexec,relatime)
pstore /sys/fs/pstore pstore (rw,nosuid,nodev,noexec,relatime)
sunrpc /var/lib/nfs/rpc_pipefs rpc_pipefs (rw,relatime)
securityfs /sys/kernel/security securityfs (rw,nosuid,nodev,noexec,relatime)
sysfs /sys sysfs (rw,nosuid,nodev,noexec,relatime)
tmpfs /sys/fs/cgroup tmpfs (ro,nosuid,nodev,noexec,mode=755)
tmpfs /dev/shm tmpfs (rw,nosuid,nodev)
tmpfs /run tmpfs (rw,nosuid,nodev,mode=755)
tmpfs /run/user/0 tmpfs (rw,nosuid,nodev,relatime,size=4660636k,mode=700)
tmpfs /run/user/1004 tmpfs (rw,nosuid,nodev,relatime,size=4660636k,mode=700,uid=1004,gid=1008)
tmpfs /run/user/1006 tmpfs (rw,nosuid,nodev,relatime,size=4660636k,mode=700,uid=1006,gid=1010)
tmpfs /run/user/42 tmpfs (rw,nosuid,nodev,relatime,size=4660636k,mode=700,uid=42,gid=42)
/dev/mapper/appvg-lvol0 /opt/oracle/oraclebase xfs (rw,relatime,attr2,inode64,noquota)
/dev/mapper/appvg-lvol1 /opt/mysql/mysqlbase xfs (rw,relatime,attr2,inode64,noquota)
/dev/mapper/appvg-lvol2 /opt/postgres/pgsqlbase xfs (rw,relatime,attr2,inode64,noquota)
/dev/mapper/appvg-lvol3 /opt/Kingbase xfs (rw,relatime,attr2,inode64,noquota)
/dev/mapper/centos-home /home xfs (rw,relatime,attr2,inode64,noquota)
/dev/mapper/centos-root / xfs (rw,relatime,attr2,inode64,noquota)
/dev/sda1 /boot xfs (rw,relatime,attr2,inode64,noquota)
root@dbhost03:~#
```

15. 网络配置信息

```
cat /etc/sysconfig/network-scripts/ifcfg-网卡名
```

```
root@dbhost03:~# cat  /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE=eth0
BOOTPROTO=none
IPV6INIT=yes
NM_CONTROLLED="yes"
ONBOOT=yes
TYPE=Ethernet
IPADDR=192.168.20.20
GATEWAY=192.168.20.2
#DNS1=192.168.201.2
DNS1=114.114.114.114
PROXY_METHOD=none
BROWSER_ONLY=no
PREFIX=24
DEFROUTE=yes
IPV4_FAILURE_FATAL=no
IPV6_AUTOCONF=no
IPV6_DEFROUTE=no
IPV6_FAILURE_FATAL=no
#NAME="System eth0"
#UUID=5fb06bd0-0bb0-7ffb-45f1-d6edd65f3e03
root@dbhost03:~#
root@dbhost03:~#
root@dbhost03:~#
```

16. **如何生成 core 文件**

以 Centos7 为例：

1）. 确认针对用户的 core 文件大小没有限制。

```
ulimit -c
```

```
[kb86@dbhost02 ~]$ ulimit -c
unlimited
```

2）. 修改/etc/abrt/abrt-action-save-package-data.conf 文件，并重启 abrtd 服务（如果不行，重启服务器）。

```
OpenGPGCheck = no
ProcessUnpackaged = yes
```

3）. 确认/var/spool/abrt 目录下有生成 ccpp* 之类的文件。

10.4.5.3　查看系统运行状态

1. **内存信息**

```
sar -r 2 2
```

```
root@dbhost03:~# sar -r 2 2
Linux 3.10.0-957.el7.x86_64 (srv20)     10/18/2021      _x86_64_        (8 CPU)

06:17:08 PM kbmemfree kbmemused  %memused kbbuffers  kbcached  kbcommit   %commit  kbactive   kbinact   kbdirty
06:17:10 PM  17507684  29098652     62.43      3476  26559512  30218288     57.13  14219440  13482040       120
06:17:12 PM  17507732  29098604     62.43      3476  26559512  30218288     57.13  14219168  13482040       144
Average:     17507708  29098628     62.43      3476  26559512  30218288     57.13  14219304  13482040       132
root@dbhost03:~#
```

2. CPU 利用率

```
sar -u 2 2
```

```
root@dbhost03:~# sar -u 2 2
Linux 3.10.0-957.el7.x86_64 (srv20)     10/18/2021      _x86_64_        (8 CPU)

06:17:04 PM     CPU     %user     %nice   %system   %iowait    %steal     %idle
06:17:06 PM     all      0.00      0.00      0.06      0.00      0.00     99.94
06:17:08 PM     all      0.06      0.00      0.13      0.00      0.00     99.81
Average:        all      0.03      0.00      0.09      0.00      0.00     99.87
root@dbhost03:~#
```

3. 操作系统总 IO 大小

```
iostat 2 2
```

```
root@dbhost03:~# iostat 2 2
Linux 3.10.0-957.el7.x86_64 (srv20)     10/18/2021      _x86_64_        (8 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.11    0.00    0.14    0.01    0.00   99.75

Device:            tps    kB_read/s    kB_wrtn/s    kB_read    kB_wrtn
sda               0.71         7.78         7.17    1133306    1044465
sdc               1.07        11.80        56.17    1717737    8177240
sdd               2.23        22.62       107.10    3292592   15592671
sde               7.68        95.04        76.07   13836969   11075345
sdb               0.29         5.62         1.36     818308     198608
scd0              0.00         0.01         0.00       1028          0
dm-0              0.72         7.63         7.12    1110199    1037077
dm-1              0.00         0.02         0.00       3356          0
dm-2              0.01         0.04         0.04       6311       5303
dm-3              0.28         5.57         1.35     810860     196560
dm-4              0.02         0.04         0.01       6000       2048
dm-5              2.60        24.47       106.13    3562934   15450788
dm-6              0.81         9.91        57.14    1442951    8319123

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.06    0.00    0.00    0.00    0.00   99.94

Device:            tps    kB_read/s    kB_wrtn/s    kB_read    kB_wrtn
sda              15.50         0.00       169.50          0        339
sdc               0.00         0.00         0.00          0          0
sdd               0.00         0.00         0.00          0          0
sde               5.00        64.00        16.00        128         32
sdb               0.00         0.00         0.00          0          0
scd0              0.00         0.00         0.00          0          0
dm-0             16.00         0.00       169.50          0        339
dm-1              0.00         0.00         0.00          0          0
dm-2              0.00         0.00         0.00          0          0
dm-3              0.00         0.00         0.00          0          0
dm-4              0.00         0.00         0.00          0          0
dm-5              0.00         0.00         0.00          0          0
dm-6              0.00         0.00         0.00          0          0

root@dbhost03:~#
```

4. TOPIO

```
iotop -b -k -P -n 4
```

```
root@dbhost03:~# iotop -b -k -P -n 4
Total DISK READ :       0.00 K/s | Total DISK WRITE :      37.73 K/s
Actual DISK READ:       0.00 K/s | Actual DISK WRITE:       7.55 K/s
   PID  PRIO  USER     DISK READ  DISK WRITE  SWAPIN      IO    COMMAND
121406 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.03 % [kworker/5:2]
 67988 be/4 grid        0.00 K/s    3.77 K/s  0.00 %  0.00 % asm_gmon_+ASM
  9888 be/4 root        0.00 K/s   33.96 K/s  0.00 %  0.00 % python /usr/sbin/iotop -b -k -P -n 4
     1 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % systemd --switched-root --system --deserialize 22
     2 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [kthreadd]
     3 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [ksoftirqd/0]
     5 be/0 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [kworker/0:0H]
     7 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [migration/0]
     8 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [rcu_bh]
     9 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [rcu_sched]
    10 be/0 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [lru-add-drain]
    11 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [watchdog/0]
    12 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [watchdog/1]
    13 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [migration/1]
    14 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [ksoftirqd/1]
    16 be/0 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [kworker/1:0H]
    17 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [watchdog/2]
    18 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [migration/2]
    19 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [ksoftirqd/2]
    21 be/0 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [kworker/2:0H]
    22 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [watchdog/3]
    23 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [migration/3]
    24 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [ksoftirqd/3]
    26 be/0 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [kworker/3:0H]
    27 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [watchdog/4]
    28 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [migration/4]
    29 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [ksoftirqd/4]
    31 be/0 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [kworker/4:0H]
    32 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [watchdog/5]
    33 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [migration/5]
    34 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [ksoftirqd/5]
    36 be/0 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [kworker/5:0H]
    37 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [watchdog/6]
    38 rt/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [migration/6]
    39 be/4 root        0.00 K/s    0.00 K/s  0.00 %  0.00 % [ksoftirqd/6]
root@dbhost03:~#
```

5. TOPCPU

```
top -o %CPU -i -b -n 4
```

```
root@dbhost03:~# top -o %CPU -i -b -n 4
top - 18:17:39 up 1 day, 16:26,  2 users,  load average: 0.10, 0.22, 0.22
Tasks: 448 total,   1 running, 447 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.2 sy,  0.0 ni, 99.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem : 46606336 total, 17505432 free,  1913288 used, 27187616 buff/cache
KiB Swap:  6291452 total,  6291452 free,        0 used. 31594444 avail Mem

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
 68061 oracle    -2   0 8825184  19268  16384 S   1.0  0.0   4:15.96 ora_vktm_+
 67955 grid      -2   0 1554224  19480  16248 S   0.7  0.0   4:16.23 asm_vktm_+
  4460 root      20   0       0      0      0 S   0.3  0.0   0:00.29 kworker/1+
  7282 grid      20   0 2574864  94736  39632 S   0.3  0.2   6:42.58 ohasd.bin
 21463 kingbase  20   0 4574772  47116  35504 S   0.3  0.1   2:32.12 kingbase
 57759 grid      20   0 2076252  98928  46024 S   0.3  0.2   6:43.25 oraagent.+
 57847 grid      20   0  433488  22460  12480 S   0.3  0.0   2:43.53 evmlogger+
 68103 oracle    20   0 8825184  20808  17916 S   0.3  0.0   0:00.97 ora_smco_+
 68196 oracle    20   0 8830320  27448  20000 S   0.3  0.1   0:00.81 ora_p004_+
 70465 oracle    20   0 8827456  21108  18060 S   0.3  0.0   0:19.25 ora_o000_+
root@dbhost03:~#
```

6. 网络流量信息

```
sar -n DEV 2 2
```

```
root@dbhost03:~# sar -n DEV 2 2
Linux 3.10.0-957.el7.x86_64 (srv20)     10/18/2021      _x86_64_        (8 CPU)

06:17:22 PM     IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s
06:17:24 PM      eth0      1.00      0.00      0.06      0.00      0.00      0.00      0.00
06:17:24 PM        lo     13.00     13.00      2.77      2.77      0.00      0.00      0.00
06:17:24 PM virbr0-nic      0.00      0.00      0.00      0.00      0.00      0.00      0.00
06:17:24 PM    virbr0      0.00      0.00      0.00      0.00      0.00      0.00      0.00

06:17:24 PM     IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s
06:17:26 PM      eth0      1.00      0.00      0.06      0.00      0.00      0.00      0.00
06:17:26 PM        lo     10.00     10.00      2.17      2.17      0.00      0.00      0.00
06:17:26 PM virbr0-nic      0.00      0.00      0.00      0.00      0.00      0.00      0.00
06:17:26 PM    virbr0      0.00      0.00      0.00      0.00      0.00      0.00      0.00

Average:        IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s
Average:         eth0      1.00      0.00      0.06      0.00      0.00      0.00      0.00
Average:           lo     11.50     11.50      2.47      2.47      0.00      0.00      0.00
Average:    virbr0-nic      0.00      0.00      0.00      0.00      0.00      0.00      0.00
Average:       virbr0      0.00      0.00      0.00      0.00      0.00      0.00      0.00
root@dbhost03:~#
```

7. 文件系统使用率

```
df -m
```

```
root@dbhost03:~# df -m
Filesystem              1M-blocks  Used Available Use% Mounted on
/dev/mapper/centos-root     36447 27187      9260  75% /
devtmpfs                    22741     0     22741   0% /dev
tmpfs                       22757   637     22121   3% /dev/shm
tmpfs                       22757    13     22745   1% /run
tmpfs                       22757     0     22757   0% /sys/fs/cgroup
/dev/sda1                    1014   166       849  17% /boot
/dev/mapper/centos-home     17794  5000     12795  29% /home
/dev/mapper/appvg-lvol0     61420 42805     18616  70% /opt/oracle/oraclebase
/dev/mapper/appvg-lvol2    102378 59364     43015  58% /opt/postgres/pgsqlbase
/dev/mapper/appvg-lvol3     61430 16695     44736  28% /opt/Kingbase
/dev/mapper/appvg-lvol1     20470  8373     12098  41% /opt/mysql/mysqlbase
tmpfs                        4552     1      4552   1% /run/user/42
tmpfs                        4552     0      4552   0% /run/user/1006
tmpfs                        4552     0      4552   0% /run/user/1004
tmpfs                        4552     0      4552   0% /run/user/0
root@dbhost03:~#
```

8. 进程内存使用排名

```
ps auxc --sort=-%mem | more
```

```
root@dbhost03:~# ps auxc --sort=-%mem |more
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
mysql       1874  0.0  0.5 2069060 382596 ?      Ssl  11:20   0:04 mysqld
root        4045  0.0  0.2 1479684 192052 ?      Ssl  11:20   0:04 deepin-devicema
buxs        4526  0.1  0.2 1580884 170140 pts/0  Sl   11:20   0:09 system_server
buxs        4094  0.0  0.2 1024080 163240 ?      Sl   11:20   0:00 uengine
buxs        2316  0.0  0.2 8966348 150604 ?      Sl   11:20   0:01 dde-desktop
root        4389  0.0  0.2 1409076 134556 pts/0  Sl   11:20   0:01 main
buxs        2314  0.2  0.1 4936704 125424 ?      Sl   11:20   0:19 dde-session-dae
root        4390  0.0  0.1 1392856 124004 pts/0  Sl   11:20   0:04 main
10020       4632  0.0  0.1 1533572 119664 pts/0  Sl   11:20   0:00 ndroid.systemui
10000       4758  0.0  0.1 1540596 114596 pts/0  Sl   11:21   0:00 rg.anbox.appmgr
buxs        9899  0.2  0.1 2345272 113800 ?      Sl   11:35   0:16 dde-lock
buxs        2633  0.0  0.1 2952580 107260 ?      Sl   11:20   0:06 dde-dock
postgres    4716  0.0  0.1 1506048 99796 pts/0   Sl   11:21   0:00 m.android.phone
buxs        2866  0.0  0.1 2273220 96340 ?       Sl   11:20   0:00 dde-osd
buxs        4729  0.0  0.1 1538008 94872 pts/0   Sl   11:21   0:00 ndroid.settings
root        2090  0.0  0.1 1945008 89752 tty1    Ssl+ 11:20   0:04 Xorg
root       24437  0.0  0.1 2552084 88160 ?       SNl  12:22   0:02 deepin-home-app
10034       5109  0.0  0.1 1501752 86772 pts/0   Sl   11:21   0:00 m.android.email
10008       5035  0.0  0.1 1490204 86612 pts/0   Sl   11:21   0:00 d.process.media
root        3734  0.0  0.1 2342816 85116 ?       Ssl  11:20   0:02 dockerd
buxs        2384  0.0  0.1 2985520 80308 ?       Sl   11:20   0:00 kwin_x11
10003       5095  0.0  0.1 1485252 77848 pts/0   Sl   11:21   0:00 viders.calendar
10027       5071  0.0  0.1 1489812 76992 pts/0   Sl   11:21   0:00 ndroid.calendar
10032       4979  0.0  0.1 1494064 76880 pts/0   Sl   11:21   0:00 droid.deskclock
kingbase   52200  0.2  0.1 1495196 76608 pts/0   Sl   13:12   0:00 droid.bluetooth
10009       4960  0.0  0.0 1482080 65108 pts/0   Sl   11:21   0:00 id.ext.services
10011       5139  0.0  0.0 1483376 65024 pts/0   Sl   11:21   0:00 gedprovisioning
buxs        9957  0.0  0.0 360788 64900 ?        Sl   11:35   0:00 onboard
10013       5156  0.0  0.0 1482016 64816 pts/0   Sl   11:21   0:00 timeinitializer
10047       4997  0.0  0.0 1482156 64036 pts/0   Sl   11:21   0:00 android.smspush
buxs        2994  0.0  0.0 2207572 60568 ?       SNsl 11:20   0:00 booster [dtkwid
buxs        2429  0.0  0.0 727688 59484 ?        Sl   11:20   0:00 sogoupinyinServ
root        1978  0.1  0.0 184900 57148 ?        Sl   11:20   0:11 deepin-anything
root        2828  0.0  0.0 327184 56252 ?        Ssl  11:20   0:00 dde-file-manage
buxs        3266  2.0  0.0 573364 50596 ?        Sl   11:20   2:20 deepin-system-m
buxs        2271  0.0  0.0 266284 49640 ?        Sl   11:20   0:00 fcitx
root        1810  0.0  0.0 2453692 48676 ?       Ssl  11:20   0:05 containerd
root        1803  0.0  0.0 368980 47216 ?        Ss   11:20   0:02 glances
```

9. 系统内存段、信号量使用

```
ipcs --human -a
```

```
iroot@dbhost03:~# ipcs --human  -a

------ Message Queues --------
key        msqid      owner      perms      size         messages

------ Shared Memory Segments --------
key        shmid      owner      perms      size       nattch     status
0xb8bfa690 1605632    grid       600          28K      26
0x00000000 1638401    oracle     600         8.2M      160
0x00000000 1671170    oracle     600           8G      80
0x00000000 1703939    oracle     600         7.8M      80
0x033ca8b9 786436     kingbase   600          56B      9
0x033c9149 229381     kingbase   600          56B      7
0x010aaa31 1409030    postgres   600          56B      6
0x03380d09 491527     postgres   600          56B      8
0xefd75be4 1736712    oracle     600          40K      80

------ Semaphore Arrays --------
key        semid      owner      perms      nsems
0x3b848e40 2883584    grid       600        250
0x3b848e41 2916353    grid       600        250
0x3551582c 3047426    oracle     600        161
0x3551582d 3080195    oracle     600        161
0x3551582e 3112964    oracle     600        161
0x3551582f 3145733    oracle     600        161
0x35515830 3178502    oracle     600        161
0x033c9149 557063     kingbase   600        17
0x033c914a 589832     kingbase   600        17
0x033c914b 622601     kingbase   600        17
0x033c914c 655370     kingbase   600        17
0x033c914d 688139     kingbase   600        17
0x033c914e 720908     kingbase   600        17
0x033c914f 753677     kingbase   600        17
0x033c9150 786446     kingbase   600        17

root@dbhost03:~#
```

10. 操作系统日志（错误信息）

```
systemctl | more
```

```
root@dbhost03:~# systemctl |more
  UNIT                                                                                                                    LOAD   ACTIVE
 SUB       DESCRIPTION
  proc-sys-fs-binfmt_misc.automount                                                                                       loaded active
 running   Arbitrary Executable File Formats File System Automount Point
  sys-devices-pci0000:00-0000:00:02.1-0000:01:00.0-usb1-1\x2d9-1\x2d9:1.0-bluetooth-hci0.device                           loaded active
 plugged   /sys/devices/pci0000:00/0000:00:02.1/0000:01:00.0/usb1/1-9/1-9:1.0/bluetooth/hci0
  sys-devices-pci0000:00-0000:00:02.1-0000:01:00.1-ata5-host4-target4:0:0-4:0:0:0-block-sda-sda1.device                   loaded active
 plugged   Samsung_SSD_870_QVO_1TB 1
  sys-devices-pci0000:00-0000:00:02.1-0000:01:00.1-ata5-host4-target4:0:0-4:0:0:0-block-sda.device                        loaded active
 plugged   Samsung_SSD_870_QVO_1TB
  sys-devices-pci0000:00-0000:00:02.1-0000:01:00.1-ata6-host5-target5:0:0-5:0:0:0-block-sdb-sdb1.device                   loaded active
 plugged   ST2000LM015-2E8174 1
  sys-devices-pci0000:00-0000:00:02.1-0000:01:00.1-ata6-host5-target5:0:0-5:0:0:0-block-sdb.device                        loaded active
 plugged   ST2000LM015-2E8174
  sys-devices-pci0000:00-0000:00:02.1-0000:01:00.2-0000:02:00.0-0000:03:00.0-net-enp3s0.device                            loaded active
 plugged   RTL8111/8168/8411 PCI Express Gigabit Ethernet Controller (Motherboard (one of many))
  sys-devices-pci0000:00-0000:00:02.1-0000:01:00.2-0000:02:01.0-0000:04:00.0-net-wlp4s0.device                            loaded active
 plugged   Wireless-AC 9260
  sys-devices-pci0000:00-0000:00:02.2-0000:07:00.0-nvme-nvme0-nvme0n1-nvme0n1p1.device                                    loaded active
 plugged   ZHIKE SSD A300 1TB EFI
  sys-devices-pci0000:00-0000:00:02.2-0000:07:00.0-nvme-nvme0-nvme0n1-nvme0n1p2.device                                    loaded active
 plugged   ZHIKE SSD A300 1TB Boot
  sys-devices-pci0000:00-0000:00:02.2-0000:07:00.0-nvme-nvme0-nvme0n1-nvme0n1p3.device                                    loaded active
 plugged   ZHIKE SSD A300 1TB Roota
  sys-devices-pci0000:00-0000:00:02.2-0000:07:00.0-nvme-nvme0-nvme0n1-nvme0n1p4.device                                    loaded active
 plugged   ZHIKE SSD A300 1TB Rootb
  sys-devices-pci0000:00-0000:00:02.2-0000:07:00.0-nvme-nvme0-nvme0n1-nvme0n1p5.device                                    loaded active
 plugged   ZHIKE SSD A300 1TB _dde_data
  sys-devices-pci0000:00-0000:00:02.2-0000:07:00.0-nvme-nvme0-nvme0n1-nvme0n1p6.device                                    loaded active
 plugged   ZHIKE SSD A300 1TB Backup
  sys-devices-pci0000:00-0000:00:02.2-0000:07:00.0-nvme-nvme0-nvme0n1-nvme0n1p7.device                                    loaded active
 plugged   ZHIKE SSD A300 1TB SWAP
  sys-devices-pci0000:00-0000:00:02.2-0000:07:00.0-nvme-nvme0-nvme0n1.device                                              loaded active
 plugged   ZHIKE SSD A300 1TB
  sys-devices-pci0000:00-0000:00:08.1-0000:08:00.0-backlight-acpi_video0.device                                           loaded active
```

11. crontab

```
cat /etc/cron.d/*
cat /var/spool/cron/*
```

```
root@dbhost03:~# cat /etc/cron.d/*
/etc/cron.d/0hourly
# Run the hourly jobs
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
01 * * * * root run-parts /etc/cron.hourly

/etc/cron.d/raid-check
# Run system wide raid-check once a week on Sunday at 1am by default
0 1 * * Sun root /usr/sbin/raid-check


/etc/cron.d/sysstat
# Run system activity accounting tool every 10 minutes
*/10 * * * * root /usr/lib64/sa/sa1 1 1
# 0 * * * * root /usr/lib64/sa/sa1 600 6 &
# Generate a daily summary of process accounting at 23:53
53 23 * * * root /usr/lib64/sa/sa2 -A


root@dbhost03:~# cat /var/spool/cron/*
/var/spool/cron/root

/var/spool/cron/kingbase
# 1 1 1 1 1

root@dbhost03:~#
```

12. core 文件收集

```
ls -l /var/spool/abrt
```

```
root@dbhost03:~#
root@dbhost03:~# ls -l /var/spool/abrt/
总用量 0
root@dbhost03:~#
```

13. 如何查看 core 文件

core 文件创建在什么位置？

在当前进程的工作目录下创建，通常与程序在相同的路径下。但如果程序中调用了 chdir 函数，则有可能改变了当前工作目录。这时 core 文件创建在 chdir 指定的路径下。Redhat7 使用 abrtd （automatically bug report daemon）服务，core 文件会在/var/spool/abrt/ccpp* 目录下。

```
[root@dbhost02 ccpp-2021-04-08-11:19:43-1807]# file coredump
coredump: ELF 64-bit LSB core file x86-64, version 1 (SYSV), SVR4-style,
from 'kingbase: system test [local] CREATE', real uid: 1103, effective uid: 1103, real gid: 1103, effective gid: 1103,
execfn: '/opt/kb86/ES/V8/Server/bin/kingbase', platform: 'x86_64'
```

使用 gdb 分析 kingbase 生成的 core 文件。

gdb kingbase 二进制路径 core 文件路径

进入交互界面后 bt 显示堆栈，bt full 显示包括参数值的堆栈

10.4.6　软件包安装及升级管理

请参考手册安装与升级相关章节

版权声明

中电科金仓（北京）科技股份有限公司（简称：电科金仓）版权所有，并保留对本手册及本声明的一切权利。

未得到电科金仓的书面许可，任何人不得以任何方式或形式对本手册内的任何部分进行复制、摘录、备份、修改、传播、翻译成其他语言、将其全部或部分用于商业用途。

免责声明

本手册内容依据现有信息制作，由于产品版本升级或其他原因，其内容有可能变更。电科金仓保留在没有任何通知或者提示的情况下对手册内容进行修改的权利。

本手册仅作为使用指导，电科金仓在编写本手册时已尽力保证其内容准确可靠，但并不确保手册内容完全没有错误或遗漏，本手册中的所有信息也不构成任何明示或暗示的担保。

技术支持

- 电科金仓官方网站：http://www.kingbase.com.cn/
- 电科金仓文档中心：http://help.kingbase.com.cn/
- 全国服务热线：400-601-1188
- 电科金仓技术支持与反馈信箱：support@kingbase.com.cn

服务周期承诺

由于市场需求在不断变化，技术创新和发展的进程不断加剧，产品的版本更迭不可避免。电科金仓对于产品版本生命周期的有效管理，有助于您提前规划项目，更好地从产品服务终止上过渡。

表 1: KingbaseES 产品生命周期里程碑

关键里程碑点	定义
产品发布日期	产品正式发布版本，即 GA（general availability）版本的发布日期。
停止销售日期	正式停止销售的日期，版本停止接受订单日。该日之后，产品将不再销售。
停止功能升级日期	在该日期之后，不再提供新特性和新硬件支持。但依旧提供错误修复、安全修复、功能维护等服务。
停止功能维护日期	在该日期之后，不再维护功能，修复问题。但依旧提供安全修复等服务
停止安全维护日期	在该日期之后，不再发布补丁版本修复中高风险漏洞，仅提供有限的支持。
产品服务终止日期	停止提供产品服务和支持的日期。包括软件维护版本，缺陷修复，以及针对该产品的所有服务支持（包括服务热线和远程/现场支持）。

服务周期策略

金仓数据库管理系统 KingbaseES 产品确保以下的服务周期：

1）产品自发布之日起至产品停止功能升级（包含新特性、新硬件支持）之日止不少于 5 年。

2）产品停止功能升级之日起至产品停止功能维护（主要包括问题修复）之日止不少于 4 年。

3）产品功能维护停止之日起至产品停止安全维护（包括中高风险漏洞修复）之日止不少于 2 年。

针对金仓数据库管理系统 KingbaseES V9 产品（发布日期：2022 年 7 月 15 日），其具体服务周期安排如下：

- 产品销售日期：以产品销售合同所约定日期为准。
- 停止升级日期：自产品销售日期起计算，满 5 年之日止。
- 服务终止日期：自产品销售日期起计算，满 11 年之日止。

服务终止策略

金仓数据库管理系统 KingbaseES 产品确保在销售后，至少提供 6 年的服务支持。

备注:

电科金仓将会综合各方因素来确定产品服务终止日期。并将在实际产品服务终止日期之前至少 90 天，通过公开方式宣布产品服务终止日期。